

Spec-Driven Coding Interviews

The Advanced Reference Manual for Software Professionals

Harinath Mallepally

Contents

I	The Spec-Driven Paradigm for Technical Interviews	1
	Prologue: The Syntax Trap	2
0.1	The Coding Round Panic	2
0.2	The Cost of Raw Coding	2
0.3	The Spec-Driven Paradigm	3
0.4	What This Book Covers	3
0.5	How to Read This Book: Persona Profiles	5
0.6	How to Use This Book	6
0.7	The 14-Day Crash Course (2 Weeks)	6
0.8	The 28-Day Deep Dive (4 Weeks)	7
0.9	Pattern Recognition Quick Reference	8
1	The Invariant-First Interview Strategy	9
1.1	The Panic of the Blank Editor	9
1.2	Escaping the Syntax Trap: A Quick-Reference Guide	9
1.3	Defining the Invariant Wall	11
1.4	The Invariant Interview Framework	12
1.5	Worked Example: Proving Binary Search	12
2	The Three System-Scale Case Studies	16
2.1	AuraPay: Core Ledger & Asynchronous Settlement (Canonical)	16
2.2	ZenithTrade: High-Frequency Matching Engine (Exercise)	19
2.3	ChiramTrust: Decentralized Identity Consent Wallet (Exercise)	21
II	Code Design and Craftsmanship	23
3	Principles of Object-Oriented Design	24
3.1	The Anemic Domain Model Anti-Pattern	24
3.2	Refactoring Walkthrough: From Anemic to Rich	26
3.3	Abstraction & Encapsulation	27
3.4	OOP Principles vs. DDD Concepts	29
3.5	Composition over Inheritance	29
3.6	Polymorphism over Conditional Logic	30
4	SOLID Principles: Enforcing Boundaries	33

4.1	SOLID in the Senior Interview	33
4.2	The SOLID Transaction Pipeline	34
4.3	Single Responsibility Principle (SRP)	36
4.4	Open/Closed Principle (OCP)	36
4.5	Liskov Substitution Principle (LSP)	37
4.6	Interface Segregation Principle (ISP)	37
4.7	Dependency Inversion Principle (DIP)	37
4.8	SOLID Violation Detector & Remedies	38
4.9	Framework Integration: SOLID in Enterprise Containers	39
5	Modern Functional Programming and Stream APIs	40
5.1	The Imperative Loop Trap	40
5.2	The AuraPay Batch Pipeline	41
5.3	Understanding Method References (:: Syntax)	43
5.4	Stream Operations Deep-Dive	44
5.5	When to Use .map() vs Collectors Directly	45
5.6	Senior Interview Critical Knowledge: Performance Pitfalls	47
5.7	Performance Comparison: Imperative vs. Streams	48
5.8	Standard Streams vs. Reactive Streams	48
5.9	Stream Practice Questions	49
5.10	Debugging Functional Pipelines	50
6	Design Patterns in Enterprise Frameworks	52
6.1	Pattern Abuse in Interviews	52
6.2	Creational Patterns	52
6.3	Structural Patterns	54
6.4	Behavioral Patterns	54
6.5	Enterprise Integration & Data Access Patterns	57
6.6	Framework Integration: Patterns in the Wild	58
III	Code Performance and Data Structures	60
7	Designing for Performance and Concurrency	61
7.1	Concurrency in System Design Interviews	61
7.2	Concurrency Models: Virtual Threads vs. Reactive	61
7.3	Database Locking: Optimistic vs. Pessimistic	63
7.4	Concurrency Control & Locking Matrix	65
7.5	Caching Patterns & Consistency Deep-Dive	66
7.6	CPU Cache Locality (L1/L2/L3) in HFT Matching Loops	67
7.7	Connection Pool Sizing: The HikariCP Formula	67
8	Core Algorithms & Assessment Tactical Guide	70
8.1	General Coding Assessment (GCA) Tactics	70
8.2	Data Structures Primer for Coding Assessments	72
8.3	Broad LeetCode Structural Patterns	76
8.4	The Eight Worked Archetypal Problems	106
8.5	Mock General Coding Assessments	111

9	Q1 Mastery — Implementation Speed, In-Place Transformations, and String Processing	117
9.1	Essential Terminology & Vocabulary	117
9.2	Reusable Code Templates	120
9.3	Solved Exemplar Problems (20 Core Implementations)	121
9.4	CodeSignal Classic Q1 Patterns (12 Additional Solved Problems)	132
9.5	Practice Problem Bank (30 Detailed Problems with Hints)	139
10	Q2 Mastery — 2D Matrix Traversal, Grid Simulations, and State Machine Processing	147
10.1	Section 1: Essential Terminology & Vocabulary	147
10.2	Section 2: Reusable Code Templates	149
10.3	Section 3: Solved Exemplar Problems	150
10.4	Section 4: CodeSignal Classic Q2 Patterns (10 Additional Solved Problems)	161
10.5	Section 5: Practice Problem Bank	168
11	Q3 Mastery — Dynamic Sliding Windows, HashMap Frequency Signatures, and Prefix Sum Analytics	172
11.1	Section 1: Essential Terminology & Vocabulary	172
11.2	Section 2: Reusable Code Templates	174
11.3	Section 3: Solved Exemplar Problems	175
11.4	Section 4: CodeSignal Classic Q3 Patterns	183
11.5	Section 5: Practice Problem Bank	187
12	Q4 Mastery — Algorithmic Optimization: Binary Search Variants, Monotonic Structures, Dynamic Programming, and Graph Algorithms	191
12.1	Section 1: Essential Terminology & Vocabulary	191
12.2	Section 2: Reusable Code Templates	195
12.3	Section 3: Solved Exemplar Problems	198
12.4	Section 4: CodeSignal Classic Q4 Patterns (10 Additional Solved Problems)	211
12.5	Section 5: Practice Problem Bank (30 Detailed Problems with Hints)	216
13	20 Exam-Grade GCA Mock Problem Sets	220
13.1	How to Use This Chapter	220
13.2	Set 1: Warm-Up Fundamentals	220
13.3	Set 2: String & Matrix Basics	221
13.4	Set 3: Pattern Recognition	222
13.5	Set 4: Boundary Logic	223
13.6	Set 5: Simulation & State	224
13.7	Set 6: Array Manipulation	225
13.8	Set 7: Hash & Frequency	226
13.9	Set 8: Window & Range	227
13.10	Set 9: Two-Pointer Mastery	228
13.11	Set 10: Greedy & Optimization	229
13.12	Set 11: Tree & Recursion Simulation	230
13.13	Set 12: Bit & Math Tricks	231
13.14	Set 13: Advanced Search	232
13.15	Set 14: Interval & Schedule	233

13.16	Set 15: Stack & Queue Patterns	234
13.17	Set 16: Graph Exploration	235
13.18	Set 17: Dynamic Programming Gauntlet	236
13.19	Set 18: Binary Search Variants	237
13.20	Set 19: Advanced Data Structures	237
13.21	Set 20: Championship Round	239
IV	System Design & Architecture at Scale	240
14	System Architecture and Design Fundamentals	241
14.1	System Design in the Senior Interview	241
14.2	Domain-Driven Design (DDD) Boundaries	241
14.3	Monolithic vs. Microservices vs. Event-Driven	243
14.4	Scaling Out: Partitioning & Consistent Hashing	245
14.5	Command Query Responsibility Segregation (CQRS)	245
14.6	CAP Theorem & Distributed Trade-offs	245
14.7	API Design & Idempotency	246
14.8	ZenithTrade Order Lifecycle Sequence	246
14.9	API Rate Limiting Strategies	248
14.10	Caching Architectures	249
14.11	System Design Mock Interview: Sharded Order Matching Engine	251
15	Enterprise Integration and Resiliency	254
15.1	The Distributed Transaction Dilemma	254
15.2	The Dual-Write Anti-Pattern	254
15.3	Event Sourcing	257
15.4	Distributed Sagas	258
15.5	Distributed Rate Limiting	259
15.6	Microservice Resiliency Patterns	260
15.7	Microservices Observability	262
16	Database Design, Compliance, and Security	263
16.1	Database Architecture in Interviews	263
16.2	ACID vs. NoSQL: Choosing the Right Engine	263
16.3	Database Sharding Strategies	265
16.4	Indexing Deep-Dive & Performance Optimization	265
16.5	PCI-DSS Compliance & Tokenization	266
16.6	GDPR vs. Immutable Ledgers: Crypto-Shredding	269
16.7	SOC2 Audit Trails & Immutable Ledgers	270
16.8	Hardening the Data Tier & Audits	271
17	Behavioral Leadership and Technical Communication	273
17.1	Technical vs. Behavioral Alignment	273
17.2	The Technical STAR Framework	273
17.3	Mock Scenario A: Architectural Disagreement (Lead / Staff Perspective)	274
17.4	Mock Scenario B: Production Crisis Management (Engineering Manager Perspective)	275
17.5	Mock Scenario C: Balancing Technical Debt vs. Features (Director Perspective)	276

17.6	Checklist for Video (Teams) & In-Person Technical Interviews	277
18	Testing and CI/CD Strategies for High-Performance Systems	278
18.1	The Testing Paradigm in Senior Interviews	278
18.2	The Testing Pyramid	279
18.3	Advanced Testing Techniques	281
18.4	Feature Flags and Progressive Delivery	282
18.5	Continuous Integration (CI/CD) Compliance Pipeline	283
18.6	Modern Deployment Strategies	283
18.7	Performance Testing & Load Validation	284
19	Distributed Event Streaming and Message Brokers	287
19.1	Event Streaming in System Design	287
19.2	Apache Kafka Internals & Sharding	288
19.3	The Ordering Invariant & Partition Keys	289
19.4	Exactly-Once Semantics (EOS)	290
19.5	Consumer Group Rebalancing and Failure Recovery	291
19.6	Event Schema Evolution	291
19.7	Kafka vs. Event-Driven Alternatives	292
19.8	Message Broker Comparison Matrix	292
20	AI/ML System Design and LLM Integration	294
20.1	AI/ML in Technical Interviews	294
20.2	The AI/ML System Design Framework	295
20.3	Model Evaluation Metrics Deep-Dive	296
20.4	Vector Databases & Semantic Search	297
20.5	LLM Integration Patterns	298
20.6	Prompt Gateway Security	299
20.7	Cost Optimization for LLM-Powered Systems	300
21	Appendix: Quick Reference Cards and Cheat Sheets	302
21.1	Big-O Complexity Quick Reference	302
21.2	The Edge-Case Checklist	303
21.3	Distributed Systems Cheat Sheet	304
21.4	Day of the Interview Checklist	305
	References	306

Part I

The Spec-Driven Paradigm for Technical Interviews

Prologue: The Syntax Trap

“The greatest threat to software craftsmanship is not the speed of the typist, but the direction of their design.”

0.1 The Coding Round Panic

You sit in front of a blank IDE, the timer ticking down. You have seventy minutes to solve four algorithmic challenges on an online assessment platform. Your heart rate rises as you scan the first problem: a convoluted description of array manipulation designed to mimic real-world financial transaction reconciliation.

Without thinking, you begin typing. You declare variables, nesting loops to handle immediate edge cases. Ten minutes in, you run the initial test suite. Out of twenty test cases, only twelve pass. You patch a conditional check here, mutate a state variable there, and run the tests again. Now, fourteen pass, but two previously passing tests fail. You are caught in the “syntax trap”—the iterative, guessing-based cycle of code modification that degrades design quality in pursuit of green checkboxes.

This is where many experienced software developers, tech leads, and engineering managers fail. They treat coding assessments as a test of speed, syntax recall, and raw typing. They forget that the primary role of a senior engineer is not to type quickly, but to design systems that are secure, reliable, and maintainable.

0.2 The Cost of Raw Coding

In professional engineering environments, the cost of the “hack-and-test” mindset is catastrophic. When software is written without defined boundaries and invariants, systems suffer from structural drift, security vulnerabilities, and logic defects.

Industry data shows that software defects discovered in production cost up to one hundred times more to resolve than those identified during the design phase (National Institute of Standards and Technology [NIST], 2002). In banking-grade environments, a single state-corruption bug in a payment ledger can lead to financial reconciliation failures, regulatory penalties, and reputational damage. Yet, when candidates enter technical interviews, they routinely throw engineering discipline out the window. They write code without pre-conditions, modify state variables without constraints, and build systems that are impossible to reason about.

This manual is a rejection of that chaos. It is a guide to cracking coding assessments and architecture interviews by applying a rigorous, **spec-driven** approach to software design.

0.3 The Spec-Driven Paradigm

The spec-driven paradigm shifts the focus of the technical interview from coding to design. Instead of jumping directly into implementation, a spec-driven engineer establishes **design invariants** before writing a single line of code.

An invariant is a condition that must always remain true during the execution of a program. By defining these boundaries first, you build an “Invariant Wall” that constrains your implementation, making errors mathematically impossible. When you write code, you are simply translating these formal boundaries into clean, structured prose in your programming language of choice.

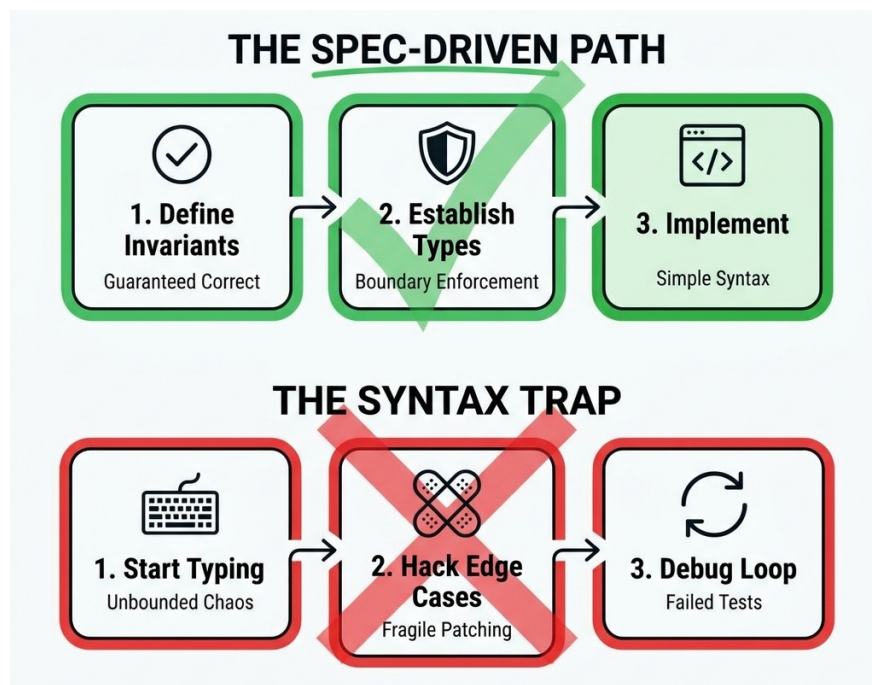


Figure 1: The Spec-Driven Path vs The Syntax Trap

0.4 What This Book Covers

This manual is organized into four parts spanning sixteen chapters, each targeting a specific dimension of the modern senior-level technical interview:

0.4.1 Part I: The Spec-Driven Paradigm for Technical Interviews

- **Chapter 1 — The Invariant-First Strategy:** How to define pre-conditions, post-conditions, and loop invariants before writing any code. Includes a step-by-step mathematical proof of Binary Search correctness.
- **Chapter 2 — The Three System-Scale Case Studies:** Introduction to the three

enterprise-grade systems (AuraPay, ZenithTrade, ChiramTrust) used as reference architectures throughout the book.

0.4.2 Part II: Code Design and Craftsmanship

- **Chapter 3 — Principles of Object-Oriented Design:** Self-validating domain entities, rich vs. anemic models, and refactoring walkthroughs.
- **Chapter 4 — SOLID Principles: Enforcing Boundaries:** Interface and dependency boundaries to keep systems modular and decoupled, with a violation detector cheat sheet.
- **Chapter 5 — Modern Functional Programming and Stream APIs:** Clean, declarative data pipelines that minimize side effects, with imperative-vs-stream comparisons and reactive stream explanations.
- **Chapter 6 — Design Patterns in Enterprise Frameworks:** GoF patterns (Builder, Singleton, Observer, State, Strategy), data access patterns (Repository, Unit of Work, DTO, Active Record vs. Data Mapper), and how frameworks like Spring and Hibernate implement them natively.

0.4.3 Part III: Code Performance and Data Structures

- **Chapter 7 — Designing for Performance and Concurrency:** Virtual threads, optimistic vs. pessimistic locking, connection pool sizing, caching strategies, and cache invalidation race conditions.
- **Chapter 8 — Core Algorithms & Assessment Tactical Guide:** The 70-minute GCA time allocation blueprint, pattern recognition flowchart, eight fully worked problems in three languages, and two full timed mock tests.

0.4.4 Part IV: System Design & Architecture at Scale

- **Chapter 9 — System Architecture and Design Fundamentals:** DDD bounded contexts, CQRS, CAP theorem trade-offs, consistent hashing, API idempotency, and a full sharded order matching engine mock interview transcript.
- **Chapter 10 — Enterprise Integration and Resiliency:** Transactional Outbox, Saga orchestration vs. choreography, event sourcing, Redis sliding window rate limiting, Circuit Breaker state machines, and OpenTelemetry distributed tracing.
- **Chapter 11 — Database Design, Compliance, and Security:** B-Tree vs. LSM-Tree storage engines, PCI-DSS tokenization vaults, SOC2 cryptographic audit trails, GDPR Crypto-Shredding, and sharding strategies.
- **Chapter 12 — Behavioral and Technical Leadership Interviews:** The Technical STAR Framework, video/Teams call checklists, and three full mock responses for senior leadership scenarios.
- **Chapter 13 — Testing and CI/CD Strategies:** The testing pyramid (unit, integration via Testcontainers, contract via Pact), and automated CI/CD release policies.
- **Chapter 14 — Distributed Event Streaming and Message Brokers:** Apache Kafka internals, partition-key sharding for in-order delivery, consumer group rebalancing, and Exactly-Once Semantics (EOS).
- **Chapter 15 — AI/ML System Design and LLM Integration:** Vector databases (HNSW vs. IVF indexes), Retrieval-Augmented Generation (RAG) pipelines, semantic caching, and prompt injection security filters.

- **Chapter 16 — Appendix and Quick-Reference Cheat Sheets:** Big-O complexity tables, edge-case checklists, system design latency numbers, and day-of-interview preparation guides.

0.5 How to Read This Book: Persona Profiles

To maximize the value of this manual, select the path that aligns with your career stage and current interview goals:

0.5.1 Persona A: The Mid-to-Senior Engineer (Target: Coding Assessments and GCA)

- **Goal:** Clear the 70-minute GCA speed run, optimize runtime performance, and handle live coding screens without panic.
- **Recommended Reading Path:**
 1. Read **Chapter 1 (Invariant-First Strategy)** to learn how to prove loops mathematically.
 2. Skip to **Part III (Chapters 7 & 8)**. Focus heavily on the GCA Time Allocation Blueprint, the 8 Worked Problems, and run the Mock Tests.
 3. Study **Part II (Chapters 3 & 5)** to learn functional stream optimizations and rich data structures.
 4. Review **Chapter 16 (Appendix)** for the Big-O cheat sheet and edge-case checklist before your assessment.

0.5.2 Persona B: The Lead / Staff Engineer (Target: System Design & Craftsmanship)

- **Goal:** Design clean microservices, establish domain boundaries, and explain complex distributed system tradeoffs to principal engineers.
- **Recommended Reading Path:**
 1. Read **Part I (Chapters 1 & 2)** to align on the core case studies.
 2. Master **Part II (Chapters 3–6)** on rich aggregate boundaries, strict SOLID inversion, and enterprise design patterns.
 3. Deep-dive into **Part IV (Chapters 9–11 & 13–15)**. Study the sharded order matching engine mock script, distributed Saga implementations, Kafka event streaming, and security compliance (PCI-DSS, SOC2, GDPR).

0.5.3 Persona C: The Engineering Manager / Director (Target: Architectural Strategy & Leadership)

- **Goal:** Evaluate team engineering standards, design resilient systems, and ensure operational compliance under regulatory frameworks.
- **Recommended Reading Path:**
 1. Read **Chapter 2 (Case Studies)** for enterprise system context.
 2. Study **Chapter 4 (SOLID boundaries)** to establish code quality metrics for your team.
 3. Focus on **Part IV (Chapters 9–11)**. Master the CAP theorem tradeoffs, disaster recovery models, rate-limiting patterns, and GDPR Crypto-Shredding architectures.

4. Read **Chapter 12 (Behavioral & Technical Leadership)** to prepare for the behavioral round with Technical STAR frameworks and full mock responses.

STAR Moment: The Invariant Principle

The best code is code that is correct by design. When you write a method, your first task is not to implement the algorithm, but to define the contract: what must be true *before* the method runs (pre-conditions), and what must be guaranteed *after* it completes (post-conditions). If you enforce these boundaries, the code inside the method almost writes itself.

0.6 How to Use This Book

This manual is designed for a dual audience. For individual engineers preparing for grueling CodeSignal General Coding Assessments (GCA), it provides a concrete, pattern-based approach to conquer algorithmic challenges under severe time constraints. For engineering leads and managers returning to coding assessments after years of management, it serves as a tactical refresher to translate high-level architectural knowledge back into executable, robust code. Treat this not just as a book, but as a systematic training plan.

0.7 The 14-Day Crash Course (2 Weeks)

For experienced engineers who need results fast. Follow this intensive schedule to rebuild your coding muscle memory quickly.

Day	Focus Area	Chapters	Practice Target	Time
1	Foundations & Mindset	Prologue, Ch 1-2 (Invariants & Case Studies)	Read and internalize the Invariant-First strategy	3-4 hrs
2	OOP & SOLID Refresher	Ch 3-4	Review patterns, do 5 practice problems mentally	2-3 hrs
3	Streams & Design Patterns	Ch 5-6	Write 3 stream pipelines from memory	2-3 hrs
4	Concurrency & Data Structures	Ch 7-8	Memorize Big-O table, implement 5 core algorithms	3-4 hrs
5	Q1 Mastery (Easy)	Ch 9	Solve 15 Q1 problems under 8-min timer each	4-5 hrs
6	Q2 Mastery (Medium)	Ch 10	Solve 10 Q2 matrix/grid problems under 15-min timer	4-5 hrs

Day	Focus Area	Chapters	Practice Target	Time
7	REST DAY	Review weak areas from Days 5-6	Light review only	1-2 hrs
8	Q3 Mastery (Medium-Hard)	Ch 11	Solve 10 Q3 sliding window/hashmap problems	4-5 hrs
9	Q4 Mastery (Hard)	Ch 12	Solve 8 Q4 DP/graph problems	4-5 hrs
10	Mock Exam Day 1	Ch 13 Sets 1-5	Full 70-min timed sessions (2 sets)	4 hrs
11	Mock Exam Day 2	Ch 13 Sets 6-10	Full 70-min timed sessions (2 sets)	4 hrs
12	System Design	Ch 14-16	Practice one mock system design interview	3-4 hrs
13	Behavioral + AI/ML	Ch 17-20	Write 5 STAR stories, review AI/ML concepts	3-4 hrs
14	Final Review & Mock	Ch 13 Sets 11-15, Appendix	Full mock exam + review weak patterns	4-5 hrs

- **Start each day** by reviewing the terminology section of the relevant chapter.
- **Time yourself on EVERY problem** — accuracy without speed is not enough for modern assessments.
- **Keep a ‘mistake log’** to track patterns you consistently get wrong.
- **On rest day**, revisit your mistake log, not new material.

0.8 The 28-Day Deep Dive (4 Weeks)

For candidates targeting Staff/Principal roles or those wanting thorough mastery, this comprehensive plan builds enduring architectural and algorithmic skills.

Week 1: Foundations & Design Thinking (Chapters 1-8) - Day 1-2: Invariants, Case Studies, OOP - Day 3-4: SOLID, Streams, Design Patterns - Day 5-6: Concurrency, Core Algorithms - Day 7: Review + implement 10 algorithms from memory

Week 2: Algorithm Mastery (Chapters 9-12) - Day 8-9: Q1 patterns — solve ALL 32 exemplar problems - Day 10-11: Q2 patterns — solve ALL 30 exemplar problems - Day 12-13: Q3 patterns — solve ALL 30 exemplar problems - Day 14: Q4 patterns — start with 15 problems

Week 3: Advanced Algorithms + System Design (Chapters 12-16) - Day 15-16: Finish Q4 patterns, solve remaining 15 problems - Day 17-18: Mock exams (Sets 1-10, two per day) -

Day 19-20: System Architecture, Resiliency, Database Design - Day 21: Review + identify weakest algorithm pattern

Week 4: Polish & Exam Readiness (Chapters 17-22 + Review) - Day 22-23: Behavioral Leadership + Testing/CI-CD - Day 24-25: Message Brokers, AI/ML + final mock exams (Sets 11-20) - Day 26-27: Full review — re-solve all problems you got wrong - Day 28: Final full mock exam under strict conditions + rest

0.9 Pattern Recognition Quick Reference

When you read a problem under pressure, use this decision tree to instantly map the specification to the correct invariant-first design:

1. **Is it asking for a single pass through an array/string?** → Two-pointer or sliding window
2. **Does it involve a sorted array or search space?** → Binary search
3. **Does it need grouping or counting?** → HashMap frequency signature
4. **Is it a 2D grid problem?** → BFS/DFS with direction vectors
5. **Does it ask for ‘minimum/maximum’ of something?** → DP or greedy
6. **Does it involve intervals?** → Sort by start/end, then merge/sweep
7. **Does ‘next greater/smaller’ appear?** → Monotonic stack
8. **Does it ask for ‘all combinations/permutations’?** → Backtracking
9. **Is it about connected components?** → Union-Find or DFS
10. **Does it have dependencies/ordering?** → Topological sort

Chapter 1

The Invariant-First Interview Strategy

“Before you build a system, draw the boundaries. Code written within correct boundaries cannot drift into error.”

1.1 The Panic of the Blank Editor

It is a common scenario in technical interviews: the interviewer presents a coding challenge, and the candidate immediately starts typing. They construct loops, initialize local counters, and write complex nested conditionals. The candidate is trying to solve the problem by writing code, using the editor as a scratching post to find a solution.

This approach is fragile. In the pressure of a live interview or a timed online assessment (like CodeSignal), writing code without a design roadmap leads to cognitive overload. You are trying to manage algorithmic logic, syntax rules, memory allocation, and edge cases simultaneously. When the first test run fails, you start modifying conditions arbitrarily—changing `<` to `<=`, adding random `+1` offsets, or introducing temporary boolean flags.

This is the “hack-and-test” methodology, and it signals to the interviewer that you lack structural discipline. A senior engineer or manager must demonstrate a systematic, predictable approach to code correctness. The solution is the **Invariant-First Strategy**.

1.2 Escaping the Syntax Trap: A Quick-Reference Guide

In high-stress technical interviews, a common failure point for senior engineers and engineering managers who do not write code every day is getting tripped up by basic language syntax or type conversions. Spending five minutes debugging why a string comparison isn’t working or how to convert an integer count into characters (chars) can derail an interview, even if your architectural approach is correct.

Below is a quick-reference guide for common type conversion and syntax pitfalls to help you stay focused on your design boundaries.

1.2.1 Number to Character Conversions

- **Char Digit to Int:**
 - *Incorrect:* `(int) '5'` returns 53 (its ASCII value).
 - *Correct:* `'5' - '0'` returns 5. Always subtract the character '0' offset.
- **Int Digit to Char:**
 - *Incorrect:* `(char) 5` returns a non-printable control character.
 - *Correct:* `(char) (5 + '0')` returns the character '5'.
- **Int to String:**
 - *Correct:* `String.valueOf(number)` or `number + ""` (using empty string concatenation).

1.2.2 Length, Size, and Count

The syntax for checking size varies across data types, and mixing them up is the most frequent syntax error under stress:

- **Array:** `arr.length` (Property — no parentheses)
- **String:** `str.length()` (Method — with parentheses)
- **Collections (List, Set, Map):** `list.size()` (Method — with parentheses)

1.2.3 String Manipulation Pitfalls

- **Immutability:** Strings in Java are immutable. You cannot modify a character in-place: `str.charAt(i) = 'c'` is a compilation error.
 - *Remedy:* Convert the string to a `StringBuilder` or a character array first:

```
StringBuilder sb = new StringBuilder(str);
sb.setCharAt(i, 'c');
String result = sb.toString();
```
- **Comparison:**
 - *Incorrect:* `str1 == str2` (compares object memory references).
 - *Correct:* `str1.equals(str2)` (compares character values).

1.2.4 Integer Division & Modulo

- **Integer Division:**
 - *Incorrect:* `5 / 2` evaluates to 2 (truncates decimals).
 - *Correct:* Cast at least one operand: `(double) 5 / 2` or `5.0 / 2` to yield 2.5.
- **Negative Modulo:**
 - *Incorrect:* `-5 % 2` evaluates to -1 in Java.
 - *Correct:* To guarantee a positive wrap-around index: `(val % N + N) % N`.

1.2.5 PriorityQueue (Heaps) Comparators

- *Incorrect:* `new PriorityQueue<>((a, b) -> a.val - b.val)` is susceptible to integer underflow/overflow if values can be negative.
- *Correct:* Use the safe comparison method: `new PriorityQueue<>((a, b) -> Integer.compare(a.val, b.val))`.

1.3 Defining the Invariant Wall

The Invariant-First Strategy requires you to define the mathematical and logical boundaries of your solution before implementing any code. In computer science, an **invariant** is a property that remains true throughout a specific phase of execution.

When you apply this to coding assessments, you construct an “Invariant Wall” composed of three layers:

1. **Pre-conditions:** Constraints on the inputs that must be true before a function or method is executed. If a caller violates a pre-condition, the method should fail immediately (e.g., throwing an `IllegalArgumentException` in Java).
2. **Post-conditions:** Guarantees that the method promises to satisfy upon successful execution. This defines what “correctness” means for the operation.
3. **Class/Data Invariants:** State rules that must always hold true for a domain object throughout its entire lifecycle.

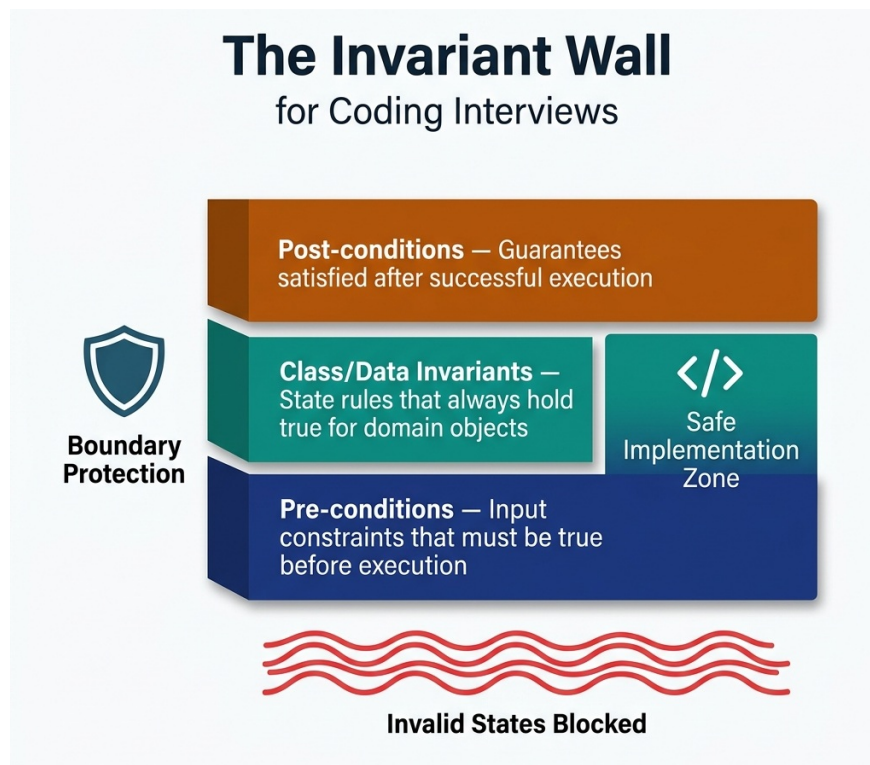


Figure 1.1: The Invariant Wall

By declaring these boundaries upfront, you decouple *what* the system must do from *how* it will do it. You establish a contract. Once the contract is clear, writing the code is simply a matter of executing that contract.

1.4 The Invariant Interview Framework

When faced with a technical coding challenge in an interview, follow this four-step spec-driven framework:

1.4.1 Define the Boundary Invariants (Clarification Phase)

Before writing any code, state the inputs, outputs, and their mathematical bounds. For example, if you are asked to process a list of transactions:

- What are the pre-conditions? Can the input list be null or empty? Can transaction amounts be negative?
- What are the post-conditions? Does the output preserve the original order of transactions? How are duplicate entries handled? Write these down as comments in the IDE or verbalize them to the interviewer.

1.4.2 Declare the Data Invariants (Type Design Phase)

Design your types to enforce invariants natively. Do not use generic types (like raw integers or strings) where a domain-specific type can prevent invalid states. For example, instead of passing a raw `double` representing a monetary amount, define a `Money` record that guarantees the amount cannot be negative and uses the correct currency scale.

1.4.3 Establish the Loop Invariants (Algorithmic Phase)

If your algorithm requires an iterative process (such as a search or a sliding window), define what remains true during each iteration of the loop. For instance, in a sliding window algorithm finding the maximum subarray sum:

- *Loop Invariant:* At the start of each iteration `i`, `current_window_sum` represents the sum of elements from index `left` to `i - 1`. If you can maintain this invariant, your loop is guaranteed to be correct, and off-by-one errors are eliminated.

1.4.4 Implement and Enforce

Write the code, beginning with explicit checks for your pre-conditions. Use modern language features to keep the code clean and expressive, ensuring that your logic never violates the declared invariants.

1.5 Worked Example: Proving Binary Search

To demonstrate the mathematical power of invariants, let us examine the classic binary search algorithm. Many developers struggle with binary search, often getting trapped in infinite loops or off-by-one errors because they guess the boundary updates (e.g., `right = mid` vs. `right = mid - 1`).

1.5.1 The Challenge

Given a sorted array of integers `nums` and a `target` value, return the index of the `target` if it exists in the array, or `-1` if it does not.

1.5.2 Define the Boundaries (Step 1)

- **Pre-condition:** `nums` is sorted in ascending order.
- **Post-condition:** The returned index `idx` satisfies `nums[idx] == target`, or if `idx == -1`, then `target ∉ nums`.

1.5.3 Establish the Loop Invariant (Step 3)

We define two pointers, `left` and `right`, defining our active search range $[left, right]$.

- **The Loop Invariant:** *If target is present in the array, it must reside within the index boundaries:*

$$\text{Invariant } P(left, right) : target \in nums[left \dots right]$$

1.5.4 Mathematical Proof of Correctness

To prove the algorithm is correct, we must prove three properties of our loop invariant:

1.5.4.1 A. Initialization

Before the loop starts, the invariant must hold true. We initialize `left = 0` and `right = nums.length - 1`.

- Since the array is sorted, if the target is in the array, it must be within the range $[0, nums.length - 1]$. The invariant holds.

1.5.4.2 B. Maintenance

If the invariant is true before an iteration, we must prove it remains true after updating our pointers. During the loop, we calculate:

$$mid = left + \frac{right - left}{2}$$

We check three cases:

Case 1: `nums[mid] == target`

The target is found, and we return `mid`, satisfying the post-condition.

Case 2: `nums[mid] < target`

Since the array is sorted, all elements at or to the left of `mid` are strictly less than the target. Therefore, the target cannot reside in the range $[left, mid]$.

We update `left = mid + 1`. The new range is $[mid + 1, right]$. If the target exists, it must lie within this new range. The invariant is maintained.

Case 3: `nums[mid] > target`

All elements at or to the right of `mid` are strictly greater than the target. The target cannot reside in the range $[mid, right]$.

We update `right = mid - 1`. The new range is $[left, mid - 1]$. The invariant is maintained.

1.5.4.3 C. Termination

When the loop terminates, the invariant must help us prove correctness. The loop terminates when `left > right`.

- If `left > right`, the search range $[left, right]$ has become empty.
- Combining this with our loop invariant (which states that if the target is present, it must lie within $[left, right]$), we prove that the target is **not** present in the array. We return `-1` with mathematical confidence.

1.5.5 Implementation (Step 4)

Because we have proved our updates mathematically, we do not need to guess the loop conditions:

```
public int binarySearch(int[] nums, int target) {
    // 1. Enforce Pre-conditions
    if (nums == null || nums.length == 0) {
        return -1;
    }

    int left = 0;
    int right = nums.length - 1;

    // Maintain Invariant: target is in nums[left...right]
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid; // Post-condition satisfied
        } else if (nums[mid] < target) {
            left = mid + 1; // Invariant maintained
        } else {
            right = mid - 1; // Invariant maintained
        }
    }

    return -1; // Search range is empty -> target not in nums
}
```

By applying this invariant-first approach, we eliminate all cognitive overhead. We do not need to “dry-run” multiple edge cases or guess boundary updates. The math guarantees the correctness of our implementation.

STAR Moment: The $O(1)$ Failure Principle

A robust system fails fast and fails explicitly. The first lines of any method should always be pre-condition validation. If an input is invalid, fail immediately. Do not allow execution to proceed with corrupted or unexpected state, as this leads to hard-to-debug failures deep inside your call stack. In an interview, writing explicit input validations shows that you design for production safety, not just passing test suites.

Chapter 2

The Three System-Scale Case Studies

“If you want to evaluate an engineer’s design skill, do not ask them about theory. Ask them to design a ledger, an exchange, or a wallet under high-concurrency and security constraints.”

2.1 AuraPay: Core Ledger & Asynchronous Settlement (Canonical)

AuraPay is the primary case study we will implement throughout this book. It is a distributed, banking-grade payment ledger and asynchronous settlement system.

2.1.1 Key System Requirements

- **Double-Entry Bookkeeping:** All ledger updates must obey double-entry rules (every debit must have a corresponding credit, and the net balance change of any transaction across the system must be exactly zero).
- **ACID Transaction Isolation:** The ledger must prevent race conditions and double-spending, maintaining strict consistency even under heavy concurrent load on “hot” accounts.
- **Asynchronous Settlement Routing:** Payments are routed to different processing networks (ACH, FedWire, Visa/Mastercard) based on speed, cost, and transaction limits.

2.1.2 Enforcing the Domain Invariants

To demonstrate the spec-driven approach, we begin by defining the core domain objects of AuraPay: the `TransactionRecord` (an immutable value object representing a transaction intent) and the `LedgerAccount` (a stateful entity enforcing balance and overdraft invariants).

Here is the immutable, self-validating transaction representation:

```
package com.aurapay.domain;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.Objects;
import java.util.UUID;
```

```

/**
 * Represents an immutable, validated financial transaction record in AuraPay.
 * Enforces pre-conditions on initialization.
 */
public record TransactionRecord(
    UUID transactionId,
    UUID sourceAccountId,
    UUID destinationAccountId,
    BigDecimal amount,
    String currency,
    Instant timestamp
) {
    public TransactionRecord {
        Objects.requireNonNull(transactionId, "Transaction ID cannot be null");
        Objects.requireNonNull(sourceAccountId, "Source Account ID cannot be
↪ null");
        Objects.requireNonNull(destinationAccountId, "Destination Account ID
↪ cannot be null");
        Objects.requireNonNull(amount, "Amount cannot be null");
        Objects.requireNonNull(currency, "Currency cannot be null");
        Objects.requireNonNull(timestamp, "Timestamp cannot be null");

        if (sourceAccountId.equals(destinationAccountId)) {
            throw new IllegalArgumentException("Source and destination accounts
↪ must be distinct");
        }
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Transaction amount must be
↪ strictly positive");
        }
        if (currency.trim().isEmpty()) {
            throw new IllegalArgumentException("Currency code cannot be empty");
        }
    }
}

```

Next, we define the stateful `LedgerAccount` that enforces balance boundaries and thread-safe operations during fund transfers:

```

package com.aurapay.domain;

import java.math.BigDecimal;
import java.util.Objects;
import java.util.UUID;

/**
 * Represents a stateful Ledger Account in AuraPay, enforcing business invariants
 * during state transitions.

```

```

*/
public class LedgerAccount {
    private final UUID accountId;
    private final String currency;
    private BigDecimal balance;
    private final BigDecimal overdraftLimit;

    public LedgerAccount(UUID accountId, String currency, BigDecimal
        ↪ initialBalance, BigDecimal overdraftLimit) {
        this.accountId = Objects.requireNonNull(accountId, "Account ID cannot be
            ↪ null");
        this.currency = Objects.requireNonNull(currency, "Currency cannot be
            ↪ null");
        Objects.requireNonNull(initialBalance, "Initial balance cannot be null");
        Objects.requireNonNull(overdraftLimit, "Overdraft limit cannot be
        ↪ negative");

        if (overdraftLimit.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Overdraft limit cannot be
                ↪ negative");
        }
        if (initialBalance.add(overdraftLimit).compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Initial balance violates the
                ↪ overdraft limit");
        }

        this.balance = initialBalance;
        this.overdraftLimit = overdraftLimit;
    }

    public UUID getAccountId() { return accountId; }
    public String getCurrency() { return currency; }
    public synchronized BigDecimal getBalance() { return balance; }

    /**
     * Credits the account (adds funds). Enforces positive credit amount.
     */
    public synchronized void credit(BigDecimal amount) {
        Objects.requireNonNull(amount, "Credit amount cannot be null");
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Credit amount must be positive");
        }
        this.balance = this.balance.add(amount);
    }

    /**
     * Debits the account (removes funds). Enforces balance invariants and
     * ↪ overdraft limits.

```

```

    */
    public synchronized void debit(BigDecimal amount) {
        Objects.requireNonNull(amount, "Debit amount cannot be null");
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Debit amount must be positive");
        }

        BigDecimal newBalance = this.balance.subtract(amount);
        // INVARIANT ENFORCEMENT: Ensure the account does not exceed its
        ↪ overdraft limit
        if (newBalance.add(this.overdraftLimit).compareTo(BigDecimal.ZERO) < 0) {
            throw new InsufficientFundsException(
                String.format("Debit of %s exceeds account overdraft boundary.
                ↪ Balance: %s, Limit: -%s",
                    amount, balance, overdraftLimit)
            );
        }
        this.balance = newBalance;
    }
}
}

```

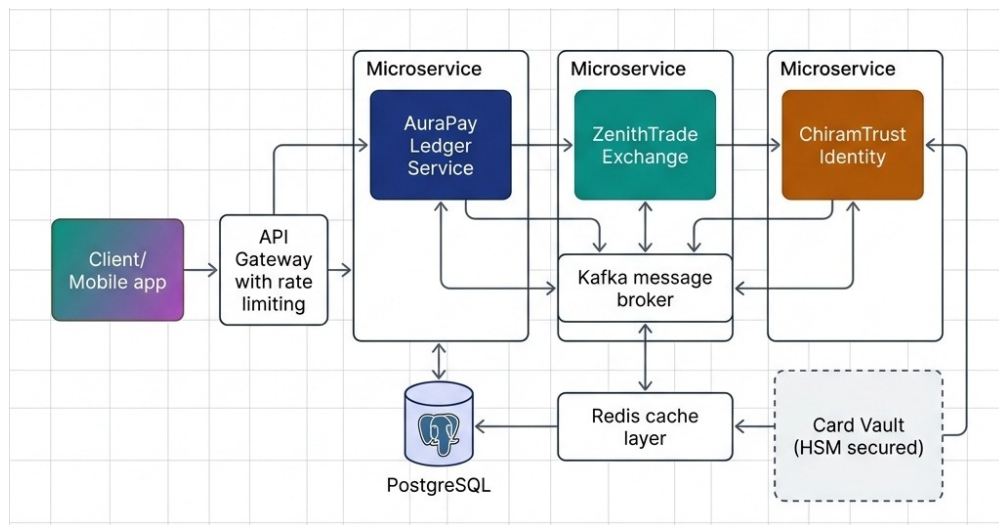


Figure 2.1: AuraPay System Architecture

In the following chapters, we will use these domain classes to demonstrate OOP design, SOLID boundary enforcement, Java Streams collection processing, and database concurrency controls.

2.2 ZenithTrade: High-Frequency Matching Engine (Exercise)

ZenithTrade is a high-frequency, low-latency order matching engine. It is designed to process incoming buy and sell limit orders and execute matches in real time.

2.2.1 Key System Requirements

- **Order Book State:** Maintains separate buy (bid) and sell (ask) order books, sorted by price (highest bid first, lowest ask first) and arrival time (FIFO).
- **Sub-Millisecond Latency:** The engine must execute order matching with minimal latency, avoiding memory allocations and garbage collection pauses.
- **Data Structure Mastery:** Utilizes custom priority queues, heaps, and double-ended queues for low-overhead bookkeeping.

2.2.2 Exercise Starter Scaffolding

To begin implementing the ZenithTrade engine, use the following `Order` entity as your starting point. It establishes the basic structure of a limit order, enforcing invariants like positive price and quantity:

```
public class Order {
    public enum Side { BUY, SELL }
    private final String id;
    private final String instrumentId;
    private final Side side;
    private final long price; // Fixed-point integer (smallest atomic unit)
    private final long quantity;

    public Order(String id, String instrumentId, Side side, long price, long
        ↪ quantity) {
        if (price <= 0) throw new IllegalArgumentException("Price must be
            ↪ positive");
        if (quantity <= 0) throw new IllegalArgumentException("Quantity must be
            ↪ positive");
        this.id = id;
        this.instrumentId = instrumentId;
        this.side = side;
        this.price = price;
        this.quantity = quantity;
    }

    public String getId() { return id; }
    public String getInstrumentId() { return instrumentId; }
    public Side getSide() { return side; }
    public long getPrice() { return price; }
    public long getQuantity() { return quantity; }
}
```

This case study is left as an exercise for the reader to apply the algorithmic patterns, concurrency models, and performance optimizations detailed in Part III.

2.3 ChiramTrust: Decentralized Identity Consent Wallet (Exercise)

ChiramTrust is a decentralized identity wallet that allows users to store credentials locally, negotiate sharing terms with verifiers, and establish consensus-based recovery.

2.3.1 Key System Requirements

- **W3C DID Compatibility:** Supports W3C Decentralized Identifiers (DIDs) for verifying cryptographic signatures on claims.
- **Granular Consent Engine:** Enforces user-defined access scopes, ensuring verifiers only receive requested claims (e.g., age verification without sharing birth dates).
- **Consensus Recovery:** Shares cryptographic key shards across a network of trusted guardians, using threshold secret sharing (Shamir's) to recover lost keys.

2.3.2 The Mechanics of Threshold Consensus (Shamir's Secret Sharing)

To implement consensus-based key recovery, the user's private key S is split into N distinct shares. We construct a random polynomial of degree $T - 1$ (where T is the threshold of guardians needed to recover the key):

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{T-1}x^{T-1} \pmod{P}$$

where $a_0 = S$ (the secret key), and the coefficients $a_1, \dots, a_{T-1}$ are randomly generated integers. The prime P defines the finite field $\mathbb{F}_P$. Each guardian i receives a coordinate point $(i, f(i))$.

By the properties of polynomial interpolation:

1. **Any T guardians** can pool their shares (x_i, y_i) and reconstruct the polynomial $f(x)$ using Lagrange interpolation, finding $f(0) = a_0 = S$:

$$S = \sum_{i=1}^T y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{P}$$

2. **Any $T-1$ or fewer guardians** possess a system of equations with infinite solutions, revealing absolutely zero information about the secret key S .

2.3.3 Exercise Starter Scaffolding

To implement the ChiramTrust wallet, use the following `DidConsentRecord` aggregate root as your starting point. It handles W3C identifier validation and thread-safe consent scope modifications:

```
public class DidConsentRecord {
    private final String did;
    private final Map<String, Boolean> consentScopes;

    public DidConsentRecord(String did, Map<String, Boolean> consentScopes) {
        if (did == null || !did.startsWith("did:")) {
            throw new IllegalArgumentException("Invalid W3C DID format");
        }
    }
}
```

```

    }
    this.did = did;
    this.consentScopes = new ConcurrentHashMap<>(consentScopes);
}

public String getDid() { return did; }

public boolean hasConsent(String scope) {
    return consentScopes.getOrDefault(scope, false);
}

public void revokeConsent(String scope) {
    consentScopes.put(scope, false);
}
}

```

2.3.4 Interview Drill: Applying Bounded Context Isolation

Here is a mock interview dialogue showing how to apply the Bounded Context Isolation rule in a real design interview:

Interviewer: *“If the AuraPay Ledger database experiences a write lag or becomes temporarily unavailable, how does that affect ZenithTrade’s matching engine? How do you prevent ledger issues from cascading and bringing down the trading platform?”*

Candidate: “We enforce strict Bounded Context Isolation. The ZenithTrade matching engine runs entirely in-memory and communicates with the AuraPay Ledger asynchronously via a transaction event stream. When an order matches, the matching engine commits the trade to its local state and publishes a `TradeExecuted` event. The Ledger service consumes this event and updates account balances.

If the Ledger database slows down or halts, the matching engine continues to process trades in memory without interruption. The event broker queues the trade events until the ledger recovers. This decoupling guarantees fault isolation and maintains a high-availability trading path.”

STAR Moment: Bounded Context Isolation

During system design interviews, explain that microservice division should mirror DDD Bounded Contexts. Say: *“We will isolate the ZenithTrade Matching Engine from the AuraPay Ledger. If the ledger experiences a database write lag, our matching engine can continue to accept and queue orders in memory, preventing system-wide downtime.”* This shows you design for fault isolation.

Part II

Code Design and Craftsmanship

Chapter 3

Principles of Object-Oriented Design

“Do not expose your state to the world. Encapsulate your data, expose your contracts, and let polymorphism handle the variance.”

3.1 The Anemic Domain Model Anti-Pattern

In many enterprise applications, domain classes are treated as passive data holders—simple collections of fields with auto-generated getters and setters. This is the **Anemic Domain Model** anti-pattern.

When your domain models are anemic, the business logic shifts into stateless service classes (e.g., `LedgerService`). The service pulls the state out of the domain model, performs validation, modifies the fields, and pushes the data back to the database. The danger of this design is that the domain object itself has no control over its state. Any developer can instantiate a ledger account, set the balance to a negative value without checks, and persist it, violating the core safety boundaries of the system.

The following code illustrates this fragile, anemic design:

```
// Anemic Account Model (Fragile Data Holder)
public class Account {
    private String id;
    private BigDecimal balance;
    private String currency;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public BigDecimal getBalance() { return balance; }
    public void setBalance(BigDecimal balance) { this.balance = balance; }
    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }
}
```

```
// Stateless Service containing business invariants (Anti-pattern)
```

```

public class LedgerService {
    public void transfer(Account from, Account to, BigDecimal amount) {
        if (from.getBalance().compareTo(amount) < 0) {
            throw new IllegalArgumentException("Insufficient funds");
        }
        if (!from.getCurrency().equals(to.getCurrency())) {
            throw new IllegalArgumentException("Currency mismatch");
        }
        from.setBalance(from.getBalance().subtract(amount));
        to.setBalance(to.getBalance().add(amount));
    }
}

```

3.1.1 Why the Anemic Model Fails in Production

1. **Lack of Encapsulation:** Any part of the application can modify the account balance directly: `account.setBalance(new BigDecimal("-1000.00"))`, bypassing the business checks entirely.
2. **Scatter-Shot Validation:** Validation logic is duplicated across multiple services (e.g., `BillingService`, `PayoutService`, `TransferService`). If a validation rule changes, you must locate and modify every instance across the codebase, risking logic drift.
3. **Concurrency Vulnerability:** In high-concurrency systems, separating state from checks leads to **Time-of-Check to Time-of-Use (TOCTOU)** race conditions, resulting in balance corruption.

In a senior coding or architecture interview, presenting an anemic model is a missed opportunity. To demonstrate true software craftsmanship, you must show how to design **rich domain models** that encapsulate state and enforce invariants.

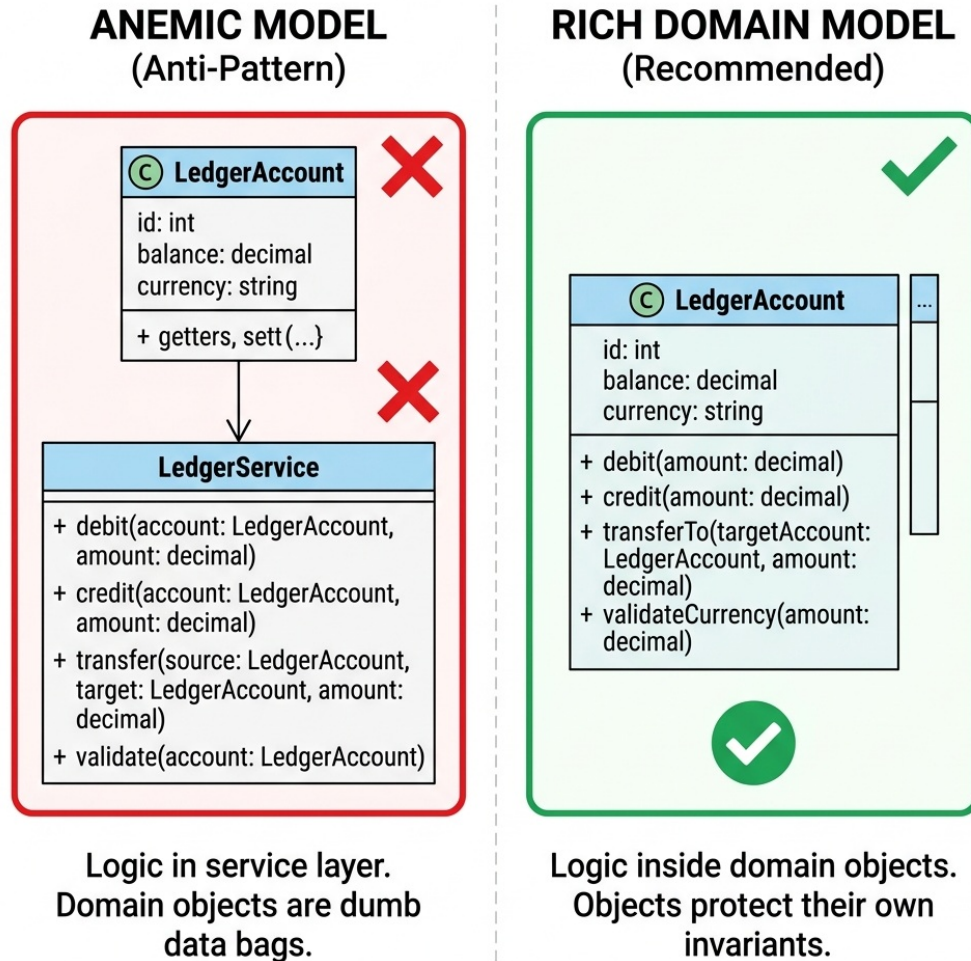


Figure 3.1: Anemic vs Rich Domain Model Comparison

3.2 Refactoring Walkthrough: From Anemic to Rich

To refactor a fragile anemic domain into a secure, self-validating rich domain model, follow these three rules:

3.2.1 Protect Domain Invariants in the Constructor

Ensure that an object can never be created in an invalid state. Validate all inputs during instantiation. If a pre-condition is violated, fail-fast immediately by throwing an exception.

3.2.2 Remove Setters and Restrict State Access

Eliminate all public setter methods. Fields should be **private** and, where possible, **final**. The only way to modify state is through explicit, domain-specific methods that protect the object's invariants.

3.2.3 Move Operations Inside the Aggregate Boundary

Instead of letting external service classes manipulate fields, encapsulate the business behavior inside the entity itself. The entity must protect its own state.

3.3 Abstraction & Encapsulation

Encapsulation is not merely the practice of making fields `private` and exposing public getters and setters. True encapsulation means that an object protects its own state, ensuring that its internal data can never enter an invalid state.

In AuraPay, our `LedgerAccount` domain model is rich. It contains its own `debit`, `credit`, and `transferTo` methods, making it impossible to perform a transfer without validating currencies, checking overdraft limits, and preventing concurrency deadlocks.

The following code illustrates this rich encapsulation:

```
package com.aurapay.domain;

import java.math.BigDecimal;
import java.util.Objects;

/**
 * Demonstrates a rich domain model encapsulating transfer logic and enforcing
 * cross-entity invariants.
 */
public class LedgerAccount {
    private final String accountId;
    private final String currency;
    private BigDecimal balance;
    private final BigDecimal overdraftLimit;

    public LedgerAccount(String accountId, String currency, BigDecimal
        ↪ initialBalance, BigDecimal overdraftLimit) {
        this.accountId = Objects.requireNonNull(accountId);
        this.currency = Objects.requireNonNull(currency);
        this.balance = Objects.requireNonNull(initialBalance);
        this.overdraftLimit = Objects.requireNonNull(overdraftLimit);
    }

    public synchronized BigDecimal getBalance() { return balance; }
    public String getCurrency() { return currency; }

    public synchronized void debit(BigDecimal amount) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Debit amount must be positive");
        }
        BigDecimal newBalance = this.balance.subtract(amount);
        if (newBalance.add(this.overdraftLimit).compareTo(BigDecimal.ZERO) < 0) {
            throw new InsufficientFundsException("Overdraft limit exceeded");
        }
    }
}
```

```

    }
    this.balance = newBalance;
}

public synchronized void credit(BigDecimal amount) {
    if (amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Credit amount must be positive");
    }
    this.balance = this.balance.add(amount);
}

/**
 * Executes a thread-safe transfer to a target account, enforcing business
 * ↪ invariants.
 * Prevents mismatched currencies (pre-condition) and double-debiting.
 */
public void transferTo(LedgerAccount target, BigDecimal amount) {
    Objects.requireNonNull(target, "Destination account cannot be null");
    Objects.requireNonNull(amount, "Transfer amount cannot be null");

    // PRE-CONDITION ENFORCEMENT: Currency matching
    if (!this.currency.equals(target.getCurrency())) {
        throw new CurrencyMismatchException(
            String.format("Cannot transfer between mismatched currencies: %s
            ↪ and %s",
                this.currency, target.getCurrency())
        );
    }

    // PRE-CONDITION ENFORCEMENT: Self-transfer check
    if (this.accountId.equals(target.accountId)) {
        throw new IllegalArgumentException("Cannot transfer to the same
        ↪ account");
    }

    // To prevent deadlocks, lock accounts in a stable global order
    LedgerAccount firstLock = this.accountId.compareTo(target.accountId) < 0
    ↪ ? this : target;
    LedgerAccount secondLock = firstLock == this ? target : this;

    synchronized (firstLock) {
        synchronized (secondLock) {
            // Execute atomic debit-credit sequence
            this.debit(amount);
            target.credit(amount);
        }
    }
}

```

```
}
```

3.3.1 Deadlock Prevention via Global Ordering

Notice the synchronization logic inside the `transferTo` method. In a high-concurrency payment engine, locking two entities simultaneously (e.g., account *A* transferring to *B*, while *B* is transferring to *A*) can lead to a circular wait deadlock.

To prevent this, the method compares the account identifiers (`this.accountId` and `target.accountId`) and locks them in a consistent, alphabetical global order. This is a classic concurrency pattern that demonstrates your readiness to design banking-grade production code.

3.4 OOP Principles vs. DDD Concepts

Object-Oriented Design and Domain-Driven Design (DDD) are deeply interconnected. When designing enterprise systems, OOD principles map directly to DDD tactical design patterns:

OOP Principle	DDD Tactical Pattern	Architectural Mapping
Encapsulation	Aggregate Root	The aggregate root acts as a consistency boundary, encapsulating internal entities and protecting invariants from external modification.
Immutability	Value Object	Objects without distinct identity (like <code>Money</code>) are designed as immutable value objects, preventing side effects during sharing.
Polymorphism	Domain Strategy	Swapping of algorithm strategies (like different fee calculations) is modeled as polymorphic strategy interfaces.
Abstraction	Repository / Service	Shielding the domain from infrastructure adapters (database, message queues) using clean interface abstractions.

3.5 Composition over Inheritance

A common mistake in object-oriented design is abusing inheritance. For example, if you are asked to support different settlement networks (ACH, FedWire, Visa), a naive developer might create a base `SettlementService` class and subclass it: `AchSettlementService`, `FedWireSettlementService`, etc.

This creates tight coupling. If you need to change how fees are calculated, or add a new network channel, you risk breaking parent behaviors. The first rule of enterprise OOP design is to **favor composition over inheritance**.

Instead of sub-classing, we compose our routing engine by injecting a collection of independent strategy routes. The core engine is decoupled from the network-specific details.

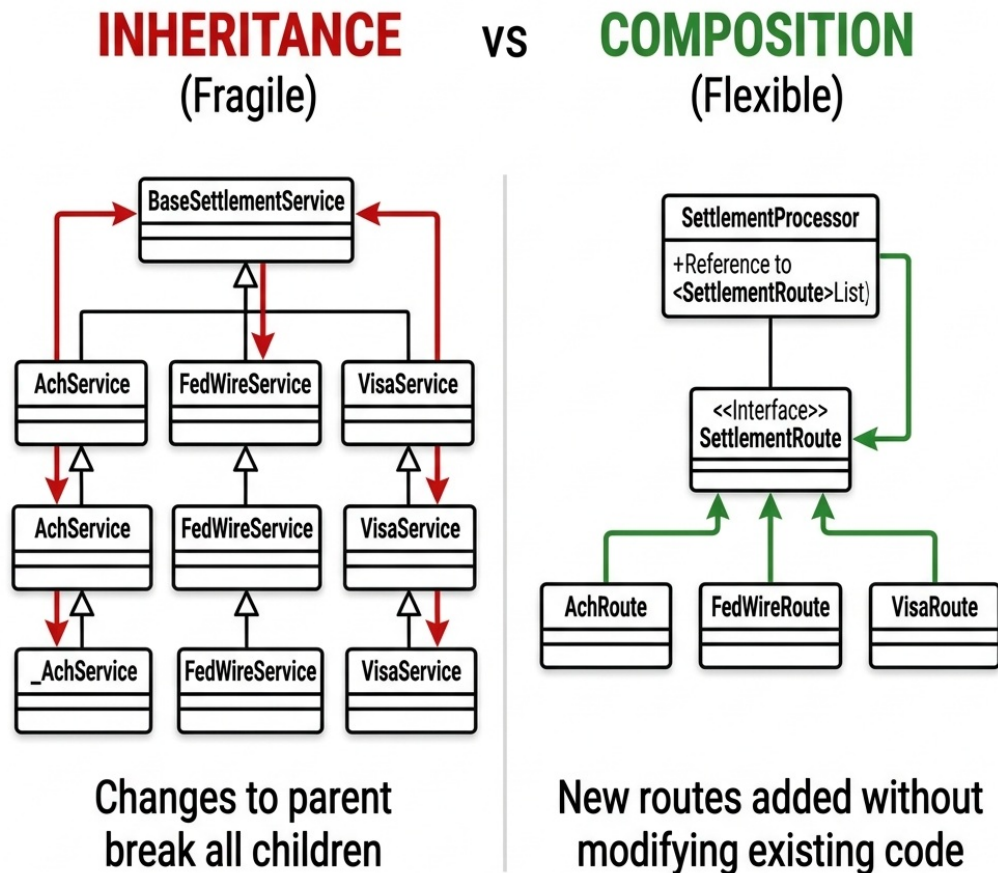


Figure 3.2: Composition over Inheritance

3.6 Polymorphism over Conditional Logic

One of the easiest ways to spot a junior candidate's code is looking for large `if-else` or `switch` blocks that inspect the type of an object to determine behavior. For example:

```
// Anti-pattern: Inspecting properties to determine routing
if (tx.getAmount().compareTo(LIMIT) > 0) {
    fedWireRoute.process(tx);
} else {
    achRoute.process(tx);
}
```

This violates the Open/Closed Principle. Every time you support a new payment network, you

must modify this routing block.

Polymorphism allows you to clean this up. By defining a generic `SettlementRoute` interface, the routing engine can iterate through all available routes, asking each route if it supports the transaction, and executing the process dynamically.

The following code defines this polymorphic settlement design:

```
package com.aurapay.settlement;

import com.aurapay.domain.TransactionRecord;
import java.math.BigDecimal;

/**
 * Interface defining the polymorphic contract for payment settlement networks.
 */
public interface SettlementRoute {
    boolean supports(TransactionRecord transaction);
    void process(TransactionRecord transaction);
    BigDecimal calculateFees(TransactionRecord transaction);
}

/**
 * Concrete implementation for the ACH network (low cost, delayed).
 */
public class AchRoute implements SettlementRoute {
    private static final BigDecimal ACH_FLAT_FEE = new BigDecimal("0.50");

    @Override
    public boolean supports(TransactionRecord transaction) {
        // ACH supports amounts up to $100,000
        return transaction.amount().compareTo(new BigDecimal("100000.00")) <= 0;
    }

    @Override
    public void process(TransactionRecord transaction) {
        System.out.println("Routing transaction " + transaction.transactionId() +
            " via ACH network.");
    }

    @Override
    public BigDecimal calculateFees(TransactionRecord transaction) {
        return ACH_FLAT_FEE;
    }
}

/**
 * Concrete implementation for the FedWire network (instant, high cost).
 */
public class FedWireRoute implements SettlementRoute {
```

```

private static final BigDecimal WIRE_FLAT_FEE = new BigDecimal("15.00");

@Override
public boolean supports(TransactionRecord transaction) {
    // FedWire is used for high-value transactions above $10,000
    return transaction.amount().compareTo(new BigDecimal("10000.00")) > 0;
}

@Override
public void process(TransactionRecord transaction) {
    System.out.println("Routing transaction " + transaction.transactionId() +
        ↪ " via FedWire network.");
}

@Override
public BigDecimal calculateFees(TransactionRecord transaction) {
    return WIRE_FLAT_FEE;
}
}

```

By utilizing this interface, the main transaction processor can execute settlements using a clean polymorphic loop, completely decoupled from specific network implementations:

```

public class SettlementProcessor {
    private final List<SettlementRoute> routes;

    public SettlementProcessor(List<SettlementRoute> routes) {
        this.routes = routes;
    }

    public void execute(TransactionRecord transaction) {
        SettlementRoute activeRoute = routes.stream()
            .filter(route -> route.supports(transaction))
            .findFirst()
            .orElseThrow(() -> new NoRouteFoundException("No supported route
                ↪ found"));

        activeRoute.process(transaction);
    }
}

```

STAR Moment: The Encapsulation Test

When designing class structures in a technical interview, ask yourself: *Can this class enter an invalid state?* If a client developer can instantiate your object and set its properties to values that violate business rules, your encapsulation has failed. Build your validation boundaries directly into the constructors and state-transition methods of your domain objects.

Chapter 4

SOLID Principles: Enforcing Boundaries

“Software architecture is the art of drawing lines between components. SOLID is the rulebook for placing those lines.”

4.1 SOLID in the Senior Interview

In senior and lead engineering interviews, you are almost guaranteed to be asked about the SOLID principles. Too many candidates respond by simply reciting the acronym: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

If you stop there, you fail to show architectural maturity. An interviewer wants to know *why* these principles matter at scale. They want to see how applying these principles prevents structural rot, allows multiple teams to work in parallel without code collisions, and ensures that a change in database technology does not break the core transaction engine.

In this chapter, we will implement the core processing pipeline of AuraPay using a design that strictly conforms to all five SOLID principles.



Figure 4.1: The Five SOLID Principles — Quick Reference

4.2 The SOLID Transaction Pipeline

To illustrate SOLID, we will examine the `TransactionProcessor` in AuraPay. This component is responsible for retrieving ledger accounts, calculating fees, updating account balances, persisting the changes to storage, and notifying external systems.

Here is the decoupled, SOLID-compliant transaction execution flow:

```
package com.aurapay.processing;

import com.aurapay.domain.LedgerAccount;
import com.aurapay.domain.TransactionRecord;
import java.math.BigDecimal;
import java.util.Objects;
import java.util.UUID;

/**
 * Abstraction for database operations (Dependency Inversion Principle).
 */
public interface LedgerRepository {
    LedgerAccount findById(UUID accountId);
    void save(LedgerAccount account);
}
```

```

/**
 * Abstraction for fee calculations (Open/Closed Principle).
 */
public interface FeeCalculator {
    BigDecimal calculate(TransactionRecord transaction);
}

/**
 * Interface Segregation Principle: Focused notification dispatch interface.
 */
public interface TransactionNotificationSender {
    void sendNotification(TransactionRecord transaction, String status);
}

/**
 * Core transaction processor showing SOLID compliance.
 */
public class TransactionProcessor {
    private final LedgerRepository repository;
    private final FeeCalculator feeCalculator;
    private final TransactionNotificationSender notificationSender;

    public TransactionProcessor(
        LedgerRepository repository,
        FeeCalculator feeCalculator,
        TransactionNotificationSender notificationSender
    ) {
        this.repository = Objects.requireNonNull(repository);
        this.feeCalculator = Objects.requireNonNull(feeCalculator);
        this.notificationSender = Objects.requireNonNull(notificationSender);
    }

    /**
     * Processes a transaction. Decoupled from repository, fee, and notification
     * ↪ details.
     */
    public void process(TransactionRecord transaction) {
        Objects.requireNonNull(transaction, "Transaction cannot be null");

        // 1. Retrieve accounts from abstraction (DIP)
        LedgerAccount source =
        ↪ repository.findById(transaction.sourceAccountId());
        LedgerAccount destination =
        ↪ repository.findById(transaction.destinationAccountId());

        if (source == null || destination == null) {
            throw new IllegalArgumentException("Source or destination account not
            ↪ found");
        }
    }
}

```

```

    }

    // 2. Calculate fee dynamically (DCP)
    BigDecimal fee = feeCalculator.calculate(transaction);
    BigDecimal totalDebit = transaction.amount().add(fee);

    // 3. Coordinate state transitions on rich domain objects (SRP / LSP)
    // Overdraft check is executed internally within source.debit()
    source.debit(totalDebit);
    destination.credit(transaction.amount());

    // 4. Persist updated states (DIP)
    repository.save(source);
    repository.save(destination);

    // 5. Notify via segregated interface (ISP)
    notificationSender.sendNotification(transaction, "SUCCESS");
}
}

```

Let us break down how this single class enforces all five design boundaries.

4.3 Single Responsibility Principle (SRP)

The Single Responsibility Principle is often summarized as “a class should do only one thing.” A more precise architectural definition is: **“a module should have one, and only one, reason to change.”**

In our transaction pipeline, the `TransactionProcessor` has one responsibility: coordinating the business workflow of a transaction. It does not contain database queries, does not know how to format SMS or Email notifications, and does not hardcode fee calculation percentages.

- If the database schema changes, only `LedgerRepository` implementations change.
- If we switch from email notifications to SMS notifications, only `TransactionNotificationSender` implementations change.
- The `TransactionProcessor` remains untouched.

4.4 Open/Closed Principle (OCP)

The Open/Closed Principle states that **software entities should be open for extension, but closed for modification.**

In AuraPay, we must support multiple fee models (e.g., flat fees for retail clients, percentage-based fees for merchants, waived fees for corporate accounts). Instead of adding nested `if-else` blocks inside the transaction processor, we inject the `FeeCalculator` interface. If we need to add a new fee model, we simply write a new class implementing `FeeCalculator` and pass it to the processor. The core processor is closed to modifications, yet the fee behavior is infinitely extendable.

4.5 Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that **subtypes must be substitutable for their base types without altering the correctness of the program.**

In financial systems, this is highly relevant when modeling different account types. For example, a `SavingsAccount` might not allow overdrafts, while a `CheckingAccount` allows up to a certain limit. If a developer creates a subclass `BlockedAccount` that throws an `UnsupportedOperationException` whenever `debit()` is called, they violate LSP. The `TransactionProcessor` assumes that any `LedgerAccount` returned by the repository can be debited and credited. LSP ensures that subclass behaviors remain consistent with the contracts defined on their parent classes, preventing runtime crashes.

4.6 Interface Segregation Principle (ISP)

The Interface Segregation Principle states that **clients should not be forced to depend on interfaces they do not use.**

In a large enterprise system, you might have a broad `NotificationService` that handles email, Slack channels, internal logging, and mobile push alerts. If the `TransactionProcessor` injected a giant `NotificationService` interface containing twenty unrelated methods, it would be coupled to changes in mobile app push logic. Instead, we define a small, segregated interface: `TransactionNotificationSender`, containing only the single `sendNotification` method. The processor only knows about what it needs to execute its task.

4.7 Dependency Inversion Principle (DIP)

The Dependency Inversion Principle states that **high-level modules should not import anything from low-level modules. Both should depend on abstractions.**

This is the most critical principle for decoupling business logic from infrastructure.

- **Low-level modules:** Database APIs, file system writers, network channels, and concrete frameworks.
- **High-level modules:** The core business rules of your application (like transaction routing and double-entry validation).

In our implementation, the `TransactionProcessor` does not import a concrete SQL database connector or Hibernate manager. It depends entirely on the `LedgerRepository` interface. The business logic is at the top of the dependency tree, and database adapters are plugged in at the bottom. This allows you to run unit tests using a mock repository in memory, completely decoupled from a database connection.

Dependency Inversion Principle (DIP)

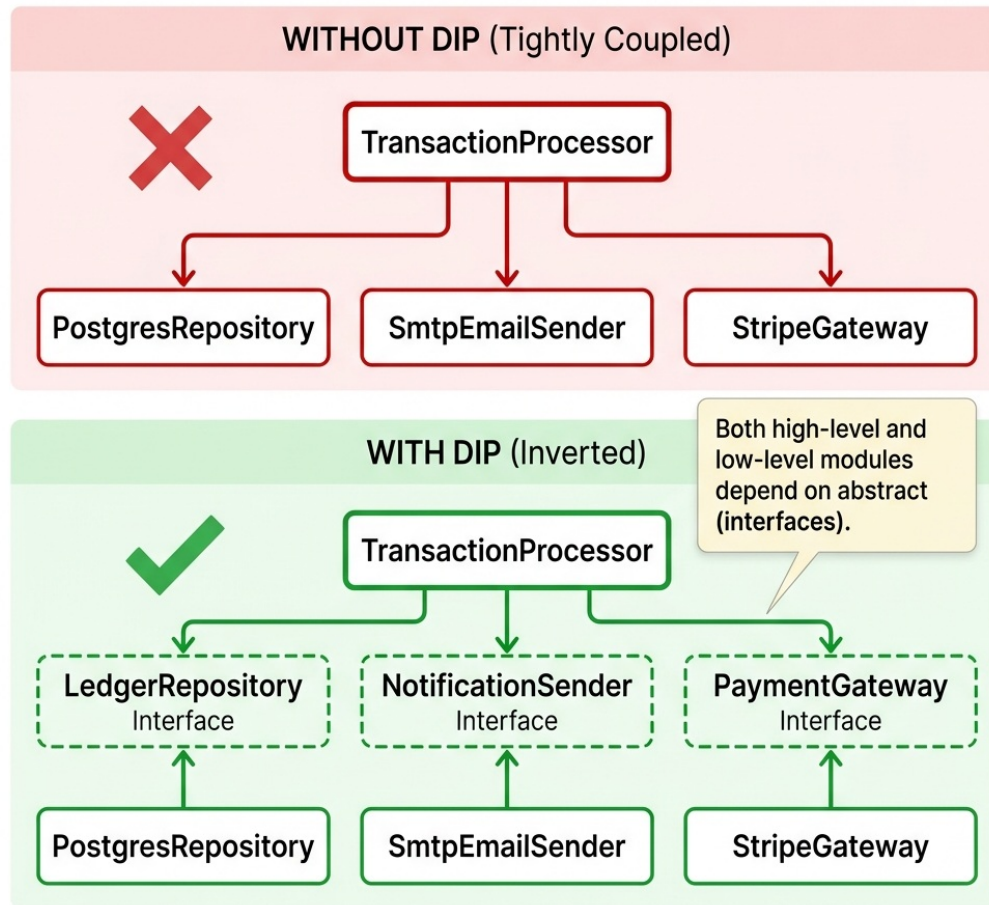


Figure 4.2: SOLID Dependency Inversion Principle — Before and After

4.8 SOLID Violation Detector & Remedies

In technical interviews, you must be able to spot structural violations in a code sample and offer clear architectural remedies:

SRP Violations

- *Red Flags:* Large source files (500+ lines). Imports both database drivers and UI libraries. Multiple developers editing the same class for unrelated features.
- *Remedy:* **Decomposition** — Split into separate, focused classes coordinated by an Orchestrator or Application Service.

OCP Violations

- *Red Flags:* Switch statements or `if-else` blocks inspecting enums/types. Modifying existing service classes to add support for new payment partners.
- *Remedy:* **Abstraction** — Define an interface and implement the Strategy pattern. Inject a collection of these strategies.

LSP Violations

- *Red Flags:* Subclass methods returning dummy values or throwing `UnsupportedOperationException`. Typecasting (`instanceof`) inside helper methods.
- *Remedy:* **Hierarchy Flattening** — Replace inheritance with composition, or split the interface into smaller, specialized interfaces.

ISP Violations

- *Red Flags:* Concrete classes implementing interfaces with empty or dummy methods. Small client classes coupled to changes in unused interface methods.
- *Remedy:* **Segregation** — Split the fat interface into multiple single-method or small role-based interfaces.

DIP Violations

- *Red Flags:* Use of the `new` keyword to instantiate databases/gateways directly inside services. Direct imports of low-level infrastructure modules.
- *Remedy:* **Dependency Injection** — Program to interfaces. Pass dependencies via Constructor Injection, managed by an IoC container.

4.9 Framework Integration: SOLID in Enterprise Containers

Modern web frameworks are designed explicitly around SOLID principles:

4.9.1 Dependency Injection (IoC) Containers

Frameworks like Spring Boot (Java), ASP.NET Core (C#), and FastAPI/Dependency Injector (Python) serve as Dependency Inversion engines. By registering interfaces and their concrete implementations in the container, the framework automates constructor injection. High-level business modules declare their dependencies as constructor interfaces, completely decoupled from concrete instantiation.

4.9.2 Aspect-Oriented Programming (AOP)

To adhere to OCP, frameworks use AOP to apply cross-cutting concerns (such as transactions, security, and logging) to service boundaries dynamically using **Proxy decorators**. For instance, adding `@Transactional` in Spring Boot or `[Transaction]` in ASP.NET Core wraps the service class in a proxy container, injecting commit and rollback logic without modifying the service's source code.

STAR Moment: The Mockability Test

The ultimate test of a SOLID design is **mockability**. In a technical interview, explain that a correctly decoupled class can be unit-tested in isolation by mocking all of its interface dependencies. If you cannot test a method without spinning up a real database, an active web server, or a third-party messaging channel, your design violates the Dependency Inversion Principle.

Chapter 5

Modern Functional Programming and Stream APIs

“A pipeline of pure functions is a system without side effects. It is a system that can be scaled, tested, and parallelized without fear.”

5.1 The Imperative Loop Trap

A classic interview task is to process a collection of records—filtering out invalid data, transforming the items, and aggregating the result. Historically, developers solved this using imperative structures: `for` loops, nested `if` statements, and mutable local variables.

```
// Imperative anti-pattern: Hard to read, mutable state, difficult to parallelize
Map<UUID, BigDecimal> volumes = new HashMap<>();
for (TransactionRecord tx : transactions) {
    if (tx.amount().compareTo(threshold) >= 0) {
        UUID merchantId = tx.destinationAccountId();
        BigDecimal currentSum = volumes.getDefault(merchantId,
            ↪ BigDecimal.ZERO);
        volumes.put(merchantId, currentSum.add(tx.amount()));
    }
}
```

While correct, this approach has drawbacks:

- It is highly **imperative**, forcing the reader to track *how* the execution runs rather than *what* is being achieved.
- It relies on **mutable state** (`volumes` map), making it unsafe to parallelize without explicit synchronization locks.
- It lacks clean boundaries, combining filtering, mapping, and aggregation into a single block of code.

Modern software engineering favors the **declarative** approach. Using functional pipelines (Java Streams, C# LINQ, Python Generators), you describe the data transformations as a sequence of

side-effect-free operations.

5.2 The AuraPay Batch Pipeline

In AuraPay, we aggregate merchant transaction volumes using functional streams. This allows us to process batches of transactions cleanly.

The following code illustrates this functional pipeline:

```
package com.aurapay.analytics;

import com.aurapay.domain.TransactionRecord;
import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.UUID;
import java.util.stream.Collectors;

/**
 * Demonstrates high-performance batch transaction analytics using Java Streams.
 */
public class TransactionAnalytics {

    /**
     * Processes a list of transactions to aggregate total volume per merchant,
     * filtering out high-risk or low-value records.
     */
    public Map<UUID, BigDecimal> aggregateMerchantVolumes(
        List<TransactionRecord> transactions,
        BigDecimal minAmountThreshold
    ) {
        Objects.requireNonNull(transactions, "Transaction list cannot be null");
        Objects.requireNonNull(minAmountThreshold, "Threshold cannot be null");

        // Declarative functional pipeline
        return transactions.stream()
            // 1. Filter: Retain only transactions meeting the value criteria
            //    ↪ (side-effect-free)
            .filter(t -> t.amount().compareTo(minAmountThreshold) >= 0)

            // 2. Collect: Group by merchant and sum the transaction volume
            .collect(Collectors.toMap(
                TransactionRecord::destinationAccountId, // Key mapper: Merchant
                TransactionRecord::amount,                // Value mapper:
                // Transaction amount
                BigDecimal::add                             // Merge function: Sum
                //    ↪ volumes
            ));
    }
}
```

```

    ));
}

/**
 * Finds the transaction IDs of all transfers exceeding a safety limit,
 * sorted chronologically.
 */
public List<UUID> getHighValueTransactionIds(
    List<TransactionRecord> transactions,
    BigDecimal limit
) {
    return transactions.stream()
        .filter(t -> t.amount().compareTo(limit) > 0)
        .map(TransactionRecord::transactionId)
        .collect(Collectors.toList());
}
}

```

Functional Stream Processing

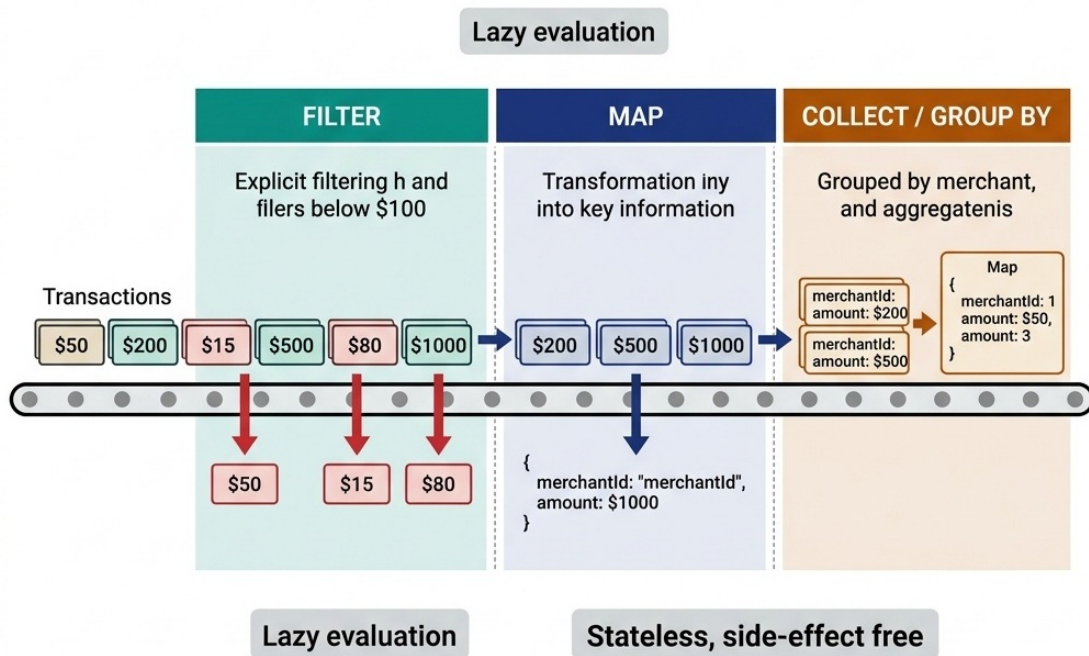


Figure 5.1: Stream Pipeline Visualization

By declaring the operations as a stream pipeline, the code becomes a readable translation of the business spec:

1. **Filter** out transaction records below the threshold.
2. **Collect** the results by grouping by the merchant ID and adding their amounts.

5.3 Understanding Method References (:: Syntax)

In Java, the `::` operator is a **method reference** — a shorthand for a lambda expression that simply delegates to an existing method. Method references make stream pipelines more readable by replacing verbose lambdas with direct method pointers.

There are four types of method references:

1. Static Method Reference — `ClassName::staticMethod`

Calls a static method. The stream element is passed as the argument.

```
// Lambda form:
.map(s -> Integer.parseInt(s))
// Method reference form:
.map(Integer::parseInt)
```

2. Instance Method on a Specific Object — `instance::method`

Calls an instance method on a specific, already-existing object.

```
TransactionValidator validator = new TransactionValidator();
// Lambda form:
.filter(tx -> validator.isValid(tx))
// Method reference form:
.filter(validator::isValid)
```

3. Instance Method on the Stream Element — `ClassName::instanceMethod`

Calls an instance method on each element flowing through the stream. The element itself becomes the `this` reference.

```
// Lambda form:
.map(tx -> tx.amount())
// Method reference form:
.map(TransactionRecord::amount)

// Lambda form:
.map(s -> s.toUpperCase())
// Method reference form:
.map(String::toUpperCase)
```

4. Constructor Reference — `ClassName::new`

Calls a constructor to create new objects from stream elements.

```
// Lambda form:
.map(name -> new Merchant(name))
// Method reference form:
.map(Merchant::new)
```

Interview Signal: Using method references consistently signals that you write idiomatic, clean functional code. When reviewing a pipeline in a live coding session, interviewers expect `Transaction::amount` over `tx -> tx.amount()`.

5.4 Stream Operations Deep-Dive

Every stream pipeline consists of three parts: a **source**, zero or more **intermediate operations** (lazy), and exactly one **terminal operation** (triggers execution).

5.4.1 Key Intermediate Operations (Lazy — Build the Pipeline)

Operation	Purpose	Example
<code>filter(Predicate)</code>	Keep elements matching a condition	<code>.filter(tx -> tx.amount() > 100)</code>
<code>map(Function)</code>	Transform each element to a new value	<code>.map(Transaction::merchantId)</code>
<code>flatMap(Function)</code>	Flatten nested collections into a single stream	<code>.flatMap(tx -> tx.items().stream())</code>
<code>distinct()</code>	Remove duplicate elements (uses <code>.equals()</code>)	<code>.distinct()</code>
<code>sorted()</code>	Sort elements (natural order or by Comparator)	<code>.sorted()</code>
<code>peek(Consumer)</code>	Inspect elements without modifying (for debugging)	<code>.peek(tx -> log.info(tx))</code>
<code>limit(n)</code>	Take only the first N elements	<code>.limit(10)</code>
<code>skip(n)</code>	Skip the first N elements	<code>.skip(5)</code>

5.4.2 Key Terminal Operations (Eager — Trigger Execution)

Operation	Purpose	Example
<code>collect(Collector)</code>	Accumulate into a collection or summary	<code>.collect(Collectors.toList())</code>
<code>forEach(Consumer)</code>	Perform an action on each element	<code>.forEach(System.out::println)</code>
<code>reduce(identity, BinaryOp)</code>	Combine all elements into a single result	<code>.reduce(BigDecimal.ZERO, BigDecimal::add)</code>
<code>count()</code>	Count elements	<code>.count()</code>
<code>findFirst()</code>	Return the first element (wrapped in Optional)	<code>.findFirst()</code>
<code>anyMatch(Predicate)</code>	Check if any element satisfies a condition	<code>.anyMatch(tx -> tx.isFraud())</code>
<code>allMatch(Predicate)</code>	Check if all elements satisfy a condition	<code>.allMatch(tx -> tx.amount() > 0)</code>
<code>toArray()</code>	Collect into an array	<code>.toArray(String[]::new)</code>

5.4.3 Collectors: The Power of `collect()`

The `Collectors` utility class provides powerful aggregation operations:

```
// Group transactions by merchant, summing amounts
```

```

Map<UUID, BigDecimal> volumeByMerchant = transactions.stream()
    .collect(Collectors.groupingBy(
        TransactionRecord::merchantId,
        Collectors.reducing(BigDecimal.ZERO, TransactionRecord::amount,
↪   BigDecimal::add)
    ));

// Partition transactions into two groups: above/below threshold
Map<Boolean, List<TransactionRecord>> partitioned = transactions.stream()
    .collect(Collectors.partitioningBy(tx -> tx.amount().compareTo(threshold) >
↪   0));

// Join merchant names into a comma-separated string
String merchantList = merchants.stream()
    .map(Merchant::name)
    .collect(Collectors.joining(", "));

// Compute statistics on amounts
DoubleSummaryStatistics stats = transactions.stream()
    .mapToDouble(tx -> tx.amount().doubleValue())
    .summaryStatistics();
// stats.getAverage(), stats.getMax(), stats.getMin(), stats.getCount()

```

5.5 When to Use .map() vs Collectors Directly

A common source of confusion is deciding whether to use `.map()` as an intermediate transformation step, or to go directly to a `Collectors.toMap()` or `Collectors.groupingBy()` call in the terminal `collect()`. The rule is straightforward:

Use `.map()` when you need ONE thing from each element into a simple collection.

The `.map()` operation transforms what is flowing through the stream. After `.map(TransactionRecord::transactionId)`, the stream is no longer `Stream<TransactionRecord>` — it becomes `Stream<UUID>`. Use this when you only need to extract a single field and collect it into a `List` or `Set`.

```

// Goal: Get a list of transaction IDs for high-value transactions
List<UUID> highValueIds = transactions.stream()
    .filter(tx -> tx.amount().compareTo(limit) > 0)
    .map(TransactionRecord::transactionId)    // Stream<TransactionRecord> ->
↪   Stream<UUID>
    .collect(Collectors.toList());           // Simple List<UUID>

```

Use `Collectors.toMap()` or `groupingBy()` when you need TWO or more things from each element into a `Map`.

When you need to extract both a key and a value from the same object, you cannot use `.map()` — mapping to one field loses access to the other. Instead, pass both extractor functions directly into the collector.

```

// Goal: Map each merchant to their total transaction volume

```

```

Map<UUID, BigDecimal> volumes = transactions.stream()
    .filter(tx -> tx.amount().compareTo(threshold) >= 0)
    .collect(Collectors.toMap(
        TransactionRecord::destinationAccountId, // key: need accountId
        TransactionRecord::amount,              // value: need amount
        BigDecimal::add                          // merge: sum on collision
    ));
// No .map() here - we need BOTH fields from the same TransactionRecord

```

5.5.1 Decision Guide

Ask yourself: *Do I need to build a Map (key -> value) from each element?*

- **Yes** -> Use `Collectors.toMap()` or `groupingBy()` directly. You need the full object to extract both key and value.
- **No** -> *Do I need to transform each element to a different type?*
 - **Yes** -> Use `.map()` then `.collect(Collectors.toList())` or `.collect(Collectors.toSet())`
 - **No** -> Just `.filter()` then `.collect(Collectors.toList())`

5.5.2 Five Patterns Side-by-Side

```

// Pattern 1: Extract one field -> List
List<UUID> ids = records.stream()
    .map(TransactionRecord::transactionId)
    .collect(Collectors.toList());

// Pattern 2: Extract one field -> Set (deduplicate)
Set<String> currencies = records.stream()
    .map(TransactionRecord::currency)
    .collect(Collectors.toSet());

// Pattern 3: Two fields -> Map (no duplicates expected)
Map<UUID, BigDecimal> balances = records.stream()
    .collect(Collectors.toMap(
        TransactionRecord::transactionId,
        TransactionRecord::amount));

// Pattern 4: Two fields -> Map with aggregation (sum duplicates)
Map<UUID, BigDecimal> totals = records.stream()
    .collect(Collectors.toMap(
        TransactionRecord::destinationAccountId,
        TransactionRecord::amount,
        BigDecimal::add));

// Pattern 5: Group into lists -> Map<Key, List<Record>>
Map<String, List<TransactionRecord>> byCurrency = records.stream()
    .collect(Collectors.groupingBy(TransactionRecord::currency));

```


Interview Tip: If an interviewer asks you to aggregate data by a key, reach for `Collectors.toMap()` with a merge function or `Collectors.groupingBy()`. If they ask you to extract or transform elements, use `.map()` followed by a simple `toList()` or `toSet()`. Explaining *why* you chose one over the other demonstrates pipeline design fluency.

5.6 Senior Interview Critical Knowledge: Performance Pitfalls

In senior developer and manager interviews, showing that you can write a stream is not enough. You must demonstrate a deep understanding of the **performance costs and runtime implications** of functional APIs.

5.6.1 The Parallel Stream Thread Starvation Trap

In Java, calling `.parallelStream()` instead of `.stream()` splits the workload across multiple threads automatically. Candidates often present this as an easy optimization.

The Danger: By default, all parallel streams in a JVM share a single, static `ForkJoinPool.commonPool()`. The size of this pool is set to `Runtime.getRuntime().availableProcessors() - 1`. If you run an I/O-bound operation (e.g., calling an external billing gateway or querying a database) inside a parallel stream, you block a thread in the common pool. If multiple requests execute these I/O tasks concurrently, the common pool becomes completely starved. Consequently, **every other parallel stream in the entire JVM application stalls**, including critical background system tasks.

Design Rule: Never run I/O-bound operations inside parallel streams. Only use parallel streams for CPU-bound computations on large collections where the overhead of thread scheduling is outweighed by the calculation size.

5.6.2 Intermediate Object Overhead & Garbage Collection

Functional pipelines construct intermediate objects for every stage of the pipeline. In high-throughput settlement engines processing millions of transactions per second, this causes significant memory overhead.

- Every `.map()` or `.filter()` operation instantiates wrapper objects.
- Primitive boxing (e.g., converting a raw `double` to a `Double` object) triggers heap allocations, putting heavy pressure on the JVM Garbage Collector.

Design Rule: In performance-critical loops, utilize primitive streams (e.g., `IntStream`, `DoubleStream`) to prevent boxing overhead, or fall back to plain array iterations if zero-allocation execution is required.

5.6.3 Lazy Evaluation and Exception Handling

Stream operations are **lazy**—they are not executed when they are declared, but only when a **terminal operation** (like `.collect()`, `.findFirst()`, or `.forEach()`) is invoked. This creates

debugging challenges. If a filter operation throws an exception, the stack trace will point to the terminal operation invocation, not where the pipeline was declared. Furthermore, standard functional interfaces do not allow checked exceptions, forcing you to write messy wrappers or handle runtime failures globally.

5.7 Performance Comparison: Imperative vs. Streams

Selecting the correct loop structure is a trade-off between readability and raw execution speed. The following table contrasts the runtime behaviors of different processing paradigms:

Metric	Imperative Loops	Sequential Streams	Parallel Streams
Execution Time	Fastest (zero overhead)	Slow to medium	Fast for massive sets; slower for small sets
Heap Allocations	None (in-place)	High (wrapper nodes, builders)	Very high (coordination nodes)
GC Pressure	Zero	Medium to high	High
Scale Limits	Single CPU core	Single CPU core	Scales with cores (CPU-bound only)
Starvation Risk	Zero	Zero	Extreme (I/O in common pool)
Readability	Low (boilerplate)	High (declarative)	High (simple conversion)

5.8 Standard Streams vs. Reactive Streams

In high-concurrency systems, candidates must distinguish between standard Java Streams and **Reactive Streams** (e.g., Spring WebFlux, Project Reactor, RxJava, C# Reactive Extensions):

- **Standard Streams (Pull-Based):** Synchronous and blocking. They operate on finite, in-memory collections. The consumer pulls data from the stream.
- **Reactive Streams (Push-Based):** Asynchronous and non-blocking. They operate on infinite event streams (e.g., live stock feeds, WebSocket connections). The producer pushes data to the consumer.
- **Backpressure:** Reactive streams support backpressure, allowing a slow consumer to signal to a fast producer to slow down, preventing the consumer from exhausting its heap memory buffer during spikes.

5.8.1 Reactive Streams Example: Real-Time Transaction Monitor

Consider AuraPay’s real-time fraud monitoring pipeline. New transactions arrive continuously from multiple payment gateways. We need to filter suspicious transactions, enrich them with account data, and emit alerts — all without blocking threads.

```
// Reactive pipeline using Project Reactor (Spring WebFlux)
Flux<FraudAlert> fraudAlerts = transactionEventStream // Infinite push-based
    ↪ stream
    .filter(tx -> tx.amount().compareTo(highValueThreshold) > 0)
```

```

.flatMap(tx -> accountService.findById(tx.accountId()) // Non-blocking DB
  ↪ call
    .map(account -> new EnrichedTransaction(tx, account)))
.filter(enriched -> enriched.riskScore() > 0.85)
.map(enriched -> new FraudAlert(enriched, Instant.now()))
.onBackpressureBuffer(1000) // Buffer up to 1000 if consumer is slow
.doOnNext(alert -> log.warn("FRAUD ALERT: {}", alert.transactionId()));

// Subscribe to start processing (nothing happens until subscribe)
fraudAlerts.subscribe(
    alert -> alertService.dispatch(alert), // onNext: process each alert
    error -> log.error("Pipeline error", error), // onError: handle failures
    () -> log.info("Stream completed") // onComplete: stream ended
);

```

The critical differences from standard streams:

Aspect	Standard Stream	Reactive Stream
Data Source	Finite collection (List, Set)	Infinite event source (Kafka, WebSocket)
Execution	Blocking (thread waits for each step)	Non-blocking (event loop, no thread waiting)
Threading	Caller's thread or ForkJoinPool	Scheduler-managed (e.g., <code>Schedulers.boundedElastic()</code>)
Error Handling	Try-catch or runtime exception	<code>.onErrorResume()</code> , <code>.retry()</code> operators
Backpressure	Not supported	Built-in (<code>onBackpressureBuffer</code> , <code>onBackpressureDrop</code>)
Lifecycle	Runs once, then garbage collected	Runs continuously until cancelled

5.9 Stream Practice Questions

The following problems are commonly asked in interviews to test functional programming fluency. Try solving each one using streams before reviewing the solution.

Q1. Find the three highest transaction amounts (no duplicates)

```

List<BigDecimal> topThree = transactions.stream()
    .map(TransactionRecord::amount)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .limit(3)
    .collect(Collectors.toList());

```

Q2. Group transactions by currency, counting how many in each

```
Map<String, Long> countByCurrency = transactions.stream()
    .collect(Collectors.groupingBy(
        TransactionRecord::currency,
        Collectors.counting()
    ));
```

Q3. Find the first transaction over \$10,000 (or return empty)

```
Optional<TransactionRecord> highValue = transactions.stream()
    .filter(tx -> tx.amount().compareTo(new BigDecimal("10000")) > 0)
    .findFirst();
```

Q4. Flatten a list of orders (each containing line items) into all items

```
List<LineItem> allItems = orders.stream()
    .flatMap(order -> order.getLineItems().stream())
    .collect(Collectors.toList());
```

Q5. Compute a comma-separated string of all merchant names, sorted alphabetically

```
String result = merchants.stream()
    .map(Merchant::name)
    .sorted()
    .collect(Collectors.joining(", "));
```

Q6. Check if all transactions are in USD

```
boolean allUsd = transactions.stream()
    .allMatch(tx -> "USD".equals(tx.currency()));
```

Q7. Convert a list of strings to a map of string -> length (handling duplicates)

```
Map<String, Integer> nameLengths = names.stream()
    .collect(Collectors.toMap(
        name -> name,
        String::length,
        (existing, replacement) -> existing // Keep first on collision
    ));
```

5.10 Debugging Functional Pipelines

Debugging streams can be difficult due to their lazy execution model. To inspect stream internals during test failures, apply these tactics:

1. **Injecting peek() for Logging:** Use the `.peek()` intermediate operation to log elements as they flow through specific stages of the pipeline:

```
transactions.stream()
    .filter(t -> t.amount() > 100)
    .peek(t -> log.debug("Passed Filter: {}", t.id()))
    .map(Transaction::merchantId)
    .collect(Collectors.toList());
```

2. **Utilizing IDE Stream Debuggers:** Modern IDEs (like IntelliJ IDEA or Visual Studio) contain visual stream debuggers. When you set a breakpoint on a stream statement, the debugger can render a visual representation of how elements are filtered and mapped at each stage.
3. **Splitting the Pipeline for Stack Traces:** If a pipeline throws an exception, temporarily break the pipeline into separate intermediate variables to isolate the throwing operation in the stack trace.

STAR Moment: The Stateless Pipeline Principle

A functional stream pipeline must never modify state variables outside the stream. If you write a `.forEach()` or `.map()` that mutates a shared list or updates a local counter, you have violated the functional contract. You lose thread safety, and your code cannot be parallelized. Keep your lambdas pure, stateless, and side-effect-free. In an interview, say: *“I use `collect()` and `reduce()` to accumulate results rather than mutating external variables, because stateless pipelines are safe to parallelize and easy to reason about.”*

Chapter 6

Design Patterns in Enterprise Frameworks

“Design patterns are not templates to copy; they are vocabulary to describe architectural relationships.”

6.1 Pattern Abuse in Interviews

Many software professionals prepare for design pattern questions by memorizing standard descriptions: “Singleton is a class with one instance,” or “Factory creates objects.”

During a senior engineering interview, this is insufficient. A senior candidate must show how patterns solve real architectural problems, such as auditing transaction status, wrapping legacy systems, or handling dynamic business rules. You must also show that you know how these patterns are integrated into the frameworks you use daily (like Spring, Hibernate, or ASP.NET Core).

In this chapter, we will examine how AuraPay utilizes design patterns, focusing on the **Observer Pattern** to audit payment settlement events for financial compliance.

6.2 Creational Patterns

Creational patterns abstract the instantiation process, decoupling your application from how objects are created and composed.

6.2.1 The Builder Pattern

When constructing complex domain objects like AuraPay’s `TransactionRecord`, constructors with ten parameters lead to unreadable code. The **Builder Pattern** solves this, allowing you to build objects step-by-step while maintaining immutability:

```
// Example of a fluent, type-safe builder for transactions
TransactionRecord tx = new TransactionRecordBuilder()
    .withId(UUID.randomUUID())
    .fromAccount(sourceId)
```

```

.toAccount(destId)
.withAmount(new BigDecimal("100.00"))
.inCurrency("USD")
.atTimestamp(Instant.now())
.build(); // Immutability and invariants are validated in build()

```

6.2.2 The Factory Pattern

When the core ledger processor needs to route a payment, it uses a **Factory Pattern** to dynamically instantiate the correct `SettlementRoute` processor based on the transaction metadata (such as routing cards via Visa vs. executing ACH).

6.2.3 The Singleton Pattern (Creational Deep-Dive)

The Singleton pattern guarantees that a class has only one instance and provides a global point of access to it. In multi-threaded enterprise engines (such as a shared connection pool managed by HikariCP), writing a thread-safe Singleton requires **Double-Checked Locking**:

```

public class LedgerConnectionPool {
    private static volatile LedgerConnectionPool instance;

    private LedgerConnectionPool() {
        // Prevent reflection instantiation
        if (instance != null) {
            throw new IllegalStateException("Already instantiated");
        }
    }

    public static LedgerConnectionPool getInstance() {
        if (instance == null) { // First check (no lock)
            synchronized (LedgerConnectionPool.class) {
                if (instance == null) { // Second check (with lock)
                    instance = new LedgerConnectionPool();
                }
            }
        }
        return instance;
    }
}

```

Warning for Senior Candidates: In cloud-native systems, classical Singletons are often considered an anti-pattern: 1. **Testing Complexity:** They introduce global mutable state, making parallel unit tests prone to side effects. 2. **Scalability limits:** A Singleton is only single per JVM instance. If your service scales out to ten microservice containers, you have ten connection pool instances, not one. 3. **IoC Managed Singletons:** Modern systems delegate singleton lifecycle management to Dependency Injection (IoC) containers rather than hardcoding static `getInstance()` methods.

6.3 Structural Patterns

Structural patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.

6.3.1 The Adapter Pattern

In banking-grade environments, you must frequently integrate with legacy core systems (e.g., COBOL-based mainframes or SOAP APIs). The **Adapter Pattern** wraps the legacy API with a clean interface that complies with your domain. For example, a `LegacySoapAdapter` implements the modern `LedgerRepository` interface, converting domain calls into SOAP requests under the hood.

6.3.2 The Decorator Pattern

If you need to add auditing, metrics, or retry behaviors to transaction execution, do not pollute the core processing code. Use a **Decorator Pattern** to wrap the transaction processor, adding the cross-cutting concerns dynamically:

```
// Wrapping the core processor with an audit logging decorator
TransactionProcessor decoratedProcessor = new
    ↪ AuditingTransactionProcessorDecorator(
        new CoreTransactionProcessor(repository, calculator, sender)
    );
```

6.4 Behavioral Patterns

Behavioral patterns identify common communication patterns between objects and realize these patterns.

6.4.1 The Strategy Pattern

AuraPay utilizes the **Strategy Pattern** to swap fee calculations dynamically. A `FlatFeeStrategy`, `TieredFeeStrategy`, and `MerchantDiscountRateStrategy` all implement `FeeCalculator`, allowing the routing engine to choose the strategy at runtime based on client profiles.

6.4.2 The Observer Pattern (Injected)

When a transaction succeeds, external systems—such as the ledger audit index, fraud detection, and SMS notification dispatchers—must be notified. Hardcoding these calls inside the core transaction loop creates tight coupling.

We solve this using the **Observer Pattern**. The `TransactionEventPublisher` manages a list of observers and notifies them of transaction success or failure.

Here is the implementation:

```
package com.aurapay.events;

import com.aurapay.domain.TransactionRecord;
import java.util.ArrayList;
```



```

import java.util.List;
import java.util.Objects;

/**
 * Interface defining the Observer contract for transaction events.
 */
public interface TransactionObserver {
    void onTransactionSuccess(TransactionRecord transaction);
    void onTransactionFailed(TransactionRecord transaction, Throwable error);
}

/**
 * Concrete Observer that writes a persistent audit trail for security
 * ↪ compliance.
 */
public class AuditTrailObserver implements TransactionObserver {

    @Override
    public void onTransactionSuccess(TransactionRecord transaction) {
        System.out.printf("AUDIT SUCCESS: Transaction %s of %s %s from %s to %s
        ↪ registered in immutable log.%n",
            transaction.transactionId(),
            transaction.amount(),
            transaction.currency(),
            transaction.sourceAccountId(),
            transaction.destinationAccountId());
    }

    @Override
    public void onTransactionFailed(TransactionRecord transaction, Throwable
    ↪ error) {
        System.err.printf("AUDIT FAILURE: Transaction %s failed. Error: %s%n",
            transaction.transactionId(),
            error.getMessage());
    }
}

/**
 * Subject class managing observers and publishing transaction status updates.
 */
public class TransactionEventPublisher {
    private final List<TransactionObserver> observers = new ArrayList<>();

    public synchronized void registerObserver(TransactionObserver observer) {
        observers.add(Objects.requireNonNull(observer));
    }

    public synchronized void deregisterObserver(TransactionObserver observer) {

```

```

        observers.remove(observer);
    }

    public void notifySuccess(TransactionRecord transaction) {
        List<TransactionObserver> targets;
        synchronized (this) {
            targets = new ArrayList<>(observers);
        }
        for (TransactionObserver observer : targets) {
            observer.onTransactionSuccess(transaction);
        }
    }

    public void notifyFailure(TransactionRecord transaction, Throwable error) {
        List<TransactionObserver> targets;
        synchronized (this) {
            targets = new ArrayList<>(observers);
        }
        for (TransactionObserver observer : targets) {
            observer.onTransactionFailed(transaction, error);
        }
    }
}

```

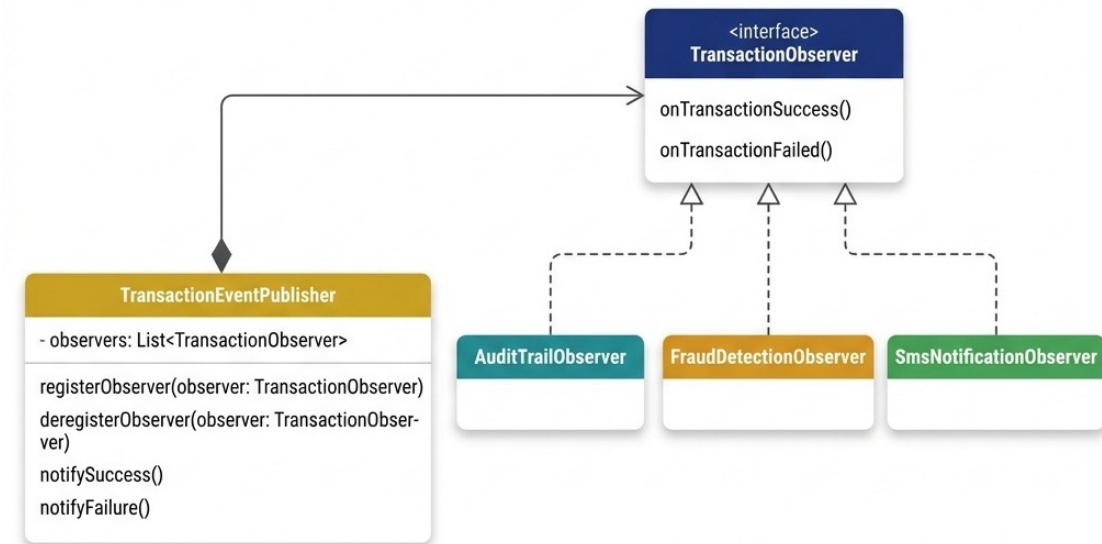


Figure 6.1: Observer Pattern Class Diagram

6.4.3 The State Pattern (Behavioral Deep-Dive)

In payment platforms, transactions transition through a strict sequence of states: **CREATED** → **PENDING** → **SETTLED** or **FAILED** → **REFUNDED**.

Instead of writing a massive, hard-to-maintain switch block inside the transaction manager:

- We apply the **State Pattern**.
- We define a **TransactionState** interface representing the allowed operations (e.g., `approve()`, `fail()`, `refund()`).
- Each state is implemented as a concrete class (e.g., `PendingState`, `SettledState`).
- The transition logic is encapsulated inside each state class, preventing invalid state jumps (e.g., you cannot refund a `CREATED` transaction, only a `SETTLED` one), enforcing business invariants at runtime.

6.5 Enterprise Integration & Data Access Patterns

In production-grade enterprise systems, designing clean persistence boundaries is as critical as GoF object coordination:

6.5.1 Repository and Unit of Work Patterns

- **The Repository Pattern:** Mediates between the domain and data mapping layers using a collection-like interface for accessing domain objects (e.g., `LedgerRepository`). The business layer remains completely ignorant of whether data is stored in Postgres, MongoDB, or an in-memory map.
- **The Unit of Work Pattern:** Tracks all database-modifying operations (inserts, updates, deletes) during a single transaction context. Instead of each repository committing changes independently, the Unit of Work coordinates the commit boundary (e.g., Spring's `@Transactional` boundary or Entity Framework's `DbContext.SaveChanges()`). This guarantees that multiple repository updates succeed or fail together, protecting transactional boundaries.

6.5.2 Data Transfer Object (DTO) Pattern

Exposing raw database entities directly over public REST/gRPC endpoints is a major security and design vulnerability. Doing so leaks internal database schemas, primary IDs, and sensitive columns (like password hashes).

- **The Solution:** Use **DTOs** (Data Transfer Objects) to define explicit data contracts for request inputs and response outputs.
- **Mapping:** Utilize mapper libraries to map entities to DTOs before serialization, decoupling internal database schemas from external API consumers.

6.5.3 Active Record vs. Data Mapper

When designing data access layers, select the persistence mapping style suited for the workload complexity:

- **Active Record (e.g., Ruby on Rails, Django ORM):** An approach where the entity class holds both the data attributes and the database access methods (e.g., `user.save()`, `user.delete()`). Very simple and fast to implement for CRUD applications. However, it violates SRP by coupling the domain model to database connection engines.
- **Data Mapper (e.g., Hibernate, JPA, Entity Framework):** An approach that completely separates data representation (the entity class) from database operations (the map-

per/repository layer). The domain object remains database-ignorant, simplifying business unit testing and maintaining clean domain boundaries.

6.6 Framework Integration: Patterns in the Wild

In senior interviews, you must connect patterns to the frameworks you use. Here is how modern enterprise engines implement them natively:

Pattern	Framework Application	How It Works
Factory	Spring Bean Container	Spring's BeanFactory instantiates beans dynamically using reflection and dependency injection maps.
Proxy	Hibernate Lazy Loading	Hibernate generates proxy wrappers for entity relationships, loading child records from the database only when getter methods are invoked (Lazy Initialization).
Observer	Spring Application Events	Publishing events via ApplicationEventPublisher and consuming them using @EventListener decouples services asynchronously.
Adapter	Spring MVC Handlers	HandlerAdapter maps incoming HTTP requests to controller methods, shielding the servlet container from concrete execution signatures.
Template Method	Spring JdbcTemplate	JdbcTemplate defines the skeleton of database execution (opening connection, statement preparation, cleanup) while letting subclasses map rows to domain objects.

Why is it called "Spring"? Rod Johnson created the Spring Framework in 2003 as a reaction to the overwhelming complexity of **J2EE** (Java 2 Enterprise Edition). He chose the name *Spring* to represent a **fresh start** 2014 a new season after the long, cold "winter" of J2EE's XML-heavy, boilerplate-ridden configuration. Spring made enterprise Java feel light and productive again, and the name perfectly captures that rebirth.

Why is it called "Hibernate"? Gavin King created Hibernate framework in 2001 and chose the name because Java objects *"hibernate"* (go dormant) inside the database and wake up when the application needs them. Just as animals hibernate through winter and emerge in spring, your domain objects are serialized into database rows and later rehydrated into live Java objects. The bear logo reinforces the metaphor.

STAR Moment: The Framework Pattern Test

During system design interviews, explain design patterns in terms of the framework concepts the interviewer already knows. Instead of drawing a generic observer diagram, say: *"We will implement this like a Spring ApplicationEventPublisher or a Kafka Event Broker, decoupling the transactional write thread from the audit and search indexing consumers."* This shows you understand patterns in modern, production-grade architectures.

Part III

Code Performance and Data Structures

Chapter 7

Designing for Performance and Concurrency

“High throughput is achieved not by making code run faster, but by eliminating waiting, contention, and coordination.”

7.1 Concurrency in System Design Interviews

When interviewing for a senior staff or engineering manager role, you will inevitably face questions about system bottlenecks. Typical candidates suggest “adding a cache” or “using multi-threading.”

An interviewer wants to hear about the trade-offs of thread management and database contention. You must be able to detail the trade-offs between **Virtual Threads** (Loom) and **Reactive Programming**, explain why a database connection pool that is too large actually degrades system throughput, and articulate exactly when to use **Optimistic** vs. **Pessimistic Concurrency Control** in high-stakes financial operations.

In this chapter, we explore how AuraPay designs its ledger persistence layers to sustain high-volume transaction throughput without risking data drift or transaction races.

7.2 Concurrency Models: Virtual Threads vs. Reactive

In Java 21+, the JVM introduces **Virtual Threads** (Project Loom). In the past, scaling web applications to handle thousands of concurrent connections required reactive frameworks (e.g., Spring WebFlux, Project Reactor).

7.2.1 The Thread-per-Request Model

Historically, web servers mapped one platform thread to one HTTP request. Since platform threads map 1-to-1 with operating system threads, they are expensive. Memory footprints (typically 1MB per thread stack) and operating system context-switching overhead capped JVM throughput at a few thousand concurrent threads.

7.2.2 The Reactive Approach

Reactive programming solved this by decoupling processing execution from threads. Event loops processed chunks of data asynchronously via non-blocking callbacks.

- **Advantage:** Extreme scalability with very low resource utilization.
- **Disadvantage:** Increased code complexity (“callback hell”), difficult stack traces, and complete incompatibility with standard Java threading tools like `ThreadLocal`.

7.2.3 The Virtual Thread Revolution

Virtual threads are lightweight threads managed by the JVM rather than the OS. They are mounted onto a small carrier pool of platform threads. When a virtual thread blocks on I/O (e.g., executing a SQL query), the JVM unmounts the virtual thread, parking it, and assigns the carrier thread to another task.

- **Impact:** You can run millions of virtual threads concurrently while writing standard, synchronous, block-on-write code that is easy to read, debug, and trace.

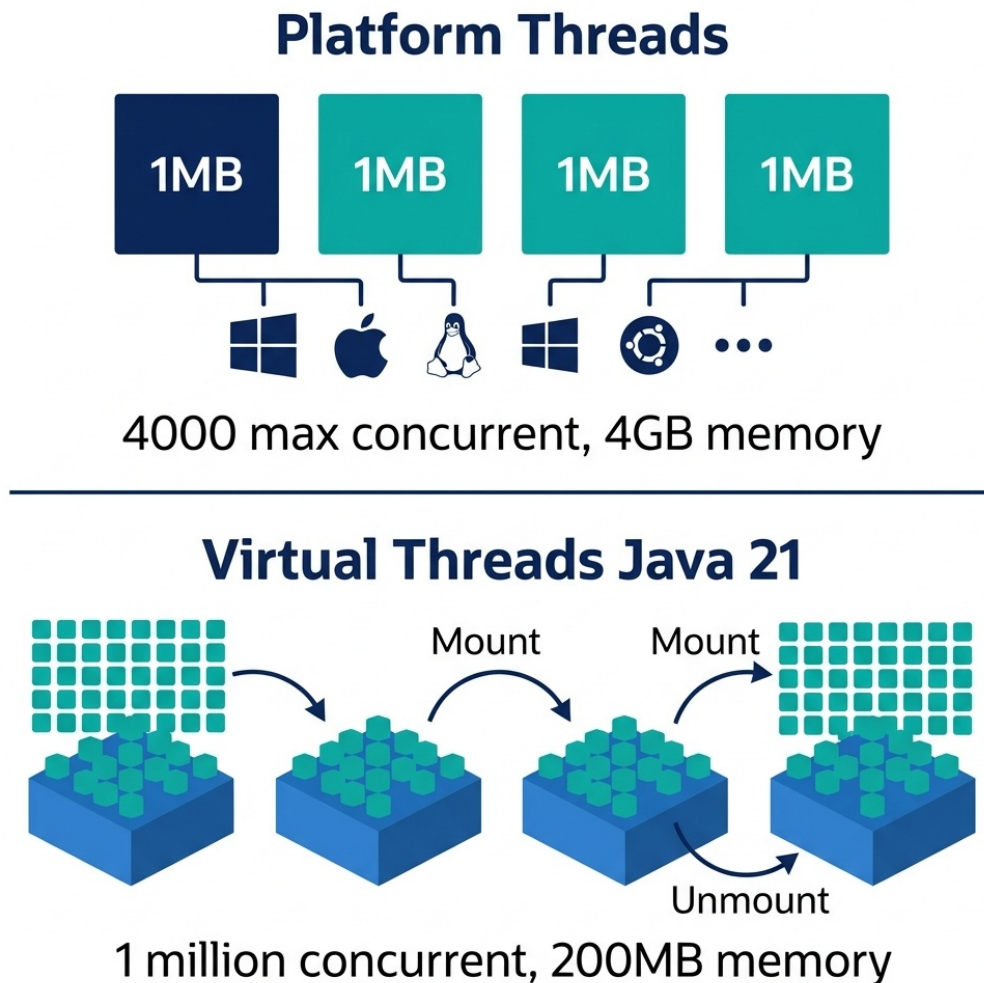


Figure 7.1: Virtual Threads vs Platform Threads

7.3 Database Locking: Optimistic vs. Pessimistic

When two concurrent transactions attempt to debit the same ledger account, we must prevent double-debiting and race conditions. This requires strict concurrency control.

7.3.1 Pessimistic Concurrency Control (PCC)

Pessimistic locking assumes that a conflict is highly likely. It blocks concurrent transactions by locking the records at the database level:

```
SELECT * FROM accounts WHERE id = ? FOR UPDATE;
```

- **Pros:** Guaranteed safety; concurrent transactions wait in line until the lock is released.
- **Cons:** High lock contention, database thread starvation, and high risk of deadlocks under load.
- **When to use:** When transaction frequency on a single account (e.g., a corporate merchant account) is extremely high, and you cannot afford transaction retries.

7.3.2 Optimistic Concurrency Control (OCC)

Optimistic locking assumes conflicts are rare. It allows concurrent threads to read and edit records without blocking. When saving the entity, the engine verifies that the record has not been modified by checking a `version` field.

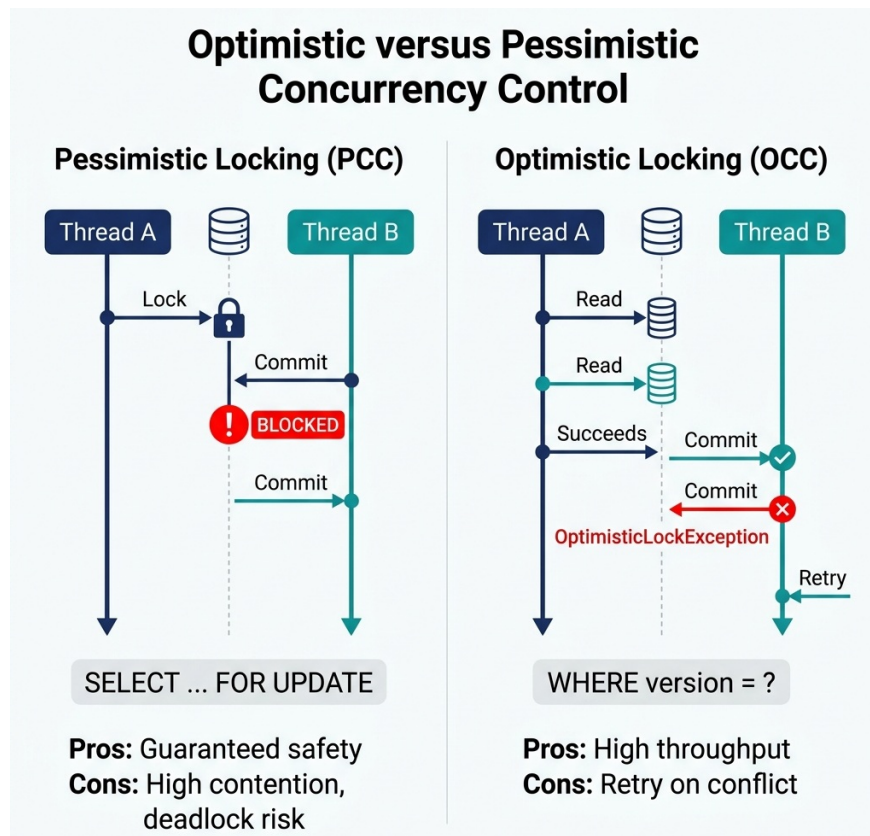


Figure 7.2: Optimistic vs Pessimistic Concurrency Control

The following code illustrates this version-checking implementation:

```
package com.aurapay.persistence;

import java.math.BigDecimal;
import java.util.Objects;
import java.util.UUID;

/**
 * Represents a database-mapped Ledger Account Entity with versioning for
 * Optimistic Concurrency Control (OCC).
 */
public class AccountEntity {
    private final UUID id;
    private BigDecimal balance;
    private final String currency;
    private long version; // Enforces OCC state check

    public AccountEntity(UUID id, BigDecimal balance, String currency, long
        ↪ version) {
        this.id = Objects.requireNonNull(id);
        this.balance = Objects.requireNonNull(balance);
        this.currency = Objects.requireNonNull(currency);
        this.version = version;
    }

    public UUID getId() { return id; }
    public BigDecimal getBalance() { return balance; }
    public String getCurrency() { return currency; }
    public long getVersion() { return version; }

    public void updateBalance(BigDecimal newBalance) {
        this.balance = Objects.requireNonNull(newBalance);
    }

    public void incrementVersion() {
        this.version++;
    }
}

/**
 * Repository implementation executing the version check update query.
 */
public class DatabaseLedgerRepository {

    /**
     * Updates the account in the database using a strict version-matching query.
     * Throws an exception if another thread modified the record concurrently.
     */
}
```

```

public void save(AccountEntity account) {
    // Under the hood, this compiles to the SQL query:
    // UPDATE accounts SET balance = ?, version = version + 1 WHERE id = ?
    //   ↪ AND version = ?;
    String query = "UPDATE accounts SET balance = :balance, version =
    //   ↪ :version + 1 " +
    //           "WHERE id = :id AND version = :version";

    int rowsUpdated = mockExecuteUpdateQuery(query, account);

    // OCC FAILURE CHECK: If no rows were updated, a concurrent transaction
    //   ↪ modified the version first.
    if (rowsUpdated == 0) {
        throw new OptimisticLockingFailureException(
            String.format("Optimistic lock conflict on account %s. Outdated
            //   ↪ version: %d",
            account.getId(), account.getVersion())
        );
    }

    account.incrementVersion();
}

private int mockExecuteUpdateQuery(String query, AccountEntity account) {
    // Simulates the DB executing the update. In a real system, the database
    //   ↪ engine
    // returns 0 if the WHERE clause (matching ID and version) matches no
    //   ↪ records.
    return 1; // Returns 1 on success, 0 on concurrent modification conflict
}
}

```

- **Pros:** High throughput; no database locks are held while executing business logic.
- **Cons:** If a conflict occurs, one of the transactions fails, forcing the application to catch the exception and retry the entire workflow.
- **When to use:** In low-to-medium contention systems where write conflicts are rare, maximizing parallel performance.

7.4 Concurrency Control & Locking Matrix

When designing financial ledgers, selecting the right locking paradigm is critical. The following matrix contrasts the three primary concurrency control options:

Criteria	Optimistic Locking (OCC)	Pessimistic Locking (PCC)	Distributed Locking (e.g., Redis Redlock)
Mechanism	Application version check (WHERE version = ?)	Database row-level locks (SELECT FOR UPDATE)	In-memory distributed key lease

Criteria	Optimistic Locking (OCC)	Pessimistic Locking (PCC)	Distributed Locking (e.g., Redis Redlock)
Complexity	Low (handled natively by ORM/SQL)	Medium (requires managing database locks)	High (requires distributed lock manager infrastructure)
Contention Cost	Low (no blocking, fails fast)	High (blocking threads waiting for lock)	Medium (spins or rejects requests)
Lock Duration	Nanoseconds (during DB UPDATE commit)	Milliseconds (entire DB transaction block)	Leased duration (typically 5–30 seconds)
Starvation Risk	High for hot accounts (constant retries)	Low (threads queue in order)	Medium (depends on retry/backoff settings)
Scale Limits	Scales with DB capacity	Hard limit based on DB connection pool size	Scales horizontally with distributed key store
Deadlock Risk	Zero	High (requires strict alphabetical locking of aggregates)	Medium (depends on lock lease expiration / release logic)

7.5 Caching Patterns & Consistency Deep-Dive

In high-throughput platforms, caching is used to offload read traffic from the primary database. However, introducing a cache creates the classic problem of **cache invalidation**.

7.5.1 Caching Architectures

1. Cache-Aside (Recommended for Ledgers):

- The application queries the cache first.
- On a *cache hit*, the application returns the cached data.
- On a *cache miss*, the application queries the database, writes the result to the cache, and returns it.

2. Write-Through:

- The application writes directly to the cache, and the cache synchronizes that write to the database synchronously.

3. Write-Behind (Write-Back):

- The application writes to the cache. The cache buffers these writes and flushes them to the database asynchronously.
- **WARNING:** Do not use Write-Behind for financial ledgers. A crash of the cache server before the buffer is flushed results in permanent data loss.

7.5.2 Cache Invalidation & Race Conditions

When updating the database, the application must invalidate the cache key.

- **Naïve Update:** Modifying the database and then updating the cache value. This introduces

a race condition: if two concurrent writes occur, they can write to the database and cache in different orders, leading to stale cache states.

- **Correct Pattern:** Always **delete** the cache key after writing to the database. By deleting the key, you force the next read operation to perform a Cache-Aside query from the source database, guaranteeing consistency.
- **Transactional Safety:** Ensure the cache key deletion occurs inside the database transaction's post-commit hook. If the database transaction rolls back, the cache key must not be deleted.

7.6 CPU Cache Locality (L1/L2/L3) in HFT Matching Loops

In ultra-low-latency matching engines (like ZenithTrade), garbage collection pauses and CPU cache misses are the primary bottlenecks. To write code that runs in the microsecond range, you must design for **cache locality**:

- **The Problem with Linked Lists:** A standard `LinkedList` contains nodes linked by memory references. These references can be scattered randomly across heap memory. When the CPU traverses a linked list to match orders, it incurs constant **L1/L2/L3 cache misses**, forcing the CPU to fetch data from physical RAM, which is up to 200 times slower than L1 cache.
- **The Array/Contiguous Layout:** To minimize cache misses, the matching loop must use contiguous memory structures. By storing orders in flat array layouts or utilizing pre-allocated object pools, the CPU can load adjacent elements into L1/L2 cache pre-emptively, accelerating execution speed.

7.7 Connection Pool Sizing: The HikariCP Formula

A common design flaw is over-allocating database connection pool sizes. If you have 500 thread workers, candidates often set the connection pool size to 500.

The Pitfall: A database engine is limited by physical resources (CPU cores, disk write I/O speed, memory). When hundreds of threads attempt to execute database operations concurrently, the database server spends more time performing CPU context switches than executing queries.

HikariCP (the industry-standard connection pool manager) uses a formula derived from PostgreSQL benchmark testing to size database pools:

$$Pool\ Size = (Core\ Count \times 2) + Effective\ Spindle\ Count$$

For example, a database server with 8 CPU cores and an SSD array (spindle count of 1) should have a pool size of:

$$(8 \times 2) + 1 = 17\ Connections$$

Setting the pool size to 17 will yield *higher* overall throughput than setting it to 100, due to the minimization of CPU context switching and disk spindle thrashing.

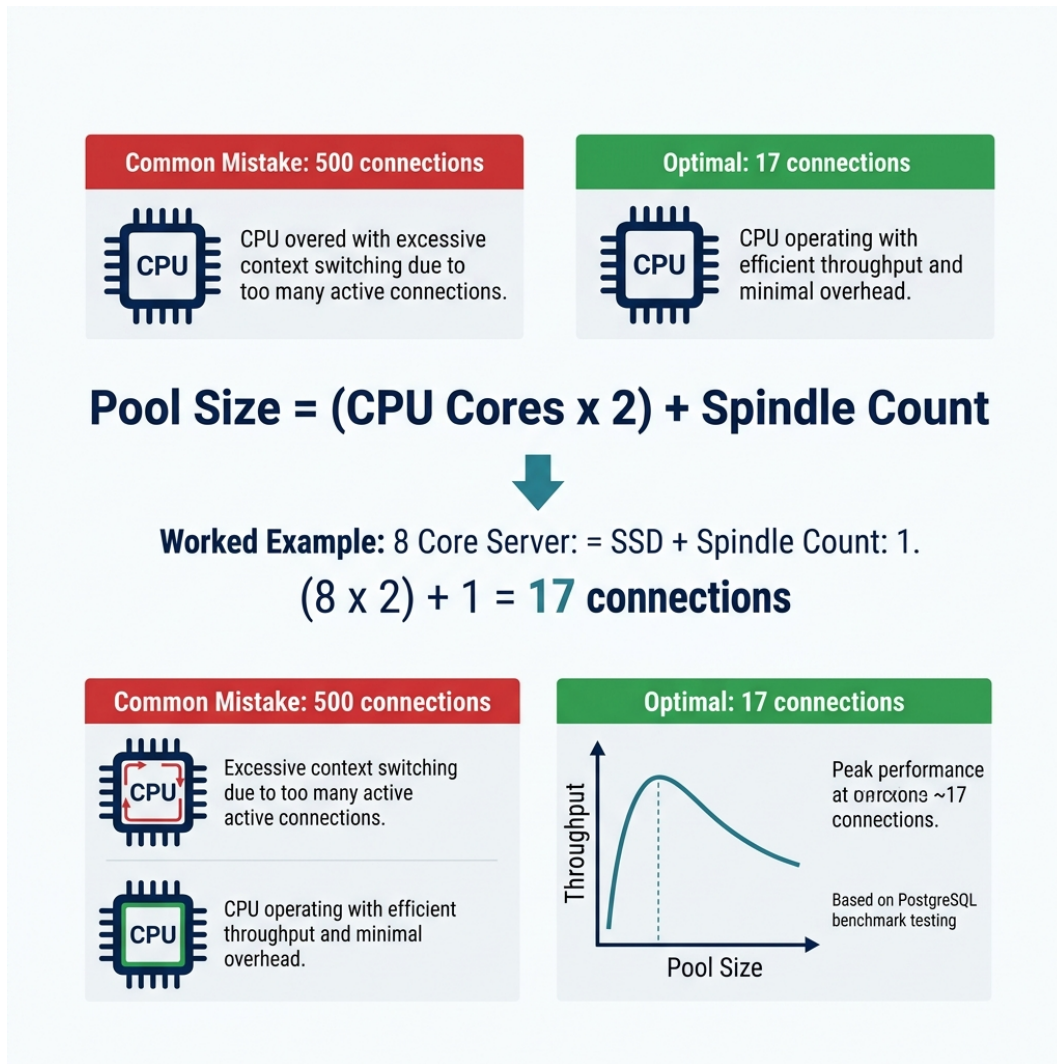


Figure 7.3: HikariCP Connection Pool Sizing

Why is it called “Hikari”? The name is not a person — **Hikari** () is the Japanese word for “**light**.” Creator Brett Wooldridge was working in Japan when he built it, frustrated by the bloat and slowness of existing connection pools (C3P0, DBCP, BoneCP). He designed HikariCP to be *light* in weight (~130KB jar, zero dependencies), *light* in speed (fastest JDBC pool ever benchmarked), and *light* in complexity. His obsession with zero-overhead engineering — using `ConcurrentBag` instead of `LinkedBlockingQueue` to eliminate lock contention, and a custom `FastList` to skip array bounds checks — made it so fast that Spring Boot adopted it as the **default connection pool** starting in version 2.0 (2018). Today, if you add `spring-boot-starter-data-jpa` to your project, HikariCP is already running under the hood. Fun fact: Japan’s famous bullet train (Shinkansen) has a service tier called *Hikari* — the name fits perfectly.

STAR Moment: The Cache Invalidation Design

When discussing performance during an interview, never say “*We will add a cache.*” Say: “*We will implement a Cache-Aside pattern using Redis. To prevent stale reads in our double-entry ledger, we will use a transactional write-through strategy, invalidating cache keys atomically inside the database commit boundary to ensure absolute consistency.*” This shows you understand caching boundaries in financial transaction systems.

Chapter 8

Core Algorithms & Assessment Tactical Guide

“Algorithms are not trivia; they are the baseline vocabulary of computational efficiency under resource constraints.”

8.1 General Coding Assessment (GCA) Tactics

Many organizations (such as Capital One, fintech firms, and major technology companies) screen candidates using standardized online coding assessments (e.g., CodeSignal GCA, HackerRank, or Codility). The most common and highest-stress format is the **70-Minute, 4-Question Speed Run**.

Candidates often fail not because they lack coding skills, but because they manage their time poorly. Under stress, they get stuck debugging a minor edge case on Question 3, leaving zero time for Question 4, which carries the highest scoring weight. To score a perfect 800+ on these assessments, you must treat the test as a tactical exercise.

8.1.1 The 4-Question Blueprint

Question	Difficulty	Archetype	Time Target	Tactical Rule
Q1	Easy	Basic array manipulation or string formatting	5–8 Min	Write clean, brute-force code immediately. Do not over-optimize.
Q2	Medium	2D matrix transformation, array rotation, or simulation	10–12 Min	Watch for array bounds and off-by-one errors. Keep helper functions simple.

Question	Difficulty	Archetype	Time Target	Tactical Rule
Q3	Medium-Hard	Hashmap state-tracking, custom key groupings, or string alignments	15–20 Min	Identify the map key early. Keep data structures simple.
Q4	Hard	Monotonic stack/queue, advanced sliding window, or binary search optimization	20–25 Min	If brute force is $O(N^2)$, look for a monotonic property to reduce to $O(N)$.

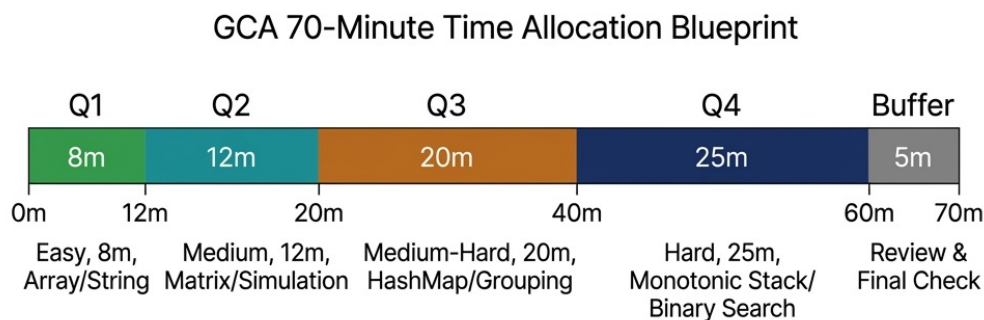


Figure 8.1: GCA 70-Minute Time Allocation Blueprint

8.1.2 The 70-Minute GCA Master Plan

To secure a top-tier score under pressure, you must follow a strict, disciplined time-boxing strategy:

- 1. The 3-Minute Limit.** If you get stuck on a compile or logic bug for more than 3 minutes, comment out your changes, revert to your last working baseline, and rethink your boundary conditions. Do not write code blindly hoping it will pass.
- 2. Never print in a loop.** Printing to standard output (`System.out.println`, `print()`, `Console.WriteLine`) inside loops kills execution speed and can cause the platform to timeout on large hidden test cases.
- 3. Submit immediately.** Once your solution passes the visible test cases, submit it and move to the next question. Do not waste time cleaning up variable names or optimizing unless a performance timeout occurs.
- 4. Solve in order (1 -> 2 -> 4 -> 3).** On platforms like CodeSignal, Q4 is often worth significantly more points than Q3, and it is usually more deterministic (e.g., a standard Monotonic Stack or Binary Search) compared to Q3, which can be a tedious simulation or string parsing problem. If you finish Q1 and Q2 quickly, scan Q4. If it matches an algorithmic archetype you recognize, solve it before tackling Q3.

8.1.3 Cracking Live Technical Rounds (Teams / Zoom / In-Person)

Unlike asynchronous online assessments, live coding rounds evaluate your communication, structured thinking, and collaborative problem-solving:

Talk Out Loud Constantly. Do not code in silence. Explain your thought process, what variables you are declaring, and why. The interviewer wants to see *how* you think.

State the Invariants First. Before writing code, state the pre-conditions, post-conditions, and loop invariants to the interviewer. This shows that you are a disciplined software engineer rather than a syntax hacker.

Dry-Run with Small Test Cases. Walk through your logic with a simple test case by tracing variable values manually on the screen before running the code.

Handle Feedback Gracefully. If the interviewer points out a bug or asks, “*What happens if this input is null?*”, do not get defensive. Acknowledge it, state the pre-condition check you will add, and implement the fix.

8.2 Data Structures Primer for Coding Assessments

Before diving into algorithm patterns, you must be fluent in the data structures they depend on. Many candidates know the algorithm conceptually but lose time during interviews because they cannot remember the correct method names or choose the wrong collection type. This section is your quick-reference guide.

8.2.1 ArrayList — Dynamic Array

The workhorse of coding interviews. An automatically resizing array with $O(1)$ random access.

```
List<Integer> list = new ArrayList<>();
list.add(42);           // Append to end -  $O(1)$  amortized
list.add(0, 99);        // Insert at index 0 -  $O(N)$  shift
list.get(0);            // Random access -  $O(1)$ 
list.set(1, 50);        // Replace at index -  $O(1)$ 
list.remove(0);         // Remove at index -  $O(N)$  shift
list.size();            // Current element count
list.contains(42);      // Linear search -  $O(N)$ 
list.isEmpty();         // Check if empty
Collections.sort(list); // Sort in-place -  $O(N \log N)$ 
```

When to use: Default choice when you need an ordered, indexable collection. Prefer over `LinkedList` for almost all interview problems.

8.2.2 HashMap and HashSet — $O(1)$ Lookup

The most critical data structure in interviews. `HashMap` maps keys to values. `HashSet` stores unique elements. Both provide $O(1)$ average-case lookup, insert, and delete.

```
// HashMap: Key -> Value mapping
Map<String, Integer> map = new HashMap<>();
map.put("apple", 3);           // Insert/update -  $O(1)$ 
```

```

map.get("apple");           // Lookup - O(1), returns null if
    ↪ missing
map.getDefault("banana", 0); // Lookup with fallback - O(1)
map.containsKey("apple");   // Key existence check - O(1)
map.remove("apple");        // Remove by key - O(1)
map.keySet();               // All keys (for iteration)
map.values();               // All values
map.entrySet();             // All key-value pairs

// Frequency counting pattern (extremely common)
for (char c : text.toCharArray()) {
    map.merge(c, 1, Integer::sum); // Increment count elegantly
}

// HashSet: Unique element storage
Set<Integer> seen = new HashSet<>();
seen.add(42);           // Add element - O(1)
seen.contains(42);      // Membership check - O(1)
seen.remove(42);        // Remove element - O(1)

```

When to use: Frequency counting, duplicate detection, two-sum lookups, graph adjacency lists, caching previously computed results (memoization).

8.2.3 Deque (ArrayDeque) — Double-Ended Queue

A Deque (pronounced “deck”) supports insertion and removal at **both ends** in $O(1)$ time. It is the backbone of sliding window problems and is also the recommended implementation for stacks and queues in modern Java.

```

Deque<Integer> deque = new ArrayDeque<>();

// --- As a double-ended queue ---
deque.offerFirst(1); // Add to front - O(1)
deque.offerLast(2);  // Add to back - O(1)
deque.peekFirst();   // View front without removing - O(1)
deque.peekLast();    // View back without removing - O(1)
deque.pollFirst();    // Remove from front - O(1)
deque.pollLast();     // Remove from back - O(1)

// --- As a Stack (LIFO) - use instead of java.util.Stack ---
deque.push(42);       // Push onto stack (adds to front)
deque.peek();         // View top element
deque.pop();          // Pop from stack (removes from front)

// --- As a Queue (FIFO) ---
deque.offer(42);       // Enqueue (adds to back)
deque.peek();          // View head
deque.poll();          // Dequeue (removes from front)

deque.isEmpty();       // Check if empty

```

```
deque.size(); // Current element count
```

Why ArrayDeque over Stack and LinkedList? Java's `java.util.Stack` class is synchronized (slow) and extends `Vector` (legacy). `LinkedList` has pointer-chasing overhead. `ArrayDeque` is backed by a resizable circular array — it is the fastest general-purpose stack and queue implementation.

When to use: Sliding window maximum/minimum (store indices), BFS (as a queue), DFS iteratively (as a stack), monotonic deque problems.

8.2.4 Queue — First-In, First-Out (FIFO)

Used primarily for BFS traversal. While `ArrayDeque` is the best implementation, you will often see `LinkedList` used in interview solutions.

```
Queue<Integer> queue = new LinkedList<>();
queue.offer(1); // Enqueue - O(1)
queue.offer(2);
queue.peek(); // View head (returns 1) - O(1)
queue.poll(); // Dequeue (removes 1) - O(1)
queue.isEmpty(); // Check if empty
queue.size(); // Current element count
```

When to use: BFS graph/tree traversal, topological sort (Kahn's algorithm), level-order processing.

8.2.5 Stack Behavior — Last-In, First-Out (LIFO)

There is no preferred standalone `Stack` class in modern Java. Use `ArrayDeque` with `push/pop/peek`.

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10); // Push - O(1)
stack.push(20);
stack.push(30);
stack.peek(); // View top (returns 30) - O(1)
stack.pop(); // Pop (removes 30) - O(1)
```

When to use: Bracket matching (valid parentheses), monotonic stack (next greater element), DFS iterative traversal, expression evaluation, undo operations.

8.2.6 PriorityQueue — Min-Heap / Max-Heap

A `PriorityQueue` is a binary heap that always keeps the smallest element at the top (min-heap by default). Insertion and removal are $O(\log N)$. Peeking at the top is $O(1)$.

```
// Min-Heap (default) - smallest element first
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(30);
minHeap.offer(10);
minHeap.offer(20);
minHeap.peek(); // Returns 10 (smallest) - O(1)
minHeap.poll(); // Removes 10 - O(log N)
```

```

// Max-Heap - largest element first
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
maxHeap.offer(30);
maxHeap.offer(10);
maxHeap.peek();           // Returns 30 (largest) - O(1)

// Custom comparator - sort by a specific field
PriorityQueue<int[]> pq = new PriorityQueue<>(
    Comparator.comparingInt(a -> a[1]) // Sort by second element
);

```

When to use: Top-K problems (K-th largest element), streaming median (dual-heap), Dijkstra's shortest path, merge K sorted lists, task scheduling by priority.

8.2.7 LinkedList — Node-Based Sequential Access

A doubly-linked list where each node points to its predecessor and successor. Rarely the best choice for array-style problems, but essential for pointer-manipulation questions.

```

LinkedList<Integer> list = new LinkedList<>();
list.addFirst(1);           // Add to head - O(1)
list.addLast(2);            // Add to tail - O(1)
list.getFirst();            // View head - O(1)
list.getLast();             // View tail - O(1)
list.removeFirst();         // Remove head - O(1)
list.removeLast();          // Remove tail - O(1)
list.get(5);                // Access by index - O(N) traversal!

```

When to use: Fast/slow pointer problems (cycle detection, find middle), LRU cache implementation (with HashMap), problems that explicitly say “linked list” in the prompt.

8.2.8 TreeMap and TreeSet — Sorted Collections

A `TreeMap` is a red-black tree that keeps keys in **sorted order**. All operations are $O(\log N)$. It provides powerful navigation methods that `HashMap` cannot.

```

TreeMap<Integer, String> map = new TreeMap<>();
map.put(10, "ten");
map.put(30, "thirty");
map.put(20, "twenty");

map.firstKey();             // Smallest key (10) - O(log N)
map.lastKey();              // Largest key (30) - O(log N)
map.floorKey(25);           // Largest key <= 25 -> returns 20
map.ceilingKey(25);         // Smallest key >= 25 -> returns 30
map.lowerKey(20);           // Largest key < 20 -> returns 10
map.higherKey(20);          // Smallest key > 20 -> returns 30
map.subMap(10, 30);         // Keys in range [10, 30)

// TreeSet - sorted unique elements

```

```

TreeSet<Integer> set = new TreeSet<>();
set.add(30); set.add(10); set.add(20);
set.first();           // 10
set.floor(25);         // 20 (largest <= 25)
set.ceiling(25);        // 30 (smallest >= 25)

```

When to use: Sliding window median, interval problems requiring sorted order, problems needing “nearest value” queries (`floor/ceiling`), calendar scheduling conflicts.

8.2.9 Quick Reference: Choosing the Right Data Structure

Problem Signal	Data Structure	Key Advantage
“Find if X exists” / “Count occurrences”	HashMap / HashSet	$O(1)$ lookup
“Sliding window max/min”	ArrayDeque	$O(1)$ add/remove both ends
“BFS” / “Level-order” / “Shortest path”	Queue (ArrayDeque)	FIFO ordering
“Matching brackets” / “Next greater element”	Stack (ArrayDeque)	LIFO ordering
“K-th largest” / “Top K” / “Merge K lists”	PriorityQueue	$O(\log N)$ min/max access
“Sorted order” / “Floor/ceiling queries”	TreeMap / TreeSet	$O(\log N)$ sorted operations
“Cycle detection” / “Find middle node”	LinkedList	Pointer manipulation
“Random access by index”	ArrayList	$O(1)$ index access

8.3 Broad LeetCode Structural Patterns

To crack senior-level assessments, you must recognize the structural pattern of the problem instantly. We organize the ten essential archetypes into two groups: patterns for linear data (arrays, strings, sequences) and patterns for relational data (graphs, trees, complex structures).

8.3.1 Group A: Arrays, Strings & Sequences

Pattern Flowchart A: Arrays, Strings & Sequences

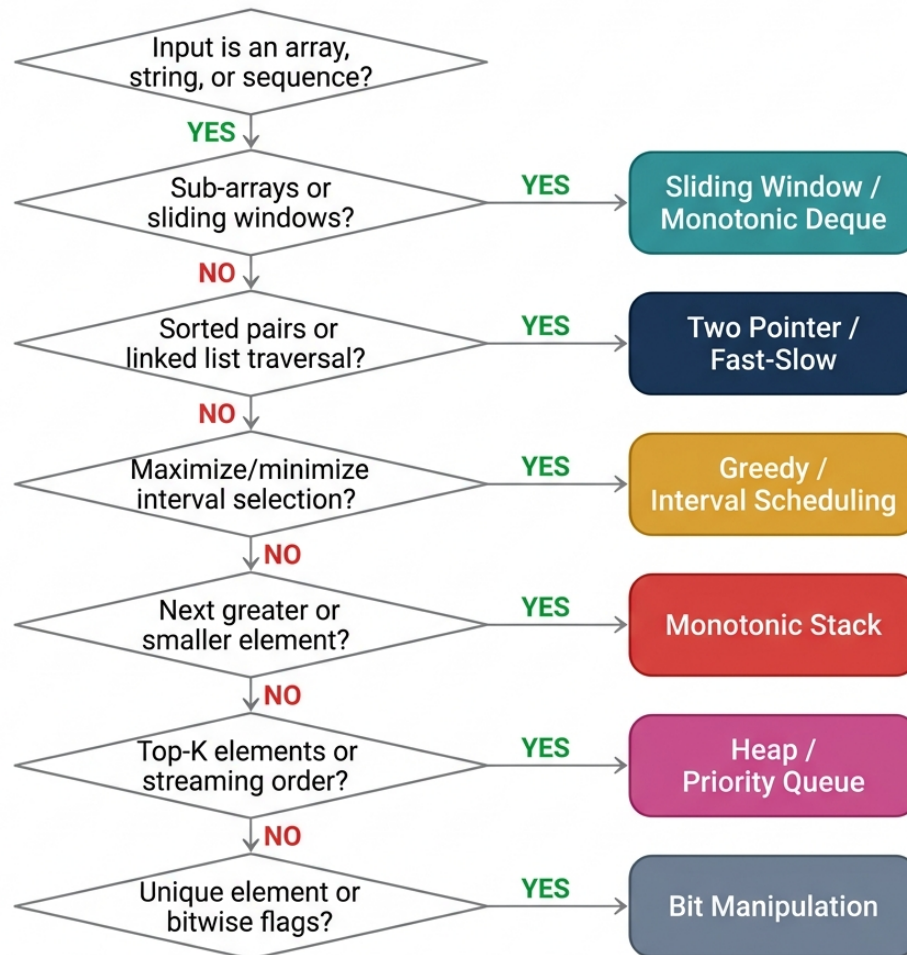


Figure 8.2: Pattern Flowchart A — Arrays, Strings & Sequences

8.3.2 Monotonic Deque & Sliding Window

When you are asked to track properties (like the maximum, minimum, or sum) of sub-arrays that slide across a larger array, you are dealing with a **Sliding Window** problem.

If the window size is K and the array size is N , a brute-force search at each step takes $O(N \times K)$ time. We can optimize this to $O(N)$ **linear time** by maintaining a **Monotonic Deque** (double-ended queue) containing array indices.

The deque maintains a strict invariant: elements corresponding to indices in the deque are stored in strictly decreasing order.

Here is the implementation:


```

package com.aurapay.algorithms;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * Implements the Sliding Window Maximum algorithm using a Monotonic Deque.
 * Time Complexity:  $O(N)$  where  $N$  is the size of the array.
 * Space Complexity:  $O(K)$  where  $K$  is the size of the sliding window.
 */
public class SlidingWindowSolver {

    /**
     * Finds the maximum value in each sliding window of size  $K$  as it moves
     * from left to right across the array.
     */
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k <= 0) {
            return new int[0];
        }

        int n = nums.length;
        int[] result = new int[n - k + 1];
        int ri = 0; // Result array index

        // Deque stores array indices.
        // Invariant: The elements corresponding to indices in the deque are
        // ↪ stored
        // in strictly decreasing order. Thus, the index of the maximum element
        // in the current window is always at the head of the deque.
        Deque<Integer> q = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            // 1. Remove indices that are out of the current window boundary
            if (!q.isEmpty() && q.peek() < i - k + 1) {
                q.poll();
            }

            // 2. Maintain monotonic invariant: Remove indices of elements
            // ↪ smaller
            // than the current element from the tail of the deque
            while (!q.isEmpty() && nums[q.peekLast()] < nums[i]) {
                q.pollLast();
            }

            // 3. Add current element's index to the tail
            q.offer(i);

            result[ri++] = nums[q.peek()];
        }

        return result;
    }
}

```



```

// 4. If window size has reached K, store the maximum in the result
if (i >= k - 1) {
    result[ri++] = nums[q.peek()];
}
}

return result;
}
}

```

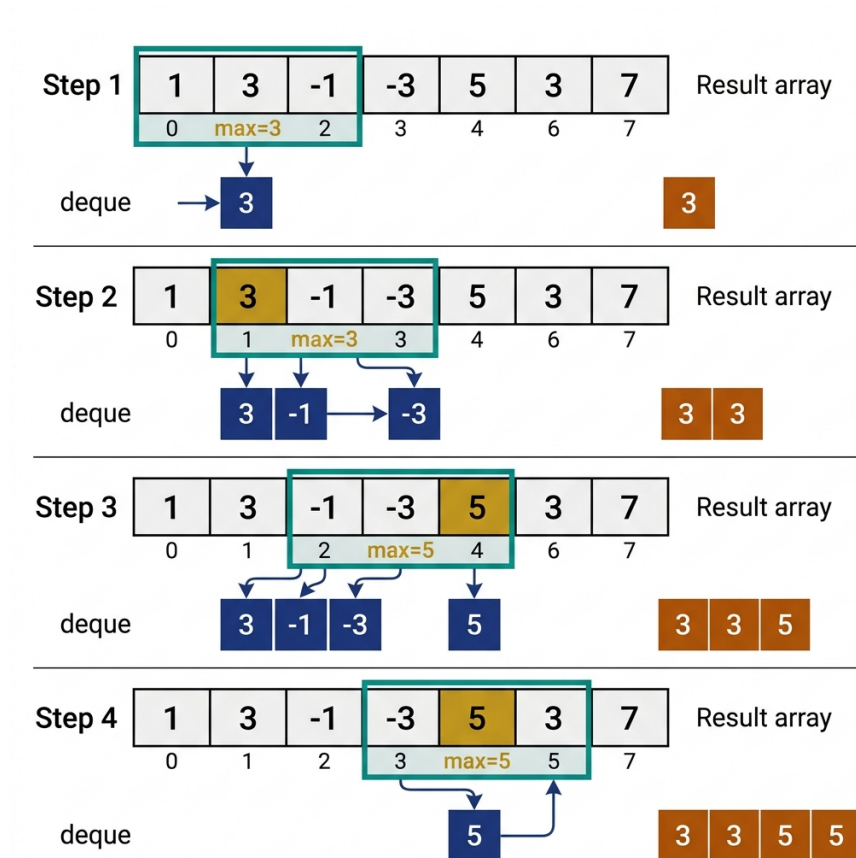


Figure 8.3: Sliding Window Algorithm — Conceptual Overview

The Core Insight: If a new element B enters the window and $B > A$ (where A is already in the deque), then A can *never* be the maximum for the current window or any future window — because B is both **larger** and **newer** (it will stay in the window longer). So A is useless, and we discard it by popping from the back of the deque. This keeps the deque values in **strictly decreasing order**, with the current maximum always at the front.

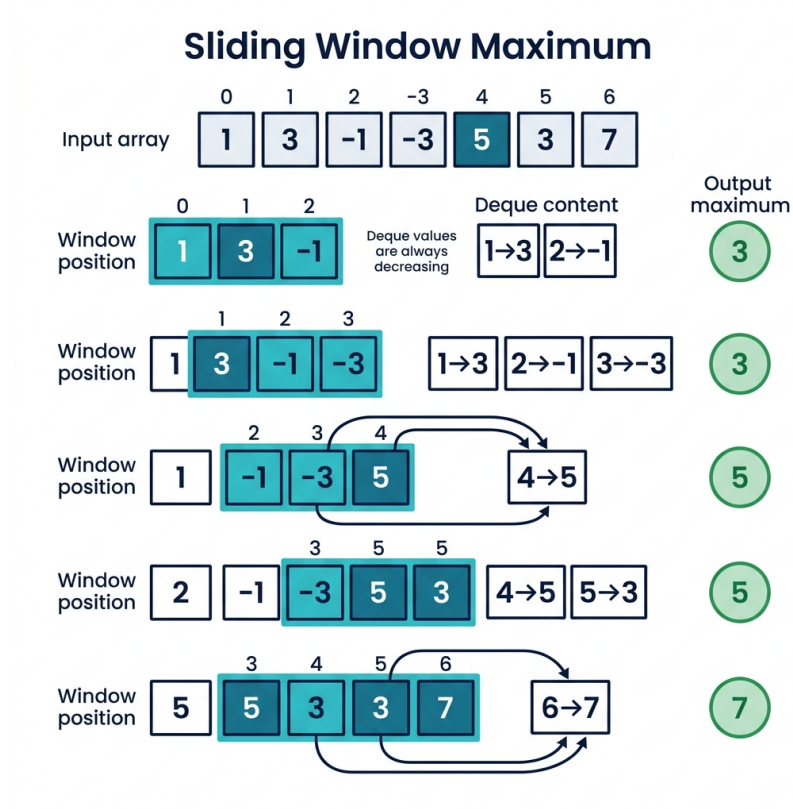


Figure 8.4: Sliding Window Maximum — Step-by-Step Execution Trace

Full Execution Trace — Array: [1, 3, -1, -3, 5, 3, 7], k=3:

Step	i	Value	Why We Do What We Do	Deque (idx->val)	Window	Max
0	0	1	Deque empty -> push index 0. Window not full yet.	[0->1]	—	—
1	1	3	3 > 1 (back). Pop 0 — 1 can never be max while 3 exists. Push 1.	[1->3]	—	—
2	2	-1	-1 < 3 (back). Keep 3 — -1 might be max after 3 leaves window. Push 2.	[1->3, 2->-1]	[1,3,-1]	3
3	3	-3	Window full! Front idx 1 still in window [1,3]. -3 < -1 (back). Push 3.	[1->3, 2->-1, 3->-3]	[3,-1,-3]	3
4	4	5	Front idx 1 out of window [2,4] -> pop front! Then 5 > -3 pop, 5 > -1 pop — both useless now. Push 4.	[4->5]	[-1,-3,5]	5

Step	i	Value	Why We Do What We Do	Deque (idx->val)	Window	Max
5	5	3	Front idx 4 in window [3,5]. $3 < 5$ (back). Push 5.	[4->5, 5->3]	[-3,5,3]	5
6	6	7	$7 > 3$ pop, $7 > 5$ pop — <i>both useless</i> . Push 6.	[6->7]	[5,3,7]	7

Result: [3, 3, 5, 5, 7]

The three rules the code follows at each step i:

1. **Evict expired:** If the front index is outside the window ($< i - k + 1$), pop it from the front.
2. **Maintain decreasing order:** While the back element $\leq$ current element, pop from the back (those elements will never be useful).
3. **Record answer:** After the window is full ($i \geq k - 1$), the front of the deque is always the index of the current maximum.

Why is this $O(N)$ despite the while loop? The while loop looks dangerous — a loop inside a loop usually means $O(N^2)$. But count the total operations *across the entire algorithm*: each of the N elements is **pushed exactly once** and **popped at most once**. That means the while loop executes at most N pops *total* across all iterations of the for loop — not N pops *per* iteration. In our 7-element example, the total pops were: Step 1 (1 pop) + Step 4 (3 pops) + Step 6 (2 pops) = **6 pops total** for 7 elements. The amortized cost per element is $O(1)$, giving $O(N)$ total.

8.3.3 The Two-Pointer & Fast/Slow Pointer Pattern

This pattern is used to process linear data structures (arrays, linked lists) using two pointer variables that move at different speeds or in different directions.

Opposite Direction. Left and right pointers moving toward the center (e.g., finding pairs in a sorted array, reversing arrays). Reduces search spaces from $O(N^2)$ to $O(N)$.

Fast/Slow Pointers. A “slow” pointer moving 1 step at a time, while a “fast” pointer moves 2 steps (e.g., Floyd’s Cycle Detection, finding the middle of a linked list).

8.3.3.1 Problem: Two Sum (Sorted Array)

Given a **sorted** array of integers **numbers** (1-indexed) and a target integer **target**, find two numbers that add up to **target**. Return their indices as [**index1**, **index2**] where **index1** < **index2**. You must use only $O(1)$ extra space.

Here is the implementation:

```
package com.aurapay.algorithms;
```

```
/**
```

```
 * Implements the Two-Pointer pattern to find two numbers that sum to a target
```

```

* in a 1-indexed sorted array.
* Time Complexity:  $O(N)$  where N is the size of the array.
* Space Complexity:  $O(1)$  auxiliary space.
*/
public class TwoPointerSolver {

    /**
     * Finds indices of the two numbers that add up to the target.
     * Uses two pointers moving from opposite ends inward.
     */
    public int[] findMatchingNumbers(int[] numbers, int target) {
        int start = 1;
        int last = numbers.length;
        int[] output = new int[2];

        while (start < last) {
            int sum = numbers[start - 1] + numbers[last - 1];
            if (sum == target) {
                output[0] = start;
                output[1] = last;
                return output;
            }
            if (sum < target) {
                start++;
            } else {
                last--;
            }
        }
        return output; // Returns [0, 0] if no match is found
    }
}

```

Opposite-Direction Two-Pointer Pattern

Two Sum problem with a sorted array

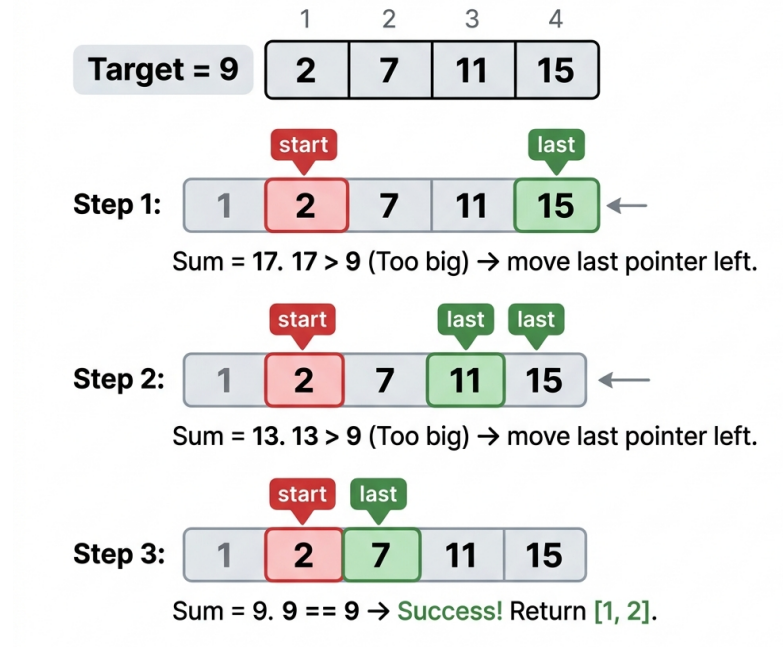


Figure 8.5: Two-Pointer Pattern — Two Sum Sorted Execution Trace

Full Execution Trace — Array: [2, 7, 11, 15], target = 9:

Step	start	last	current_sum	Decision Logic	Pointers	Output
1	1 (val 2)	4 (val 15)	17	17 > 9 (too large) → decrement last	start=1, last=4	—
2	1 (val 2)	3 (val 11)	13	13 > 9 (too large) → decrement last	start=1, last=3	—
3	1 (val 2)	2 (val 7)	9	9 == 9 → Target matched!	start=1, last=2	[1, 2]

Elimination Logic: Because the array is sorted, each pointer movement eliminates an entire set of invalid pairs. For instance, in Step 1, since the sum of $2 + 15 = 17$ is too large, the sum of 15 with *any* other element in the array is guaranteed to be larger than target. Thus, we can safely eliminate the index of 15 entirely by decrementing `last`.

8.3.3.2 Problem: Container With Most Water

Given n non-negative integers representing an elevation map where the width of each bar is 1, find two lines that together with the x-axis form a container, such that the container contains the most water.

Two Pointer Pattern: Container With Most Water

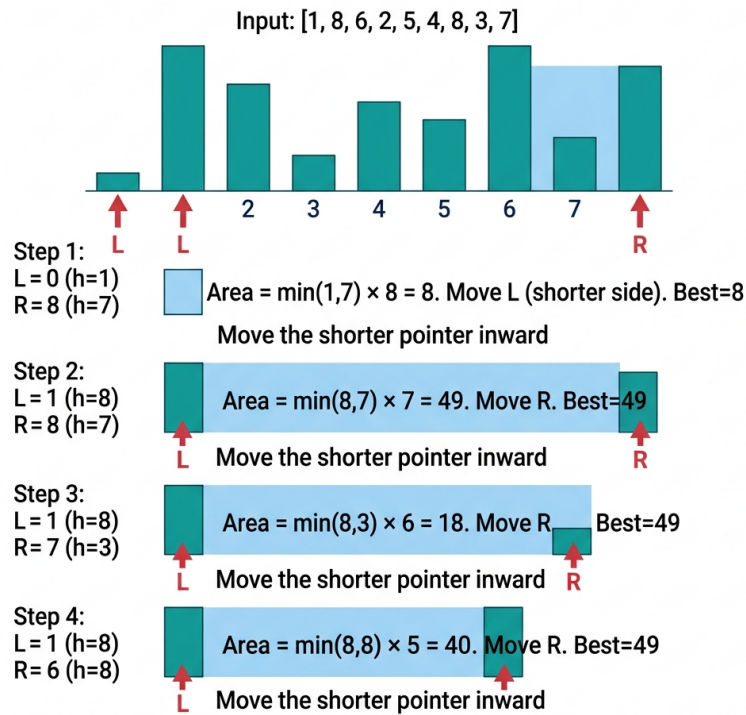


Figure 8.6: Two Pointer Pattern — Container With Most Water Execution Trace

How to read this trace: Two pointers start at opposite ends. At each step, we calculate the area between them. We then move the pointer pointing to the shorter bar inward — because moving the taller bar can never improve the area (the width shrinks and the height is still limited by the shorter bar). This greedy elimination guarantees we never miss the optimal pair.

8.3.3.3 Problem: Linked List Cycle Detection (Fast/Slow Pointers)

Given the head of a singly linked list, determine if the linked list has a cycle in it. A cycle exists if some node can be reached again by continuously following the `next` pointer.

Here is the implementation:

```
package com.aurapay.algorithms;

/**
 * Implements the Fast/Slow Pointer (Tortoise and Hare) pattern to detect cycles
 * in a singly linked list.
 * Time Complexity:  $O(N)$  where  $N$  is the number of nodes.
 * Space Complexity:  $O(1)$  auxiliary space.
 */
public class CycleDetector {
```

```

public static class ListNode {
    int val;
    ListNode next;
    ListNode(int x) {
        val = x;
        next = null;
    }
}

/**
 * Detects if a linked list contains a cycle.
 * Moves slow pointer by 1 step, fast pointer by 2 steps.
 * If they meet, a cycle exists.
 */
public boolean hasCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;           // Tortoise: 1 step
        fast = fast.next.next;      // Hare: 2 steps

        if (slow == fast) {
            return true; // Fast pointer caught up to slow pointer -> cycle!
        }
    }

    return false; // Fast pointer reached the end -> no cycle
}
}

```

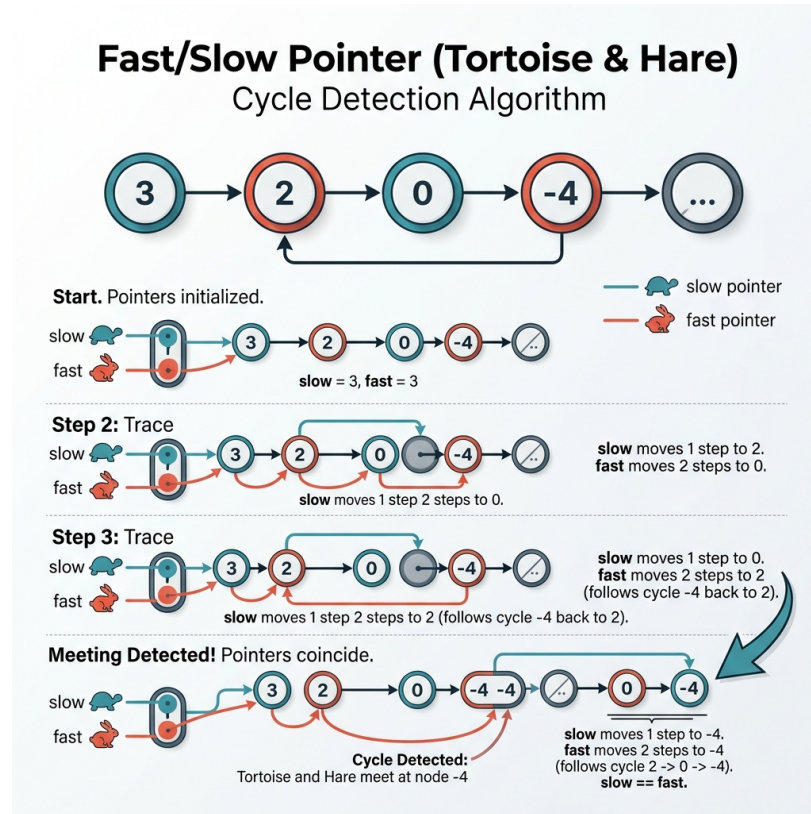



Figure 8.7: Fast/Slow Pointer Pattern — Cycle Detection Execution Trace

Full Execution Trace — List: 3 -> 2 -> 0 -> -4 (cycle from -4 back to 2):

Step	slow (Tortoise)	fast (Hare)	slow == fast	State / Decision Logic
1	3	3	True	Initial state. Skip check on Step 1 to avoid instant termination.
2	2	0	False	slow moves 1 step → 2. fast moves 2 steps → 0.
3	0	2	False	slow moves 1 step → 0. fast moves 2 steps → 2 (loops back).
4	-4	-4	True	slow moves 1 step → -4. fast moves 2 steps → -4. Meeting detected!

Cycle Detection Logic: If there is no cycle, **fast** will eventually reach **null** and terminate the algorithm in $O(N)$ time. If a cycle exists, **fast** enters the cycle first. Since **fast** reduces the distance between itself and **slow** by 1 node at each step, they are guaranteed to meet inside the cycle. This ensures $O(1)$ space complexity as we only track reference memory pointers without storing nodes.

Mathematical Proof of Start of Cycle: Let A be the distance from head to start of cycle, B be the distance from start of cycle to meeting point, and C be the cycle length. The distance slow traveled is $A+B$. The distance fast traveled is $2(A+B)$. Since fast traveled some integer number of loops k more than slow, $2(A+B) = A+B+kC \rightarrow A+B = kC \rightarrow A = kC - B$. This mathematically proves that if we reset one pointer to head and keep the other at the meeting point, moving both at speed 1 will cause them to meet exactly at the start of the cycle (A steps later).

8.3.3.4 Dual Application: Finding a Duplicate in an Array (Floyd's on Arrays)

A classic advanced coding interview problem asks you to find a duplicate number in an array under strict constraints:

1. Do **not modify** the input array (forbidding Cyclic Sort or in-place sorting).
2. Use only $O(1)$ **auxiliary space** (forbidding HashSets or frequency arrays).
3. The array contains $N + 1$ integers, where each integer is strictly in the range $[1, N]$.

Design Insight: Floyd's vs. Cyclic Sort

While Cyclic Sort is the most intuitive $O(N)$ way to find duplicates, it requires swapping elements in-place. If an interviewer adds a constraint banning array modification, you must shift your perspective and treat the array values as memory pointers to indices (i.e., an implicit linked list where index i points to index `nums[i]`).

This technique uses **Floyd's Cycle-Finding Algorithm** (named after Turing Award winner Robert W. Floyd who published it in 1967). It is also known as the **Tortoise and Hare** algorithm because it uses two pointers moving at different speeds to detect the cycle's entrance (which represents the duplicate number) without modifying a single element.

Here is the implementation:

```
package com.aurapay.algorithms;

/**
 * Finds the duplicate number in an array using Floyd's Cycle Detection.
 * Time Complexity:  $O(N)$  where  $N$  is the size of the array.
 * Space Complexity:  $O(1)$  auxiliary space.
 * Constraint: The array must contain  $N + 1$  elements, each between 1 and  $N$ .
 */
public class DuplicateArrayFinder {

    public int findDuplicate(int[] nums) {
        // Phase 1: Detect cycle (meeting point)
        int slow = nums[0];
        int fast = nums[0];

        do {
            slow = nums[slow];           // Move 1 step
            fast = nums[nums[fast]];     // Move 2 steps
        } while (slow != fast);
    }
}
```

```

        // Phase 2: Find cycle entrance (duplicate value)
        slow = nums[0]; // Reset slow to start
        while (slow != fast) {
            slow = nums[slow]; // Move 1 step
            fast = nums[fast]; // Move 1 step
        }

        return slow; // The duplicate value
    }
}

```

8.3.3.5 Problem: In-place Reversal of a Linked List

Reversing the pointers of a singly linked list in-place without allocating new nodes is a fundamental coding pattern. It forms the basis of many harder list manipulation questions (like reversing sub-lists or checking if a list is a palindrome).

Here is the implementation:

```

package com.aurapay.algorithms;

/**
 * Reverses a singly linked list in-place.
 * Time Complexity:  $O(N)$  where  $N$  is the number of nodes.
 * Space Complexity:  $O(1)$  auxiliary space.
 */
public class LinkedListReversal {

    public static class ListNode {
        int val;
        ListNode next;
        ListNode(int x) {
            val = x;
            next = null;
        }
    }

    /**
     * Reverses the linked list and returns the new head node.
     */
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;

        while (curr != null) {
            ListNode nextTemp = curr.next; // 1. Save the next node
            curr.next = prev;              // 2. Reverse the current node's
            ↪ pointer                        // 3. Move prev one step forward
            prev = curr;
        }
    }
}

```

```

        curr = nextTemp;                                // 4. Move curr one step forward
    }

    return prev; // New head node
}
}

```

Reversal Invariant: At each step, we maintain three pointers: `prev` (already reversed part), `curr` (node currently being reversed), and `nextTemp` (temporarily stores the rest of the list so we don't lose it when we break the link). We simply point `curr.next` to `prev`, then slide both `prev` and `curr` one node forward.

8.3.3.6 Problem: Cyclic Sort (In-place Array Sorting)

When you are given an array of numbers in a contiguous range from 1 to N (or 0 to N), you can sort the array in $O(N)$ time and $O(1)$ space using **Cyclic Sort**. It is the go-to pattern for finding missing or duplicate numbers.

Here is the implementation:

```

package com.aurapay.algorithms;

/**
 * Sorts an array containing numbers from 1 to N in-place.
 * Time Complexity:  $O(N)$  where N is the size of the array.
 * Space Complexity:  $O(1)$  auxiliary space.
 */
public class CyclicSort {

    /**
     * Sorts the array using the Cyclic Sort pattern.
     */
    public void sort(int[] nums) {
        int i = 0;
        while (i < nums.length) {
            int correctIndex = nums[i] - 1; // In a 1-to-N range, value X belongs
            // at index X-1
            if (nums[i] != nums[correctIndex]) {
                swap(nums, i, correctIndex); // Swap to its correct position
            } else {
                i++; // Only increment when the current element is in its correct
                // place
            }
        }
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}

```

```
    }
}
```

Sorting Invariant: Since elements are in the range $[1, N]$, the number X belongs exactly at index $X - 1$. We iterate through the array. If the current element is not at its correct index, we swap it with the element at its correct index. We only increment our loop pointer i when the element at index i is already correct. Since each swap places at least one element in its final correct position, the algorithm finishes in at most $2N$ steps — giving $O(N)$ linear time.

8.3.4 Interval Scheduling & Greedy Algorithms

Greedy algorithms make the locally optimal choice at each step with the hope of finding a global optimum. A classic implementation is **Interval Scheduling** (e.g., matching non-overlapping transaction batches or scheduling CPU tasks).

To maximize the number of non-overlapping intervals, you must apply the **earliest deadline first** heuristic: sort the intervals by their end times, and greedily select the next interval that starts after the end of the previously selected one.

Here is the implementation:

```
package com.aurapay.algorithms;

import java.util.Arrays;
import java.util.Comparator;

/**
 * Solves the Interval Scheduling problem using a Greedy approach.
 * Minimizes overlaps, finding the maximum number of non-overlapping transactions
 * ↪
 * that can be processed.
 * Time Complexity:  $O(N \log N)$  due to sorting.
 * Space Complexity:  $O(1)$  or  $O(N)$  depending on sort implementation.
 */
public class IntervalScheduler {

    public static class TransactionInterval {
        int start;
        int end;

        public TransactionInterval(int start, int end) {
            this.start = start;
            this.end = end;
        }
    }

    /**
     * Returns the maximum number of non-overlapping transaction intervals.
     */
    public int maxNonOverlappingTransactions(TransactionInterval[] intervals) {
```

```

    if (intervals == null || intervals.length == 0) {
        return 0;
    }

    // GREEDY INVARIANT: Sort intervals by their end time.
    // Selecting the interval that ends earliest leaves the maximum space
    // for subsequent transactions.
    Arrays.sort(intervals, Comparator.comparingInt(a -> a.end));

    int count = 1; // Always select the first interval
    int lastSelectedEnd = intervals[0].end;

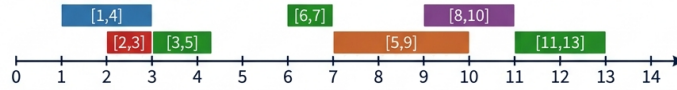
    for (int i = 1; i < intervals.length; i++) {
        // If the start time is greater than or equal to the end time of the
        // last selected interval, select this transaction
        if (intervals[i].start >= lastSelectedEnd) {
            count++;
            lastSelectedEnd = intervals[i].end;
        }
    }

    return count;
}
}

```

Greedy Pattern: Interval Scheduling (Earliest Deadline First)

Input intervals (unsorted)



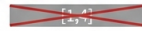
Step 1. Sorted by end time



Step 2. Select $[2,3]$ ✓ since its first interval, set lastEnd = 3



Step 3. Skip $[1,4]$ ✗ since start (1) is less than lastEnd (3), overlaps.



Step 4. Select $[3,5]$ ✓ since start (3) $\geq$ lastEnd (3), lastEnd = 5



Step 5. Select $[6,7]$ ✓ since start (6) $\geq$ lastEnd (5), set lastEnd = 7



Step 6. Skip $[5,9]$ ✗ since start (5) less than lastEnd (7), overlap



Step 7. Select $[8,10]$ ✓ as start (8) $\geq$ lastEnd (7), all set lastEnd = 10



Step 8. Select $[11,13]$ ✓ with start (11) $\geq$ lastEnd (10), set lastEnd = 13



Figure 8.8: Greedy Interval Scheduling — Step-by-Step Execution Trace

How to read this trace: After sorting intervals by end time, we greedily pick the next interval whose start time does not overlap with the last selected interval's end time. Green intervals are selected; red intervals are skipped because they overlap.

8.3.5 Group B: Graphs, Trees & Complex Structures

Pattern Flowchart B: Graphs, Trees & Complex Structures

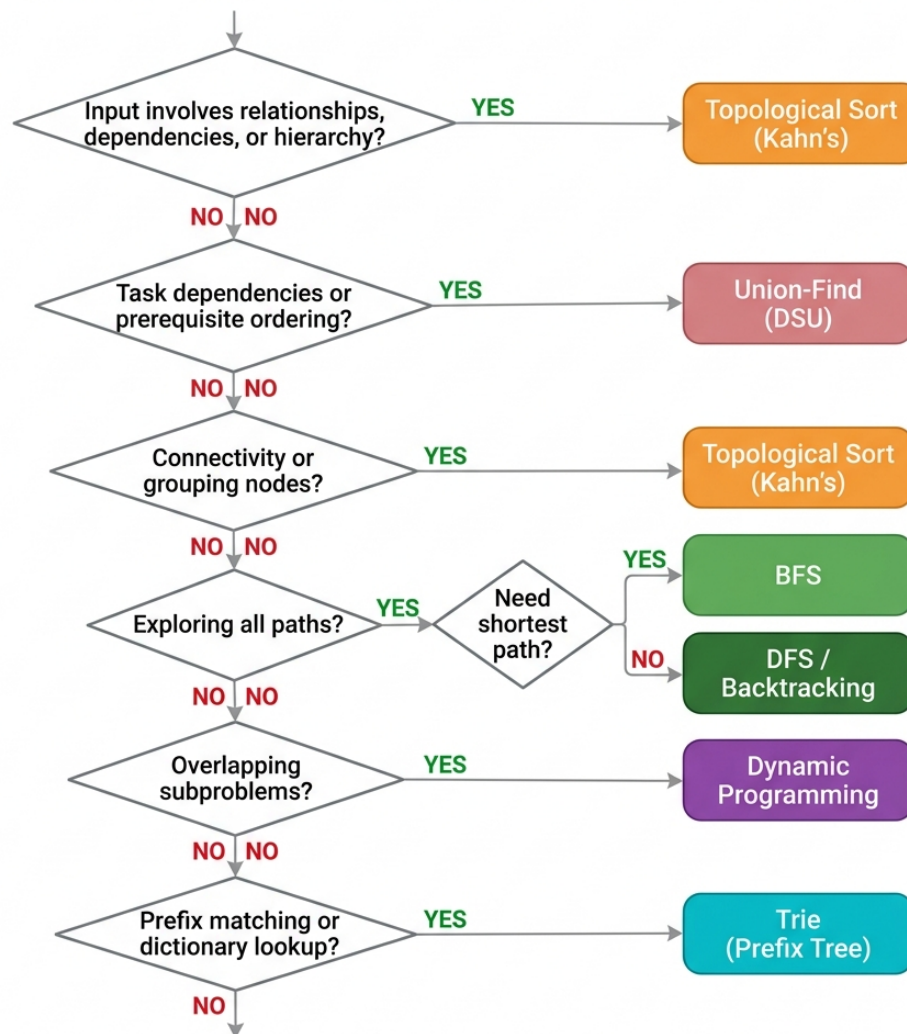


Figure 8.9: Pattern Flowchart B — Graphs, Trees & Complex Structures

8.3.6 Backtracking, DFS, and BFS

These patterns are used to traverse trees, graphs, or search spaces.

Breadth-First Search (BFS). Uses a queue to explore nodes level-by-level. Always use BFS when you need to find the **shortest path** or minimum steps in an unweighted graph.

Depth-First Search (DFS). Uses recursion (call stack) to explore branches as deeply as possible before backtracking.

Backtracking. A refined DFS that prunes invalid search branches early. For example, when generating all valid payment routing paths, if a path violates a limit constraint (pre-condition failure), backtrack immediately rather than continuing down that branch.

BFS vs DFS: Graph Traversal Comparison

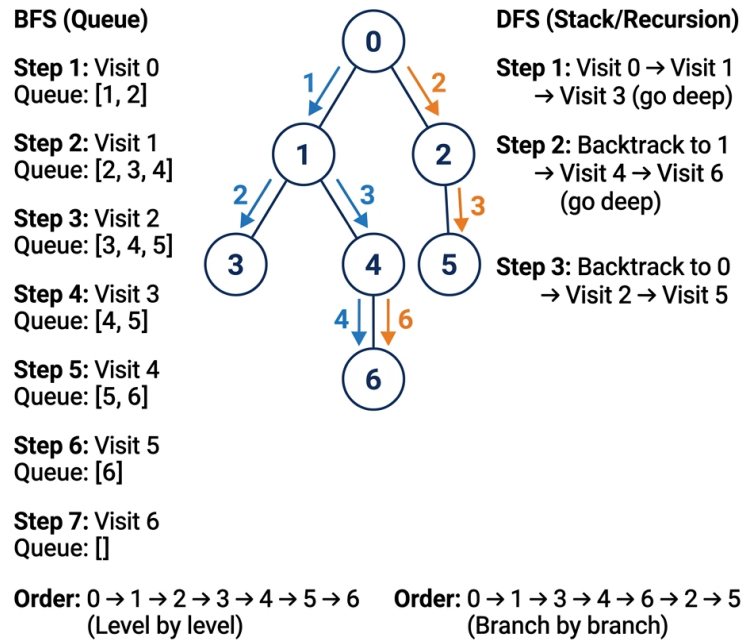


Figure 8.10: BFS vs DFS — Graph Traversal Comparison

How to read this trace: BFS (left) uses a queue — it visits all nodes at distance 1 before distance 2, guaranteeing the shortest path. DFS (right) uses recursion — it dives as deep as possible along one branch before backtracking. Choose BFS for shortest-path problems, DFS for exhaustive search or cycle detection.

Island Pattern (Matrix Traversal). A very common assessment pattern involves traversing a 2D grid (matrix) where cells are connected horizontally or vertically (e.g., counting “islands” of connected 1s in a sea of 0s). We treat the grid as an implicit graph where each cell (r, c) has up to four neighbors: $(r \pm 1, c)$ and $(r, c \pm 1)$. When visiting a cell, we mark it as visited (or sink it by changing 1 to 0) and recursively trigger DFS or BFS to traverse all connected cells. ### Dynamic Programming (DP)

Dynamic Programming is used to solve optimization problems by breaking them down into overlapping subproblems, solving each subproblem once, and storing the results (memoization).

8.3.6.1 The Case Study: Climbing Stairs

To master DP, let’s look at the classic problem: **Climbing Stairs**. > *You are climbing a staircase. It takes N steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?*

8.3.6.2 Naive DFS (Decision Trees)

Every DP problem begins with a decision. If you are standing at step N , you could have arrived there in one of two ways:

1. By climbing **1 step** from step $N - 1$.
2. By climbing **2 steps** from step $N - 2$.

Therefore, the total ways to reach step N is:

$$\text{climb}(N) = \text{climb}(N - 1) + \text{climb}(N - 2)$$

This is a simple recursive relationship. Here is the implementation using naive DFS:

```
public int climbStairs(int n) {  
    if (n <= 1) return 1; // Base case: 1 way to stay on step 0 or 1  
    return climbStairs(n - 1) + climbStairs(n - 2);  
}
```

8.3.6.2.1 Naive DFS Complexity Analysis:

- **Time Complexity:** $O(2^N)$. The recursion tree doubles in size at each level.
- **Space Complexity:** $O(N)$. The maximum depth of the call stack is N .

Why is this $O(2^N)$ time complexity a disaster? Look at the recursion tree for $N = 5$:

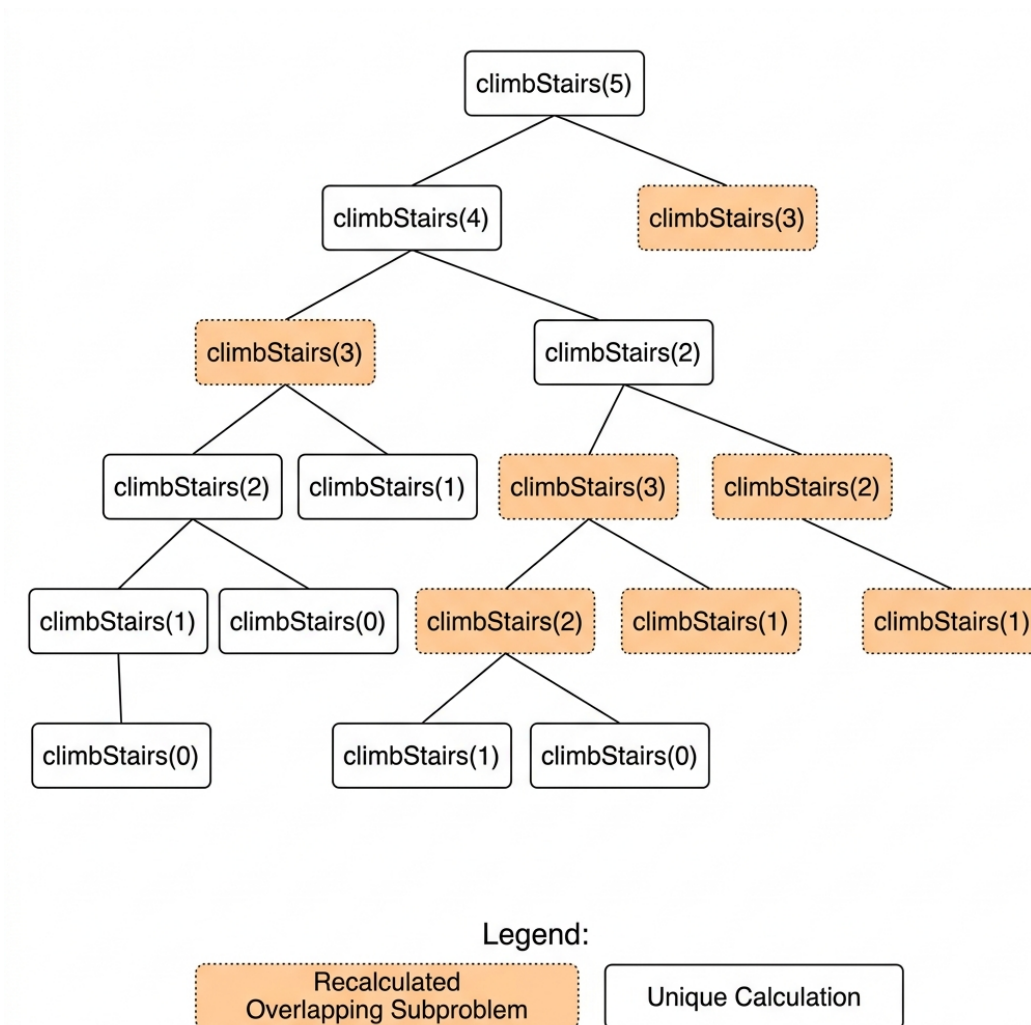


Figure 8.11: Overlapping Subproblems in Climbing Stairs Recursion Tree

Notice that `climbStairs(3)` is calculated **2 separate times**, `climbStairs(2)` is calculated **3 separate times**, and `climbStairs(1)` is calculated **5 separate times**! As N grows, this redundant calculation causes the program to halt.

8.3.6.3 Top-Down DP (Memoization)

To fix the $O(2^N)$ time complexity, we simply **remember the past**. We add a cache (a memoization array `memo`) to store the result of `climbStairs(i)` the first time we calculate it. If the recursion ever visits that step again, we return the cached value in $O(1)$ time.

```
public int climbStairs(int n) {
    int[] memo = new int[n + 1];
    return dfs(n, memo);
}
```

```
private int dfs(int n, int[] memo) {
    if (n <= 1) return 1;

    // Return cached result if already calculated
    if (memo[n] != 0) {
        return memo[n];
    }

    // Store in cache before returning
    memo[n] = dfs(n - 1, memo) + dfs(n - 2, memo);
    return memo[n];
}
```

8.3.6.3.1 Memoized Complexity Analysis:

- **Time Complexity:** $O(N)$. We calculate each step value exactly once.
 - **Space Complexity:** $O(N)$. We use $O(N)$ space for the cache array and $O(N)$ space on the recursive call stack.
-

8.3.6.4 Bottom-Up DP (Tabulation)

While Memoization is fast, it relies on **recursion**. In Java, every recursive call creates a new stack frame on the call stack. If N is very large (e.g. 10,000), recursive DFS will crash with a `StackOverflowError` because the call stack size is limited.

To prevent this, we use **Tabulation**. Instead of working top-down from N , we start at the bottom (base cases 0 and 1) and fill a table iteratively using a simple `for` loop:

```
public int climbStairs(int n) {
    if (n <= 1) return 1;

    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```

8.3.6.4.1 Tabular Complexity Analysis:

- **Time Complexity:** $O(N)$. A single loop from 2 to N .
- **Space Complexity:** $O(N)$. The `dp` array stores $N + 1$ values in heap memory, and there is no recursive call stack.

8.3.6.5 Space-Optimized DP

Look closely at the tabulation loop: $dp[i] = dp[i - 1] + dp[i - 2]$

To compute $dp[i]$, we only need the values of the **last two elements** ($dp[i-1]$ and $dp[i-2]$). We do not need the rest of the historical values in the array! We can discard the array and simply track those two values using two variables:

```
public int climbStairs(int n) {
    if (n <= 1) return 1;

    int prev2 = 1; // Represents dp[i - 2]
    int prev1 = 1; // Represents dp[i - 1]

    for (int i = 2; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1; // Move prev2 forward
        prev1 = curr;  // Move prev1 forward
    }

    return prev1;
}
```

8.3.6.5.1 Space-Optimized Complexity Analysis:

- **Time Complexity:** $O(N)$.
 - **Space Complexity:** $O(1)$. We only use two variables regardless of how large N is.
-

8.3.6.6 Understanding the General DP State Formula

When preparing for system design or advanced coding loops, you will often see generalized DP formulas that look highly abstract. Let's demystify the classic **0/1 Knapsack** formula so you can correlate its variables directly:

$$DP[i][w] = \max(DP[i - 1][w], DP[i - 1][w - weight_i] + value_i)$$

Where do i and w come from? Imagine you are packing a bag (knapsack) that has a maximum weight capacity W . You have a list of items, each with a specific **weight** and **value**. You want to find the combination of items that gives the maximum value without breaking your bag.

- **i (The Item Choice):** The index of the item you are currently evaluating (e.g. item 3).
- **w (The Remaining Capacity):** The amount of weight capacity left in your bag.
- **$DP[i][w]$:** The maximum value you can achieve using the first i items when your bag has w capacity remaining.

The Decision logic: At item i , you have exactly two choices:

1. **Exclude the item (Leave it):** The bag's capacity stays at w . Your max value is simply whatever you could achieve using the previous $i - 1$ items: $DP[i - 1][w]$.
2. **Include the item (Take it):** The bag's remaining capacity drops by the item's weight ($w - weight_i$). You gain the item's value ($value_i$). Your total value is: $DP[i - 1][w - weight_i] + value_i$.

The transition equation simply takes the $\max()$ of these two choices.

Tactical Tip. In an interview, start by writing a simple recursive DFS solution. Once it works, add a cache (memoization table) to optimize it. This is a “top-down” approach, which is often easier to write under pressure than a “bottom-up” iterative DP table.

1D Dynamic Programming table for climbStairs(5)

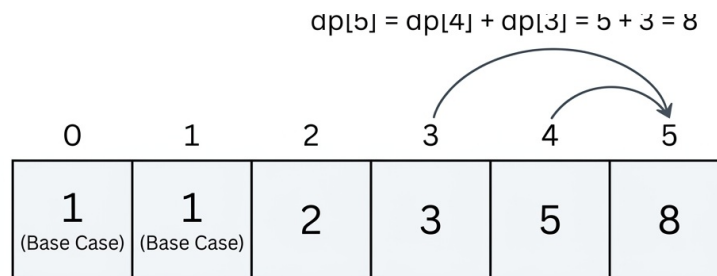


Figure 8.12: 1D Dynamic Programming Tabulation Table for Climbing Stairs

Dynamic Programming: Longest Common Subsequence (LCS)

Text1: "ABCDE"

Text2: "ACE"

	"	A	B	C	D	E
"	0	0	0	0	0	0
A	0	1	1	1	1	1
C	0	1	1	2	2	2
E	0	1	1	2	2	3

LCS = ACE, Length = 3

Figure 8.13: Dynamic Programming — LCS Step-by-Step Table Fill Trace

How to read this trace: The DP table is filled row by row. When the characters match (diagonal green arrow), we take the diagonal value + 1. When they don't match (gray), we take the maximum of the cell above or to the left. The backtrack path (highlighted) reveals the LCS itself.

8.3.7 Topological Sort (Dependency Ordering)

Topological Sort produces a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every edge (u, v) , vertex u comes before v . It is the standard answer for dependency resolution problems — course prerequisites, build system task ordering, and microservice deployment sequencing.

When to use it: The problem mentions “prerequisites,” “dependencies,” “ordering constraints,” or asks you to detect cycles in a directed graph.

The Invariant. A node is only processed after all of its incoming dependencies have been resolved. In Kahn's algorithm, a node enters the queue only when its in-degree reaches zero.

```
public List<Integer> topologicalSort(int numNodes, int[][] edges) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] inDegree = new int[numNodes];
    for (int i = 0; i < numNodes; i++) adj.add(new ArrayList<>());
```

```

for (int[] edge : edges) {
    adj.get(edge[0]).add(edge[1]);
    inDegree[edge[1]]++;
}

Queue<Integer> queue = new LinkedList<>();
for (int i = 0; i < numNodes; i++) {
    if (inDegree[i] == 0) queue.offer(i);
}

List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor : adj.get(node)) {
        inDegree[neighbor]--;
        if (inDegree[neighbor] == 0) queue.offer(neighbor);
    }
}

if (order.size() != numNodes) {
    throw new IllegalStateException("Cycle detected");
}
return order;
}

```

Interview Signal: If the result list size is less than the total node count, a cycle exists in the graph. This is how you detect circular dependencies in build systems or deadlocks in task schedulers.

8.3.8 Union-Find (Disjoint Set Union)

Union-Find tracks which elements belong to the same connected group. It supports two operations in near-constant time: `find(x)` (which group does `x` belong to?) and `union(x, y)` (merge the groups of `x` and `y`).

When to use it: The problem asks about connectivity, connected components, grouping, or redundant connections in an undirected graph. Classic problems include “Number of Provinces,” “Redundant Connection,” and network clustering.

The Invariant. Every element points to a representative (root) of its group. Path compression flattens the tree on every `find()` call, keeping operations amortized $O(\alpha(N))$ — effectively constant.

```

public class UnionFind {
    private int[] parent;
    private int[] rank;
    private int componentCount;

    public UnionFind(int n) {

```

```

    parent = new int[n];
    rank = new int[n];
    componentCount = n;
    for (int i = 0; i < n; i++) parent[i] = i;
}

public int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]); // Path compression
    }
    return parent[x];
}

public boolean union(int x, int y) {
    int rootX = find(x), rootY = find(y);
    if (rootX == rootY) return false;
    if (rank[rootX] < rank[rootY]) { int tmp = rootX; rootX = rootY; rootY =
        tmp; }
    parent[rootY] = rootX;
    if (rank[rootX] == rank[rootY]) rank[rootX]++;
    componentCount--;
    return true;
}

public int getComponentCount() { return componentCount; }
}

```

8.3.9 Tries (Prefix Trees)

A Trie is a tree-shaped data structure where each node represents a character, and paths from root to leaf form complete words. It enables $O(L)$ prefix lookup (where L is the word length), regardless of how many words are stored.

When to use it: The problem involves autocomplete, spell checking, prefix matching, word search in a grid, or dictionary operations. Classic problems include “Implement Trie,” “Word Search II,” and “Design Search Autocomplete System.”

The Invariant. Every path from the root to a node marked `isEnd = true` represents a valid word in the dictionary.

```

public class Trie {
    private TrieNode root = new TrieNode();

    private static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    public void insert(String word) {
        TrieNode node = root;

```



```

        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = findNode(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return findNode(prefix) != null;
    }

    private TrieNode findNode(String s) {
        TrieNode node = root;
        for (char c : s.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return null;
            node = node.children[idx];
        }
        return node;
    }
}

```

8.3.10 Heaps and Priority Queues (Top-K Problems)

A heap (min-heap or max-heap) maintains a partially sorted structure that allows $O(\log N)$ insertion and $O(1)$ access to the smallest (or largest) element. It is the standard answer for “Top-K” problems.

When to use it: The problem asks for the K-th largest element, K most frequent elements, median from a data stream, or any scenario requiring efficient access to extreme values while processing a continuous flow of data.

The Invariant. A min-heap of size K always contains the K largest elements seen so far. The heap’s root is the K-th largest.

```

public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    return minHeap.peek();
}

```

Dual-Heap Pattern for Streaming Median. Maintain a max-heap (lower half) and a min-heap (upper half). The median is either the top of the max-heap or the average of both tops. This is a frequently asked senior-level problem.

K-way Merge Pattern. Merging K sorted lists or arrays into a single sorted list is a very common priority queue pattern. It is the engine behind external sorting algorithms and distributed log mergers.

Here is the implementation:

```
package com.aurapay.algorithms;

import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;

/**
 * Merges K sorted lists into one sorted list using a Min-Heap.
 * Time Complexity:  $O(N \log K)$  where  $N$  is total elements,  $K$  is number of lists.
 * Space Complexity:  $O(K)$  auxiliary space for the heap.
 */
public class KWayMerge {

    public static class HeapNode {
        int val;
        int listIndex;
        int elementIndex;

        HeapNode(int val, int listIndex, int elementIndex) {
            this.val = val;
            this.listIndex = listIndex;
            this.elementIndex = elementIndex;
        }
    }

    /**
     * Merges K sorted lists into a single sorted list.
     */
    public List<Integer> mergeKLists(List<List<Integer>> lists) {
        PriorityQueue<HeapNode> minHeap = new PriorityQueue<>((a, b) -> a.val -
            b.val);
        List<Integer> result = new ArrayList<>();

        // 1. Initialize heap with the first element of each list
        for (int i = 0; i < lists.size(); i++) {
            if (lists.get(i) != null && !lists.get(i).isEmpty()) {
                minHeap.offer(new HeapNode(lists.get(i).get(0), i, 0));
            }
        }
    }
}
```

```

// 2. Repeatedly extract the minimum value and push the next element from
↪ that list
while (!minHeap.isEmpty()) {
    HeapNode curr = minHeap.poll();
    result.add(curr.val);

    int nextElementIdx = curr.elementIndex + 1;
    if (nextElementIdx < lists.get(curr.listIndex).size()) {
        minHeap.offer(new HeapNode(
            lists.get(curr.listIndex).get(nextElementIdx),
            curr.listIndex,
            nextElementIdx
        ));
    }
}

return result;
}
}

```

Merge Invariant: A Min-Heap of size K holds the current smallest unprocessed element from each of the K sorted lists. We pop the smallest element from the heap, append it to our result, and then insert the *next* element from that same list into the heap. This maintains a running frontier of sorted candidates, completing in $O(N \log K)$ time.

8.3.11 Bit Manipulation

Bit manipulation uses bitwise operators (&, |, ^, ~, <<, >>) to solve problems in $O(1)$ space and often $O(N)$ time. While less common, it appears in high-stakes assessments to test low-level thinking.

When to use it: The problem involves finding a single unique number among duplicates, power-of-two checks, counting set bits, or toggling flags without extra memory.

Key Bit Tricks:

XOR for finding the unique element. XOR of a number with itself is zero. XOR of a number with zero is itself. So XORing all elements cancels out duplicates.

```

public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num; // Duplicates cancel: a ^ a = 0, 0 ^ b = b
    }
    return result;
}

```

Power of two check. A number is a power of two if and only if it has exactly one bit set: $n > 0$ && $(n \& (n - 1)) == 0$.

8.4 The Eight Worked Archetypal Problems

To reinforce these structural patterns, we will walk through eight canonical LeetCode problems. For each problem, we define the invariants, analyze the design boundaries, and provide optimal multi-language implementations.

8.4.1 Problem 1: Sliding Window Maximum (Hard)

Given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position. Return the max sliding window.

8.4.1.1 Design Invariants

1. **Window Boundaries:** The indices in the deque must always reside within the range $[i - k + 1, i]$.
2. **Decreasing Order:** The deque must store indices such that their corresponding values are in strictly descending order. Thus, `deque.peekFirst()` always returns the index of the maximum element in the current window.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || nums.length == 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
            deque.pollFirst();
        }
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
            deque.pollLast();
        }
        deque.offerLast(i);
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }
    return result;
}
```

8.4.2 Problem 2: Container With Most Water (Medium)

Given `n` non-negative integers $a_1, a_2, \dots, a_n$, where each represents a point at coordinate (i, a_i) . `n` vertical lines are drawn such that the two endpoints of the line i is at (i, a_i) and $(i, 0)$. Find two lines, which, together with the x-axis forms a container, such that the container contains the most water.

8.4.2.1 Design Invariants

1. **Search Space:** The maximum area must lie within the current boundaries $[left, right]$.

2. **Greedy Elimination:** The pointer pointing to the shorter vertical line can be safely moved inward because maintaining it can never yield a larger area (as width decreases and height is limited by the shorter line).

```
public int maxArea(int[] height) {
    int maxVal = 0;
    int left = 0;
    int right = height.length - 1;
    while (left < right) {
        int width = right - left;
        int currentHeight = Math.min(height[left], height[right]);
        maxVal = Math.max(maxVal, width * currentHeight);
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }
    return maxVal;
}
```

8.4.3 Problem 3: Merge Intervals (Medium)

Given an array of intervals where $intervals[i] = [start_i, end_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

8.4.3.1 Design Invariants

1. **Sorting Invariant:** Sorting intervals by their start times ensures that overlapping intervals are contiguous in the sorted list.
2. **Overlap Condition:** An overlap occurs if and only if $interval[start] \leq current_merged[end]$.

```
public int[][] merge(int[][] intervals) {
    if (intervals.length <= 1) return intervals;
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> merged = new ArrayList<>();
    int[] current = intervals[0];
    merged.add(current);
    for (int[] interval : intervals) {
        if (interval[0] <= current[1]) {
            current[1] = Math.max(current[1], interval[1]);
        } else {
            current = interval;
            merged.add(current);
        }
    }
    return merged.toArray(new int[merged.size()][]);
}
```

8.4.4 Problem 4: Word Search (Medium)

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` if `word` exists in the grid. The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

8.4.4.1 Design Invariants

1. **Grid Boundaries:** Recursion must immediately terminate if row r or column c is outside board boundaries.
2. **Path Uniqueness:** Cells in the current search path must be marked (e.g., replaced with '#') to prevent reuse, and restored (backtracked) once the path exploration finishes.

```
public boolean exist(char[][] board, String word) {
    int m = board.length;
    int n = board[0].length;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (dfs(board, word, i, j, 0)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] board, String word, int r, int c, int index) {
    if (index == word.length()) return true;
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length ||
        board[r][c] != word.charAt(index)) {
        return false;
    }
    char temp = board[r][c];
    board[r][c] = '#'; // Mark visited
    boolean found = dfs(board, word, r + 1, c, index + 1)
        || dfs(board, word, r - 1, c, index + 1)
        || dfs(board, word, r, c + 1, index + 1)
        || dfs(board, word, r, c - 1, index + 1);
    board[r][c] = temp; // Backtrack
    return found;
}
```

8.4.5 Problem 5: Longest Common Subsequence (Medium)

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

8.4.5.1 Design Invariants

1. **DP State:** `dp[i][j]` represents the length of the longest common subsequence of `text1[0...i-1]` and `text2[0...j-1]`.

2. **Transition Rule:** If $text1[i-1] == text2[j-1]$, then $dp[i][j] = dp[i-1][j-1] + 1$. Otherwise, $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length();
    int n = text2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

8.4.6 Problem 6: Number of Islands (Medium)

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

8.4.6.1 Design Invariants

1. **Island Boundary:** Once an island cell '1' is discovered, all connected land cells must be sunk (set to '0') via DFS to ensure the island is only counted once.

```
public int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j);
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c]
        ↪ != '1') {
        return;
    }
    grid[r][c] = '0'; // Sink the island
}
```

```

    dfs(grid, r + 1, c);
    dfs(grid, r - 1, c);
    dfs(grid, r, c + 1);
    dfs(grid, r, c - 1);
}

```

8.4.7 Problem 7: Daily Temperatures (Medium)

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the i -th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

8.4.7.1 Design Invariants

1. **Monotonic Stack Invariant:** The stack stores indices of temperatures in strictly descending order.
2. **Trigger Condition:** If the current temperature exceeds the temperature at the index stored at the top of the stack, we resolve that day's wait time and pop the stack.

```

public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>();
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i])
            ↪ {
                int idx = stack.pop();
                result[idx] = i - idx;
            }
        stack.push(i);
    }
    return result;
}

```

8.4.8 Problem 8: Search in Rotated Sorted Array (Medium)

There is an integer array `nums` sorted in ascending order (with distinct values). Prior to being passed to your function, `nums` is possibly rotated at an unknown pivot index. Given the array `nums` after the rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

8.4.8.1 Design Invariants

1. **Sorted Half:** In any rotated sorted array split in half, at least one half of the array must be sorted.
2. **Search Boundary:** If the sorted half contains the target, narrow search to that half; otherwise, search the other half.

```

public int search(int[] nums, int target) {
    int left = 0;

```



```

int right = nums.length - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] == target) return mid;
    if (nums[left] <= nums[mid]) {
        if (target >= nums[left] && target < nums[mid]) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    } else {
        if (target > nums[mid] && target <= nums[right]) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
}
return -1;
}

```

8.5 Mock General Coding Assessments

To replicate the pressure of a real speed run, practice with these two timed mock assessments.

8.5.1 Mock GCA Test 1 (Target Time: 70 Minutes)

8.5.1.1 Q1: String Compression (Easy)

Given an array of characters `chars`, compress it using the following algorithm: Begin with an empty string `s`. For each group of consecutive repeating characters in `chars`: If the group's length is 1, append the character to `s`. Otherwise, append the character followed by the group's length. Return the new length of the array after compression. The design must update the input array in-place.

- **Spec-Driven Analysis:** Use a two-pointer technique: one `read` pointer to scan the groups and one `write` pointer to write the compressed characters in-place.
- **Invariants:** The `write` pointer must always be less than or equal to the `read` pointer.

```

public int compress(char[] chars) {
    int write = 0;
    int read = 0;
    while (read < chars.length) {
        char currentChar = chars[read];
        int count = 0;
        while (read < chars.length && chars[read] == currentChar) {
            read++;
            count++;
        }
        chars[write++] = currentChar;
        if (count > 1) {

```

```

        for (char c : Integer.toString(count).toCharArray()) {
            chars[write++] = c;
        }
    }
    return write;
}

```

8.5.1.2 Q2: Rotate Image (Medium)

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise) in-place.

- **Spec-Driven Analysis:** To rotate a matrix 90 degrees clockwise in-place, transpose the matrix (swap `matrix[i][j]` with `matrix[j][i]`) and then reverse each row.
- **Invariants:** Transposition only swaps elements where $j \geq i$ to prevent double-swapping back to original positions.

```

public void rotate(int[][] matrix) {
    int n = matrix.length;
    // Transpose matrix
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse each row
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n / 2; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[i][n - 1 - j];
            matrix[i][n - 1 - j] = temp;
        }
    }
}

```

8.5.1.3 Q3: Group Anagrams (Medium-Hard)

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

- **Spec-Driven Analysis:** Anagrams share the exact same character counts. Sort each string's characters to generate a unique canonical key for a HashMap.
- **Invariants:** All strings mapped to the same HashMap key must be anagrams.

```

public List<List<String>> groupAnagrams(String[] strs) {
    if (strs == null || strs.length == 0) return new ArrayList<>();
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {

```

```

        char[] ca = s.toCharArray();
        Arrays.sort(ca);
        String key = String.valueOf(ca);
        if (!map.containsKey(key)) map.put(key, new ArrayList<>());
        map.get(key).add(s);
    }
    return new ArrayList<>(map.values());
}

```

8.5.1.4 Q4: Sliding Window Median (Hard)

Given an integer array `nums` and an integer `k`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. Return the median array for each window position.

- **Spec-Driven Analysis:** Track the median of the window dynamically. Balance elements between a Max-Heap (lower half) and a Min-Heap (upper half).
- **Invariants:** `leftHeap.size() == rightHeap.size()` (even window) or `leftHeap.size() == rightHeap.size() + 1` (odd window).

```

public double[] medianSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    double[] result = new double[n - k + 1];
    // Max heap for lower half
    PriorityQueue<Integer> left = new
        ↳ PriorityQueue<>(Collections.reverseOrder());
    // Min heap for upper half
    PriorityQueue<Integer> right = new PriorityQueue<>();

    for (int i = 0; i < n; i++) {
        if (left.isEmpty() || nums[i] <= left.peek()) {
            left.add(nums[i]);
        } else {
            right.add(nums[i]);
        }
        balance(left, right);

        if (i >= k - 1) {
            if (left.size() == right.size()) {
                result[i - k + 1] = ((double)left.peek() + right.peek()) / 2.0;
            } else {
                result[i - k + 1] = left.peek();
            }

            // Remove element sliding out of window
            int elementToRemove = nums[i - k + 1];
            if (elementToRemove <= left.peek()) {
                left.remove(elementToRemove);
            } else {
                right.remove(elementToRemove);
            }
        }
    }
}

```

```

        balance(left, right);
    }
}
return result;
}

private void balance(PriorityQueue<Integer> left, PriorityQueue<Integer> right) {
    if (left.size() > right.size() + 1) {
        right.add(left.poll());
    } else if (left.size() < right.size()) {
        left.add(right.poll());
    }
}
}

```

8.5.2 Mock GCA Test 2 (Target Time: 70 Minutes)

8.5.2.1 Q1: Valid Parentheses (Easy)

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

- **Spec-Driven Analysis:** Use a stack to track open brackets. A closing bracket must match the most recently opened bracket.
- **Invariants:** The stack contains unmatched open brackets in LIFO order.

```

public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == '}' && top != '{') return false;
            if (c == ']' && top != '[') return false;
        }
    }
    return stack.isEmpty();
}

```

8.5.2.2 Q2: Spiral Matrix (Medium)

Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

- **Spec-Driven Analysis:** Define four boundaries: `r1` (top row), `r2` (bottom row), `c1` (left col), and `c2` (right col). Traverse boundaries clockwise, updating limits.
- **Invariants:** Traversal terminates once `r1 > r2` or `c1 > c2`.

```

public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> result = new ArrayList<>();

```

```

    if (matrix.length == 0) return result;
    int r1 = 0, r2 = matrix.length - 1;
    int c1 = 0, c2 = matrix[0].length - 1;
    while (r1 <= r2 && c1 <= c2) {
        for (int c = c1; c <= c2; c++) result.add(matrix[r1][c]);
        for (int r = r1 + 1; r <= r2; r++) result.add(matrix[r][c2]);
        if (r1 < r2 && c1 < c2) {
            for (int c = c2 - 1; c > c1; c--) result.add(matrix[r2][c]);
            for (int r = r2; r > r1; r--) result.add(matrix[r][c1]);
        }
        r1++;
        r2--;
        c1++;
        c2--;
    }
    return result;
}

```

8.5.2.3 Q3: Subarray Sum Equals K (Medium-Hard)

Given an array of integers `nums` and an integer `k`, return the total number of continuous subarrays whose sum equals to `k`.

- **Spec-Driven Analysis:** A subarray sum from index i to j is computed as $PrefixSum[j] - PrefixSum[i - 1]$. Track prefix sums and their frequency in a `HashMap`.
- **Invariants:** For any index j , if $PrefixSum[j] - k$ is present in the map, a matching subarray exists.

```

public int subarraySum(int[] nums, int k) {
    int count = 0, sum = 0;
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1);
    for (int num : nums) {
        sum += num;
        if (map.containsKey(sum - k)) {
            count += map.get(sum - k);
        }
        map.put(sum, map.getOrDefault(sum, 0) + 1);
    }
    return count;
}

```

8.5.2.4 Q4: Median of Two Sorted Arrays (Hard)

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays. The overall run time complexity should be $O(\log(m + n))$.

- **Spec-Driven Analysis:** Binary search on the partition split of the smaller array. Partition the arrays such that the left half has the same size as the right half.

- **Invariants:** Left partitions must be smaller than or equal to right partitions: $A[i-1] \leq B[j]$ and $B[j-1] \leq A[i]$.

```
public double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) {
        return findMedianSortedArrays(B, A); // Ensure A is the shorter array
    }
    int m = A.length;
    int n = B.length;
    int left = 0, right = m;
    while (left <= right) {
        int i = left + (right - left) / 2;
        int j = (m + n + 1) / 2 - i;

        int aLeft = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
        int aRight = (i == m) ? Integer.MAX_VALUE : A[i];
        int bLeft = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
        int bRight = (j == n) ? Integer.MAX_VALUE : B[j];

        if (aLeft <= bRight && bLeft <= aRight) {
            if ((m + n) % 2 == 1) {
                return Math.max(aLeft, bLeft);
            }
            return (Math.max(aLeft, bLeft) + Math.min(aRight, bRight)) / 2.0;
        } else if (aLeft > bRight) {
            right = i - 1;
        } else {
            left = i + 1;
        }
    }
    return 0.0;
}
```

STAR Moment: The Monotonic Stack Shortcut

In coding tests, if a problem asks you to find the “*next greater element*” or “*next smaller element*” for every item in an array, it is a **Monotonic Stack** problem. Do not write nested loops. Push elements onto a stack, popping elements off when you find a value that exceeds the stack’s tail. This reduces the time complexity from $O(N^2)$ to $O(N)$ instantly.

Chapter 9

Q1 Mastery — Implementation Speed, In-Place Transformations, and String Processing

The first question (Q1) on the CodeSignal General Coding Assessment (GCA) is designed to evaluate fundamental implementation speed, boundary correctness, and memory hygiene. You have roughly **8 minutes** to solve Q1. While categorized as “Easy,” Q1 is where candidates most frequently drop valuable points — not because the problem is hard, but because they rush and introduce off-by-one errors, forget null checks, or use inefficient string concatenation. A perfect Q1 score is the foundation of a 750+ GCA result.

This chapter teaches you the core vocabulary, the reusable pointer archetypes, 20 fully solved exemplar problems with detailed explanations, and 30 concrete practice problems with strategic hints.

9.1 Essential Terminology & Vocabulary

Before solving any Q1 problem, you must internalize these foundational concepts. Each one maps directly to a class of problems you will encounter on the exam.

9.1.1 In-Place Mutation

An algorithm is **in-place** if it transforms the input using $\mathcal{O}(1)$ auxiliary space (excluding the input itself). In Java, arrays are mutable references — you can overwrite `arr[i]` directly. Strings, however, are **immutable objects** — every modification creates a new heap allocation.

Why it matters on Q1: Many Q1 problems explicitly require in-place modification. If you allocate a new array when the spec says “in-place,” you lose points even if the output is correct.

9.1.2 Read/Write Pointer Pattern

A two-pointer technique where: - The **read pointer** scans every element sequentially (always moves forward). - The **write pointer** only advances when an element passes a filter condition.

After the loop, `arr[0..write-1]` contains the filtered result. This pattern solves: *Remove Element*, *Move Zeros*, *Remove Duplicates from Sorted Array*, and *String Compression*.

9.1.3 Character Frequency Array (`int[256]` or `int[26]`)

A fixed-size integer array indexed by character ASCII value. `counts['a']++` increments the counter at index 97. This provides: - $\mathcal{O}(1)$ per lookup/update (direct array access, no hashing) - Zero heap allocations (lives on the stack) - Deterministic performance (no hash collisions)

Use `int[26]` when input is guaranteed lowercase English letters only (`c - 'a'`). Use `int[256]` when input may contain any ASCII character.

Comparison with HashMap:

Attribute	<code>int[256]</code>	<code>HashMap<Character, Integer></code>
Access Time	$\mathcal{O}(1)$ direct	$\mathcal{O}(1)$ amortized (hash collisions possible)
Memory	1 KB fixed on stack	Variable heap allocations
GC Pressure	Zero	High (autoboxing <code>char</code> $\rightarrow$ <code>Character</code>)
When to Use	ASCII text, known char range	Unicode, arbitrary key types

9.1.4 Symmetrical Two-Pointer Convergence

Two pointers start at opposite ends (`left = 0`, `right = len - 1`) and move toward each other. The loop condition is `while (left < right)`. This pattern solves: *Palindrome Check*, *Reverse String*, *Two Sum in Sorted Array*, and *Container With Most Water*.

9.1.5 Run-Length Encoding (RLE)

Compress consecutive identical elements into (`element`, `count`) pairs. "aaabbc" becomes "a3b2c1". The read pointer tracks the current run; the write pointer emits compressed output. This is a classic Q1 problem that combines the Read/Write pattern with counting.

9.1.6 String Immutability & StringBuilder

In Java, `String` is immutable. The expression `s += char` inside a loop creates a **new String object on every iteration**, copying all previous characters. For a string of length N , this produces $\mathcal{O}(N^2)$ total character copies. Always use `StringBuilder` for loop-based string construction — it maintains a resizable `char[]` buffer internally and runs in amortized $\mathcal{O}(N)$.

9.1.7 XOR Bit Manipulation for Uniqueness

The XOR operator ($\wedge$) has two key properties: $a \wedge a = 0$ (same values cancel) and $a \wedge 0 = a$ (zero is identity). XOR-ing all elements in an array where every value appears twice except one

produces the unique value. This runs in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space with zero branching.

9.1.8 Prefix Sum / Running Total

A technique where you compute cumulative sums to answer range queries in $\mathcal{O}(1)$. For pivot index problems: `leftSum == totalSum - leftSum - nums[i]` identifies the balance point without nested loops.

9.1.9 Integer Overflow & Boundary Guarding

This involves handling `Integer.MAX_VALUE` and `Integer.MIN_VALUE` constraints. It requires implementing safe comparisons before executing arithmetic operations to prevent exceeding limits. Why it matters: Reverse-integer and palindrome-number problems require overflow detection.

9.1.10 Digit Extraction Loop

This technique involves using the modulo operator `num % 10` to get the last digit of a number. You then use integer division `num / 10` to remove that digit for the next iteration. Why it matters: This is fundamental for reverse integer, digit sum, and palindrome number checks.

9.1.11 ASCII Arithmetic

This technique leverages character ASCII values for mathematical operations. Using `c - 'a'` converts a letter to a 0-25 index, `c - '0'` converts a char digit to an integer, and `(char)(i + 'a')` converts it back. Why it matters: This pattern maps characters to array indices without relying on a HashMap.

9.1.12 Boolean Flag / Sentinel Pattern

This involves using a single boolean variable to track whether a specific event has occurred across a scan. It monitors state changes cleanly throughout an iteration. Why it matters: It simplifies complex conditions into clean single-variable tracking.

9.1.13 Edge Case Taxonomy

This categorizes common input boundaries such as null, empty array/string, single element, all-same values, all-different values, and maximum integer values. Understanding this taxonomy ensures comprehensive test coverage. Why it matters: You systematically test these BEFORE writing the main loop to catch 80% of bugs.

9.1.14 Stack-Based Matching

This approach involves pushing opening delimiters onto a stack during traversal. Upon encountering a closing delimiter, you pop from the stack and verify the match. Why it matters: This is the universal pattern for bracket, parentheses, and tag validation problems.

9.1.15 Two-Pass Strategy

This algorithm design splits processing into two distinct phases. The first pass collects necessary data like counts, maximums, or positions, and the second pass acts on that collected information. Why it matters: It avoids complex single-pass logic and significantly reduces bugs.

9.1.16 Greedy Forward Scan

This strategy involves processing an array from left to right sequentially. At each step, you make the locally optimal choice without looking back. Why it matters: It is heavily used in array change problems (like bumping each element above the previous) and similar Q1 tasks.

9.1.17 Modular Arithmetic Basics

This encompasses foundational modulo operations for cyclic or remainder logic. Examples include using $n \% 2$ for parity, $n \% k$ for divisibility, and $(a + b - 1) / b$ for ceiling division. Why it matters: It avoids floating-point arithmetic entirely and handles circular increments efficiently.

9.1.18 In-Place Swap

This is the standard programming idiom for swapping two variables using a temporary holder. It uses the `temp = a; a = b; b = temp;` pattern. Why it matters: It serves as a fundamental building block for partitioning, reversing, and Dutch National Flag problems.

9.2 Reusable Code Templates

These are the two most important templates to have memorized before the exam.

9.2.1 Template A: Read/Write In-Place Filter

```
// Retains elements satisfying a condition, overwrites array in-place
int write = 0;
for (int read = 0; read < arr.length; read++) {
    if (keepCondition(arr[read])) {
        arr[write] = arr[read];
        write++;
    }
}
// Result is arr[0..write-1], return write as the new length
```

Used by: Remove Element, Move Zeros, Remove Duplicates, Squeeze Spaces.

9.2.2 Template B: Symmetric Converging Pointers

```
int left = 0, right = arr.length - 1;
while (left < right) {
    // Process or compare arr[left] and arr[right]
    // Optionally skip invalid elements
    left++;
    right--;
}
```

Used by: Palindrome Check, Reverse Array, Two Sum (sorted), Sort Colors.

9.3 Solved Exemplar Problems (20 Core Implementations)

9.3.1 Problem 1: First Non-Repeating Character

Specification: Given a string `s`, find the first character that appears exactly once. Return its 0-based index. If no unique character exists, return `-1`.

Example: `"leetcode"` → 0 (the character `'l'` appears once and is the first such character).

Pattern: Two-pass frequency array. First pass counts; second pass finds the first count of 1.

Why two passes? A single pass cannot determine uniqueness because later characters might duplicate earlier ones. The frequency array decouples counting from searching.

```
public int firstUniqChar(String s) {
    if (s == null || s.isEmpty()) return -1;

    // Pass 1: Count frequency of each character
    int[] counts = new int[256];
    for (int i = 0; i < s.length(); i++) {
        counts[s.charAt(i)]++;
    }

    // Pass 2: Find first character with frequency exactly 1
    for (int i = 0; i < s.length(); i++) {
        if (counts[s.charAt(i)] == 1) return i;
    }

    return -1; // All characters repeat
}
// Time: O(N), Space: O(1) - the int[256] is constant size
```

9.3.2 Problem 2: In-Place String Compression (Run-Length Encoding)

Specification: Given a character array `chars`, compress it in-place using RLE. Consecutive duplicate characters are replaced by the character followed by the count (only if count > 1). Return the new length. You must modify `chars` in-place — no new array allocation.

Example: `['a','a','b','b','c','c','c']` → modified to `['a','2','b','2','c','3']`, return 6.

Pattern: Read/Write pointers with a nested counting loop.

Critical edge case: When count exceeds 9 (e.g., count = 12), you must write `'1'` then `'2'` as separate characters.

```
public int compress(char[] chars) {
    if (chars == null || chars.length == 0) return 0;

    int write = 0; // Write pointer for compressed output
    int read = 0;  // Read pointer scanning input
```

```

while (read < chars.length) {
    char current = chars[read];
    int count = 0;

    // Count consecutive occurrences of current character
    while (read < chars.length && chars[read] == current) {
        read++;
        count++;
    }

    // Write the character itself
    chars[write++] = current;

    // Write the count digits (only if count > 1)
    if (count > 1) {
        // Convert count to individual digit characters
        for (char digit : Integer.toString(count).toCharArray()) {
            chars[write++] = digit;
        }
    }
}

return write;
}
// Time: O(N), Space: O(1) auxiliary

```

9.3.3 Problem 3: Valid Palindrome with Non-Alphanumeric Skipping

Specification: Given a string `s`, return `true` if it is a palindrome considering only alphanumeric characters and ignoring case. An empty string is a valid palindrome.

Example: "A man, a plan, a canal: Panama" → `true`.

Pattern: Symmetric converging pointers with skip logic for non-alphanumeric characters.

Common mistake: Forgetting to check `left < right` inside the skip-while loops, causing `ArrayIndexOutOfBoundsException` on strings like `".,,"`.

```

public boolean isPalindrome(String s) {
    if (s == null) return false;

    int left = 0, right = s.length() - 1;

    while (left < right) {
        // Skip non-alphanumeric from the left
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
            left++;
        }
    }
}

```

```

    // Skip non-alphanumeric from the right
    while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
        right--;
    }

    // Compare characters (case-insensitive)
    if (Character.toLowerCase(s.charAt(left)) !=
        Character.toLowerCase(s.charAt(right))) {
        return false;
    }

    left++;
    right--;
}

return true;
}
// Time: O(N), Space: O(1)

```

9.3.4 Problem 4: Move Zeros to End

Specification: Given an integer array `nums`, move all 0s to the end while maintaining the relative order of non-zero elements. Must be done in-place.

Example: `[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]`.

Pattern: Read/Write pointer. Non-zero elements are copied forward; remaining positions are filled with zeros.

Why not swap? Swapping works too, but the two-pass approach (copy then fill) is cleaner and less error-prone under time pressure.

```

public void moveZeroes(int[] nums) {
    if (nums == null || nums.length == 0) return;

    // Pass 1: Copy all non-zero elements to the front
    int write = 0;
    for (int read = 0; read < nums.length; read++) {
        if (nums[read] != 0) {
            nums[write++] = nums[read];
        }
    }

    // Pass 2: Fill remaining positions with zeros
    while (write < nums.length) {
        nums[write++] = 0;
    }
}
// Time: O(N), Space: O(1)

```

9.3.5 Problem 5: Remove Duplicates from Sorted Array

Specification: Given a sorted integer array `nums`, remove duplicates in-place so each element appears only once. Return the number of unique elements. The first `k` elements of `nums` should hold the result.

Example: `[1, 1, 2] → [1, 2, _]`, return 2.

Pattern: Read/Write pointer. Since the array is sorted, duplicates are always adjacent. The write pointer advances only when `nums[read] != nums[write - 1]`.

```
public int removeDuplicates(int[] nums) {
    if (nums == null || nums.length == 0) return 0;

    int write = 1; // First element is always unique
    for (int read = 1; read < nums.length; read++) {
        if (nums[read] != nums[write - 1]) {
            nums[write++] = nums[read];
        }
    }

    return write;
}
// Time: O(N), Space: O(1)
```

9.3.6 Problem 6: Single Number (XOR Uniqueness)

Specification: Given a non-empty array where every element appears exactly twice except one, find the single element. Must run in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.

Example: `[4, 1, 2, 1, 2] → 4`.

Pattern: XOR accumulation. $a \oplus a = 0$ cancels pairs; $a \oplus 0 = a$ preserves the unique element.

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num; // Pairs cancel, unique value survives
    }
    return result;
}
// Time: O(N), Space: O(1)
```

9.3.7 Problem 7: Valid Parentheses

Specification: Given a string containing only `(, ), {, }, [, ]`, determine if the input is valid. Every open bracket must be closed by the same type in correct order.

Example: "() [] {}" $\rightarrow$ true. "[]" $\rightarrow$ false.

Pattern: Stack-based matching. On open bracket, push the expected closing bracket. On close bracket, pop and compare.

Optimization: Use a char[] as a manual stack to avoid java.util.Stack overhead.

```
public boolean isValid(String s) {
    if (s == null || s.length() % 2 != 0) return false;

    char[] stack = new char[s.length()];
    int top = -1;

    for (char c : s.toCharArray()) {
        if (c == '(') stack[++top] = ')';
        else if (c == '{') stack[++top] = '}';
        else if (c == '[') stack[++top] = ']';
        else {
            if (top == -1 || stack[top--] != c) return false;
        }
    }

    return top == -1; // Stack must be empty
}
// Time: O(N), Space: O(N) worst case for the stack
```

9.3.8 Problem 8: Reverse String In-Place

Specification: Reverse a character array in-place using $\mathcal{O}(1)$ extra memory.

Example: ['h','e','l','l','o'] $\rightarrow$ ['o','l','l','e','h'].

Pattern: Symmetric converging pointers with swap.

```
public void reverseString(char[] s) {
    if (s == null || s.length <= 1) return;

    int left = 0, right = s.length - 1;
    while (left < right) {
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        left++;
        right--;
    }
}
// Time: O(N), Space: O(1)
```

9.3.9 Problem 9: Pivot Index (Balance Point)

Specification: Given array `nums`, find the leftmost index where the sum of elements to its left equals the sum of elements to its right. If no such index exists, return `-1`. The element at the pivot is excluded from both sums.

Example: `[1, 7, 3, 6, 5, 6]` $\rightarrow$ 3 (left sum $1+7+3 = 11$, right sum $5+6 = 11$).

Pattern: Prefix sum. Compute total sum first, then scan left-to-right maintaining a running left sum. At each index: `rightSum = totalSum - leftSum - nums[i]`.

```
public int pivotIndex(int[] nums) {
    if (nums == null) return -1;

    int totalSum = 0;
    for (int num : nums) totalSum += num;

    int leftSum = 0;
    for (int i = 0; i < nums.length; i++) {
        // rightSum = totalSum - leftSum - nums[i]
        if (leftSum == totalSum - leftSum - nums[i]) return i;
        leftSum += nums[i];
    }

    return -1;
}
// Time: O(N), Space: O(1)
```

9.3.10 Problem 10: Check Array Monotonicity

Specification: Return `true` if the array is entirely non-decreasing or entirely non-increasing.

Example: `[1, 2, 2, 3]` $\rightarrow$ `true`. `[6, 5, 4, 4]` $\rightarrow$ `true`. `[1, 3, 2]` $\rightarrow$ `false`.

Pattern: Dual boolean flags. Track both `isIncreasing` and `isDecreasing`. If an adjacent pair violates one direction, set its flag to `false`. Return `true` if either flag survives.

```
public boolean isMonotonic(int[] nums) {
    if (nums == null || nums.length <= 2) return true;

    boolean increasing = true;
    boolean decreasing = true;

    for (int i = 0; i < nums.length - 1; i++) {
        if (nums[i] > nums[i + 1]) increasing = false;
        if (nums[i] < nums[i + 1]) decreasing = false;
    }

    return increasing || decreasing;
}
```


// Time: O(N), Space: O(1)

9.3.11 Problem 11: Neighbor Sum Transformation

Specification: Given array A, return array B where $B[i] = A[i-1] + A[i] + A[i+1]$. Treat out-of-bounds indices as 0.

Example: $[4, 0, 1, -2, 3] \rightarrow [4, 5, -1, 2, 1]$.

Pattern: Boundary-safe neighbor access with ternary guards.

Why a new array? Modifying A in-place would corrupt values needed for subsequent index calculations.

```
public int[] neighborSum(int[] a) {
    if (a == null) return new int[0];
    int n = a.length;
    int[] b = new int[n];

    for (int i = 0; i < n; i++) {
        int leftVal = (i > 0) ? a[i - 1] : 0;
        int rightVal = (i < n - 1) ? a[i + 1] : 0;
        b[i] = leftVal + a[i] + rightVal;
    }

    return b;
}
// Time: O(N), Space: O(N) for output array
```

9.3.12 Problem 12: Maximum Subarray Sum of Fixed Window K

Specification: Given integer array nums and integer k, find the maximum sum among all contiguous subarrays of exactly size k.

Example: $\text{nums} = [2, 1, 5, 1, 3, 2], k = 3 \rightarrow 9$ (subarray $[5, 1, 3]$).

Pattern: Fixed-size sliding window. Initialize window sum with first k elements, then slide by adding the entering element and subtracting the leaving element.

```
public int maxSumSubarray(int[] nums, int k) {
    if (nums == null || nums.length < k || k <= 0) return 0;

    // Initialize sum of first window
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];

    int maxSum = windowSum;

    // Slide the window: add right element, remove left element
```

```

    for (int i = k; i < nums.length; i++) {
        windowSum += nums[i] - nums[i - k];
        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}
// Time: O(N), Space: O(1)

```

9.3.13 Problem 13: Find the Added Character

Specification: String `t` is created by shuffling string `s` and inserting one extra character at a random position. Find and return that added character.

Example: `s = "abcd", t = "abcde" → 'e'`.

Pattern: XOR accumulation. XOR every character in both strings together. Paired characters cancel to zero; the extra character remains.

```

public char findTheDifference(String s, String t) {
    char result = 0;
    for (char c : s.toCharArray()) result ^= c;
    for (char c : t.toCharArray()) result ^= c;
    return result; // Only the unpaired character survives
}
// Time: O(N), Space: O(1)

```

9.3.14 Problem 14: Capitalize or Reverse by Word Length Parity

Specification: Given an array of words, transform each word: if the word's length is odd, convert to uppercase; if even, reverse its characters.

Example: `["Hello", "Data"] → ["HELLO", "ataD"]`.

Pattern: Per-element transformation with parity branching.

```

public String[] transformWords(String[] words) {
    if (words == null) return new String[0];
    String[] result = new String[words.length];

    for (int i = 0; i < words.length; i++) {
        if (words[i].length() % 2 != 0) {
            result[i] = words[i].toUpperCase();
        } else {
            result[i] = new StringBuilder(words[i]).reverse().toString();
        }
    }

    return result;
}

```

```
}
// Time:  $O(N * K)$  where  $K$  is average word length, Space:  $O(N * K)$  for output
```

9.3.15 Problem 15: Check Equal Character Frequencies

Specification: Return `true` if every character in string `s` appears the exact same number of times.

Example: "abacbc" $\rightarrow$ `true` (each of a, b, c appears 2 times). "aaabb" $\rightarrow$ `false`.

Pattern: Frequency array + validation scan. Count all characters, then verify every non-zero count matches.

```
public boolean areOccurrencesEqual(String s) {
    if (s == null || s.isEmpty()) return true;

    int[] counts = new int[26];
    for (char c : s.toCharArray()) counts[c - 'a']++;

    int expected = 0;
    for (int count : counts) {
        if (count > 0) {
            if (expected == 0) expected = count;
            else if (count != expected) return false;
        }
    }

    return true;
}

// Time:  $O(N)$ , Space:  $O(1)$ 
```

9.3.16 Problem 16: Remove Element In-Place

Specification: Given integer array `nums` and integer `val`, remove all occurrences of `val` in-place. Return the count of elements not equal to `val`. The first `k` positions of `nums` should contain the remaining elements.

Example: `nums = [3, 2, 2, 3]`, `val = 3` $\rightarrow$ return 2, array becomes `[2, 2, ...]`.

Pattern: Read/Write pointer — identical structure to Move Zeros.

```
public int removeElement(int[] nums, int val) {
    if (nums == null) return 0;

    int write = 0;
    for (int read = 0; read < nums.length; read++) {
        if (nums[read] != val) {
            nums[write++] = nums[read];
        }
    }
}
```

```

    return write;
}
// Time: O(N), Space: O(1)

```

9.3.17 Problem 17: Parity Alternation Validation

Specification: Given an integer array, return `true` if every adjacent pair alternates between odd and even (i.e., no two adjacent elements share the same parity).

Example: `[1, 2, 3, 4] → true`. `[1, 3, 2] → false` (1 and 3 are both odd).

Pattern: Linear scan comparing `nums[i] % 2` with `nums[i+1] % 2`.

Edge case with negatives: `(-3) % 2` in Java returns `-1`, not `1`. Use `Math.abs(nums[i] % 2)` for safe parity checks.

```

public boolean isAlternatingParity(int[] nums) {
    if (nums == null || nums.length <= 1) return true;

    for (int i = 0; i < nums.length - 1; i++) {
        // Use Math.abs for safety with negative numbers
        if (Math.abs(nums[i] % 2) == Math.abs(nums[i + 1] % 2)) {
            return false;
        }
    }

    return true;
}
// Time: O(N), Space: O(1)

```

9.3.18 Problem 18: Two Sum (Unsorted Array)

Specification: Given an array of integers `nums` and an integer `target`, return the indices of two elements that add up to `target`. Each input has exactly one solution. You may not use the same element twice.

Example: `nums = [2, 7, 11, 15], target = 9 → [0, 1]`.

Pattern: HashMap complement lookup. For each element, check if `target - nums[i]` has been seen. If yes, return both indices. If no, store `nums[i] → i` in the map.

```

public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[]{seen.get(complement), i};
        }
    }
}

```

```

    }
    seen.put(nums[i], i);
}

return new int[]{}; // Should not reach here per problem guarantee
}
// Time: O(N), Space: O(N)

```

9.3.19 Problem 19: Majority Element

Specification: Given an array `nums` of size `n`, return the element that appears more than $\lfloor n/2 \rfloor$ times. The majority element is guaranteed to exist.

Example: `[2, 2, 1, 1, 1, 2, 2] → 2`.

Pattern: Boyer–Moore Voting Algorithm. Maintain a candidate and a count. When count drops to zero, switch candidates. The majority element will always survive because it appears more than half the time.

```

public int majorityElement(int[] nums) {
    int candidate = nums[0];
    int count = 1;

    for (int i = 1; i < nums.length; i++) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}
// Time: O(N), Space: O(1)

```

9.3.20 Problem 20: Plus One (Large Number as Array)

Specification: Given a large integer represented as an array of digits (most significant digit first), increment the integer by one and return the resulting array.

Example: `[1, 2, 3] → [1, 2, 4]`. `[9, 9, 9] → [1, 0, 0, 0]`.

Pattern: Right-to-left carry propagation. Process digits from the least significant end. If a digit becomes 10, set it to 0 and carry. If no carry remains, return immediately.

Edge case: All 9s (`[9, 9, 9]`) require a new array of length `n + 1` with a leading 1.

```

public int[] plusOne(int[] digits) {
    for (int i = digits.length - 1; i >= 0; i--) {
        digits[i]++;
        if (digits[i] < 10) {
            return digits; // No further carry needed
        }
        digits[i] = 0; // Carry to next position
    }

    // All digits were 9 - need a new array [1, 0, 0, ..., 0]
    int[] result = new int[digits.length + 1];
    result[0] = 1;
    return result;
}
// Time: O(N), Space: O(1) amortized (O(N) only for all-9s edge case)

```

9.4 CodeSignal Classic Q1 Patterns (12 Additional Solved Problems)

The following problems are drawn directly from the CodeSignal Arcade and GCA Q1 question bank. They emphasize boundary arithmetic, simple simulations, and filter-sort-reinsert patterns that appear frequently on actual assessments.

9.4.1 Problem 21: Maximum Adjacent Element Product

Specification: Given an array of integers, find the pair of adjacent elements that has the largest product. Return that product.

Example: [3, 6, -2, -5, 7, 3] → 21 (from pair [7, 3]).

Invariant: The maximum adjacent product can only occur between `arr[i]` and `arr[i+1]` for some valid `i`. A single linear scan tracking the running max is sufficient.

Common mistake: Forgetting that two large negative numbers produce a large positive product (e.g., [-5, -4] → 20).

```

public int adjacentElementsProduct(int[] inputArray) {
    if (inputArray == null || inputArray.length < 2) return 0;

    int maxProd = inputArray[0] * inputArray[1];

    for (int i = 1; i < inputArray.length - 1; i++) {
        int prod = inputArray[i] * inputArray[i + 1];
        if (prod > maxProd) {
            maxProd = prod;
        }
    }
}

```

```

    return maxProd;
}
// Time: O(N), Space: O(1)

```

9.4.2 Problem 22: Century From Year

Specification: Given a year, return the century it belongs to. The first century spans year 1 through 100 inclusive, the second spans 101 through 200, etc.

Example: 1905 → 20. 1700 → 17. 2000 → 20. 2001 → 21.

Pattern: Integer ceiling division. The formula $(\text{year} + 99) / 100$ computes the ceiling of $\text{year} / 100$ using only integer arithmetic, avoiding floating-point rounding errors.

```

public int centuryFromYear(int year) {
    return (year + 99) / 100;
}
// Time: O(1), Space: O(1)

```

9.4.3 Problem 23: All Longest Strings

Specification: Given an array of strings, return a new array containing all strings that share the maximum length.

Example: ["aba", "aa", "ad", "vcd", "aba"] → ["aba", "vcd", "aba"].

Pattern: Two-pass filter. Pass 1 finds the maximum string length. Pass 2 collects all strings matching that length.

Why two passes? A single pass would require backtracking to remove shorter strings discovered before the true maximum is known.

```

public String[] allLongestStrings(String[] inputArray) {
    // Pass 1: Find the maximum length
    int maxLength = 0;
    for (String s : inputArray) {
        if (s.length() > maxLength) {
            maxLength = s.length();
        }
    }

    // Pass 2: Collect strings matching the max length
    List<String> result = new ArrayList<>();
    for (String s : inputArray) {
        if (s.length() == maxLength) {
            result.add(s);
        }
    }
}

```

```

    return result.toArray(new String[0]);
}
// Time: O(N), Space: O(N) for output

```

9.4.4 Problem 24: Common Character Count

Specification: Given two strings `s1` and `s2`, find the number of common characters between them. Each character match consumes one occurrence from each string.

Example: `s1 = "aabcc", s2 = "adcaa" → 3` (common: 'a', 'a', 'c').

Pattern: Dual frequency arrays with element-wise minimum. Build `int[26]` for each string. The number of shared instances of character `c` is `Math.min(count1[c], count2[c])`.

```

public int commonCharacterCount(String s1, String s2) {
    int[] count1 = new int[26];
    int[] count2 = new int[26];

    for (char c : s1.toCharArray()) count1[c - 'a']++;
    for (char c : s2.toCharArray()) count2[c - 'a']++;

    int common = 0;
    for (int i = 0; i < 26; i++) {
        common += Math.min(count1[i], count2[i]);
    }

    return common;
}
// Time: O(N + M), Space: O(1) - fixed 26-element arrays

```

9.4.5 Problem 25: Lucky Ticket (Digit Sum Halves)

Specification: A ticket number (even number of digits) is “lucky” if the sum of its first-half digits equals the sum of its second-half digits. Determine if a given number is lucky.

Example: `1230 → true` (`1 + 2 = 3, 3 + 0 = 3`). `239017 → false` (`2+3+9 = 14, 0+1+7 = 8`).

Pattern: Convert to string for digit access. Split at midpoint. Sum each half independently.

```

public boolean isLucky(int n) {
    String s = String.valueOf(n);
    int mid = s.length() / 2;
    int sum1 = 0, sum2 = 0;

    for (int i = 0; i < mid; i++) {
        sum1 += s.charAt(i) - '0';           // First half digit
        sum2 += s.charAt(i + mid) - '0';    // Second half digit
    }
}

```



```

    return sum1 == sum2;
}
// Time:  $O(D)$  where  $D$  is digit count, Space:  $O(D)$  for string conversion

```

9.4.6 Problem 26: Sort By Height (Obstacles in Place)

Specification: People are standing in a row with immovable trees (represented by -1) between them. Sort the people by height in non-descending order without moving the trees.

Example: [-1, 150, 190, 170, -1, -1, 160, 180] → [-1, 150, 160, 170, -1, -1, 180, 190].

Pattern: Filter-Sort-Reinsert. Extract non-tree values into a separate list, sort that list, then write the sorted values back into the original array at non-tree positions only.

Invariant: Tree positions (-1) are never touched. Only human positions are modified.

```

public int[] sortByHeight(int[] a) {
    // Step 1: Extract all non-tree heights
    List<Integer> heights = new ArrayList<>();
    for (int h : a) {
        if (h != -1) heights.add(h);
    }

    // Step 2: Sort the extracted heights
    Collections.sort(heights);

    // Step 3: Reinsert sorted heights at non-tree positions
    int index = 0;
    for (int i = 0; i < a.length; i++) {
        if (a[i] != -1) {
            a[i] = heights.get(index++);
        }
    }

    return a;
}
// Time:  $O(N \log N)$  for sorting, Space:  $O(N)$  for extracted list

```

9.4.7 Problem 27: Alternating Team Sums

Specification: People in a row are divided into two teams by alternating index: person 0 → Team 1, person 1 → Team 2, person 2 → Team 1, etc. Return the total weight of each team as [team1Sum, team2Sum].

Example: [50, 60, 60, 45, 70] → [180, 105].

Pattern: Index parity accumulation. $i \% 2 == 0$ accumulates into Team 1, $i \% 2 == 1$ into Team 2.

```
public int[] alternatingSums(int[] a) {
    int team1 = 0, team2 = 0;

    for (int i = 0; i < a.length; i++) {
        if (i % 2 == 0) {
            team1 += a[i];
        } else {
            team2 += a[i];
        }
    }

    return new int[]{team1, team2};
}
// Time: O(N), Space: O(1)
```

9.4.8 Problem 28: Add Border to Character Matrix

Specification: Given a rectangular array of strings (representing rows of a character matrix), add a border of asterisks (*) around it. Return the new bordered matrix.

Example: ["abc", "ded"] $\rightarrow$ ["*****", "*abc*", "*ded*", "*****"].

Pattern: String construction with dimensional arithmetic. New width = original width + 2. New height = original height + 2. First and last rows are full asterisk strings. Middle rows are wrapped with * on each side.

```
public String[] addBorder(String[] picture) {
    int newWidth = picture[0].length() + 2;
    String[] result = new String[picture.length + 2];

    // Build the border row
    StringBuilder borderRow = new StringBuilder();
    for (int i = 0; i < newWidth; i++) borderRow.append('*');
    String border = borderRow.toString();

    // Top border
    result[0] = border;

    // Wrap each interior row with side asterisks
    for (int i = 0; i < picture.length; i++) {
        result[i + 1] = "*" + picture[i] + "*";
    }

    // Bottom border
    result[result.length - 1] = border;
}
```

```

    return result;
}
// Time: O(rows * cols), Space: O(rows * cols) for output

```

9.4.9 Problem 29: Array Change (Minimum Moves for Strict Increase)

Specification: Given an integer array, find the minimum number of single-increment moves needed to make the sequence strictly increasing (every element must be greater than the previous one).

Example: $[1, 1, 1] \rightarrow 3$ (sequence becomes $[1, 2, 3]$). $[3, 2] \rightarrow 2$ (sequence becomes $[3, 4]$).

Pattern: Greedy forward scan. At each position i , if $\text{arr}[i] \leq \text{arr}[i-1]$, compute the deficit $\text{arr}[i-1] - \text{arr}[i] + 1$, increment $\text{arr}[i]$ by that amount, and accumulate the moves.

Invariant: After processing index i , the constraint $\text{arr}[i] > \text{arr}[i-1]$ is guaranteed. The greedy minimum at each step is globally optimal because increasing $\text{arr}[i]$ to $\text{arr}[i-1] + 1$ (the smallest valid value) minimizes cascading costs downstream.

```

public int arrayChange(int[] inputArray) {
    int moves = 0;

    for (int i = 1; i < inputArray.length; i++) {
        if (inputArray[i] <= inputArray[i - 1]) {
            // Calculate the minimum increment needed
            int deficit = inputArray[i - 1] - inputArray[i] + 1;
            inputArray[i] += deficit;
            moves += deficit;
        }
    }

    return moves;
}
// Time: O(N), Space: O(1)

```

9.4.10 Problem 30: Matrix Elements Sum (Haunted Rooms)

Specification: A building is represented as a 2D matrix where each element is the rent price of a room. Rooms directly below a free room (value 0) on any floor are also considered “haunted” and should be excluded from the total. Calculate the sum of all non-haunted rooms.

Example: $[[0, 1, 1, 2], [0, 5, 0, 0], [2, 0, 3, 3]] \rightarrow 9$ (rooms below any 0 in the column above are excluded).

Pattern: Column-wise top-down scan with a boolean “poisoned” flag per column. Once a 0 is encountered in a column, all values below it in that column are skipped.

```

public int matrixElementsSum(int[][] matrix) {
    int rows = matrix.length;

```

```

int cols = matrix[0].length;
int total = 0;

for (int c = 0; c < cols; c++) {
    for (int r = 0; r < rows; r++) {
        if (matrix[r][c] == 0) {
            break; // All rooms below are haunted - skip rest of column
        }
        total += matrix[r][c];
    }
}

return total;
}
// Time: O(rows * cols), Space: O(1)

```

9.4.11 Problem 31: Almost Increasing Sequence

Specification: Given a sequence of integers, determine whether it is possible to obtain a strictly increasing sequence by removing no more than one element.

Example: $[1, 3, 2, 1] \rightarrow \text{false}$. $[1, 3, 2] \rightarrow \text{true}$ (remove 3 $\rightarrow [1, 2]$).

Pattern: Count violations (positions where $\text{arr}[i] \geq \text{arr}[i+1]$). If zero violations, it is already increasing. If exactly one violation at position i , check two removal candidates: removing $\text{arr}[i]$ or removing $\text{arr}[i+1]$. If either removal produces a valid increasing sequence around the gap, return **true**. If more than one violation, return **false**.

This is one of the trickiest Q1 problems. The naive approach of “just remove one element and re-check” is $\mathcal{O}(N^2)$. The optimal approach is $\mathcal{O}(N)$.

```

public boolean almostIncreasingSequence(int[] sequence) {
    int count = 0; // Number of violations
    int badIdx = -1; // Index of first violation

    for (int i = 0; i < sequence.length - 1; i++) {
        if (sequence[i] >= sequence[i + 1]) {
            count++;
            badIdx = i;
            if (count > 1) return false; // More than one violation
        }
    }

    if (count == 0) return true; // Already strictly increasing

    // Try removing element at badIdx
    if (badIdx == 0 || sequence[badIdx - 1] < sequence[badIdx + 1]) {
        return true;
    }
}

```

```

    // Try removing element at badIdx + 1
    if (badIdx + 2 >= sequence.length || sequence[badIdx] < sequence[badIdx + 2])
        ↪ {
            return true;
        }

    return false;
}
// Time: O(N), Space: O(1)

```

9.4.12 Problem 32: Reverse Parentheses (Nested String Reversal)

Specification: Given a string `s` with lowercase letters and parentheses, reverse the strings in each pair of matching parentheses, starting from the innermost pair. Remove the parentheses from the result.

Example: `"(abcd)" → "dcba"`. `"(u(love)i)" → "iloveu"`. `"(ed(et(oc))el)" → "leetcod"`.

Pattern: Stack-based simulation. Use a stack of `StringBuilders`. When `(` is encountered, push a new builder. When `)` is encountered, pop the top builder, reverse it, and append its contents to the new top of the stack.

```

public String reverseInParentheses(String s) {
    Deque<StringBuilder> stack = new ArrayDeque<>();
    stack.push(new StringBuilder());

    for (char c : s.toCharArray()) {
        if (c == '(') {
            stack.push(new StringBuilder()); // Start new nested context
        } else if (c == ')') {
            StringBuilder inner = stack.pop(); // Pop innermost context
            inner.reverse(); // Reverse it
            stack.peek().append(inner); // Append to enclosing context
        } else {
            stack.peek().append(c); // Accumulate character
        }
    }

    return stack.peek().toString();
}
// Time: O(N^2) worst case for nested reversals, Space: O(N)

```

9.5 Practice Problem Bank (30 Detailed Problems with Hints)

The following 30 problems cover every Q1 pattern you may encounter on the CodeSignal GCA. Each includes a full specification, concrete examples, input constraints, and a strategic hint pointing you

toward the correct pattern.

Practice 1: Reverse Words in a Sentence

Specification: Given a string `s` containing words separated by single spaces, reverse the order of the words. Leading/trailing spaces should be removed, and multiple spaces between words should be reduced to a single space.

Example: " the sky is blue " $\rightarrow$ "blue is sky the".

Constraints: $1 \leq |s| \leq 10^4$. `s` contains English letters, digits, and spaces.

Hint: Split on whitespace, filter empty strings, reverse the resulting array, and join with single spaces. Alternatively, reverse the entire string, then reverse each word individually for an in-place solution.

Practice 2: Rotate Array by K Steps

Specification: Given an integer array `nums`, rotate the array to the right by `k` steps in-place.

Example: `nums = [1,2,3,4,5,6,7]`, `k = 3` $\rightarrow$ `[5,6,7,1,2,3,4]`.

Constraints: $1 \leq n \leq 10^5$, $0 \leq k \leq 10^5$. Handle `k > n` by taking `k % n`.

Hint: Three-reverse trick: reverse entire array, reverse first `k` elements, reverse remaining `n - k` elements. All in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.

Practice 3: Contains Duplicate

Specification: Given an integer array `nums`, return `true` if any value appears at least twice.

Example: `[1, 2, 3, 1]` $\rightarrow$ `true`. `[1, 2, 3, 4]` $\rightarrow$ `false`.

Constraints: $1 \leq n \leq 10^5$, $-10^9 \leq \text{nums}[i] \leq 10^9$.

Hint: Use a `HashSet`. Add each element — if `add()` returns `false`, a duplicate exists. $\mathcal{O}(N)$ time.

Practice 4: Length of Last Word

Specification: Given a string `s` of words and spaces, return the length of the last word. A word is a maximal substring of non-space characters.

Example: "Hello World" $\rightarrow$ 5. " fly me to the moon " $\rightarrow$ 4.

Constraints: $1 \leq |s| \leq 10^4$. `s` contains only English letters and spaces.

Hint: Scan backwards from the end. Skip trailing spaces, then count consecutive non-space characters. No need to split the entire string.

Practice 5: Roman Numeral to Integer

Specification: Convert a Roman numeral string to its integer value. Roman numerals: I=1, V=5, X=10, L=50, C=100, D=500, M=1000. Subtractive cases: IV=4, IX=9, XL=40, XC=90, CD=400, CM=900.

Example: "MCMXCIV" $\rightarrow$ 1994.

Constraints: $1 \leq |s| \leq 15$. Input is guaranteed valid.

Hint: Scan left-to-right. If the current symbol's value is less than the next symbol's value, subtract it (subtractive case). Otherwise, add it. Use a `Map<Character, Integer>` or switch statement for value lookup.

Practice 6: Missing Number in Range

Specification: Given an array `nums` containing n distinct numbers in the range $[0, n]$, return the one number in the range that is missing.

Example: $[3, 0, 1] \rightarrow 2$. $[0, 1] \rightarrow 2$.

Constraints: $n = \text{nums.length}$, $0 \leq \text{nums}[i] \leq n$. All numbers are unique.

Hint: Use Gauss's formula: $\text{expected sum} = n \times (n+1)/2$. Subtract the actual sum. The difference is the missing number. $\mathcal{O}(N)$ time, $\mathcal{O}(1)$ space. Alternatively, XOR all indices and values.

Practice 7: Merge Two Sorted Arrays into One

Specification: Given two sorted integer arrays `nums1` (length $m + n$ with trailing zeros as placeholders) and `nums2` (length n), merge `nums2` into `nums1` in-place so the result is sorted.

Example: `nums1 = [1,2,3,0,0,0]`, $m = 3$, `nums2 = [2,5,6]`, $n = 3 \rightarrow \text{nums1} = [1,2,2,3,5,6].$

Constraints: $0 \leq m, n \leq 200$.

Hint: Merge from the back (right-to-left). Compare `nums1[m-1]` with `nums2[n-1]` and place the larger one at `nums1[m+n-1]`. This avoids overwriting unprocessed elements.

Practice 8: Find Numbers with Even Number of Digits

Specification: Given an array of integers, return the count of elements that have an even number of digits.

Example: $[12, 345, 2, 6, 7896] \rightarrow 2$ (only 12 and 7896 have an even number of digits).

Constraints: $1 \leq n \leq 500$, $1 \leq \text{nums}[i] \leq 10^5$.

Hint: For each number, count digits using `Integer.toString(num).length()` or repeated division by 10. Check if the digit count is even.

Practice 9: Implement strStr() — Find First Occurrence

Specification: Given two strings `haystack` and `needle`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if not found.

Example: `haystack = "sadbutsad", needle = "sad" → 0.`

Constraints: $1 \leq |\text{haystack}|, |\text{needle}| \leq 10^4$.

Hint: Iterate from index 0 to `haystack.length() - needle.length()`. At each position, check if the substring matches using `haystack.substring(i, i + needle.length()).equals(needle)` or a character-by-character comparison loop.

Practice 10: Intersection of Two Arrays (Unique Elements)

Specification: Given two integer arrays `nums1` and `nums2`, return an array of their unique intersection. Each element must appear at most once in the result.

Example: `nums1 = [1,2,2,1], nums2 = [2,2] → [2].`

Constraints: $1 \leq n \leq 1000$.

Hint: Add all elements of `nums1` into a `HashSet`. Iterate `nums2` and check membership. Use a second `HashSet` for the result to avoid duplicates.

Practice 11: Valid Anagram

Specification: Given two strings `s` and `t`, return `true` if `t` is an anagram of `s` (same characters, same frequencies, different order).

Example: `s = "anagram", t = "nagaram" → true.`

Constraints: $1 \leq |s|, |t| \leq 5 \times 10^4$.

Hint: Use an `int[26]` frequency array. Increment for characters in `s`, decrement for characters in `t`. If all counts are zero at the end, it is an anagram.

Practice 12: Remove All Adjacent Duplicates in String

Specification: Repeatedly remove adjacent pairs of equal characters until no more removals are possible. Return the final string.

Example: `"abbaca" → remove "bb" → "aaca" → remove "aa" → "ca".`

Constraints: $1 \leq |s| \leq 10^5$.

Hint: Use a `StringBuilder` as a stack. For each character, if it matches the last character in the builder, pop (delete last). Otherwise, append. Single pass, $\mathcal{O}(N)$.

Practice 13: Best Time to Buy and Sell Stock (Single Transaction)

Specification: Given array `prices` where `prices[i]` is the price on day `i`, find the maximum profit from buying on one day and selling on a later day. If no profit is possible, return 0.

Example: `[7, 1, 5, 3, 6, 4]` $\rightarrow$ 5 (buy at 1, sell at 6).

Constraints: $1 \leq n \leq 10^5$.

Hint: Track `minPriceSoFar` as you scan left-to-right. At each day, compute `profit = prices[i] - minPriceSoFar` and update `maxProfit`. Single pass, $\mathcal{O}(N)$.

Practice 14: Jewels and Stones

Specification: Given string `jewels` (types of jewel stones, each unique) and string `stones` (stones you have), count how many of your stones are jewels.

Example: `jewels = "aA"`, `stones = "aAAbbbb"` $\rightarrow$ 3.

Constraints: $1 \leq |\text{jewels}|, |\text{stones}| \leq 50$.

Hint: Put all jewel characters in a `HashSet`. Iterate through `stones`, count membership matches. $\mathcal{O}(J + S)$ time.

Practice 15: Squeeze Multiple Spaces to Single Space

Specification: Given a string with multiple consecutive spaces between words, replace each sequence of spaces with a single space. Trim leading and trailing spaces.

Example: `" hello world "` $\rightarrow$ `"hello world"`.

Constraints: $1 \leq |s| \leq 10^4$.

Hint: Use the Read/Write pattern on a `char[]`. Write a space only if the previous written character is not already a space. Skip leading spaces by initializing a flag or checking `write == 0`.

Practice 16: Check if Array is Sorted and Rotated

Specification: Given an array `nums`, return `true` if the array was originally sorted in non-decreasing order and then rotated some number of positions (including zero).

Example: `[3, 4, 5, 1, 2]` $\rightarrow$ `true`. `[2, 1, 3, 4]` $\rightarrow$ `false`.

Constraints: $1 \leq n \leq 100$.

Hint: Count the number of “descents” (positions where `nums[i] > nums[i+1]`, wrapping around to compare `nums[n-1]` with `nums[0]`). A valid sorted-and-rotated array has at most one descent.

Practice 17: Running Sum of 1D Array

Specification: Given array `nums`, return an array where `result[i] = sum(nums[0]..nums[i])`.

Example: `[1, 2, 3, 4]` $\rightarrow$ `[1, 3, 6, 10]`.

Constraints: $1 \leq n \leq 1000$.

Hint: Modify in-place: `nums[i] += nums[i-1]` for `i >= 1`. Single pass, $\mathcal{O}(N)$, $\mathcal{O}(1)$ extra space.

Practice 18: Count Common Characters in Array of Strings

Specification: Given an array of lowercase strings, find all characters that appear in every string (including duplicates). Return them as a list.

Example: `["bella", "label", "roller"]` $\rightarrow$ `["e", "l", "l"]`.

Constraints: $1 \leq n \leq 100$, $1 \leq |s_i| \leq 100$.

Hint: Use an `int[26]` initialized to `Integer.MAX_VALUE`. For each string, compute its own `int[26]` frequency, then take the element-wise minimum with the global array. The final array represents the minimum frequency of each character across all strings.

Practice 19: Maximum Consecutive Ones

Specification: Given a binary array `nums`, return the maximum number of consecutive 1s.

Example: `[1, 1, 0, 1, 1, 1]` $\rightarrow$ 3.

Constraints: $1 \leq n \leq 10^5$.

Hint: Track `currentStreak` and `maxStreak`. When `nums[i] == 1`, increment `currentStreak`. When `nums[i] == 0`, reset `currentStreak` to 0. Update `maxStreak` at each step.

Practice 20: Determine if String Halves Are Alike

Specification: A string `s` of even length is split into two halves. Return `true` if both halves contain the same number of vowels (`a`, `e`, `i`, `o`, `u` — case-insensitive).

Example: `"book"` $\rightarrow$ `true` (`"bo"` has 1 vowel, `"ok"` has 1 vowel).

Constraints: $2 \leq |s| \leq 1000$, `|s|` is even.

Hint: Count vowels in `s[0..n/2-1]` and `s[n/2..n-1]`. Compare counts. $\mathcal{O}(N)$.

Practice 21: Sign of the Product of an Array

Specification: Return 1 if the product of all elements is positive, -1 if negative, 0 if zero. Do not compute the actual product (it may overflow).

Example: `[-1, -2, -3, -4, 3, 2, 1]` $\rightarrow$ 1 (product is positive).

Constraints: $1 \leq n \leq 1000$.

Hint: If any element is 0, return 0. Otherwise, count negative numbers. If the count is even, the product is positive; if odd, negative.

Practice 22: Replace Elements with Greatest on Right Side

Specification: Given array `arr`, replace each element with the greatest element to its right. The last element should be replaced with `-1`.

Example: `[17, 18, 5, 4, 6, 1] → [18, 6, 6, 6, 1, -1]`.

Constraints: $1 \leq n \leq 10^4$.

Hint: Scan right-to-left tracking `maxSoFar`. At each position, the answer is `maxSoFar`, then update `maxSoFar = Math.max(maxSoFar, originalValue)`.

Practice 23: Sort Array by Parity

Specification: Rearrange array so all even elements come before all odd elements. Relative order within even/odd groups does not matter.

Example: `[3, 1, 2, 4] → [2, 4, 3, 1]` (any valid ordering).

Constraints: $1 \leq n \leq 5000$.

Hint: Read/Write pointer: write pointer tracks the next “even slot.” Swap `nums[write]` with `nums[read]` whenever `nums[read]` is even.

Practice 24: Convert Sorted Array to Binary Search Tree

Specification: Given a sorted integer array, create a height-balanced Binary Search Tree (BST).

Example: `[-10, -3, 0, 5, 9] → BST` with root 0, left subtree `[-10, -3]`, right subtree `[5, 9]`.

Constraints: $1 \leq n \leq 10^4$.

Hint: Recursive binary split. The middle element becomes the root. Left half becomes the left subtree; right half becomes the right subtree. Base case: empty range returns null.

Practice 25: Richest Customer Wealth

Specification: Given 2D array `accounts` where `accounts[i][j]` is the amount of money customer `i` has in bank `j`, return the wealth of the richest customer (sum of all bank balances).

Example: `[[1,2,3],[3,2,1]] → 6`.

Constraints: $1 \leq m, n \leq 50$.

Hint: For each customer, compute row sum. Track the maximum row sum. Two nested loops, $\mathcal{O}(m \times n)$.

Practice 26: Number of Good Pairs

Specification: Given array `nums`, count the number of pairs `(i, j)` where `i < j` and `nums[i] == nums[j]`.

Example: `[1, 2, 3, 1, 1, 3] → 4`.

Constraints: $1 \leq n \leq 100$, $1 \leq \text{nums}[i] \leq 100$.

Hint: Use a frequency array. For each number, if it has been seen c times before, it forms c new pairs. Increment count by c , then increment frequency.

Practice 27: Check if N and Its Double Exist

Specification: Given array `arr`, check if there exist two indices i and j such that $i \neq j$ and `arr[i] == 2 * arr[j]`.

Example: `[10, 2, 5, 3] → true` ($10 = 2 * 5$).

Constraints: $2 \leq n \leq 500$.

Hint: Use a `HashSet`. For each element, check if $2 * \text{num}$ or $\text{num} / 2$ (when num is even) is already in the set. Then add num to the set.

Practice 28: Truncate Sentence

Specification: Given a sentence s and integer k , truncate s to contain only the first k words.

Example: $s = \text{"Hello how are you Contestant"}$, $k = 4 \rightarrow \text{"Hello how are you"}$.

Constraints: $1 \leq |s| \leq 500$, $1 \leq k \leq \text{word count}$.

Hint: Scan character-by-character counting spaces. When the space count reaches k , return `s.substring(0, i)`. Or split and rejoin the first k tokens.

Practice 29: Cells in Range on an Excel Sheet

Specification: Given a string s representing a cell range like `"K1:L2"`, return all cells in the range in row-major order.

Example: `"K1:L2" → ["K1", "K2", "L1", "L2"]`.

Constraints: Column is a single uppercase letter, row is a single digit.

Hint: Parse start column, start row, end column, end row. Nested loop: outer on columns (`col = s.charAt(0)` to `s.charAt(3)`), inner on rows. Build each cell string.

Practice 30: Sum of Digits Until Single Digit

Specification: Given a non-negative integer num , repeatedly add its digits until the result is a single digit. Return that digit.

Example: $38 \rightarrow 3 + 8 = 11 \rightarrow 1 + 1 = 2$. Return 2.

Constraints: $0 \leq \text{num} \leq 2^{31} - 1$.

Hint: Iterative approach: extract digits with $\text{num} \% 10$, accumulate sum, reduce with $\text{num} = \text{sum}$. Repeat until $\text{num} < 10$. Mathematical shortcut: Digital Root formula $1 + (\text{num} - 1) \% 9$ for $\mathcal{O}(1)$.

Chapter 10

Q2 Mastery — 2D Matrix Traversal, Grid Simulations, and State Machine Processing

This chapter covers Q2 of the CodeSignal GCA (Medium difficulty, ~15 minutes target time). Q2 tests multidimensional array processing, grid boundary control, BFS/DFS flood fill, and step-by-step state machine simulation.

10.1 Section 1: Essential Terminology & Vocabulary

- **Row-Major vs Column-Major layout:** Row-major layout stores 2D arrays row by row in memory (used in Java, C/C++), while column-major stores them column by column (Fortran, MATLAB). In Java, `matrix[r][c]` means row `r`, column `c`. Traversing row-major arrays by row is cache-friendly and faster.
- **In-Place Matrix Transposition:** The process of flipping a matrix over its main diagonal without allocating a new matrix. Mathematical formula: $A^T[i][j] = A[j][i]$. For an $N \times N$ matrix, iterate `i` from 0 to `N-1` and `j` from `i+1` to `N-1`, swapping `matrix[i][j]` and `matrix[j][i]`.
- **90-Degree Clockwise/Counter-Clockwise Rotation Theorem:** Rotating a grid 90° can be done with two simpler operations. Clockwise: Transpose the matrix, then reverse each row. Counter-Clockwise: Transpose the matrix, then reverse each column.
- **Spiral Matrix Boundary Contraction:** A traversal technique using four pointer boundaries (`top`, `bottom`, `left`, `right`). We traverse the perimeter, then shrink the boundaries (e.g., `top++`, `right--`) and repeat until the boundaries overlap.
- **Coordinate Direction Vectors:** Pre-defined arrays to cleanly iterate through grid neighbors. Standard 4-directional setup: `int[] dr = {-1, 1, 0, 0}; int[] dc = {0, 0, -1, 1};`. This prevents writing four repetitive `if` statements for North, South, West, East.
- **Flood Fill / BFS vs DFS on grids:** Techniques to traverse connected components in a matrix. DFS uses recursion (call stack) to go deep, which is easier to write but can cause stack overflow on massive grids. BFS uses a `Queue` to process level-by-level, ideal for shortest path calculations.
- **2D Prefix Sum:** A precomputation technique where `prefix[i][j]` stores the sum of all

elements in the submatrix from (0,0) to (i-1,j-1). Allows answering arbitrary submatrix sum queries in $\mathcal{O}(1)$ time using inclusion-exclusion.

- **State Machine Simulation:** Problems where you process a sequence of commands or instructions step-by-step. Often requires maintaining a “current state” (e.g., direction, coordinate, phase) and applying transition logic based on the input stream.
- **Toeplitz Matrix:** A matrix in which every diagonal descending from left to right has constant values. Property to check: `matrix[i][j] == matrix[i-1][j-1]` for all valid $i > 0, j > 0$.
- **In-Place State Encoding:** A trick to update states simultaneously without a copy grid. We use bits or sentinel values to encode both “old state” and “new state” (e.g., 0=dead, 1=live, 2=was dead now live, 3=was live now dead) and later decode it with modulo/division.

10.1.1 Row-Major Index Linearization

This technique converts 2D coordinates into a 1D index using `index = r * cols + c`. It can also reverse the process using `r = index / cols` and `c = index % cols`. Why it matters: It is needed for matrix reshape operations and binary searching in a sorted matrix.

10.1.2 BFS Level-by-Level Tracking

This approach uses an inner loop based on `int size = queue.size()` inside the standard BFS `while` loop. This ensures the algorithm processes one full level of nodes before advancing depth. Why it matters: It is crucial for calculating minimum steps, rotting oranges, and shortest path problems.

10.1.3 Visited Set vs In-Place Marking

This evaluates the trade-off between allocating a separate `boolean[][] visited` array and modifying grid cells directly, such as setting `grid[r][c] = '#'`. In-place marking avoids extra memory allocation but destroys the original matrix. Why it matters: In-place marking saves memory but mutates the input, which is a key discussion point in interviews.

10.1.4 Diagonal Traversal Pattern

This property states that elements sharing the same $r + c$ sum belong to the same anti-diagonal. Conversely, elements sharing the same $r - c$ difference are on the same main diagonal. Why it matters: This pattern is key for zigzag traversals and Toeplitz matrix verification.

10.1.5 Boundary Validation Helper

This is the practice of extracting boundary logic into a separate `boolean inBounds(r, c, rows, cols)` utility method. It centralizes coordinate checks during grid traversal. Why it matters: It eliminates repetitive boundary checks and significantly reduces bugs in grid BFS/DFS.

10.1.6 Multi-Source BFS

Instead of running BFS individually from each source, this technique seeds the initial queue with ALL starting positions simultaneously. The search then expands outwards concurrently from multiple origins. Why it matters: It solves rotting oranges and walls-and-gates problems in a single, highly efficient BFS pass.

10.2 Section 2: Reusable Code Templates

10.2.1 Template A: Spiral Boundary Traversal

```
int top = 0, bottom = matrix.length - 1;
int left = 0, right = matrix[0].length - 1;
while (top <= bottom && left <= right) {
    for (int j = left; j <= right; j++) { /* process matrix[top][j] */ }
    top++;
    for (int i = top; i <= bottom; i++) { /* process matrix[i][right] */ }
    right--;
    if (top <= bottom) {
        for (int j = right; j >= left; j--) { /* process matrix[bottom][j] */ }
        bottom--;
    }
    if (left <= right) {
        for (int i = bottom; i >= top; i--) { /* process matrix[i][left] */ }
        left++;
    }
}
```

10.2.2 Template B: 4-Directional BFS/DFS Grid Walk

```
int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};

void dfs(int[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] ==
        -1) return;
    grid[r][c] = -1; // mark visited
    for (int i = 0; i < 4; i++) {
        dfs(grid, r + dr[i], c + dc[i]);
    }
}
```

10.2.3 Template C: 2D Prefix Sum Construction + Query

```
// Construction
int[][] sum = new int[R + 1][C + 1];
for (int r = 1; r <= R; r++) {
    for (int c = 1; c <= C; c++) {
        sum[r][c] = matrix[r-1][c-1] + sum[r-1][c] + sum[r][c-1] - sum[r-1][c-1];
    }
}

// Query from (r1, c1) to (r2, c2)
int query(int r1, int c1, int r2, int c2) {
    return sum[r2+1][c2+1] - sum[r1][c2+1] - sum[r2+1][c1] + sum[r1][c1];
}
```

10.3 Section 3: Solved Exemplar Problems

10.3.1 1. Rotate Matrix 90° Clockwise

Specification: You are given an $n \times n$ 2D matrix representing an image. Rotate the image by 90 degrees (clockwise) in-place. **Example:** Input: $[[1,2],[3,4]]$ -> Output: $[[3,1],[4,2]]$ **Pattern:** Transpose + Reverse Rows. **Explanation:** Rotating 90 degrees clockwise is mathematically equivalent to transposing the matrix (swapping i, j with j, i) and then reversing the elements of each row. This avoids needing complex 4-way coordinate swaps.

```
public void rotate(int[][] matrix) {
    int n = matrix.length;
    // Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse each row
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n / 2; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[i][n - 1 - j];
            matrix[i][n - 1 - j] = temp;
        }
    }
}
// Time:  $\mathcal{O}(N^2)$  / Space:  $\mathcal{O}(1)$ 
```

10.3.2 2. Spiral Matrix Traversal

Specification: Given an $m \times n$ matrix, return all elements of the matrix in spiral order. **Example:** Input: $[[1,2,3],[4,5,6],[7,8,9]]$ -> Output: $[1,2,3,6,9,8,7,4,5]$ **Pattern:** Boundary Contraction. **Explanation:** Maintain top, bottom, left, right pointers. Traverse the top row, increment top. Traverse right col, decrement right. Traverse bottom row (if top <= bottom), decrement bottom. Traverse left col (if left <= right), increment left.

```
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    int t = 0, b = matrix.length - 1, l = 0, r = matrix[0].length - 1;
    while (t <= b && l <= r) {
        for (int j = l; j <= r; j++) res.add(matrix[t][j]); // Top
        t++;
        for (int i = t; i <= b; i++) res.add(matrix[i][r]); // Right
        r--;
        for (int j = r; j >= l; j--) res.add(matrix[b][j]); // Bottom
        b--;
        for (int i = b; i >= t; i--) res.add(matrix[i][l]); // Left
        l++;
    }
    return res;
}
```



```

    r--;
    if (t <= b) {
        for (int j = r; j >= 1; j--) res.add(matrix[b][j]); // Bottom
        b--;
    }
    if (l <= r) {
        for (int i = b; i >= t; i--) res.add(matrix[i][l]); // Left
        l++;
    }
}
return res;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.3 3. Set Matrix Zeros

Specification: Given an $m \times n$ integer matrix, if an element is 0, set its entire row and column to 0's in-place. **Example:** Input: $[[1,1,1],[1,0,1],[1,1,1]]$ -> Output: $[[1,1,1],[0,0,0],[1,1,1]]$ **Pattern:** In-Place State Encoding (using first row/col as markers). **Explanation:** We use the first row and first column to store information about whether that row or column should be zeroed out. We need a separate variable for the first column to avoid overlapping state.

```

public void setZeroes(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    boolean firstColZero = false;
    // Mark zeros on first row/col
    for (int i = 0; i < m; i++) {
        if (matrix[i][0] == 0) firstColZero = true;
        for (int j = 1; j < n; j++) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }
    // Zero out based on marks
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;
        }
    }
    // Handle first row/col specifically
    if (matrix[0][0] == 0) {
        for (int j = 0; j < n; j++) matrix[0][j] = 0;
    }
    if (firstColZero) {
        for (int i = 0; i < m; i++) matrix[i][0] = 0;
    }
}

```

```

    }
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.4 4. Diagonal Matrix Traversal

Specification: Given an $m \times n$ matrix, return an array of all its elements arranged in a diagonal zigzag order. **Example:** Input: $[[1,2,3],[4,5,6],[7,8,9]]$ -> Output: $[1,2,4,7,5,3,6,8,9]$

Pattern: Zigzag Direction Switching. **Explanation:** In a diagonal traversal, the sum of indices $(i+j)$ is constant for each diagonal. For even sums, we move Up-Right. For odd sums, we move Down-Left. Boundary conditions handle when we hit the edges.

```

public int[] findDiagonalOrder(int[][] mat) {
    int m = mat.length, n = mat[0].length;
    int[] res = new int[m * n];
    int r = 0, c = 0;
    for (int i = 0; i < m * n; i++) {
        res[i] = mat[r][c];
        if ((r + c) % 2 == 0) { // Moving Up-Right
            if (c == n - 1) r++;
            else if (r == 0) c++;
            else { r--; c++; }
        } else { // Moving Down-Left
            if (r == m - 1) c++;
            else if (c == 0) r++;
            else { r++; c--; }
        }
    }
    return res;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.5 5. Matrix Reshape Validation

Specification: In MATLAB, `reshape` changes an $m \times n$ matrix into an $r \times c$ matrix. If impossible, return original. Otherwise, fill row by row. **Example:** Input: `mat = [[1,2],[3,4]]`, `r = 1`, `c = 4` -> Output: $[[1,2,3,4]]$ **Pattern:** Row-Major Index Mapping. **Explanation:** A 2D matrix can be flattened logically. The 1D index k maps to 2D coordinates $(k / \text{cols}, k \% \text{cols})$. We map the original matrix into the new shape using a single counter k .

```

public int[][] matrixReshape(int[][] mat, int r, int c) {
    int m = mat.length, n = mat[0].length;
    if (m * n != r * c) return mat; // Invalid shape

    int[][] res = new int[r][c];
    for (int i = 0; i < m * n; i++) {
        res[i / c][i % c] = mat[i / n][i % n];
    }
}

```

```

    }
    return res;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(R \times C)$ 

```

10.3.6 6. Rotate Matrix 90° Counter-Clockwise

Specification: Rotate an $N \times N$ matrix by 90 degrees counter-clockwise in place. **Example:** Input: $[[1,2],[3,4]] \rightarrow$ Output: $[[2,4],[1,3]]$ **Pattern:** Transpose + Reverse Columns. **Explanation:** Counter-clockwise rotation is similar to clockwise. We transpose first, then reverse the columns (top to bottom swap) instead of rows.

```

public void rotateCounter(int[] [] matrix) {
    int n = matrix.length;
    // Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse each column
    for (int j = 0; j < n; j++) {
        for (int i = 0; i < n / 2; i++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[n - 1 - i][j];
            matrix[n - 1 - i][j] = temp;
        }
    }
}
// Time:  $\mathcal{O}(N^2)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.7 7. Search in Row-Column Sorted Matrix

Specification: Write an efficient algorithm that searches for a value in an $m \times n$ matrix where each row and column is sorted in ascending order. **Example:** Input: $\text{mat} = [[1,4],[2,5]]$, $\text{target} = 2 \rightarrow$ Output: **true** **Pattern:** Staircase Search from Top-Right. **Explanation:** Start at the top-right corner. If target is smaller than the current value, it can't be in this column (move left). If target is larger, it can't be in this row (move down).

```

public boolean searchMatrix(int[] [] matrix, int target) {
    int r = 0, c = matrix[0].length - 1;
    while (r < matrix.length && c >= 0) {
        if (matrix[r][c] == target) return true;
        else if (matrix[r][c] > target) c--;
        else r++;
    }
}

```

```

    }
    return false;
}
// Time:  $\mathcal{O}(M + N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.8 8. Game of Life

Specification: Given a board of 0s (dead) and 1s (live), compute the next state based on Conway's Game of Life rules simultaneously. **Example:** Rules: <2 neighbors dies, 2-3 lives, >3 dies. Dead with 3 lives. **Pattern:** In-Place State Encoding. **Explanation:** To update in-place without a copy, encode transitions. Let 2 mean "was dead, now live", and -1 mean "was live, now dead". When counting neighbors, check if `abs(val) == 1`. After updating all, decode the states.

```

public void gameOfLife(int[][] board) {
    int m = board.length, n = board[0].length;
    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            int live = 0;
            for (int i = -1; i <= 1; i++) {
                for (int j = -1; j <= 1; j++) {
                    if (i == 0 && j == 0) continue;
                    int nr = r + i, nc = c + j;
                    if (nr >= 0 && nr < m && nc >= 0 && nc < n && Math.abs(board[nr][nc])
                        == 1) live++;
                }
            }
            if (board[r][c] == 1 && (live < 2 || live > 3)) board[r][c] = -1;
            if (board[r][c] == 0 && live == 3) board[r][c] = 2;
        }
    }
    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            if (board[r][c] > 0) board[r][c] = 1;
            else board[r][c] = 0;
        }
    }
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.9 9. Toeplitz Matrix Verification

Specification: Given an $m \times n$ matrix, return true if the matrix is Toeplitz. A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements. **Example:** Input: `[[1,2],[3,1]]` -> Output: `true` **Pattern:** Matrix Traversal Property. **Explanation:** Simply check every cell `matrix[i][j]` against its top-left neighbor `matrix[i-1][j-1]`. If they mismatch, return false.

```

public boolean isToeplitzMatrix(int[] [] matrix) {
    for (int i = 1; i < matrix.length; i++) {
        for (int j = 1; j < matrix[0].length; j++) {
            if (matrix[i][j] != matrix[i-1][j-1]) {
                return false;
            }
        }
    }
    return true;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.10 10. Spiral Matrix Construction

Specification: Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order. **Example:** Input: $n = 3$ -> Output: $[[1,2,3],[8,9,4],[7,6,5]]$ **Pattern:** Boundary Contraction (Write mode). **Explanation:** Similar to spiral traversal, but instead of reading, we write an incrementing counter $val++$ into the boundaries, contracting inwards until we fill n^2 elements.

```

public int[] [] generateMatrix(int n) {
    int[] [] mat = new int[n][n];
    int t = 0, b = n - 1, l = 0, r = n - 1;
    int val = 1;
    while (t <= b && l <= r) {
        for (int j = l; j <= r; j++) mat[t][j] = val++;
        t++;
        for (int i = t; i <= b; i++) mat[i][r] = val++;
        r--;
        if (t <= b) {
            for (int j = r; j >= l; j--) mat[b][j] = val++;
            b--;
        }
        if (l <= r) {
            for (int i = b; i >= t; i--) mat[i][l] = val++;
            l++;
        }
    }
    return mat;
}
// Time:  $\mathcal{O}(N^2)$  / Space:  $\mathcal{O}(N^2)$ 

```

10.3.11 11. Flood Fill

Specification: An image is an $m \times n$ grid. Perform a flood fill starting from (sr, sc) replacing the connected old color with a color. **Example:** Input: $img = [[1,1,1],[1,1,0],[1,0,1]]$,

sr=1,sc=1, color=2 -> Output: `[[2,2,2],[2,2,0],[2,0,1]]` **Pattern:** DFS Recursive 4-Directional. **Explanation:** We check if the starting pixel is already the target color. If not, we recursively replace all adjacent cells of the original color with the new color using DFS.

```
public int[][] floodFill(int[][] image, int sr, int sc, int color) {
    if (image[sr][sc] != color) {
        dfs(image, sr, sc, image[sr][sc], color);
    }
    return image;
}

private void dfs(int[][] img, int r, int c, int oldC, int newC) {
    if (r < 0 || r >= img.length || c < 0 || c >= img[0].length || img[r][c] !=
        ↪ oldC) return;
    img[r][c] = newC; // mark and fill
    dfs(img, r-1, c, oldC, newC);
    dfs(img, r+1, c, oldC, newC);
    dfs(img, r, c-1, oldC, newC);
    dfs(img, r, c+1, oldC, newC);
}

// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 
```

10.3.12 12. Transpose Rectangular Matrix

Specification: Given a 2D integer array matrix, return the transpose of matrix. Matrix may not be square. **Example:** Input: `[[1,2,3],[4,5,6]]` -> Output: `[[1,4],[2,5],[3,6]]` **Pattern:** Allocation + Row-Major Mapping. **Explanation:** Since the matrix isn't square, we cannot transpose in place. We allocate a new matrix of size $C \times R$, and assign `ans[j][i] = matrix[i][j]`.

```
public int[][] transpose(int[][] matrix) {
    int r = matrix.length;
    int c = matrix[0].length;
    int[][] ans = new int[c][r];
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            ans[j][i] = matrix[i][j];
        }
    }
    return ans;
}

// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 
```

10.3.13 13. Valid Sudoku

Specification: Determine if a 9×9 Sudoku board is valid. Only the filled cells need to be validated according to standard rules. **Example:** Input: Standard Sudoku grid with duplicates in row 1 -> Output: `false` **Pattern:** HashSet Encoding Trick. **Explanation:** We iterate through the grid.

For each cell, we create a string encoding its presence in its row, column, and block (e.g., "5 in row 0"). If `HashSet.add()` returns false, a duplicate exists.

```
public boolean isValidSudoku(char[][] board) {
    Set<String> seen = new HashSet<>();
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            char number = board[i][j];
            if (number != '.') {
                if (!seen.add(number + " in row " + i) ||
                    !seen.add(number + " in col " + j) ||
                    !seen.add(number + " in block " + i/3 + "-" + j/3))
                    return false;
            }
        }
    }
    return true;
}
// Time:  $\mathcal{O}(1)$  (fixed 9x9) / Space:  $\mathcal{O}(1)$ 
```

10.3.14 14. Island Perimeter

Specification: You are given row x col grid representing a map where 1 is land and 0 is water. Calculate the perimeter of the island. **Example:** Input: `[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]` -> Output: 16 **Pattern:** Neighbor Subtraction. **Explanation:** Each land cell adds 4 to the perimeter. For each land cell, we check its left and top neighbors. If they are also land, they share an edge, meaning we subtract 2 from the total perimeter (1 for each cell).

```
public int islandPerimeter(int[][] grid) {
    int perimeter = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == 1) {
                perimeter += 4;
                if (i > 0 && grid[i - 1][j] == 1) perimeter -= 2;
                if (j > 0 && grid[i][j - 1] == 1) perimeter -= 2;
            }
        }
    }
    return perimeter;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 
```

10.3.15 15. Maximum $K \times K$ Submatrix Sum

Specification: Given an $M \times N$ matrix and integer K , find the max sum of a contiguous $K \times K$ submatrix. **Example:** Input: `mat=[[1,2],[3,4]]`, `K=1` -> Output: 4 **Pattern:** 2D Prefix Sum. **Explanation:** Construct a 2D prefix sum array. Then iterate through all possible bottom-right corners (i,j) of size $K \times K$, extracting the sum in $\mathcal{O}(1)$ time.

```
public int maxSum(int[] [] mat, int k) {
    int m = mat.length, n = mat[0].length;
    int[] [] pre = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            pre[i][j] = mat[i-1][j-1] + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1];
        }
    }
    int max = Integer.MIN_VALUE;
    for (int i = k; i <= m; i++) {
        for (int j = k; j <= n; j++) {
            int sum = pre[i][j] - pre[i-k][j] - pre[i][j-k] + pre[i-k][j-k];
            max = Math.max(max, sum);
        }
    }
    return max;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 
```

10.3.16 16. Number of Islands

Specification: Given an $m \times n$ grid of '1's (land) and '0's (water), return the number of islands (connected components). **Example:** Input: `[["1","1","0"],["0","0","1"]]` -> Output: 2 **Pattern:** BFS/DFS Connected Components. **Explanation:** Iterate over every cell. When a '1' is found, increment the island count, and launch a DFS/BFS to mark all connected '1's as '0' to avoid recounting.

```
public int numIslands(char[] [] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j);
            }
        }
    }
    return count;
}

private void dfs(char[] [] grid, int r, int c) {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] == '0') return;
    ↪ '0') return;
```



```

    grid[r][c] = '0';
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 

```

10.3.17 17. Flip and Invert Image

Specification: Given an $n \times n$ binary matrix, flip the image horizontally, then invert it. Flipping means reversing the row. Inverting means changing 0 to 1 and 1 to 0. **Example:** Input: `[[1,1,0]]` -> Output: `[[1,0,0]]` **Pattern:** Two-Pointer XOR + Reverse. **Explanation:** In a single pass per row, we can use two pointers `i` and `j`. We assign `row[i] = row[j] ^ 1` and `row[j] = temp ^ 1`. Note the middle element when length is odd.

```

public int[][] flipAndInvertImage(int[][] image) {
    for (int[] row : image) {
        int left = 0, right = row.length - 1;
        while (left <= right) {
            int temp = row[left] ^ 1;
            row[left] = row[right] ^ 1;
            row[right] = temp;
            left++; right--;
        }
    }
    return image;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.3.18 18. Shift 2D Grid

Specification: Given a 2D grid of size $m \times n$ and an integer `k`, shift the grid `k` times. Shifting means element at `(i,j)` moves to `(i, j+1)`, last column moves to next row, bottom-right moves to `(0,0)`. **Example:** Input: `[[1,2],[3,4]]`, `k=1` -> Output: `[[4,1],[2,3]]` **Pattern:** Modular Index Arithmetic (1D Flattening). **Explanation:** Map the grid to a 1D array conceptually of size $M \times N$. The new position of an element at index `i` is `(i + k) % (M * N)`. We can construct a new result grid based on this mapping.

```

public List<List<Integer>> shiftGrid(int[][] grid, int k) {
    int m = grid.length, n = grid[0].length;
    int total = m * n;
    k %= total;
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < m; i++) {
        res.add(new ArrayList<>(Collections.nCopies(n, 0)));
    }
    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {

```

```

        int new1D = (r * n + c + k) % total;
        res.get(new1D / n).set(new1D % n, grid[r][c]);
    }
}
return res;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 

```

10.3.19 19. Word Search in Grid

Specification: Given an $m \times n$ grid of characters and a word, return true if the word exists. The word can be constructed from letters of sequentially adjacent cells (horizontally or vertically). **Example:** Input: `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word="ABCCED" -> Output: true **Pattern:** DFS Backtracking. **Explanation:** Iterate over all cells. If the first character matches, launch DFS. Temporarily mark cells (e.g., #) during recursion to prevent reuse, and restore them after the recursive call returns.

```

public boolean exist(char[][] board, String word) {
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            if (dfs(board, i, j, word, 0)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] b, int r, int c, String word, int idx) {
    if (idx == word.length()) return true;
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || b[r][c] !=
        word.charAt(idx)) return false;
    char temp = b[r][c];
    b[r][c] = '#';
    boolean found = dfs(b, r+1, c, word, idx+1) || dfs(b, r-1, c, word, idx+1) ||
        dfs(b, r, c+1, word, idx+1) || dfs(b, r, c-1, word, idx+1);
    b[r][c] = temp;
    return found;
}
// Time:  $\mathcal{O}(M \times N \times 4^L)$  / Space:  $\mathcal{O}(L)$ 

```

10.3.20 20. Determine If Matrix Can Be Obtained By Rotation

Specification: Given two $n \times n$ binary matrices `mat` and `target`, return true if it is possible to make `mat` equal to `target` by rotating `mat` in 90-degree increments. **Example:** Input: `mat = [[0,1],[1,0]]`, `target = [[1,0],[0,1]]` -> Output: true **Pattern:** Multiple Rotation Validation. **Explanation:** A matrix can be rotated at most 3 times (90, 180, 270 degrees). We compare `mat` to `target` up to 4 times, rotating `mat` by 90 degrees each time.

```

public boolean findRotation(int[][] mat, int[][] target) {

```

```

    for (int k = 0; k < 4; k++) {
        if (Arrays.deepEquals(mat, target)) return true;
        rotate(mat); // uses function from Problem 1
    }
    return false;
}

private void rotate(int[][] mat) {
    int n = mat.length;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int t = mat[i][j]; mat[i][j] = mat[j][i]; mat[j][i] = t;
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n/2; j++) {
            int t = mat[i][j]; mat[i][j] = mat[i][n-1-j]; mat[i][n-1-j] = t;
        }
    }
}

// Time:  $\mathcal{O}(N^2)$  / Space:  $\mathcal{O}(1)$ 

```

10.4 Section 4: CodeSignal Classic Q2 Patterns (10 Additional Solved Problems)

10.4.1 21. Chess Board Cell Color

Specification: Given two cell strings on a standard chessboard (e.g. "A1", "C3"), determine if they are the same color. **Example:** Input: cell1 = "A1", cell2 = "C3" -> Output: true **Pattern:** Parity Check. **Explanation:** Convert the column letter and row number to integers. The color of a cell (x, y) is uniquely determined by $(x + y) \% 2$. Compare the parity.

```

public boolean solution(String cell1, String cell2) {
    int sum1 = (cell1.charAt(0) - 'A') + (cell1.charAt(1) - '1');
    int sum2 = (cell2.charAt(0) - 'A') + (cell2.charAt(1) - '1');
    return (sum1 % 2) == (sum2 % 2);
}

// Time:  $\mathcal{O}(1)$  / Space:  $\mathcal{O}(1)$ 

```

10.4.2 22. Minesweeper Click Reveal

Specification: Given a Minesweeper board and a click coordinate, if it's a mine 'M', turn to 'X'. If empty 'E' with no adjacent mines, turn to 'B' and recursively reveal neighbors. If empty with mines, turn to digit. **Example:** Input: board=[['E','E'],['E','M']], click=[0,0] -> Output: [['1','1'],['1','M']] **Pattern:** BFS/DFS Simulation with 8 Directions. **Explanation:**

Count adjacent mines (8 directions). If > 0 , set to digit. If $= 0$, set to 'B' and DFS to 8 adjacent 'E' neighbors.

```
public char[][] updateBoard(char[][] board, int[] click) {
    int r = click[0], c = click[1];
    if (board[r][c] == 'M') {
        board[r][c] = 'X';
        return board;
    }
    dfs(board, r, c);
    return board;
}

private void dfs(char[][] b, int r, int c) {
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || b[r][c] != 'E')
        ↪ return;
    int mines = 0;
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            int nr = r + i, nc = c + j;
            if (nr >= 0 && nr < b.length && nc >= 0 && nc < b[0].length && b[nr][nc] ==
                ↪ 'M') mines++;
        }
    }
    if (mines > 0) {
        b[r][c] = (char)(mines + '0');
    } else {
        b[r][c] = 'B';
        for (int i = -1; i <= 1; i++) {
            for (int j = -1; j <= 1; j++) dfs(b, r+i, c+j);
        }
    }
}

// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 
```

10.4.3 23. Battleship Placement Validation

Specification: Given an $m \times n$ matrix where 'X' are ships and '.' are water. Count valid battleships. They can only be placed horizontally or vertically. Ships are separated by at least one cell.

Example: Input: `[["X",".",".","X"],[".",".",".","X"]]` -> Output: 2 **Pattern:** Top-Left Identifier Traversal. **Explanation:** Instead of a full DFS, just count the “top-left” cell of every battleship. A cell is a top-left if it is 'X' and has no 'X' above or to the left of it.

```
public int countBattleships(char[][] board) {
    int count = 0;
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            if (board[i][j] == 'X') {
                if (i > 0 && board[i-1][j] == 'X') continue;
            }
        }
    }
}
```

```

        if (j > 0 && board[i][j-1] == 'X') continue;
        count++;
    }
}
}
return count;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(1)$ 

```

10.4.4 24. Box Blur

Specification: Apply a box blur algorithm to an image. Each pixel in the blurred image is the average of a 3×3 block centered at that pixel (rounded down). **Example:** Input: 3×3 matrix. Output: 1×1 matrix with average. **Pattern:** Sliding Window Matrix Accumulation. **Explanation:** The output matrix size is $(M - 2) \times (N - 2)$. We iterate over these valid centers and compute the sum of the 3×3 area.

```

public int[][] boxBlur(int[][] image) {
    int m = image.length, n = image[0].length;
    int[][] res = new int[m-2][n-2];
    for (int i = 1; i < m - 1; i++) {
        for (int j = 1; j < n - 1; j++) {
            int sum = 0;
            for (int di = -1; di <= 1; di++) {
                for (int dj = -1; dj <= 1; dj++) {
                    sum += image[i + di][j + dj];
                }
            }
            res[i-1][j-1] = sum / 9;
        }
    }
    return res;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 

```

10.4.5 25. Zigzag String Conversion

Specification: The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows. Read line by line to return the result. **Example:** Input: s = "PAYPALISHIRING", numRows = 3 -> Output: "PAHNAPLSIIGYIR" **Pattern:** Simulation with Direction Vector. **Explanation:** Maintain a row index and a direction. Add characters to `StringBuilder[]` corresponding to each row. When hitting top or bottom row, reverse direction.

```

public String convert(String s, int numRows) {
    if (numRows == 1) return s;
    StringBuilder[] rows = new StringBuilder[Math.min(numRows, s.length())];
    for (int i = 0; i < rows.length; i++) rows[i] = new StringBuilder();

```

```

int curRow = 0;
boolean goingDown = false;
for (char c : s.toCharArray()) {
    rows[curRow].append(c);
    if (curRow == 0 || curRow == numRows - 1) goingDown = !goingDown;
    curRow += goingDown ? 1 : -1;
}

StringBuilder ret = new StringBuilder();
for (StringBuilder row : rows) ret.append(row);
return ret.toString();
}
// Time:  $\mathcal{O}(N)$  / Space:  $\mathcal{O}(N)$ 

```

10.4.6 26. Simulate Robot Commands on Grid

Specification: A robot is on a (0,0) facing North. It receives commands: -2 (turn left), -1 (turn right), 1..9 (move forward). There are obstacles. Find max distance squared from origin. **Example:** Input: commands = [4,-1,3], obstacles = [] -> Output: 25 **Pattern:** State Machine Simulation (Direction Matrix). **Explanation:** Encode North, East, South, West using dx and dy. Turn right is $\text{dir} = (\text{dir} + 1) \% 4$. Move step by step checking against an obstacle HashSet.

```

public int robotSim(int[] commands, int[][] obstacles) {
    int[] dx = {0, 1, 0, -1}, dy = {1, 0, -1, 0};
    Set<String> obs = new HashSet<>();
    for (int[] o : obstacles) obs.add(o[0] + "," + o[1]);

    int x = 0, y = 0, dir = 0, maxDist = 0;
    for (int cmd : commands) {
        if (cmd == -2) dir = (dir + 3) % 4;
        else if (cmd == -1) dir = (dir + 1) % 4;
        else {
            for (int k = 0; k < cmd; k++) {
                int nx = x + dx[dir], ny = y + dy[dir];
                if (obs.contains(nx + "," + ny)) break;
                x = nx; y = ny;
                maxDist = Math.max(maxDist, x*x + y*y);
            }
        }
    }
    return maxDist;
}
// Time:  $\mathcal{O}(C + O)$  / Space:  $\mathcal{O}(O)$ 

```

10.4.7 27. Matrix Water Flow (Pacific Atlantic)

Specification: Grid representing island heights. Pacific touches left/top, Atlantic touches right/bottom. Find coordinates where water can flow to BOTH oceans (must go to equal or lower height). **Example:** Input: `[[1,2],[3,1]]` -> Output: `[[0,1],[1,0]]` **Pattern:** Reverse Multi-Source DFS. **Explanation:** Instead of going downhill from every cell, go UPHILL from the ocean borders to mark reachable cells. Intersection of Pacific-reachable and Atlantic-reachable is the answer.

```
public List<List<Integer>> pacificAtlantic(int[][] heights) {
    int m = heights.length, n = heights[0].length;
    boolean[][] pac = new boolean[m][n], atl = new boolean[m][n];
    for (int i = 0; i < m; i++) { dfs(heights, pac, i, 0); dfs(heights, atl, i,
        ↪ n-1); }
    for (int j = 0; j < n; j++) { dfs(heights, pac, 0, j); dfs(heights, atl, m-1,
        ↪ j); }

    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (pac[i][j] && atl[i][j]) res.add(Arrays.asList(i, j));
        }
    }
    return res;
}

private void dfs(int[][] h, boolean[][] v, int r, int c) {
    v[r][c] = true;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr>=0 && nr<h.length && nc>=0 && nc<h[0].length && !v[nr][nc] &&
            ↪ h[nr][nc] >= h[r][c])
            dfs(h, v, nr, nc);
    }
}

// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 
```

10.4.8 28. Rotting Oranges

Specification: 0=empty, 1=fresh orange, 2=rotten. Every minute, fresh oranges adjacent to rotten ones become rotten. Return min minutes to rot all, or -1. **Example:** Input: `[[2,1,1],[1,1,0],[0,1,1]]` -> Output: 4 **Pattern:** Multi-Source BFS. **Explanation:** Add all initially rotten oranges to a queue. Use BFS level-by-level to rot adjacent oranges. Track minutes. Finally, check if any fresh oranges remain.

```
public int orangesRotting(int[][] grid) {
    Queue<int[]> q = new LinkedList<>();
    int fresh = 0, m = grid.length, n = grid[0].length;
```

```

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (grid[i][j] == 2) q.offer(new int[]{i, j});
        else if (grid[i][j] == 1) fresh++;
    }
}
if (fresh == 0) return 0;
int mins = 0;
int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
while (!q.isEmpty()) {
    int size = q.size();
    boolean rotted = false;
    for (int k = 0; k < size; k++) {
        int[] curr = q.poll();
        for (int[] d : dirs) {
            int r = curr[0] + d[0], c = curr[1] + d[1];
            if (r >= 0 && r < m && c >= 0 && c < n && grid[r][c] == 1) {
                grid[r][c] = 2; fresh--;
                q.offer(new int[]{r, c});
                rotted = true;
            }
        }
    }
    if (rotted) mins++;
}
return fresh == 0 ? mins : -1;
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 

```

10.4.9 29. Surrounded Regions

Specification: Given a grid of 'X' and 'O', capture all regions surrounded by 'X' by flipping 'O' to 'X'. A region is surrounded if no 'O' is on the border. **Example:** Input: `[['X','X','X'],['X','O','X'],['X','X','X']]` -> Output: all 'X'. **Pattern:** Border DFS. **Explanation:** Any 'O' connected to a border 'O' cannot be captured. DFS from all border 'O's and mark them as safe ('#'). Flip all remaining 'O' to 'X', then revert '#' to 'O'.

```

public void solve(char[][] board) {
    int m = board.length, n = board[0].length;
    for (int i = 0; i < m; i++) { dfs(board, i, 0); dfs(board, i, n-1); }
    for (int j = 0; j < n; j++) { dfs(board, 0, j); dfs(board, m-1, j); }

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j] == 'O') board[i][j] = 'X';
            else if (board[i][j] == '#') board[i][j] = 'O';
        }
    }
}

```



```

}
private void dfs(char[][] b, int r, int c) {
    if (r<0 || r>=b.length || c<0 || c>=b[0].length || b[r][c] != '0') return;
    b[r][c] = '#';
    dfs(b, r+1, c); dfs(b, r-1, c); dfs(b, r, c+1); dfs(b, r, c-1);
}
// Time:  $\mathcal{O}(M \times N)$  / Space:  $\mathcal{O}(M \times N)$ 

```

10.4.10 30. Path with Minimum Effort

Specification: You are a hiker traversing an $m \times n$ matrix of heights. Effort is the maximum absolute difference in heights between two consecutive cells. Return min effort to go (0,0) to $(m-1, n-1)$. **Example:** Input: $[[1,2,2],[3,8,2],[5,3,5]]$ -> Output: 2 **Pattern:** Binary Search + BFS. **Explanation:** We can binary search the answer range $[0, 10^6]$. For a chosen effort limit K, use BFS. If BFS reaches the end using only edges $\leq K$, then K is possible, so search lower. Else, search higher.

```

public int minimumEffortPath(int[][] heights) {
    int left = 0, right = 1000000, ans = right;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (canReach(heights, mid)) {
            ans = mid; right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return ans;
}

private boolean canReach(int[][] h, int limit) {
    int m = h.length, n = h[0].length;
    boolean[][] vis = new boolean[m][n];
    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{0, 0}); vis[0][0] = true;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};

    while (!q.isEmpty()) {
        int[] curr = q.poll();
        if (curr[0] == m-1 && curr[1] == n-1) return true;
        for (int[] d : dirs) {
            int r = curr[0]+d[0], c = curr[1]+d[1];
            if (r>=0 && r<m && c>=0 && c<n && !vis[r][c]) {
                if (Math.abs(h[r][c] - h[curr[0]][curr[1]]) <= limit) {
                    vis[r][c] = true;
                    q.offer(new int[]{r, c});
                }
            }
        }
    }
}

```

```

    }
    return false;
}
// Time:  $\mathcal{O}(M \times N \times \log(\text{MaxH}))$  / Space:
 $\hookrightarrow \mathcal{O}(M \times N)$ 

```

10.5 Section 5: Practice Problem Bank

- Snake Traversal Verification** *Specification:* Given an $N \times N$ matrix and a 1D array representing a path, verify if the array follows a strict snake-like (zigzag) traversal of the matrix row by row. *Example:* Input: $[[1,2],[4,3]]$, Path: $[1,2,3,4]$. Output: `true`. *Constraints:* $N \leq 100$. Path length equals N^2 . *Hint:* Use zigzag string conversion pattern. Flip the column iteration direction based on `row % 2`.
- Diagonal Submatrix Sums** *Specification:* Given a square matrix, compute the sum of the primary diagonal and secondary diagonal. If they intersect at a center element, do not double-count the center. *Example:* Input: $[[1,2,3],[4,5,6],[7,8,9]]$. Output: 25. *Constraints:* $N \leq 500$. *Hint:* Only one loop `i` from 0 to $N - 1$ is needed. Primary is (i, i) , secondary is $(i, N-1-i)$.
- Check Matrix Symmetries** *Specification:* Return true if a binary matrix is symmetrically identical horizontally, vertically, and diagonally (both diagonals). *Example:* Input: $[[1,0,1],[0,1,0],[1,0,1]]$. Output: `true`. *Constraints:* $N \leq 100$. *Hint:* Check `mat[i][j]` against `mat[N-1-i][j]`, `mat[i][N-1-j]`, `mat[j][i]`.
- K-Rotations of Matrix** *Specification:* Given an $N \times N$ matrix and integer K , return the matrix rotated clockwise K times by 90 degrees. *Example:* Input: $[[1,2],[3,4]]$, $K = 5$. Output: $[[3,1],[4,2]]$. *Constraints:* $0 \leq K \leq 10^9$. *Hint:* Rotate in-place. $K \% 4$ gives the true number of rotations needed.
- Local Minima Grid Search** *Specification:* A local minimum in a matrix is strictly less than its up to 4 neighbors. Find any local minimum's coordinates and return it. *Example:* Input: $[[9,8,7],[6,1,2]]$. Output: $[1,1]$. *Constraints:* $M, N \leq 1000$. All elements unique. *Hint:* Use DFS or a greedy walk. Always move to a strictly smaller neighbor until trapped.
- Matrix Block Sum (K-Radius)** *Specification:* Return a matrix `answer` where `answer[i][j]` is the sum of all elements `mat[r][c]` for $i - K \leq r \leq i + K, j - K \leq c \leq j + K$. *Example:* Input: $[[1,2,3],[4,5,6],[7,8,9]]$, $K=1$. Output: $[[12,21,16], \dots]$. *Constraints:* Matrix size up to 100×100 . *Hint:* Use Template C (2D Prefix Sum) to answer each cell's block sum in $\mathcal{O}(1)$.
- Count Submatrices with All Ones** *Specification:* Given a binary matrix, count how many rectangular submatrices consist entirely of 1s. *Example:* Input: $[[1,1],[1,1]]$. Output: 9 (four 1×1 , two 1×2 , two 2×1 , one 2×2). *Constraints:* $M, N \leq 150$. *Hint:* For each cell, count contiguous 1s on the left, then scan upwards to form rectangles.
- Sparse Matrix Multiplication** *Specification:* Multiply two sparse matrices A and B . Return the result matrix. *Example:* Input: $A = [[1,0],[0,1]]$, $B = [[2,0],[0,2]]$. Output: $[[2,0],[0,2]]$. *Constraints:* Matrices up to 100×100 . *Hint:* Only multiply and accumulate $A[i][k] * B[k][j]$ if $A[i][k]$ is non-zero.

9. **Maximum Path Sum in Grid** *Specification:* Given an $M \times N$ grid, find the path from top-left to bottom-right that minimizes the sum of its values. You can only move right or down. *Example:* Input: $[[1,3,1],[1,5,1],[4,2,1]]$. Output: 7. *Constraints:* Contains positive integers. *Hint:* This is DP but simulates grid walks. State transition: $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$.
10. **Robot Bounded in Circle** *Specification:* A robot follows a string of instructions ("G", "L", "R"). After executing the instructions infinitely, does it stay in a bounded circle? *Example:* Input: "GGLGG". Output: true. *Constraints:* String length ≤ 100 . *Hint:* State Machine Simulation. If after one cycle the robot is at (0,0) OR not facing North, it is bounded.
11. **Grid Game (Two Robots)** *Specification:* A $2 \times N$ grid of points. Robot 1 goes $(0,0) \rightarrow (1, N-1)$ setting visited cells to 0. Robot 2 does the same, trying to maximize its points. Robot 1 plays optimally to MINIMIZE Robot 2's points. Return Robot 2's score. *Example:* Input: $[[2,5,4],[1,5,1]]$. Output: 4. *Constraints:* $N \leq 5 \times 10^4$. *Hint:* Robot 1 only has 1 turn to drop down. Use Prefix and Suffix arrays to simulate the remaining paths for Robot 2.
12. **As Far from Land as Possible** *Specification:* Grid of 0s (water) and 1s (land). Find a water cell such that its distance to the nearest land is maximized. Return this distance. *Example:* Input: $[[1,0,1],[0,0,0],[1,0,1]]$. Output: 2. *Constraints:* $M, N \leq 100$. *Hint:* Multi-Source BFS. Add all 1s to queue, then BFS outwards. The last layer reached is the answer.
13. **Spiral Matrix III** *Specification:* Start at (rStart, cStart) in an $R \times C$ grid facing East. Walk in a spiral shape. Return coordinates of all cells visited in the grid. *Example:* Input: $R=1, C=4, rStart=0, cStart=0$. Output: $[[0,0],[0,1],[0,2],[0,3]]$. *Constraints:* $1 \leq R, C \leq 100$. *Hint:* Step sequence is 1, 1, 2, 2, 3, 3... Simulate the walk, only adding valid in-bound coordinates to the result.
14. **Enclaves (Number of Closed Islands)** *Specification:* Binary matrix (0=land, 1=water). A closed island is completely surrounded by 1s (no land touches borders). Count them. *Example:* Input: $[[1,1,1],[1,0,1],[1,1,1]]$. Output: 1. *Constraints:* $N \leq 100$. *Hint:* Border DFS. Eliminate all 0s connected to the grid borders. Then count remaining components of 0s.
15. **Ant on a Grid (Langton's Ant)** *Specification:* Simulate K steps of an ant on an infinite white grid. White square $\rightarrow$ turn right, flip to black, move forward. Black square $\rightarrow$ turn left, flip to white, move. *Example:* Input: $K = 10$. Output: Return bounds of modified grid. *Constraints:* $K \leq 10^5$. *Hint:* State Machine Simulation using a HashSet to store coordinates of black squares. Track max/min X and Y.
16. **Shortest Bridge** *Specification:* An $N \times N$ matrix contains exactly two islands (1s). Find the shortest water bridge (0s to flip) to connect them. *Example:* Input: $[[0,1],[1,0]]$. Output: 1. *Constraints:* $N \leq 100$. *Hint:* Find the first island with DFS and push all its cells to a queue. Then BFS from that queue to find the second island.
17. **Count Negative Numbers in Sorted Matrix** *Specification:* Matrix is sorted in decreasing order row-wise and column-wise. Count the negative numbers. *Example:* Input: $[[4,3,-1],[2,1,-2]]$. Output: 2. *Constraints:* $M, N \leq 100$. *Hint:* Staircase search. Start at bottom-left or top-right and eliminate rows/columns.
18. **Build Matrix with Conditions** *Specification:* Build a $K \times K$ matrix with numbers 1 to

- K . Given row condition array and column condition array representing relative ordering (like u must appear before v). *Example:* Input: $K=3$, `rowConditions=[[1,2]]`, `colConditions=[[2,1]]`. Output: valid placement grid. *Constraints:* $K \leq 400$. *Hint:* Topological Sort. Apply Kahn's Algorithm independently for rows and columns to find the exact coordinate for each number.
19. **Determine Matrix is Magic Square** *Specification:* Given a 3×3 grid of integers, determine if it is a magic square (distinct numbers 1-9, rows/cols/diagonals sum to 15). *Example:* Input: `[[4,3,8],[9,5,1],[2,7,6]]`. Output: `true`. *Constraints:* Grid is always 3×3 . *Hint:* HashSet to check uniqueness (1-9), and 8 sums (3 rows, 3 cols, 2 diagonals) must equal 15.
 20. **Coloring a Border** *Specification:* Given a grid, (r, c) , and `color`. Color the border of the connected component at (r, c) . A border cell touches a cell outside the component or the grid edge. *Example:* Input: `[[1,1],[1,2]]`, $(0,0)$, 3. Output: `[[3,3],[3,2]]`. *Constraints:* $M, N \leq 50$. *Hint:* DFS. If a cell has a neighbor of a different original color or is on the boundary, it's a border cell.
 21. **Rotate Grid by K Steps** *Specification:* Rotate the layers of an $M \times N$ grid counter-clockwise K times independently. *Example:* Input: `[[1,2],[3,4]]`, $K=1$. Output: `[[2,4],[1,3]]`. *Constraints:* Layers are concentric rectangles. *Hint:* Extract each spiral layer into a 1D array, perform 1D cyclic shift, and write it back using Boundary Contraction.
 22. **Max Area of Island** *Specification:* Grid of 0s and 1s. Find the maximum area of a single connected component of 1s. *Example:* Input: `[[1,1,0],[1,0,0]]`. Output: 3. *Constraints:* $M, N \leq 50$. *Hint:* DFS returning integer size. `return 1 + dfs(up) + dfs(down) + dfs(left) + dfs(right)`.
 23. **Reshape the Matrix to 1D** *Specification:* Convert a 2D jagged array (rows of different lengths) into a strict 1D array in row-major order. *Example:* Input: `[[1,2],[3],[4,5,6]]`. Output: `[1,2,3,4,5,6]`. *Constraints:* Total elements $\leq 10^5$. *Hint:* Sequential iteration for `(int[] row : matrix) for (int val : row)`.
 24. **Find the Winner of Tic-Tac-Toe** *Specification:* Given an array of moves (coordinates), determine the winner ("A", "B", "Draw", or "Pending"). *Example:* Input: `[[0,0],[1,1],[0,1],[0,2],[1,0],[2,0]]`. Output: "B". *Constraints:* Standard 3×3 grid. *Hint:* Maintain arrays `rows[3]`, `cols[3]`, `diag`, `anti_diag`. Player A adds 1, B adds -1. Check for `sum == 3` or `-3`.
 25. **Maximal Square** *Specification:* Find the largest square submatrix containing only 1s and return its area. *Example:* Input: `[[1,1],[1,1]]`. Output: 4. *Constraints:* $M, N \leq 300$. *Hint:* DP. `dp[i][j] = min(dp[i-1][j-1], dp[i][j-1], dp[i-1][j]) + 1` if cell is '1'.
 26. **Bomb Enemy** *Specification:* Grid with '0' (empty), 'E' (enemy), 'W' (wall). Place a bomb at an empty cell to kill max enemies in its row/col until a wall is hit. *Example:* Input: `[[ "0", "E", "0", "0"], ["E", "0", "W", "E"]]`. Output: 3. *Constraints:* $M, N \leq 500$. *Hint:* State Encoding. Cache the row kill count and column kill count. Recalculate row hits only when crossing a wall.
 27. **Check if Move is Legal (Othello/Reversi)** *Specification:* Given an 8×8 board, an (r, c) position, and `color`, return true if placing the stone forms a valid Reversi line. *Example:* Input: Board state. Output: `true`. *Constraints:* Exactly 8×8 . *Hint:* Raycasting simulation.

Cast a ray in all 8 directions. It must pass through ≥ 1 opponent stones before hitting a friendly stone.

28. **Minimum Knight Moves** *Specification:* Infinite chessboard. Starting at $(0,0)$, find min moves for a Knight to reach (x,y) . *Example:* Input: $x = 2, y = 1$. Output: 1. *Constraints:* $|x|, |y| \leq 300$. *Hint:* BFS with 8 knight direction vectors. Use a `Set<String>` for visited coordinates. Leverage symmetry (absolute values of x, y) to bound search.
29. **Matrix Diagonal Sort** *Specification:* Sort each $i - j$ diagonal of an $M \times N$ matrix in ascending order. *Example:* Input: $[[3,3,1],[2,2,1],[1,1,1]]$. Output: $[[1,1,1],[1,2,2],[2,3,3]]$. *Constraints:* $M, N \leq 100$. *Hint:* Use a `HashMap<Integer, PriorityQueue<Integer>>` where key is $i - j$. Add all elements, then write them back out.
30. **Game of Life 3D (Infinite Space)** *Specification:* Similar to Game of Life but in 3D. Find active cells after 6 cycles. Start with 2D plane in 3D space. *Example:* Input: $[[0,1],[1,1]]$. Output: count of active cells. *Constraints:* Fixed 6 cycles. *Hint:* State Machine Simulation using a `HashSet` of string coordinates " x,y,z ". Only simulate neighbors of currently active cells.

Chapter 11

Q3 Mastery — Dynamic Sliding Windows, HashMap Frequency Signatures, and Prefix Sum Analytics

11.1 Section 1: Essential Terminology & Vocabulary

Dynamic Sliding Window A technique where a window expands to the right to include elements and contracts from the left when a specific invariant or constraint is violated. It matters because it optimizes $\mathcal{O}(N^2)$ brute-force subarray checks into $\mathcal{O}(N)$ operations by avoiding redundant recalculations. Use when searching for the longest/shortest contiguous subarray satisfying a condition.

Fixed-Size Sliding Window vs Dynamic Sliding Window | Feature | Fixed-Size Window | Dynamic Sliding Window |  | **Window Size** | Constant (e.g., length K). | Variable (expands and contracts). | | **Movement** | Move both left and right pointers together. | Move right continuously, move left only to fix invariants. | | **Use Case** | Anagrams in a fixed window, max sum of K elements. | Longest substring with K distinct chars, minimum subarray sum. |

HashMap Frequency Signature Creating a unique key for a group of items (like anagrams) based on their character frequencies rather than sorting. Usually represented as a mapped string of an `int[26]` array. This avoids the $\mathcal{O}(N \log N)$ sorting cost, providing an $\mathcal{O}(N)$ way to group items.

Prefix Sum Array & Cumulative Matching An array where `pref[i]` stores the sum of elements from index 0 to i . The trick `pref[j] - pref[i] = K` allows finding a subarray sum K in $\mathcal{O}(1)$ time by rearranging to `pref[i] = pref[j] - K` and looking up previously seen prefix sums.

Prefix Sum with HashMap A pattern combining prefix sums with a HashMap to count the occurrences of each prefix sum. This allows counting how many subarrays sum to a specific value K in $\mathcal{O}(N)$ time.

Two-Pointer for Sorted Arrays Using two pointers, usually starting at the beginning and end of a sorted array, that converge towards the middle. Used to find pairs summing to a target in $\mathcal{O}(N)$ time without extra space.

Interval Merging and Insertion Sorting a collection of intervals by start time and iterating through them to combine overlapping ranges (where `current.start <= previous.end`).

Monotonic Stack/Queue A data structure that maintains elements in a strictly increasing or decreasing order. Useful for finding the “next greater element” or managing the maximum/minimum in a sliding window in $\mathcal{O}(N)$ time.

Character Frequency Signature Using a fixed-size array (like `int[26]` for lowercase English letters) to count character occurrences. By converting this array to a string (or checking array equality), it acts as a canonical $\mathcal{O}(1)$ space key for anagrams.

Modular Arithmetic in Prefix Sums Using the modulo operator with prefix sums. If `pref[i] % K == pref[j] % K`, then the subarray between i and j has a sum divisible by K .

11.1.1 ‘At Most K’ to ‘Exactly K’ Conversion

This mathematical reduction calculates exact occurrences using cumulative bounds. It uses the formula `exactly(K) = atMost(K) - atMost(K-1)`. Why it matters: This is the standard trick for counting subarrays with exactly K distinct elements.

11.1.2 Index Negation Trick

This technique marks elements as ‘seen’ by negating the value at the corresponding index, such as `nums[abs(val)-1] = -nums[abs(val)-1]`. It only works for array values bounded within the range $[1, N]$. Why it matters: It provides $\mathcal{O}(1)$ space duplicate or missing element detection without using extra data structures.

11.1.3 Cyclic Sort / Index Placement

This sorting pattern places each value v precisely at its correct target index `nums[v-1]`. It continuously swaps elements until the current position holds the correct value. Why it matters: It is the optimal strategy to find the first missing positive integer in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.

11.1.4 Expand-Around-Center

This technique treats each index (and the space between indices) as a potential palindrome center. It then expands outwards as long as the mirrored characters match. Why it matters: It is a simple and reliable $\mathcal{O}(N^2)$ approach for the longest palindromic substring problem.

11.1.5 Frequency Bucket Sort

This sorting alternative groups elements by their frequency into buckets ranging from 0 to N . You then scan these buckets in reverse order to collect the most frequent items. Why it matters: It solves Top- K frequent elements problems in $\mathcal{O}(N)$ time without requiring a heap.

11.1.6 Deferred Deletion / Lazy Invalidation

Instead of immediately removing items from a data structure, this technique marks entries as invalid. The actual cleanup happens later during traversal or retrieval. Why it matters: It avoids `ConcurrentModificationExceptions` and heavily simplifies priority queue update patterns.

11.1.7 Contribution Counting

Instead of iterating through all possible subarrays, this mathematical approach computes exactly how many subarrays a specific element contributes to. It aggregates the total across all individual element contributions. Why it matters: It dramatically transforms $O(N^2)$ brute force summation logic into a highly optimal $O(N)$ pass.

11.1.8 Greedy Interval Scheduling

This algorithm sorts given intervals by their end times first. It then greedily picks the next non-overlapping interval to maximize total count. Why it matters: It is a provably optimal approach for finding the maximum number of non-overlapping intervals.

11.2 Section 2: Reusable Code Templates

Template A: Dynamic Sliding Window

```
int left = 0, maxLen = 0;
for (int right = 0; right < arr.length; right++) {
    // 1. Add arr[right] to window state
    while (/* window state violates invariant */) {
        // 2. Remove arr[left] from window state
        left++;
    }
    // 3. Update maxLen or minLen
    maxLen = Math.max(maxLen, right - left + 1);
}
```

Template B: Fixed-Size Sliding Window

```
int k = 3, sum = 0, max = 0;
for (int i = 0; i < arr.length; i++) {
    sum += arr[i]; // Add current element
    if (i >= k - 1) {
        max = Math.max(max, sum); // Update result
        sum -= arr[i - (k - 1)]; // Remove leftmost element for next iteration
    }
}
```

Template C: Prefix Sum + HashMap Counter

```
Map<Integer, Integer> map = new HashMap<>();
map.put(0, 1); // Base case for subarrays starting at index 0
int sum = 0, count = 0;
for (int num : nums) {
    sum += num;
    if (map.containsKey(sum - k)) {
        count += map.get(sum - k);
    }
    map.put(sum, map.getOrDefault(sum, 0) + 1);
}
```



```
}
```

Template D: HashMap Frequency Grouping

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    String key = Arrays.toString(count);
    map.putIfAbsent(key, new ArrayList<>());
    map.get(key).add(s);
}
```

11.3 Section 3: Solved Exemplar Problems

1. Longest Substring Without Repeating Characters Specification: Given a string, find the length of the longest substring without repeating characters. **Example:** `s = "abcabcbb"` -> Output: 3 ("abc") **Pattern:** Dynamic Sliding Window + HashMap **Explanation:** We expand the right pointer. If the character is in the set, we contract the left pointer until the duplicate is removed, ensuring the window always contains unique characters.

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, max = 0;
    for (int right = 0; right < s.length(); right++) {
        // Contract if duplicate found
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left++));
        }
        set.add(s.charAt(right)); // Add current char
        max = Math.max(max, right - left + 1);
    }
    return max;
}
// Time Complexity: O(N) | Space Complexity: O(min(N, M))
```

2. Subarray Sum Equals K Specification: Find the total number of continuous subarrays whose sum equals to K. **Example:** `nums = [1,1,1], k = 2` -> Output: 2 **Pattern:** Prefix Sum + HashMap **Explanation:** We maintain a running sum. If `sum - k` exists in our frequency map, it means there is a subarray ending at the current index that sums to K.

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1); // Base case
    int sum = 0, count = 0;
    for (int num : nums) {
        sum += num;
        // Check if required prefix exists
        if (map.containsKey(sum - k)) count += map.get(sum - k);
    }
}
```

```

        map.put(sum, map.getOrDefault(sum, 0) + 1);
    }
    return count;
}
// Time Complexity: O(N) | Space Complexity: O(N)

```

3. Group Anagrams Specification: Group strings that are anagrams of each other. **Example:** ["eat","tea","tan","ate","nat","bat"] -> Output: [["bat"],["nat","tan"],["ate","eat","tea"]] **Pattern:** HashMap Frequency Signature **Explanation:** Generate a 26-element character count array for each string, convert it to a string key, and use it in a HashMap to group anagrams together.

```

public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) count[c - 'a']++; // Build signature
        String key = Arrays.toString(count);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
// Time Complexity: O(N * L) | Space Complexity: O(N * L)

```

4. Find All Anagram Start Indices Specification: Find all start indices of p's anagrams in s. **Example:** s = "cbaebabacd", p = "abc" -> Output: [0, 6] **Pattern:** Fixed-Size Sliding Window + Frequency Array **Explanation:** Use a window of size p.length(). Keep arrays of character frequencies for p and the current window in s. If they match, add the index.

```

public List<Integer> findAnagrams(String s, String p) {
    List<Integer> res = new ArrayList<>();
    if (s.length() < p.length()) return res;
    int[] pCount = new int[26], sCount = new int[26];
    for (char c : p.toCharArray()) pCount[c - 'a']++;
    for (int i = 0; i < s.length(); i++) {
        sCount[s.charAt(i) - 'a']++;
        if (i >= p.length()) sCount[s.charAt(i - p.length()) - 'a']--; //
        ↪ Contract
        if (Arrays.equals(pCount, sCount)) res.add(i - p.length() + 1); // Match
    }
    return res;
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

5. Longest Substring with At Most K Distinct Characters Specification: Find the length of the longest substring with at most K distinct characters. **Example:** s = "eceba", k = 2 -> Output: 3 ("ece") **Pattern:** Dynamic Sliding Window **Explanation:** Use a HashMap to track character frequencies. When map size exceeds K, shrink window from left until size is K again.

```

public int lengthOfLongestSubstringKDistinct(String s, int k) {
    Map<Character, Integer> map = new HashMap<>();

```

```

    int left = 0, max = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        map.put(c, map.getOrDefault(c, 0) + 1);
        while (map.size() > k) { // Invariant broken
            char leftChar = s.charAt(left++);
            map.put(leftChar, map.get(leftChar) - 1);
            if (map.get(leftChar) == 0) map.remove(leftChar);
        }
        max = Math.max(max, right - left + 1);
    }
    return max;
}
// Time Complexity: O(N) | Space Complexity: O(K)

```

6. Minimum Window Substring Specification: Given strings *s* and *t*, find the minimum substring of *s* containing all characters in *t*. **Example:** *s* = "ADOBECODEBANC", *t* = "ABC" -> Output: "BANC" **Pattern:** Dynamic Sliding Window **Explanation:** Track required characters in a map. Expand right until all required characters are in the window, then contract left to minimize the window.

```

public String minWindow(String s, String t) {
    int[] map = new int[128];
    for (char c : t.toCharArray()) map[c]++;
    int left = 0, count = t.length(), minLen = Integer.MAX_VALUE, minStart = 0;
    for (int right = 0; right < s.length(); right++) {
        if (map[s.charAt(right)]-- > 0) count--; // Found required char
        while (count == 0) { // All chars found
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minStart = left;
            }
            if (++map[s.charAt(left++)] > 0) count++; // Removed required char
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart + minLen);
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

7. Group Shifted Strings Specification: Group strings that can be formed by shifting characters uniformly. **Example:** ["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"] -> Output groups ["abc","bcd","xyz"], etc. **Pattern:** Difference-Based Signature **Explanation:** Calculate the relative distance between adjacent characters. Use this sequence of differences as the HashMap key.

```

public List<List<String>> groupStrings(String[] strings) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strings) {
        StringBuilder key = new StringBuilder();
    }

```

```

    for (int i = 1; i < s.length(); i++) {
        int diff = (s.charAt(i) - s.charAt(i-1) + 26) % 26; // Circular
        ↪ difference
        key.append(diff).append(",");
    }
    map.computeIfAbsent(key.toString(), k -> new ArrayList<>()).add(s);
}
return new ArrayList<>(map.values());
}
// Time Complexity: O(N * L) | Space Complexity: O(N * L)

```

8. Contiguous Array Equal 0s and 1s Specification: Find the maximum length of a contiguous subarray with an equal number of 0s and 1s. **Example:** [0, 1, 0] -> Output: 2 **Pattern:** Prefix Sum (+1/-1 trick) **Explanation:** Treat 0s as -1. If the running sum is seen again, it means the subarray between those two indices sums to 0, implying equal 0s and 1s.

```

public int findMaxLength(int[] nums) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, -1);
    int sum = 0, max = 0;
    for (int i = 0; i < nums.length; i++) {
        sum += nums[i] == 0 ? -1 : 1; // Map 0 to -1
        if (map.containsKey(sum)) {
            max = Math.max(max, i - map.get(sum));
        } else {
            map.put(sum, i); // Store first occurrence
        }
    }
    return max;
}
// Time Complexity: O(N) | Space Complexity: O(N)

```

9. Subarray Product Less Than K Specification: Count contiguous subarrays where the product is strictly less than K. **Example:** nums = [10,5,2,6], k = 100 -> Output: 8 **Pattern:** Dynamic Sliding Window **Explanation:** Maintain a running product. If product >= k, shrink from left. Number of valid subarrays ending at right is right - left + 1.

```

public int numSubarrayProductLessThanK(int[] nums, int k) {
    if (k <= 1) return 0;
    int prod = 1, left = 0, count = 0;
    for (int right = 0; right < nums.length; right++) {
        prod *= nums[right];
        while (prod >= k) prod /= nums[left++]; // Shrink
        count += right - left + 1; // Add valid subarrays
    }
    return count;
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

10. Permutation in String Specification: Return true if s2 contains a permutation of s1. **Example:** s1 = "ab", s2 = "eidbaooo" -> Output: true **Pattern:** Fixed-Size Window Fre-

quency Match **Explanation:** Same logic as Anagram Start Indices. Maintain a window of size `s1.length()` and compare character counts.

```
public boolean checkInclusion(String s1, String s2) {
    if (s1.length() > s2.length()) return false;
    int[] s1map = new int[26], s2map = new int[26];
    for (char c : s1.toCharArray()) s1map[c - 'a']++;
    for (int i = 0; i < s2.length(); i++) {
        s2map[s2.charAt(i) - 'a']++;
        if (i >= s1.length()) s2map[s2.charAt(i - s1.length()) - 'a']--;
        if (Arrays.equals(s1map, s2map)) return true;
    }
    return false;
}
// Time Complexity: O(N) | Space Complexity: O(1)
```

11. Maximum Erasure Value Specification: Find the maximum score (sum) from a subarray of unique elements. **Example:** `nums = [4,2,4,5,6]` -> Output: 17 **Pattern:** Dynamic Sliding Window + HashSet **Explanation:** Use a set to track uniqueness. Expand right, add to sum. If duplicate found, shrink from left, subtracting from sum until unique.

```
public int maximumUniqueSubarray(int[] nums) {
    Set<Integer> set = new HashSet<>();
    int sum = 0, max = 0, left = 0;
    for (int right = 0; right < nums.length; right++) {
        while (set.contains(nums[right])) {
            set.remove(nums[left]);
            sum -= nums[left++]; // Remove duplicate
        }
        set.add(nums[right]);
        sum += nums[right];
        max = Math.max(max, sum);
    }
    return max;
}
// Time Complexity: O(N) | Space Complexity: O(N)
```

12. Longest Repeating Character Replacement Specification: Longest substring of same letters after replacing at most `k` chars. **Example:** `s = "AABABBA"`, `k = 1` -> Output: 4 **Pattern:** Window with Max Frequency Tracking **Explanation:** If `window size - max_freq_char_count > k`, we have too many differing chars, so we shrink the window.

```
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int maxCount = 0, left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        maxCount = Math.max(maxCount, ++count[s.charAt(right) - 'A']);
        if (right - left + 1 - maxCount > k) { // Invalid window
            count[s.charAt(left++) - 'A']--;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
}
```

```

    }
    return maxLen;
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

13. Fruit Into Baskets Specification: Max fruit collected with 2 baskets (equivalent to max substring with ≤ 2 distinct characters). **Example:** [1,2,3,2,2] -> Output: 4 **Pattern:** Dynamic Sliding Window **Explanation:** Keep a frequency map. When distinct fruit types exceed 2, increment left pointer to shrink.

```

public int totalFruit(int[] fruits) {
    Map<Integer, Integer> count = new HashMap<>();
    int left = 0, max = 0;
    for (int right = 0; right < fruits.length; right++) {
        count.put(fruits[right], count.getDefault(fruits[right], 0) + 1);
        while (count.size() > 2) {
            count.put(fruits[left], count.get(fruits[left]) - 1);
            if (count.get(fruits[left]) == 0) count.remove(fruits[left]);
            left++;
        }
        max = Math.max(max, right - left + 1);
    }
    return max;
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

14. Continuous Subarray Sum Multiple of K Specification: Check if a subarray of length ≥ 2 has a sum multiple of K. **Example:** nums = [23,2,4,6,7], k = 6 -> Output: true **Pattern:** Prefix Sum Modular Math **Explanation:** If $\text{pref}[i] \% k == \text{pref}[j] \% k$, the sum between i and j is a multiple of K . Store remainder and its first seen index.

```

public boolean checkSubarraySum(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, -1);
    int sum = 0;
    for (int i = 0; i < nums.length; i++) {
        sum += nums[i];
        int mod = k == 0 ? sum : sum % k;
        if (map.containsKey(mod)) {
            if (i - map.get(mod) > 1) return true; // Length >= 2
        } else {
            map.put(mod, i);
        }
    }
    return false;
}
// Time Complexity: O(N) | Space Complexity: O(min(N, K))

```

15. Max Consecutive Ones III Specification: Longest contiguous 1s after flipping at most K zeros. **Example:** nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2 -> Output: 6 **Pattern:** Window with Zero-Flip Budget **Explanation:** Expand window. If 0 encountered, decrease K. If $K < 0$,

shrink window until a 0 is excluded.

```
public int longestOnes(int[] nums, int k) {
    int left = 0;
    for (int right = 0; right < nums.length; right++) {
        if (nums[right] == 0) k--;
        if (k < 0) { // Over budget
            if (nums[left++] == 0) k++;
        }
    }
    return nums.length - left; // Trick to return max valid length seen
}
// Time Complexity: O(N) | Space Complexity: O(1)
```

16. Find All Duplicates in Array Specification: Find elements appearing twice in an array containing integers in range $[1, n]$. **Example:** $[4,3,2,7,8,2,3,1] \rightarrow$ Output: $[2,3]$ **Pattern:** Index Negation Trick **Explanation:** Use the array itself as a hash table. Mark the number at index $\text{abs}(\text{num}) - 1$ negative. If it's already negative, it's a duplicate.

```
public List<Integer> findDuplicates(int[] nums) {
    List<Integer> res = new ArrayList<>();
    for (int num : nums) {
        int idx = Math.abs(num) - 1;
        if (nums[idx] < 0) res.add(Math.abs(num)); // Found duplicate
        else nums[idx] = -nums[idx]; // Mark seen
    }
    return res;
}
// Time Complexity: O(N) | Space Complexity: O(1)
```

17. Task Scheduler CPU Units Specification: Minimum CPU intervals to finish tasks given a cooldown of n between identical tasks. **Example:** $\text{tasks} = ["A","A","A","B","B","B"], n = 2 \rightarrow$ Output: 8 **Pattern:** Frequency Math **Explanation:** Calculate idle slots based on the most frequent task. $\text{maxIdle} = (\text{maxFreq} - 1) * n$. Fill slots with other tasks.

```
public int leastInterval(char[] tasks, int n) {
    int[] count = new int[26];
    int max = 0, maxCount = 0;
    for (char c : tasks) {
        count[c - 'A']++;
        if (count[c - 'A'] == max) maxCount++;
        else if (count[c - 'A'] > max) { max = count[c - 'A']; maxCount = 1; }
    }
    int emptySlots = (max - 1) * (n - (maxCount - 1));
    int availableTasks = tasks.length - max * maxCount;
    int idles = Math.max(0, emptySlots - availableTasks);
    return tasks.length + idles;
}
// Time Complexity: O(N) | Space Complexity: O(1)
```

18. Insert & Merge Overlapping Intervals Specification: Insert a new interval into a sorted

list and merge if necessary. **Example:** `[[1,3],[6,9]]`, `new = [2,5]` -> Output: `[[1,5],[6,9]]`
Pattern: Interval Merging **Explanation:** Three phases: Add all before new, merge overlapping with new, add all after new.

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < newInterval[0]) res.add(intervals[i++]); //
    ↪ Before
    while (i < n && intervals[i][0] <= newInterval[1]) { // Merge
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.add(newInterval);
    while (i < n) res.add(intervals[i++]); // After
    return res.toArray(new int[res.size()][]);
}
// Time Complexity: O(N) | Space Complexity: O(N)
```

19. Top K Frequent Elements Specification: Return the K most frequent elements. **Example:** `nums = [1,1,1,2,2,3]`, `k = 2` -> Output: `[1,2]` **Pattern:** HashMap + Min-Heap
Explanation: Count frequencies in a map, then keep a min-heap of size K based on frequencies.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int n : nums) count.put(n, count.getOrDefault(n, 0) + 1);
    PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> count.get(a) -
    ↪ count.get(b));
    for (int n : count.keySet()) {
        heap.add(n);
        if (heap.size() > k) heap.poll(); // Keep size K
    }
    return heap.stream().mapToInt(i -> i).toArray();
}
// Time Complexity: O(N log K) | Space Complexity: O(N)
```

20. First Missing Positive Integer Specification: Find the smallest missing positive integer in an unsorted array. **Example:** `[3,4,-1,1]` -> Output: `2` **Pattern:** Cyclic Sort (Index placement)
Explanation: Place number x at index x-1. Then scan to find the first index that doesn't have i+1.

```
public int firstMissingPositive(int[] nums) {
    int i = 0;
    while (i < nums.length) {
        // Swap to correct position if valid
        if (nums[i] > 0 && nums[i] <= nums.length && nums[nums[i] - 1] !=
        ↪ nums[i]) {
            int temp = nums[nums[i] - 1];
            nums[nums[i] - 1] = nums[i];
            nums[i] = temp;
        }
    }
    for (int i = 0; i < nums.length; i++) {
        if (i + 1 != nums[i]) return i + 1;
    }
    return nums.length + 1;
}
```



```

    } else {
        i++;
    }
}
for (i = 0; i < nums.length; i++) {
    if (nums[i] != i + 1) return i + 1; // Missing
}
return nums.length + 1;
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

11.4 Section 4: CodeSignal Classic Q3 Patterns

21. Minimum Size Subarray Sum Specification: Min length of subarray with sum $\geq$ target. **Example:** target = 7, nums = [2,3,1,2,4,3] -> Output: 2 **Pattern:** Dynamic Window with Target Sum **Explanation:** Keep expanding until sum $\geq$ target, then shrink to find minimum.

```

public int minSubArrayLen(int target, int[] nums) {
    int left = 0, sum = 0, min = Integer.MAX_VALUE;
    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];
        while (sum >= target) {
            min = Math.min(min, right - left + 1);
            sum -= nums[left++];
        }
    }
    return min == Integer.MAX_VALUE ? 0 : min;
}
// Time Complexity: O(N) | Space Complexity: O(1)

```

22. Substring with Concatenation of All Words Specification: Find starting indices of substrings that are a concatenation of all words in an array exactly once. **Example:** s = "barfoothefoobarman", words = ["foo","bar"] -> Output: [0, 9] **Pattern:** Fixed-Size Window with Inner HashMap **Explanation:** Use a map for word counts. Slide a window of length words.length * wordLen and verify word counts inside.

```

public List<Integer> findSubstring(String s, String[] words) {
    List<Integer> res = new ArrayList<>();
    if (s.isEmpty() || words.length == 0) return res;
    int wordLen = words[0].length(), totalLen = wordLen * words.length;
    Map<String, Integer> counts = new HashMap<>();
    for (String w : words) counts.put(w, counts.getOrDefault(w, 0) + 1);

    for (int i = 0; i <= s.length() - totalLen; i++) {
        Map<String, Integer> seen = new HashMap<>();
        int j = 0;
        while (j < words.length) {
            String w = s.substring(i + j * wordLen, i + (j + 1) * wordLen);

```

```

        if (counts.containsKey(w)) {
            seen.put(w, seen.getOrDefault(w, 0) + 1);
            if (seen.get(w) > counts.get(w)) break;
        } else break;
        j++;
    }
    if (j == words.length) res.add(i);
}
return res;
}
// Time Complexity: O(N * M * L) / Space Complexity: O(M)

```

23. Contains Duplicate II Specification: Check if array has duplicates within distance k. **Example:** [1,2,3,1], k = 3 -> Output: true **Pattern:** Sliding Window Set **Explanation:** Keep a sliding set of size k. If add fails, duplicate found.

```

public boolean containsNearbyDuplicate(int[] nums, int k) {
    Set<Integer> set = new HashSet<>();
    for (int i = 0; i < nums.length; i++) {
        if (i > k) set.remove(nums[i - k - 1]);
        if (!set.add(nums[i])) return true;
    }
    return false;
}
// Time Complexity: O(N) / Space Complexity: O(K)

```

24. Count Number of Nice Subarrays Specification: Count subarrays with exactly k odd numbers. **Example:** nums = [1,1,2,1,1], k = 3 -> Output: 2 **Pattern:** Prefix Sum of Odds **Explanation:** Treat odds as 1s, evens as 0s. Same as subarray sum equals K.

```

public int numberOfSubarrays(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1);
    int sum = 0, count = 0;
    for (int num : nums) {
        sum += num % 2;
        count += map.getOrDefault(sum - k, 0);
        map.put(sum, map.getOrDefault(sum, 0) + 1);
    }
    return count;
}
// Time Complexity: O(N) / Space Complexity: O(N)

```

25. Frequency of Most Frequent Element Specification: Max frequency of an element after incrementing at most K operations. **Example:** [1,2,4], k = 5 -> Output: 3 **Pattern:** Sort + Sliding Window **Explanation:** Sort first. To make all elements in window equal to nums[right], we need nums[right] * window_length - window_sum <= k.

```

public int maxFrequency(int[] nums, int k) {
    Arrays.sort(nums);
    int left = 0;

```

```

    long sum = 0;
    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];
        if ((long)nums[right] * (right - left + 1) - sum > k) {
            sum -= nums[left++];
        }
    }
    return nums.length - left;
}
// Time Complexity: O(N log N) / Space Complexity: O(1)

```

26. Subarrays with K Different Integers Specification: Count subarrays with exactly K distinct integers. **Example:** [1,2,1,2,3], K = 2 -> Output: 7 **Pattern:** At-Most-K Trick **Explanation:** Exactly(K) = AtMost(K) - AtMost(K-1).

```

public int subarraysWithKDistinct(int[] nums, int k) {
    return atMostK(nums, k) - atMostK(nums, k - 1);
}

private int atMostK(int[] nums, int k) {
    int[] count = new int[nums.length + 1];
    int left = 0, res = 0, distinct = 0;
    for (int right = 0; right < nums.length; right++) {
        if (count[nums[right]]++ == 0) distinct++;
        while (distinct > k) {
            if (--count[nums[left++]] == 0) distinct--;
        }
        res += right - left + 1;
    }
    return res;
}
// Time Complexity: O(N) / Space Complexity: O(N)

```

27. Longest Palindromic Substring Specification: Find the longest substring that reads same backwards. **Example:** "babad" -> Output: "bab" **Pattern:** Expand Around Center **Explanation:** Treat each character and between-character as a center and expand outwards to check for palindrome.

```

public String longestPalindrome(String s) {
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i);
        int len2 = expand(s, i, i + 1);
        int len = Math.max(len1, len2);
        if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}

```

```
private int expand(String s, int L, int R) {
    while (L >= 0 && R < s.length() && s.charAt(L) == s.charAt(R)) { L--; R++; }
    return R - L - 1;
}
// Time Complexity: O(N^2) / Space Complexity: O(1)
```

28. 3Sum Specification: Find all unique triplets that sum to zero. **Example:** [-1,0,1,2,-1,-4] -> Output: [[-1,-1,2],[-1,0,1]] **Pattern:** Sort + Two Pointer **Explanation:** Sort array. Iterate i, and use two pointers L and R to find pairs summing to -nums[i]. Skip duplicates.

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        int L = i + 1, R = nums.length - 1;
        while (L < R) {
            int sum = nums[i] + nums[L] + nums[R];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i], nums[L], nums[R]));
                while (L < R && nums[L] == nums[L+1]) L++;
                while (L < R && nums[R] == nums[R-1]) R--;
                L++; R--;
            }
            else if (sum < 0) L++;
            else R--;
        }
    }
    return res;
}
// Time Complexity: O(N^2) / Space Complexity: O(1)
```

29. 4Sum Specification: Find unique quadruplets summing to target. **Example:** nums = [1,0,-1,0,-2,2], target = 0 -> Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]] **Pattern:** Sort + Nested Two Pointer **Explanation:** Extend 3Sum by adding one more outer loop.

```
public List<List<Integer>> fourSum(int[] nums, int target) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 3; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        for (int j = i + 1; j < nums.length - 2; j++) {
            if (j > i + 1 && nums[j] == nums[j-1]) continue;
            int L = j + 1, R = nums.length - 1;
            while (L < R) {
                long sum = (long)nums[i] + nums[j] + nums[L] + nums[R];
                if (sum == target) {
                    res.add(Arrays.asList(nums[i], nums[j], nums[L], nums[R]));
                    while (L < R && nums[L] == nums[L+1]) L++;
                    while (L < R && nums[R] == nums[R-1]) R--;
                }
            }
        }
    }
    return res;
}
```

```

        L++; R--;
    }
    else if (sum < target) L++;
    else R--;
}
}
}
return res;
}
// Time Complexity: O(N^3) | Space Complexity: O(1)

```

30. Number of Distinct Islands Specification: Count number of uniquely shaped islands in a grid. **Example:** Grid with two identical 2x2 islands -> Output: 1 **Pattern:** DFS + Path Signature Hashing **Explanation:** Record the direction moved (U, D, L, R) during DFS traversal. Store path strings in a HashSet to deduplicate identical shapes.

```

public int numDistinctIslands(int[][] grid) {
    Set<String> set = new HashSet<>();
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == 1) {
                StringBuilder sb = new StringBuilder();
                dfs(grid, i, j, "S", sb); // Start with 'S'
                set.add(sb.toString());
            }
        }
    }
    return set.size();
}

private void dfs(int[][] grid, int r, int c, String dir, StringBuilder sb) {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c]
        == 0) return;
    grid[r][c] = 0; // mark visited
    sb.append(dir);
    dfs(grid, r + 1, c, "D", sb);
    dfs(grid, r - 1, c, "U", sb);
    dfs(grid, r, c + 1, "R", sb);
    dfs(grid, r, c - 1, "L", sb);
    sb.append("B"); // Backtrack to distinguish paths
}
// Time Complexity: O(R * C) | Space Complexity: O(R * C)

```

11.5 Section 5: Practice Problem Bank

1. Contiguous Subarray Max Vowels Specification: Given string s and length k , find the maximum number of vowels in a substring of length k . **Example:** $s = \text{"abciidef"}$, $k = 3$ -> Output: 3 (for "iii") **Constraints:** $1 \leq s.length \leq 10^5$, $1 \leq k \leq s.length$. Lowercase English

letters. **Strategic Hint:** Fixed-Size Sliding Window counting vowels as it slides.

2. Number of Subarrays with Bounded Maximum Specification: Count subarrays such that the maximum value in the subarray is between `left` and `right` inclusive. **Example:** `nums = [2,1,4,3]`, `left = 2`, `right = 3` -> Output: 3 ([2], [2, 1], [3]) **Constraints:** $1 \leq \text{nums.length} \leq 10^5$, $0 \leq \text{nums}[i] \leq 10^9$. **Strategic Hint:** Two-pointer tracking valid range start and last valid number seen.

3. Grumpy Bookstore Owner Specification: Maximize customers satisfied over an array. The owner is grumpy at some indices. You have one `minutes` long secret technique to keep them not grumpy. **Example:** `customers=[1,0,1,2,1,1,7,5]`, `grumpy=[0,1,0,1,0,1,0,1]`, `minutes=3` -> Output: 16 **Constraints:** Arrays equal length $\leq 2 \times 10^4$. **Strategic Hint:** Fixed-size window tracking the max potential customer gain.

4. Minimum Flips to Make Binary String Alternating Specification: You can remove the first char and append it to the end. Find the min operations to change the string into an alternating string of 0s and 1s. **Example:** "111000" -> Output: 2 **Constraints:** $1 \leq s.length \leq 10^5$. **Strategic Hint:** Double the string (`s+s`) and use a Fixed-Size Sliding Window of length N .

5. Maximize the Confusion of an Exam Specification: Given a string of 'T' and 'F', flip at most K answers to maximize consecutive identical answers. **Example:** "TTFF", $k=2$ -> Output: 4 **Constraints:** $1 \leq len \leq 5 \times 10^4$. **Strategic Hint:** Apply Max Consecutive Ones III logic separately for 'T' and 'F'.

6. Longest Subarray of 1s After Deleting One Element Specification: You must delete exactly one element. Find the max continuous 1s remaining. **Example:** [1,1,0,1] -> Output: 3 **Constraints:** $1 \leq \text{nums.length} \leq 10^5$. **Strategic Hint:** Dynamic Window with a zero-budget of exactly 1.

7. Repeated DNA Sequences Specification: Find all 10-letter-long sequences occurring more than once. **Example:** "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT" -> Output: ["AAAAACCCC", "CCCCCAAAAA"] **Constraints:** $1 \leq s.length \leq 10^5$. **Strategic Hint:** Fixed window length 10 hashing strings or bitmask signatures.

8. K-diff Pairs in an Array Specification: Count unique pairs (i, j) such that $|\text{nums}[i] - \text{nums}[j]| == k$. **Example:** [3,1,4,1,5], $k=2$ -> Output: 2 (1,3 and 3,5) **Constraints:** Array length $\leq 10^4$. **Strategic Hint:** HashMap counting frequencies. Check `num + k` for $k > 0$ and frequency > 1 for $k = 0$.

9. Check if Array Pairs Are Divisible by k Specification: Can we pair up all elements such that every pair sum is divisible by k ? **Example:** [1,2,3,4,5,10,6,7,8,9], $k=5$ -> Output: true **Constraints:** Array length even, $\leq 10^5$. **Strategic Hint:** Modulo arithmetic counts array. Count of `x` must equal count of `k - x`.

10. Count Vowel Substrings of a String Specification: Substrings containing only vowels, and at least one of each ('a','e','i','o','u'). **Example:** "aeiouu" -> Output: 2 **Constraints:** $1 \leq s.length \leq 100$. **Strategic Hint:** Dynamic Window with vowel frequency tracking.

11. Subarray Sums Divisible by K Specification: Count subarrays whose sum is divisible by K . **Example:** [4,5,0,-2,-3,1], $k = 5$ -> Output: 7 **Constraints:** $1 \leq \text{nums.length} \leq 3 \times 10^4$. **Strategic Hint:** Prefix Sum + Modulo HashMap grouping.

12. Find the Longest Substring Containing Vowels in Even Counts Specification: Max

length substring with all vowels appearing an even number of times. **Example:** "eleetmini-coworoep" -> Output: 13 **Constraints:** $1 \leq s.length \leq 5 \times 10^5$. **Strategic Hint:** Prefix Sum with Bitmask (5 bits) mapped to first occurrence indices.

13. Matrix Block Sum Specification: Compute sum of elements in a submatrix defined by a distance K . **Example:** 3×3 grid, $K = 1$. Output is block sums. **Constraints:** Matrix dimensions ≤ 100 . **Strategic Hint:** 2D Prefix Sum Array. $pref[i][j] = val + pref[i-1][j] + pref[i][j-1] - pref[i-1][j-1]$.

14. Replace the Substring for Balanced String Specification: String has 'Q', 'W', 'E', 'R'. Replace a minimal substring to make counts exactly $N/4$. **Example:** "QWER" -> Output: 0. "QQWE" -> Output: 1. **Constraints:** length is multiple of 4. **Strategic Hint:** Dynamic Window matching missing characters needed outside the window.

15. Longest Substring Of All Vowels in Order Specification: Substring must contain all 5 vowels in alphabetical order. **Example:** "aeiaaioaaaaeiiiiouuuooaauuaeiu" -> Output: 13 **Constraints:** string size $\leq 5 \times 10^5$. **Strategic Hint:** Dynamic Window resetting when order is broken or char is not a vowel.

16. Count Good Meals Specification: Number of pairs of items whose sum is a power of two. **Example:** [1,3,5,7,9] -> Output: 4 **Constraints:** Elements $\leq 2^{20}$. **Strategic Hint:** Two Sum with target looping through all 22 powers of two.

17. Largest Subarray of 0's and 1's Specification: Exact same as 'Contiguous Array', formulated differently. **Example:** [0,1] -> Output: 2 **Constraints:** size $\leq 10^5$. **Strategic Hint:** Prefix sum converting 0 to -1, check map for first occurrence.

18. Sort Characters By Frequency Specification: Sort string based on character frequencies descending. **Example:** "tree" -> Output: "eert" or "eetr" **Constraints:** $1 \leq len \leq 5 \times 10^5$. **Strategic Hint:** HashMap for frequencies, then PriorityQueue or Bucket Sort.

19. Number of Pairs of Strings With Concatenation Equal to Target Specification: Given array of strings, count pairs (i, j) where $nums[i] + nums[j] == target$. **Example:** $nums = ["777", "7", "77", "77"]$, $target = "7777"$ -> Output: 4 **Constraints:** ≤ 100 strings. **Strategic Hint:** Hashmap string frequencies. Check target prefixes and suffixes.

20. Arithmetic Slices Specification: Count contiguous subarrays forming arithmetic progressions of length ≥ 3 . **Example:** [1,2,3,4] -> Output: 3 **Constraints:** size ≤ 5000 . **Strategic Hint:** Dynamic Window or DP tracking consecutive diffs.

21. Number of Submatrices That Sum to Target Specification: 2D version of Subarray Sum Equals K. **Example:** 2×2 grid, target 0. **Constraints:** Matrix $\leq 100 \times 100$. **Strategic Hint:** 2D Prefix Sums flattened into 1D for every pair of rows + HashMap.

22. Count Trippets That Can Form Two Arrays of Equal XOR Specification: $a = arr[i] \dots arr[j-1]$, $b = arr[j] \dots arr[k]$. Count (i, j, k) where $a == b$. **Example:** [2,3,1,6,7] -> Output: 4 **Constraints:** length ≤ 300 . **Strategic Hint:** Prefix XOR. $a == b \implies arr[i \dots k] == 0$.

23. Maximum Number of Vowels in a Substring of Given Length Specification: Another variation of vowel counting with fixed K. **Example:** "leetcode", $k=3$ -> Output: 2 **Constraints:** length $\leq 10^5$. **Strategic Hint:** Fixed window $O(N)$ linear scan.

- 24. Subarray With Given Sum Specification:** Non-negative integers, find continuous subarray summing to S. (Return bounds). **Example:** [1,2,3,7,5], S=12 -> Output: [2,4] (1-based) **Constraints:** Elements > 0. **Strategic Hint:** Dynamic Window (since all positive, monotonic sum).
- 25. Minimum Operations to Reduce X to Zero Specification:** Remove elements from either left or right ends to make target X. Minimum ops. **Example:** nums = [1,1,4,2,3], x = 5 -> Output: 2 **Constraints:** Elements > 0. **Strategic Hint:** Find max length subarray summing to TotalSum - X.
- 26. Distinct Numbers in Each Subarray Specification:** Count distinct numbers in every window of size K. **Example:** [1,2,3,2,2,1,3], k=3 -> Output: [3,2,2,2,3] **Constraints:** $1 \leq n \leq 10^5$. **Strategic Hint:** Fixed size window with HashMap counting frequencies.
- 27. Shortest Subarray with Sum at Least K Specification:** Like minimum size subarray sum but array can have negatives! **Example:** [2,-1,2], k=3 -> Output: 3 **Constraints:** Length $\leq 10^5$. **Strategic Hint:** Prefix sum + Monotonic Deque to maintain increasing prefix sums.
- 28. Make Sum Divisible by P Specification:** Remove smallest subarray so remaining array sum is divisible by P. **Example:** [3,1,4,2], p=6 -> Output: 1 (remove [4]) **Constraints:** Length $\leq 10^5$. **Strategic Hint:** Target remainder is total_sum % P. Find shortest subarray with this mod.
- 29. Continuous Subarrays Specification:** Subarrays where absolute diff between any two elements is ≤ 2 . **Example:** [5,4,2,4] -> Output: 8 **Constraints:** Length $\leq 10^5$. **Strategic Hint:** Dynamic Window with TreeMap or Monotonic Queues to track min/max.
- 30. Longest Subarray With Maximum Bitwise AND Specification:** Find max bitwise AND possible, then find longest subarray with that value. **Example:** [1,2,3,3,2,2] -> Output: 2 **Constraints:** Length $\leq 10^5$. **Strategic Hint:** Max AND is just the max element. Find longest contiguous sequence of the max element.

Chapter 12

Q4 Mastery — Algorithmic Optimization: Binary Search Variants, Monotonic Structures, Dynamic Programming, and Graph Algorithms

This chapter covers Q4 of the CodeSignal GCA (Hard difficulty, ~25 minutes target time). Q4 is the most challenging question testing optimal $\mathcal{O}(\log N)$ or $\mathcal{O}(N)$ solutions, DP state transitions, and graph algorithms.

12.1 Section 1: Essential Terminology & Vocabulary

12.1.1 Rotated Sorted Array & Monotonic Partition Invariant

12.1.1.1 1. Conceptual Definition: What is a Rotated Sorted Array?

A **Rotated Sorted Array** is an array that was originally sorted in ascending order (with unique elements), but has been shifted (rotated) at some unknown pivot index K .

For example, consider the original sorted array:

Original Sorted Array: $[0, 1, 2, 4, 5, 6, 7]$

If we rotate this array at pivot index $K = 3$ (shifting elements from index 3 onwards to the front), we get:

Rotated Sorted Array: $[4, 5, 6, 7, 0, 1, 2]$

Notice what happened: - The single monotonically increasing sequence is split into **two sorted sub-arrays**: $[4, 5, 6, 7]$ (the left segment) and $[0, 1, 2]$ (the right segment). - The array is no longer sorted overall, so standard Binary Search (which assumes `nums[left] <= nums[right]`) fails if implemented naively.

12.1.1.2 2. The Core Mathematical Invariant

The key insight that allows us to achieve $\mathcal{O}(\log N)$ time complexity is the **Monotonic Partition Invariant**:

The Fundamental Invariant: Whenever you split a Rotated Sorted Array into two halves using a midpoint $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$, **AT LEAST ONE OF THE TWO HALVES IS GUARANTEED TO BE STRICTLY MONOTONICALLY SORTED.**

- If $\text{nums}[\text{left}] \leq \text{nums}[\text{mid}]$: The **LEFT** half $[\text{left} \dots \text{mid}]$ is monotonically sorted.
- If $\text{nums}[\text{left}] > \text{nums}[\text{mid}]$: The **RIGHT** half $[\text{mid} \dots \text{right}]$ is monotonically sorted.

12.1.1.3 3. Step-by-Step Binary Search Decision Rule

Because one half is always sorted, we can easily check if our **target** lies within the boundaries of that sorted half:

1. Calculate $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$.
2. If $\text{nums}[\text{mid}] == \text{target}$, return mid immediately.
3. Check which half is sorted:
 - **Case A: Left Half $[\text{left} \dots \text{mid}]$ is Sorted ($\text{nums}[\text{left}] \leq \text{nums}[\text{mid}]$)**
 - Is **target** in range $[\text{nums}[\text{left}] \dots \text{nums}[\text{mid}]]$?
 - * If **Yes**: Eliminate the right half $\rightarrow \text{right} = \text{mid} - 1$.
 - * If **No**: Eliminate the left half $\rightarrow \text{left} = \text{mid} + 1$.
 - **Case B: Right Half $[\text{mid} \dots \text{right}]$ is Sorted ($\text{nums}[\text{left}] > \text{nums}[\text{mid}]$)**
 - Is **target** in range $[\text{nums}[\text{mid}] \dots \text{nums}[\text{right}]]$?
 - * If **Yes**: Eliminate the left half $\rightarrow \text{left} = \text{mid} + 1$.
 - * If **No**: Eliminate the right half $\rightarrow \text{right} = \text{mid} - 1$.

12.1.1.4 4. Complete Worked Execution Trace ($\text{nums} = [4, 5, 6, 7, 0, 1, 2]$, $\text{target} = 0$)

Let's trace searching for $\text{target} = 0$:

- **Iteration 1:**
 - $\text{left} = 0$ (val 4), $\text{right} = 6$ (val 2).
 - $\text{mid} = 0 + (6 - 0) / 2 = 3 \rightarrow \text{nums}[3] = 7$.
 - Is $\text{nums}[\text{mid}] == \text{target}$? $7 == 0$ (False).
 - Check left half sortedness: $\text{nums}[0] \ (4) \leq \text{nums}[3] \ (7) \rightarrow$ **Left Half $[4, 5, 6, 7]$ IS SORTED.**
 - Is **target** (0) within $[4 \dots 7]$? No ($0 < 4$).
 - Action: Eliminate left half $\rightarrow \text{left} = \text{mid} + 1 = 4$.
- **Iteration 2:**
 - $\text{left} = 4$ (val 0), $\text{right} = 6$ (val 2).
 - $\text{mid} = 4 + (6 - 4) / 2 = 5 \rightarrow \text{nums}[5] = 1$.
 - Is $\text{nums}[\text{mid}] == \text{target}$? $1 == 0$ (False).
 - Check left half sortedness: $\text{nums}[4] \ (0) \leq \text{nums}[5] \ (1) \rightarrow$ **Left Half $[0, 1]$ IS SORTED.**
 - Is **target** (0) within $[0 \dots 1]$? Yes! ($0 \geq 0$ and $0 \leq 1$).

- Action: Eliminate right half $\rightarrow \text{right} = \text{mid} - 1 = 4$.
- **Iteration 3:**
 - $\text{left} = 4$ (val 0), $\text{right} = 4$ (val 0).
 - $\text{mid} = 4 + (4 - 4) / 2 = 4 \rightarrow \text{nums}[4] = 0$.
 - Is $\text{nums}[\text{mid}] == \text{target}$? $0 == 0$ (True!).
 - **Return index 4!** (Exact $\mathcal{O}(\log N)$ solution reached in 3 steps).

12.1.2 Binary Search on Answer Space (Parametric Binary Search)

Definition: A technique where we search for an optimal value (the “answer”) within a known range $[\text{low}, \text{high}]$ instead of searching for a specific element in an array. We use a monotonic predicate function (e.g., `canFulfill(mid)`) to determine whether a given value `mid` is feasible.

Why it matters: It transforms optimization problems (e.g., “find the minimum capacity”) into a series of simpler decision problems (e.g., “is capacity X sufficient?”), enabling $\mathcal{O}(N \log(\text{max} - \text{min}))$ solutions. **When to use:** When the answer space is bounded, the feasibility function is monotonic (if x is valid, $x + 1$ is also valid, or vice versa), and calculating feasibility takes linear time $\mathcal{O}(N)$.

12.1.3 Monotonic Stack & Deque

Definition: A stack or double-ended queue (deque) where elements are maintained in strictly increasing or strictly decreasing order. **Why it matters:** It provides $\mathcal{O}(1)$ amortized time complexity for range maximum/minimum lookups or finding the “next greater element”. Elements are pushed and popped at most once. **When to use:** Finding the next greater/smaller element, sliding window maximum/minimum, and calculating histogram areas.

12.1.4 Dynamic Programming State Transition (1D, 2D, Interval DP)

Definition: The mathematical rule or formula that relates the solution of a larger problem to its smaller overlapping subproblems. - **1D DP:** The state depends on a single variable (e.g., index i). Transition: $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$. - **2D DP:** The state depends on two variables (e.g., strings of length i and j). Transition: $\text{dp}[i][j] = \dots$. - **Interval DP:** The state is defined by a range $[i, j]$. Subproblems are smaller intervals within the range. **Why it matters:** Properly defining the state and transition is the core of any DP solution. It turns exponential $\mathcal{O}(2^N)$ backtracking into polynomial time $\mathcal{O}(N)$ or $\mathcal{O}(N^2)$ solutions.

12.1.5 Memoization vs Tabulation

Definition: The two primary methods for implementing Dynamic Programming. | Feature | Memoization (Top-Down) | Tabulation (Bottom-Up) | | — | — | — | | **Direction** | Start from the main problem, recursively call subproblems. | Start from base cases, iteratively build up to the main problem. | | **State Storage** | Hash Map or Array. | N-dimensional Array. | | **Overhead** | Recursive stack overhead (potential StackOverflow). | No recursive overhead, generally faster constant time. | | **When to use** | When not all subproblems need to be evaluated. | When all subproblems will definitely be evaluated. |

12.1.6 Knapsack Variants

Definition: A family of combinatorial optimization problems involving packing items into a capacity-constrained space to maximize value. - **0/1 Knapsack:** Each item can be chosen at

most once. Transition relies on picking or skipping: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$. - **Unbounded Knapsack:** Each item can be chosen infinitely many times. - **Subset Sum:** A specialized 0/1 Knapsack where we want to know if a subset sums exactly to **target**. **Why it matters:** They form the basis for numerous resource allocation and subset combination problems in technical interviews.

12.1.7 Topological Sort

Definition: A linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge $U \rightarrow V$, vertex U comes before V in the ordering. **Why it matters:** Kahn's Algorithm (using an in-degree array and queue) processes dependencies efficiently in $\mathcal{O}(V + E)$ time. **When to use:** Task scheduling, resolving prerequisites (like courses or build systems), finding dependency cycles.

12.1.8 BFS Shortest Path

Definition: Breadth-First Search traversal to find the shortest path in an **unweighted** graph. It processes nodes level-by-level using a Queue. **Why it matters:** It guarantees that the first time a target node is reached, it is via the shortest possible path (fewest edges). **When to use:** Shortest path on grids or unweighted graphs, state transitions requiring fewest moves (like word ladders or minimum jumps).

12.1.9 Two-pointer

Definition: Using two indices (usually **left** and **right**) to traverse a sequence simultaneously. **Why it matters:** It optimally narrows down search spaces without requiring extra memory, often reducing $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$. **When to use:** Finding pairs in sorted arrays, bounding areas (like trapping rain water or container with most water), and cycle detection.

12.1.10 Greedy

Definition: Making the locally optimal choice at each step with the hope that these local choices lead to a globally optimal solution. **Why it matters:** When a greedy choice property can be proven (e.g., via contradiction or exchange arguments), the algorithm is extremely fast and space-efficient. **When to use:** Interval scheduling, jump games, Huffman coding, minimum spanning trees.

12.1.11 DP State Compression

When processing a dynamic programming grid where the current row only depends on the previous row, this technique replaces $dp[N][M]$ with two 1D arrays **prev[]** and **curr[]**. **Why it matters:** It halves memory usage and often reduces $\mathcal{O}(N \cdot M)$ space to $\mathcal{O}(M)$.

12.1.12 Binary Search Loop Termination

This deals with the crucial choice between **while (lo < hi)** and **while (lo <= hi)** loops, paired with $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ to prevent overflow. Choosing the wrong termination condition causes infinite loops. **Why it matters:** Boundary conditions and termination logic are the #1 source of binary search bugs.

12.1.13 Interval DP Framework

This DP pattern defines the state `dp[i][j]` as the optimal solution for a subarray from `i` to `j`. It enumerates a split point `k` to divide the interval into smaller subproblems. Why it matters: It is essential for solving burst balloons, matrix chain multiplication, and palindrome partitioning.

12.1.14 Union-Find (Disjoint Set Union)

This data structure tracks elements partitioned into disjoint subsets. It combines a `find()` method using path compression with a `union()` method utilizing union-by-rank to achieve near $O(1)$ amortized operations. Why it matters: It is the optimal structure for connected components, cycle detection, and Kruskal's Minimum Spanning Tree.

12.1.15 Dijkstra's Algorithm

This is an optimal pathfinding algorithm that utilizes a priority queue for a breadth-first search on weighted edges. It processes nodes in order of shortest accumulated distance in $O((V+E) \log V)$ time. Why it matters: It is the gold standard for solving single-source shortest path problems with non-negative weights.

12.1.16 Backtracking Template

This pattern generates all possible configurations by exhaustively exploring decision trees. It strictly follows a “choose, explore, unchoose” structured layout with early pruning. Why it matters: It is universally used to generate all combinations, permutations, and subsets in optimization problems.

12.1.17 LRU Cache Architecture

This system design paradigm combines a `HashMap<Key, Node>` with a doubly linked list. The hash map provides instant access, while the list maintains temporal usage order. Why it matters: It allows $O(1)$ get and put operations by seamlessly combining hash lookup with ordered eviction.

12.1.18 Fibonacci DP Recognition

This refers to identifying when a problem's state perfectly maps to the linear recurrence `dp[i] = dp[i-1] + dp[i-2]`. The entire array state can be compressed into two variables. Why it matters: Problems like climbing stairs, decode ways, and tiling can be instantly recognized and compressed to $O(1)$ space.

12.2 Section 2: Reusable Code Templates

12.2.1 Template A: Binary Search

```
// Standard Binary Search
int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
```

```

        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// Binary Search on Answer Space (Leftmost valid)
int binarySearchAnswerSpace(int min, int max) {
    int left = min, right = max;
    int best = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (isValid(mid)) {
            best = mid;
            right = mid - 1; // Try to find a smaller valid answer
        } else {
            left = mid + 1;
        }
    }
    return best;
}

```

12.2.2 Template B: Monotonic Stack

```

public int[] nextGreaterElement(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // stores indices
    for (int i = 0; i < n; i++) {
        // Maintain strictly decreasing stack
        while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) {
            int prevIndex = stack.pop();
            result[prevIndex] = nums[i]; // Found next greater!
        }
        stack.push(i);
    }
    return result;
}

```

12.2.3 Template C: 1D DP with State Compression

```

public int dpStateCompression(int[] nums) {
    if (nums.length == 0) return 0;
    int prev2 = 0; // dp[i-2]
    int prev1 = nums[0]; // dp[i-1]
    for (int i = 1; i < nums.length; i++) {
        int curr = Math.max(prev1, prev2 + nums[i]);
    }
}

```

```

        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

```

12.2.4 Template D: BFS with Level Tracking

```

public int bfsLevel(Node start, Node target) {
    Queue<Node> queue = new LinkedList<>();
    Set<Node> visited = new HashSet<>();
    queue.offer(start);
    visited.add(start);

    int level = 0;
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            Node curr = queue.poll();
            if (curr.equals(target)) return level;

            for (Node neighbor : curr.neighbors) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    queue.offer(neighbor);
                }
            }
        }
        level++; // Increment level after exploring all nodes at current depth
    }
    return -1;
}

```

12.2.5 Template E: Topological Sort (Kahn's Algorithm)

```

public List<Integer> topologicalSort(int numNodes, int[][] edges) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] inDegree = new int[numNodes];
    for (int i = 0; i < numNodes; i++) adj.add(new ArrayList<>());

    for (int[] edge : edges) {
        adj.get(edge[1]).add(edge[0]); // edge[1] -> edge[0]
        inDegree[edge[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numNodes; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }
}

```

```

List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int curr = queue.poll();
    order.add(curr);
    for (int neighbor : adj.get(curr)) {
        if (--inDegree[neighbor] == 0) {
            queue.offer(neighbor);
        }
    }
}
return order.size() == numNodes ? order : new ArrayList<>(); // Empty if
    ↪ cycle exists
}

```

12.3 Section 3: Solved Exemplar Problems

12.3.1 1. Search in Rotated Sorted Array

Specification: Given an integer array sorted in ascending order (with distinct values) and rotated at an unknown pivot, find the index of **target**. **Example:** `nums = [4,5,6,7,0,1,2]`, `target = 0` → output 4. **Pattern:** Rotated Binary Search **Explanation:** We use the monotonic partition invariant. At any midpoint, at least one half of the array is strictly sorted. We identify the sorted half and check if the target falls within its range.

```

public int search(int[] nums, int target) {
    if (nums == null || nums.length == 0) return -1;
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;

        // Left half is sorted
        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1; // Target is in the sorted left half
            } else {
                left = mid + 1; // Target must be in the right half
            }
        }
        // Right half is sorted
        else {
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1; // Target is in the sorted right half
            } else {
                right = mid - 1; // Target must be in the left half
            }
        }
    }
}

```



```

    }
}
return -1;
}
// Time Complexity: O(log N)
// Space Complexity: O(1)

```

12.3.2 2. Sliding Window Maximum

Specification: Return an array of the maximum values in every sliding window of size K. **Example:** `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3` → output `[3,3,5,5,6,7]`. **Pattern:** Monotonic Deque **Explanation:** We maintain a deque of indices such that the values are in strictly decreasing order. The front of the deque always holds the maximum element's index for the current window. We remove elements from the front that fall out of the window.

```

public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] res = new int[n - k + 1];
    int resIndex = 0;
    Deque<Integer> q = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        // Remove indices outside the current window
        if (!q.isEmpty() && q.peekFirst() < i - k + 1) {
            q.pollFirst();
        }
        // Remove smaller elements (maintain decreasing order)
        while (!q.isEmpty() && nums[q.peekLast()] < nums[i]) {
            q.pollLast();
        }
        q.offerLast(i);

        // Record max for the window
        if (i >= k - 1) {
            res[resIndex++] = nums[q.peekFirst()];
        }
    }
    return res;
}
// Time Complexity: O(N) since each element is pushed/popped at most once
// Space Complexity: O(K) for the deque

```

12.3.3 3. Longest Common Subsequence

Specification: Return the length of the longest common subsequence between two strings. **Example:** `text1 = "abcde"`, `text2 = "ace"` → output 3 ("ace"). **Pattern:** 2D DP **Explanation:** `dp[i][j]` represents the LCS of the prefixes of length `i` and `j`. If characters match, we add 1 to

the result of `dp[i-1][j-1]`. If not, we take the max of skipping a character in either string.

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1; // Match found
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]); // Skip char
            }
        }
    }
    return dp[m][n];
}
// Time Complexity: O(M * N)
// Space Complexity: O(M * N)
```

12.3.4 4. Burst Balloons

Specification: Maximize coins by bursting balloons. Bursting `nums[i]` yields `nums[i-1] * nums[i] * nums[i+1]` coins. **Example:** `nums = [3,1,5,8]` → output 167. **Pattern:** Interval DP **Explanation:** We think backwards: what is the LAST balloon to be burst in an interval `[left, right]`? This allows us to split the problem into independent subproblems. `dp[i][j]` is the max coins obtained from bursting balloons strictly between `i` and `j`.

```
public int maxCoins(int[] nums) {
    int n = nums.length;
    int[] arr = new int[n + 2];
    arr[0] = 1; arr[n + 1] = 1; // Padding with 1s
    for (int i = 0; i < n; i++) arr[i + 1] = nums[i];

    int[][] dp = new int[n + 2][n + 2];

    // len is the length of the interval strictly between i and j
    for (int len = 1; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len + 1;
            // k is the index of the LAST balloon to burst in (i, j)
            for (int k = i + 1; k < j; k++) {
                int coins = arr[i] * arr[k] * arr[j] + dp[i][k] + dp[k][j];
                dp[i][j] = Math.max(dp[i][j], coins);
            }
        }
    }
    return dp[0][n + 1];
}
// Time Complexity: O(N^3)
```

// Space Complexity: $O(N^2)$

12.3.5 5. Maximum Product Subarray

Specification: Find a contiguous non-empty subarray with the maximum product. **Example:** `nums = [2,3,-2,4]` → output 6 (subarray `[2,3]`). **Pattern:** 1D DP (Min/Max Tracking) **Explanation:** Since multiplying two negative numbers yields a positive number, we must track BOTH the maximum product and the minimum product ending at the current position.

```
public int maxProduct(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    int maxVal = nums[0], minVal = nums[0], result = nums[0];

    for (int i = 1; i < nums.length; i++) {
        // If current is negative, max and min will swap roles
        if (nums[i] < 0) {
            int temp = maxVal;
            maxVal = minVal;
            minVal = temp;
        }
        maxVal = Math.max(nums[i], maxVal * nums[i]);
        minVal = Math.min(nums[i], minVal * nums[i]);
        result = Math.max(result, maxVal);
    }
    return result;
}
// Time Complexity:  $O(N)$ 
// Space Complexity:  $O(1)$ 
```

12.3.6 6. Median of Two Sorted Arrays

Specification: Find the median of two sorted arrays in $\mathcal{O}(\log(M + N))$ time. **Example:** `nums1 = [1,3]`, `nums2 = [2]` → output 2.0. **Pattern:** Binary Search on Partitions **Explanation:** We binary search for the correct partition index in the smaller array such that the left halves of both arrays contain exactly half the total elements, and the largest element on the left is $\leq$ the smallest element on the right.

```
public double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) return findMedianSortedArrays(B, A); // ensure A is
    ↪ smaller
    int m = A.length, n = B.length;
    int left = 0, right = m;

    while (left <= right) {
        int i = (left + right) / 2; // partition A
        int j = (m + n + 1) / 2 - i; // partition B

        int maxLeftA = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
        int minRightA = (i == m) ? Integer.MAX_VALUE : A[i];
        int maxLeftB = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
```

```

int minRightB = (j == n) ? Integer.MAX_VALUE : B[j];

if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
    // Correct partition found
    if ((m + n) % 2 == 0) {
        return (Math.max(maxLeftA, maxLeftB) + Math.min(minRightA,
            ↪ minRightB)) / 2.0;
    } else {
        return Math.max(maxLeftA, maxLeftB);
    }
} else if (maxLeftA > minRightB) {
    right = i - 1; // move partition left in A
} else {
    left = i + 1; // move partition right in A
}
}
return 0.0;
}
// Time Complexity: O(log(min(M, N)))
// Space Complexity: O(1)

```

12.3.7 7. Trapping Rain Water

Specification: Calculate how much rain water can be trapped after raining. **Example:** height = [0,1,0,2,1,0,1,3,2,1,2,1] → output 6. **Pattern:** Two-Pointer **Explanation:** The amount of water above a bar depends on min(max_left, max_right). We use two pointers from both ends, safely moving the pointer that points to the strictly smaller max bound, adding water along the way.

```

public int trap(int[] height) {
    if (height == null || height.length == 0) return 0;
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0, totalWater = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) leftMax = height[left];
            else totalWater += leftMax - height[left];
            left++;
        } else {
            if (height[right] >= rightMax) rightMax = height[right];
            else totalWater += rightMax - height[right];
            right--;
        }
    }
    return totalWater;
}
// Time Complexity: O(N)
// Space Complexity: O(1)

```

12.3.8 8. Daily Temperatures

Specification: Find the number of days you have to wait after each day to get a warmer temperature. **Example:** [73,74,75,71,69,72,76,73] → output [1,1,4,2,1,1,0,0]. **Pattern:** Monotonic Stack **Explanation:** We maintain a stack of indices representing days where we haven't found a warmer day yet (decreasing order). When we find a warmer day, we pop from the stack and compute the wait time.

```
public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] res = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        // While current temp is greater than temp at stack top
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()])
            ↪ {
                int prevIndex = stack.pop();
                res[prevIndex] = i - prevIndex;
            }
        stack.push(i);
    }
    return res;
}
// Time Complexity: O(N)
// Space Complexity: O(N)
```

12.3.9 9. Edit Distance / Levenshtein

Specification: Minimum insertions, deletions, substitutions to convert word1 to word2. **Example:** word1 = "horse", word2 = "ros" → output 3. **Pattern:** 2D DP **Explanation:** dp[i][j] is the edit distance between word1 prefix length i and word2 prefix length j. If characters match, cost is dp[i-1][j-1]. Otherwise, cost is 1 + min(insert, delete, replace).

```
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    // Base cases
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1]; // No op
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], // Replace
                                         Math.min(dp[i - 1][j], // Delete
                                                    dp[i][j - 1])); // Insert
            }
        }
    }
}
```

```

    }
}
return dp[m][n];
}
// Time Complexity:  $O(M * N)$ 
// Space Complexity:  $O(M * N)$ 

```

12.3.10 10. LRU Cache

Specification: Design a cache with Least Recently Used eviction policy supporting `get` and `put` in $\mathcal{O}(1)$ time. **Pattern:** HashMap + Doubly Linked List **Explanation:** The HashMap provides $\mathcal{O}(1)$ access to nodes. The Doubly Linked List maintains the eviction order. Moving a node to the head of the list designates it as most recently used.

```

public class LRUCache {
    class Node {
        int key, val;
        Node prev, next;
    }
    private Map<Integer, Node> map = new HashMap<>();
    private int capacity;
    private Node head, tail;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Node();
        tail = new Node();
        head.next = tail;
        tail.prev = head; // Connect dummy head and tail
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node); // Move to head (MRU)
        insert(node);
        return node.val;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            remove(map.get(key));
        }
        if (map.size() == capacity) {
            map.remove(tail.prev.key);
            remove(tail.prev); // Evict LRU
        }
        Node node = new Node();
    }
}

```

```

        node.key = key;
        node.val = value;
        insert(node);
        map.put(key, node);
    }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void insert(Node node) { // Insert right after head
        node.next = head.next;
        node.next.prev = node;
        head.next = node;
        node.prev = head;
    }
}

// Time Complexity: O(1) for both get and put
// Space Complexity: O(Capacity)

```

12.3.11 11. Maximal Rectangle in Binary Matrix

Specification: Find the largest rectangle containing only 1s in a 2D binary matrix. **Example:** `[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]` → output 6. **Pattern:** Histogram Reduction + Monotonic Stack **Explanation:** We treat each row as the base of a histogram and update heights. We then run the $\mathcal{O}(N)$ “Largest Rectangle in Histogram” algorithm using a monotonic stack on each row.

```

public int maximalRectangle(char[][] matrix) {
    if (matrix == null || matrix.length == 0) return 0;
    int cols = matrix[0].length;
    int[] heights = new int[cols];
    int maxArea = 0;

    for (char[] row : matrix) {
        // Update histogram heights
        for (int c = 0; c < cols; c++) {
            heights[c] = (row[c] == '1') ? heights[c] + 1 : 0;
        }
        maxArea = Math.max(maxArea, maxHistogram(heights));
    }
    return maxArea;
}

private int maxHistogram(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int max = 0, n = heights.length;
    for (int i = 0; i <= n; i++) {

```

```

    int h = (i == n) ? 0 : heights[i];
    while (!stack.isEmpty() && h < heights[stack.peek()]) {
        int height = heights[stack.pop()];
        int width = stack.isEmpty() ? i : i - stack.peek() - 1;
        max = Math.max(max, height * width);
    }
    stack.push(i);
}
return max;
}
// Time Complexity: O(R * C)
// Space Complexity: O(C)

```

12.3.12 12. Word Ladder

Specification: Find the shortest sequence of word mutations from `beginWord` to `endWord`, changing one letter at a time, using a dictionary. **Example:** `begin = "hit", end = "cog", list = ["hot","dot","dog","lot","log","cog"]` → output 5. **Pattern:** BFS **Explanation:** We use BFS because we want the shortest path in an unweighted graph. For each word, we generate all valid next mutations and enqueue them, tracking the level.

```

public int ladderLength(String beginWord, String endWord, List<String> wordList)
↪ {
    Set<String> set = new HashSet<>(wordList);
    if (!set.contains(endWord)) return 0;

    Queue<String> queue = new LinkedList<>();
    queue.offer(beginWord);
    int level = 1;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) { // Level-by-level processing
            String curr = queue.poll();
            char[] chars = curr.toCharArray();
            for (int j = 0; j < chars.length; j++) {
                char orig = chars[j];
                for (char c = 'a'; c <= 'z'; c++) { // Try all mutations
                    if (c == orig) continue;
                    chars[j] = c;
                    String next = new String(chars);
                    if (next.equals(endWord)) return level + 1;
                    if (set.remove(next)) { // remove serves as 'visited' check
                        queue.offer(next);
                    }
                }
                chars[j] = orig; // Backtrack
            }
        }
    }
}

```



```

        level++;
    }
    return 0;
}
// Time Complexity:  $O(M^2 * N)$  where  $M$  is word length,  $N$  is number of words
// Space Complexity:  $O(M * N)$ 

```

12.3.13 13. Coin Change

Specification: Find the minimum number of coins needed to make up a given amount. **Example:** coins = [1,2,5], amount = 11 → output 3. **Pattern:** 1D DP (Unbounded Knapsack) **Explanation:** dp[i] is the minimum coins needed for amount i. We iterate through amounts and coins, taking the min of using the coin or not: $dp[i] = \min(dp[i], dp[i - coin] + 1)$.

```

public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // Fill with max invalid value
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
// Time Complexity:  $O(\text{Amount} * N)$ 
// Space Complexity:  $O(\text{Amount})$ 

```

12.3.14 14. House Robber

Specification: Maximum money you can rob from houses where you cannot rob adjacent houses. **Example:** [2,7,9,3,1] → output 12. **Pattern:** 1D DP with State Compression **Explanation:** The transition is $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$. We only need to store the previous two values, saving space.

```

public int rob(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    int prev1 = 0; // max so far excluding current
    int prev2 = 0; // max so far including current (-2)

    for (int num : nums) {
        int temp = Math.max(prev1, prev2 + num); // rob or don't rob
        prev2 = prev1;
        prev1 = temp;
    }

    return prev1;
}

```

```
// Time Complexity: O(N)
// Space Complexity: O(1)
```

12.3.15 15. Regular Expression Matching

Specification: Implement regex matching with support for `.` (any single char) and `*` (zero or more of the preceding char). **Example:** `s = "ab", p = ".*"` → output `true`. **Pattern:** 2D DP **Explanation:** Complex transition logic based on whether we see a `*`. We either treat `*` as zero occurrences (`dp[i][j-2]`) or multiple occurrences (`dp[i-1][j]` if the preceding char matches).

```
public boolean isMatch(String s, String p) {
    int m = s.length(), n = p.length();
    boolean[][] dp = new boolean[m + 1][n + 1];
    dp[0][0] = true;

    // Match empty string with patterns like a*b*
    for (int j = 1; j <= n; j++) {
        if (p.charAt(j - 1) == '*') dp[0][j] = dp[0][j - 2];
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (p.charAt(j - 1) == '.' || p.charAt(j - 1) == s.charAt(i - 1)) {
                dp[i][j] = dp[i - 1][j - 1]; // Single char match
            } else if (p.charAt(j - 1) == '*') {
                dp[i][j] = dp[i][j - 2]; // Match zero times
                // If preceding char matches, match one or more times
                if (p.charAt(j - 2) == '.' || p.charAt(j - 2) == s.charAt(i - 1))
                    dp[i][j] = dp[i][j] || dp[i - 1][j];
            }
        }
    }

    return dp[m][n];
}

// Time Complexity: O(M * N)
// Space Complexity: O(M * N)
```

12.3.16 16. Course Schedule II

Specification: Return the ordering of courses you should take to finish all courses given prerequisite pairs `[course, prereq]`. **Example:** `num = 4, prereqs = [[1,0],[2,0],[3,1],[3,2]]` → output `[0,1,2,3]`. **Pattern:** Topological Sort (Kahn's) **Explanation:** We count the in-degree of each course. A course with in-degree 0 has no prerequisites and can be taken. We enqueue it, take it, and decrement the in-degree of its neighbors.

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    int[] inDegree = new int[numCourses];
    List<List<Integer>> adj = new ArrayList<>();
```

```

for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());

for (int[] p : prerequisites) {
    adj.get(p[1]).add(p[0]);
    inDegree[p[0]]++;
}

Queue<Integer> q = new LinkedList<>();
for (int i = 0; i < numCourses; i++) {
    if (inDegree[i] == 0) q.offer(i);
}

int[] res = new int[numCourses];
int idx = 0;
while (!q.isEmpty()) {
    int curr = q.poll();
    res[idx++] = curr;
    for (int next : adj.get(curr)) {
        if (--inDegree[next] == 0) q.offer(next);
    }
}
return idx == numCourses ? res : new int[0]; // If not all courses taken,
↳ cycle exists
}
// Time Complexity: O(V + E)
// Space Complexity: O(V + E)

```

12.3.17 17. Partition Equal Subset Sum

Specification: Determine if an array can be partitioned into two subsets with equal sums. **Example:** nums = [1,5,11,5] → output true. **Pattern:** 0/1 Knapsack DP **Explanation:** The problem translates to: “Is there a subset that sums exactly to total_sum / 2?” We use a 1D DP array where dp[j] is true if a sum j is achievable.

```

public boolean canPartition(int[] nums) {
    int sum = 0;
    for (int num : nums) sum += num;
    if (sum % 2 != 0) return false;

    int target = sum / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;

    for (int num : nums) {
        // Iterate backwards to avoid reusing the same element
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }
}

```

```

    return dp[target];
}
// Time Complexity: O(N * Target)
// Space Complexity: O(Target)

```

12.3.18 18. Decode Ways

Specification: Given a string of digits, return the number of ways it can be decoded (A=1, Z=26). **Example:** s = "226" → output 3 (BZ, VF, BBF). **Pattern:** 1D DP **Explanation:** Very similar to Fibonacci. The number of ways to decode up to i is the ways to decode up to i-1 (if single digit valid) plus the ways to decode up to i-2 (if two digits valid).

```

public int numDecodings(String s) {
    if (s == null || s.isEmpty() || s.charAt(0) == '0') return 0;
    int n = s.length();
    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        int oneDigit = Integer.parseInt(s.substring(i - 1, i));
        int twoDigits = Integer.parseInt(s.substring(i - 2, i));

        if (oneDigit >= 1 && oneDigit <= 9) {
            dp[i] += dp[i - 1];
        }
        if (twoDigits >= 10 && twoDigits <= 26) {
            dp[i] += dp[i - 2];
        }
    }
    return dp[n];
}
// Time Complexity: O(N)
// Space Complexity: O(N) which can be optimized to O(1)

```

12.3.19 19. Stock Span

Specification: Design a class that calculates the stock's span (consecutive days prior where price was $\leq$ today). **Example:** [100, 80, 60, 70, 60, 75, 85] → output [1, 1, 1, 2, 1, 4, 6]. **Pattern:** Monotonic Stack **Explanation:** Maintain a stack of pairs {price, span}. If the incoming price is greater than the top of the stack, pop the stack and accumulate the span. This maintains a strictly decreasing stack.

```

public class StockSpanner {
    // Array holds {price, span}
    private Deque<int[]> stack = new ArrayDeque<>();

    public int next(int price) {
        int span = 1;
        while (!stack.isEmpty() && stack.peek()[0] <= price) {

```

```

        span += stack.pop()[1]; // Accumulate previous spans
    }
    stack.push(new int[]{price, span});
    return span;
}
// Time Complexity: Amortized O(1) per next() call
// Space Complexity: O(N)

```

12.3.20 20. Longest Increasing Subsequence

Specification: Find the length of the longest strictly increasing subsequence in an array. **Example:** `nums = [10,9,2,5,3,7,101,18]` → output 4 (`[2,3,7,101]`). **Pattern:** DP + Binary Search **Explanation:** We maintain an array `tails` where `tails[i]` stores the smallest tail of all increasing subsequences of length `i+1`. We binary search the position to update in `tails`.

```

public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int x : nums) {
        int left = 0, right = size;
        while (left != right) {
            int mid = left + (right - left) / 2;
            if (tails[mid] < x) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        tails[left] = x;
        if (left == size) size++; // Found a larger element, expand LIS
    }
    return size;
}
// Time Complexity: O(N log N)
// Space Complexity: O(N)

```

12.4 Section 4: CodeSignal Classic Q4 Patterns (10 Additional Solved Problems)

12.4.1 21. Find Minimum in Rotated Sorted Array

Specification: Return the minimum element in a rotated sorted array in $\mathcal{O}(\log N)$. **Example:** `[3,4,5,1,2]` → output 1. **Pattern:** Binary Search **Explanation:** If `nums[mid] > nums[right]`, the minimum is in the right half. Else, the minimum is in the left half (including mid).

```

public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;

```

```

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right]) left = mid + 1;
        else right = mid;
    }
    return nums[left];
}
// Time Complexity: O(log N)
// Space Complexity: O(1)

```

12.4.2 22. Kth Smallest Element in Sorted Matrix

Specification: Find the K-th smallest element in a matrix where rows and columns are sorted.

Example: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8 → output 13. **Pattern:** Binary Search on Answer Space **Explanation:** Binary search the value space [min, max]. Count how many elements are $\leq$ mid. If count < k, left = mid + 1. Else right = mid.

```

public int kthSmallest(int[][] matrix, int k) {
    int n = matrix.length;
    int left = matrix[0][0], right = matrix[n-1][n-1];
    while (left < right) {
        int mid = left + (right - left) / 2;
        int count = countLessEqual(matrix, mid);
        if (count < k) left = mid + 1;
        else right = mid;
    }
    return left;
}

private int countLessEqual(int[][] matrix, int target) {
    int n = matrix.length, i = n - 1, j = 0, count = 0;
    while (i >= 0 && j < n) {
        if (matrix[i][j] <= target) { count += i + 1; j++; }
        else { i--; }
    }
    return count;
}
// Time Complexity: O(N log(Max - Min))
// Space Complexity: O(1)

```

12.4.3 23. Jump Game II

Specification: Return minimum jumps to reach the last index. You can jump up to nums[i] steps from index i. **Example:** [2,3,1,1,4] → output 2. **Pattern:** Greedy BFS levels **Explanation:** We maintain the farthest reach for the current jump level. When i == currentEnd, we must make a jump and update currentEnd = farthest.

```

public int jump(int[] nums) {
    int jumps = 0, currentEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
    }
}

```

```

        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}
// Time Complexity: O(N)
// Space Complexity: O(1)

```

12.4.4 24. Unique Paths

Specification: Count ways to reach bottom-right from top-left moving only right and down. **Example:** $m = 3, n = 7 \rightarrow$ output 28. **Pattern:** 2D DP **Explanation:** $dp[i][j] = dp[i-1][j] + dp[i][j-1]$.

```

public int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++) dp[i][0] = 1;
    for (int j = 0; j < n; j++) dp[0][j] = 1;
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[m-1][n-1];
}
// Time Complexity: O(M * N)
// Space Complexity: O(M * N) (can be optimized to O(N))

```

12.4.5 25. Maximum Subarray / Kadane's Algorithm

Specification: Find contiguous subarray with largest sum. **Example:** $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow$ output 6. **Pattern:** DP / Greedy **Explanation:** At each step, either add the current element to the previous sum, or start a new subarray if the previous sum is negative.

```

public int maxSubArray(int[] nums) {
    int maxSum = nums[0], currentSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
// Time Complexity: O(N)
// Space Complexity: O(1)

```

12.4.6 26. Climbing Stairs

Specification: Number of ways to climb n stairs (taking 1 or 2 steps). **Example:** $n = 3 \rightarrow$ output 3. **Pattern:** Fibonacci DP **Explanation:** $dp[i] = dp[i-1] + dp[i-2]$.

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
// Time Complexity: O(N)
// Space Complexity: O(1)
```

12.4.7 27. Largest Rectangle in Histogram

Specification: Find area of largest rectangle in histogram. **Example:** $[2,1,5,6,2,3] \rightarrow$ output 10. **Pattern:** Monotonic Stack **Explanation:** Stack stores indices of strictly increasing heights. Pop when a smaller height is found, calculating area using the popped height as the bottleneck.

```
public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int maxArea = 0, n = heights.length;
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek()]) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
// Time Complexity: O(N)
// Space Complexity: O(N)
```

12.4.8 28. Merge K Sorted Lists

Specification: Merge K sorted linked lists into one sorted list. **Example:** $[[1,4,5],[1,3,4],[2,6]] \rightarrow$ output $[1,1,2,3,4,4,5,6]$. **Pattern:** Min-Heap **Explanation:** Put all list heads into a PriorityQueue. Extract the min, append to result, and insert the next node from the extracted list.

```
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b) -> a.val - b.val);
    for (ListNode head : lists) {
```



```

        if (head != null) pq.offer(head);
    }
    ListNode dummy = new ListNode(0), curr = dummy;
    while (!pq.isEmpty()) {
        ListNode minNode = pq.poll();
        curr.next = minNode;
        curr = curr.next;
        if (minNode.next != null) pq.offer(minNode.next);
    }
    return dummy.next;
}
// Time Complexity: O(N log K)
// Space Complexity: O(K)

```

12.4.9 29. Longest Valid Parentheses

Specification: Find length of longest valid (well-formed) parentheses substring. **Example:** ")()())" → output 4. **Pattern:** DP **Explanation:** dp[i] is the length of longest valid substring ending at i. If s[i] == ')' and s[i-1] == '(', dp[i] = dp[i-2] + 2. If s[i-1] == ')', match earlier part.

```

public int longestValidParentheses(String s) {
    int maxLen = 0;
    int[] dp = new int[s.length()];
    for (int i = 1; i < s.length(); i++) {
        if (s.charAt(i) == ')') {
            if (s.charAt(i - 1) == '(') {
                dp[i] = (i >= 2 ? dp[i - 2] : 0) + 2;
            } else if (i - dp[i - 1] > 0 && s.charAt(i - dp[i - 1] - 1) == '(') {
                dp[i] = dp[i - 1] + ((i - dp[i - 1]) >= 2 ? dp[i - dp[i - 1] - 2]
                : 0) + 2;
            }
            maxLen = Math.max(maxLen, dp[i]);
        }
    }
    return maxLen;
}
// Time Complexity: O(N)
// Space Complexity: O(N)

```

12.4.10 30. Container With Most Water

Specification: Find two lines that together with x-axis forms a container holding the most water. **Example:** [1,8,6,2,5,4,8,3,7] → output 49. **Pattern:** Two-pointer **Explanation:** Area is width * min(h[L], h[R]). Move the pointer pointing to the shorter line to potentially find a taller line.

```

public int maxArea(int[] height) {
    int maxArea = 0;
    int left = 0, right = height.length - 1;

```

```

while (left < right) {
    int w = right - left;
    int h = Math.min(height[left], height[right]);
    maxArea = Math.max(maxArea, w * h);
    if (height[left] < height[right]) left++;
    else right--;
}
return maxArea;
}
// Time Complexity: O(N)
// Space Complexity: O(1)

```

12.5 Section 5: Practice Problem Bank (30 Detailed Problems with Hints)

31. Capacity To Ship Packages Within D Days Specification: A conveyor belt has packages that must be shipped in D days. The i-th package has weight `weights[i]`. Each day, you load the ship with packages in the order given up to the ship's max weight capacity. Return the least weight capacity of the ship. **Example:** `weights = [1,2,3,4,5,6,7,8,9,10]`, `D = 5` → output 15. **Constraints:** `1 <= D <= weights.length <= 5*104` **Hint:** Use Binary Search on Answer Space (`[max(weights), sum(weights)]`).

32. Russian Doll Envelopes Specification: Given a 2D array of envelopes `[width, height]`, you can put one inside another if both width and height of the inner are strictly smaller. Find the maximum number of envelopes you can Russian doll. **Example:** `envelopes = [[5,4],[6,4],[6,7],[2,3]]` → output 3. **Constraints:** `1 <= envelopes.length <= 105` **Hint:** Sort by width ASC and height DESC, then apply Longest Increasing Subsequence logic with DP + Binary Search.

33. Course Schedule Specification: There are `numCourses` to take. Some courses have prerequisites. Determine if it is possible to finish all courses. **Example:** `numCourses = 2`, `prerequisites = [[1,0],[0,1]]` → output false. **Constraints:** `1 <= numCourses <= 2000` **Hint:** Use Kahn's Algorithm for Topological Sort to detect cycles in the DAG.

34. Next Greater Element II Specification: Given a circular integer array, return the next greater number for every element. If it doesn't exist, return -1. **Example:** `nums = [1,2,1]` → output `[2,-1,2]`. **Constraints:** `1 <= nums.length <= 104` **Hint:** Use a Monotonic Stack and loop through the array twice to simulate circularity.

35. Koko Eating Bananas Specification: Koko wants to eat all bananas in H hours. Return her minimum eating speed K bananas per hour. **Example:** `piles = [3,6,7,11]`, `H = 8` → output 4. **Constraints:** `1 <= piles.length <= 104` **Hint:** Use Binary Search on Answer Space with bounds `[1, max(piles)]`.

36. Palindrome Partitioning II Specification: Given a string, partition it such that every substring is a palindrome. Return the minimum cuts needed. **Example:** `s = "aab"` → output 1 ("aa", "b"). **Constraints:** `1 <= s.length <= 2000` **Hint:** 1D DP where `dp[i]` is min cuts for suffix `s[i..n]`. Expand from centers to find palindromes.

- 37. Search a 2D Matrix Specification:** Write an efficient algorithm that searches for a value in an $m \times n$ matrix. Each row is sorted from left to right, and the first integer of each row is greater than the last integer of the previous row. **Example:** `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3` → output `true`. **Constraints:** `m == matrix.length`, `n == matrix[i].length`, `1 <= m`, `n <= 100` **Hint:** Treat the 2D matrix as a flat 1D array and use standard Binary Search.
- 38. Minimum Path Sum Specification:** Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right which minimizes the sum of all numbers along its path. **Example:** `grid = [[1,3,1],[1,5,1],[4,2,1]]` → output 7. **Constraints:** `1 <= m`, `n <= 200` **Hint:** 2D DP modifying the grid in-place: `grid[i][j] += min(grid[i-1][j], grid[i][j-1])`.
- 39. Perfect Squares Specification:** Given an integer n , return the least number of perfect square numbers that sum to n . **Example:** `n = 12` → output 3 ($4 + 4 + 4$). **Constraints:** `1 <= n <= 10^4` **Hint:** 1D DP similar to Coin Change where coins are perfect squares up to `sqrt(n)`.
- 40. Combination Sum IV Specification:** Given an array of distinct integers and a target, return the number of possible combinations that add up to target. **Example:** `nums = [1,2,3]`, `target = 4` → output 7. **Constraints:** `1 <= nums.length <= 200` **Hint:** 1D DP where `dp[i] += dp[i - num]` for all valid `num` in `nums`.
- 41. Split Array Largest Sum Specification:** Split an array into k non-empty contiguous subarrays such that the largest sum among these subarrays is minimized. **Example:** `nums = [7,2,5,10,8]`, `k = 2` → output 18. **Constraints:** `1 <= nums.length <= 1000` **Hint:** Binary Search on Answer Space where `left = max(nums)` and `right = sum(nums)`.
- 42. Trapping Rain Water II Specification:** Given an $m \times n$ integer matrix of heights, return the volume of water it can trap after raining. **Example:** `heightMap = [[1,4,3,1,3,2],[3,2,1,3,2,4],[2,3,3,2,3,1]]` → output 4. **Constraints:** `1 <= m`, `n <= 200` **Hint:** Use a Min-Heap starting with boundary cells and simulate a rising water level using BFS.
- 43. Maximize Distance to Closest Person Specification:** In a row of seats, 1 means occupied, 0 means empty. Find a seat to maximize distance to the closest person. **Example:** `seats = [1,0,0,0,1,0,1]` → output 2. **Constraints:** `2 <= seats.length <= 20000` **Hint:** Two-pointer approach counting zeros between ones, with edge cases for edges of the array.
- 44. Minimum Window Substring Specification:** Given two strings s and t , return the minimum window substring of s such that every character in t is included in the window. **Example:** `s = "ADOBE CODEBANC"`, `t = "ABC"` → output "BANC". **Constraints:** `1 <= s.length`, `t.length <= 10^5` **Hint:** Sliding Window Two-Pointer with a character frequency map to track fulfillment.
- 45. Largest Divisible Subset Specification:** Given a set of distinct positive integers, find the largest subset such that every pair (S_i, S_j) satisfies $S_i \% S_j == 0$ or $S_j \% S_i == 0$. **Example:** `nums = [1,2,3]` → output `[1,2]` or `[1,3]`. **Constraints:** `1 <= nums.length <= 1000` **Hint:** Sort first. Use 1D DP `dp[i]` representing the max subset ending with `nums[i]`, tracking parents to reconstruct.
- 46. Interleaving String Specification:** Given strings s_1 , s_2 , and s_3 , find whether s_3 is formed by an interleaving of s_1 and s_2 . **Example:** `s1 = "aabcc"`, `s2 = "dbbca"`, `s3 = "aadb-bcbcac"` → output `true`. **Constraints:** `0 <= s1.length`, `s2.length <= 100` **Hint:** 2D DP

where `dp[i][j]` means if `s3.substring(0, i+j)` can be formed by `s1.substring(0, i)` and `s2.substring(0, j)`.

47. Shortest Path in Binary Matrix Specification: Find the shortest clear path from top-left to bottom-right in a grid. **Example:** `grid = [[0,1],[1,0]]` → output 2. **Constraints:** $1 \leq n \leq 100$ **Hint:** BFS with Level Tracking since we want the shortest path in an unweighted grid with 8 directions.

48. Task Scheduler Specification: Given an array of CPU tasks and a cooldown `n`, return the least number of intervals needed to finish all tasks. **Example:** `tasks = ["A","A","A","B","B","B"]`, `n = 2` → output 8. **Constraints:** $1 \leq \text{tasks.length} \leq 10^4$ **Hint:** Greedy approach or Math formula based on the frequency of the most common task.

49. 132 Pattern Specification: Given an array of integers, find if there is a 132 pattern ($i < j < k$ and `nums[i] < nums[k] < nums[j]`). **Example:** `nums = [3,1,4,2]` → output `true`. **Constraints:** $1 \leq \text{nums.length} \leq 2 * 10^5$ **Hint:** Traverse backwards maintaining a Monotonic Stack to find the `nums[k]` value while keeping track of `max nums[k]`.

50. Minimum Size Subarray Sum Specification: Return the minimal length of a contiguous subarray of which the sum is greater than or equal to `target`. **Example:** `target = 7`, `nums = [2,3,1,2,4,3]` → output 2. **Constraints:** $1 \leq \text{nums.length} \leq 10^5$ **Hint:** Sliding Window Two-Pointer. Expand right until sum is met, then contract left to minimize.

51. Frog Jump Specification: A frog crosses a river with stones. If the last jump was `k` units, the next jump must be `k-1`, `k`, or `k+1`. Can it reach the last stone? **Example:** `stones = [0,1,3,5,6,8,12,17]` → output `true`. **Constraints:** $2 \leq \text{stones.length} \leq 2000$ **Hint:** 2D DP or Hash Map of sets where `map.get(stone)` contains all possible jump lengths that reached this stone.

52. Rotting Oranges Specification: Every minute, any fresh orange adjacent to a rotten one becomes rotten. Return minimum minutes until no cell has a fresh orange. **Example:** `grid = [[2,1,1],[1,1,0],[0,1,1]]` → output 4. **Constraints:** $1 \leq m, n \leq 10$ **Hint:** Multi-source BFS starting with all rotten oranges in the queue at minute 0.

53. Word Break Specification: Given a string and a dictionary, determine if the string can be segmented into a space-separated sequence of dictionary words. **Example:** `s = "leetcode"`, `wordDict = ["leet", "code"]` → output `true`. **Constraints:** $1 \leq \text{s.length} \leq 300$ **Hint:** 1D DP `dp[i]` is true if `s.substring(0, i)` can be broken down.

54. Max Consecutive Ones III Specification: Given a binary array and an integer `k`, return the max number of consecutive 1s if you can flip at most `k` 0s. **Example:** `nums = [1,1,1,0,0,0,1,1,1,1,0]`, `k = 2` → output 6. **Constraints:** $1 \leq \text{nums.length} \leq 10^5$ **Hint:** Sliding Window. The window can contain at most `k` zeros.

55. Jump Game Specification: Determine if you can reach the last index starting from the first. **Example:** `nums = [2,3,1,1,4]` → output `true`. **Constraints:** $1 \leq \text{nums.length} \leq 10^4$ **Hint:** Greedy approach tracking the maximum reachable index `maxReach = max(maxReach, i + nums[i])`.

56. Wiggle Subsequence Specification: Find length of longest subsequence that alternates between strictly increasing and strictly decreasing. **Example:** `nums = [1,7,4,9,2,5]` → output 6. **Constraints:** $1 \leq \text{nums.length} \leq 1000$ **Hint:** 1D DP tracking the longest ending with an “up” transition and a “down” transition.

57. Predict the Winner Specification: Two players pick numbers from either end of an array. Determine if Player 1 can guarantee a win. **Example:** `nums = [1, 5, 2]` → output `false`. **Constraints:** `1 <= nums.length <= 20` **Hint:** Interval DP `dp[i][j]` representing the max score difference a player can achieve taking from `[i, j]`.

58. Find K-th Smallest Pair Distance Specification: Find the K-th smallest distance among all pairs `(nums[i], nums[j])` in an array. **Example:** `nums = [1,3,1]`, `k = 1` → output 0. **Constraints:** `n <= 10^4` **Hint:** Binary Search on Answer Space with sliding window to count pairs with distance $\leq$ mid.

59. Cheapest Flights Within K Stops Specification: Find the cheapest price from `src` to `dst` with up to `k` stops. **Example:** `n = 3`, `flights = [[0,1,100],[1,2,100],[0,2,500]]`, `src = 0`, `dst = 2`, `k = 1` → output 200. **Constraints:** `1 <= n <= 100` **Hint:** Bellman-Ford or BFS with level tracking (up to K levels) tracking minimum costs.

60. Remove K Digits Specification: Given a string representing a non-negative integer, remove `k` digits to form the smallest possible integer. **Example:** `num = "1432219"`, `k = 3` → output "1219". **Constraints:** `1 <= num.length <= 10^5` **Hint:** Monotonic Stack (increasing). Pop strictly larger digits while `k > 0`.

Chapter 13

20 Exam-Grade GCA Mock Problem Sets

13.1 How to Use This Chapter

This chapter provides 20 full, four-question exam mock sets (80 problems total) modeled directly after the General Coding Assessment (GCA) blueprint. Each set is designed to simulate the rigorous 70-minute assessment environment you will face during a real coding interview. The problems strictly adhere to the expected difficulty curve: Q1 tests basic implementation and traversal (Easy, 5-8 minutes), Q2 focuses on 2D matrices and simulation (Medium, 12-15 minutes), Q3 requires algorithmic pattern recognition like HashMaps or sliding windows (Medium-Hard, 18-20 minutes), and Q4 challenges you with dynamic programming, graphs, or advanced data structures (Hard, 20-25 minutes).

To get the most out of these mock exams, strictly time yourself. Set a timer for 70 minutes and attempt all four questions in order. Do not look up syntax or external resources. If you get stuck on Q3 or Q4, practice timeboxing: move on and secure partial credit where possible. After the 70 minutes expire, review your performance. Use the provided hints to guide your post-exam study sessions, identifying which specific patterns (e.g., sliding window, BFS, monotonic stack) require further review.

Remember, there is no code in this chapter—this is your practice arena. Read the specifications, analyze the test cases, check the constraints, and write your own optimal solutions.

13.2 Set 1: Warm-Up Fundamentals

- **Q1 (Easy): Vowel Starting Words**
 - *Specification:* Given a string of text containing words separated by single spaces, calculate the total number of words that begin with a vowel. Vowels are defined as 'a', 'e', 'i', 'o', and 'u', and the check should be case-insensitive. Ignore any punctuation, assuming the string consists only of alphabetical characters and spaces. Return the final integer count of qualifying words.
 - *Sample Test Case:* Input: "Apple banana Orange umbrella" -> Output: 3.

- *Constraints:* String length $1 \leq L \leq 10^5$.
 - *Hint:* Use standard string splitting to isolate words, then check the first character of each token against a predefined set of vowels.
 - **Q2 (Medium): Rotate Rectangular Image**
 - *Specification:* You are given an $M \times N$ 2D matrix representing an image, where each cell holds a pixel value. Your task is to rotate the image 90 degrees clockwise. Unlike square matrix rotation, this matrix is rectangular, meaning the dimensions of the resulting matrix will swap to $N \times M$. You must allocate a new matrix to hold the rotated values and populate it correctly.
 - *Sample Test Case:* Input: `[[1, 2, 3], [4, 5, 6]]` -> Output: `[[4, 1], [5, 2], [6, 3]]`.
 - *Constraints:* $1 \leq M, N \leq 1000$.
 - *Hint:* The element at `matrix[r][c]` in the original matrix moves to `new_matrix[c][M - 1 - r]` in the rotated matrix.
 - **Q3 (Medium-Hard): K-Frequency Substring**
 - *Specification:* Given a string and an integer K, find the length of the longest contiguous substring where no character appears more than K times. You must process the string and keep track of character frequencies dynamically. If the frequency of any character exceeds K, you must shrink the valid sequence until the condition is met again. Return the maximum length observed.
 - *Sample Test Case:* Input: `s = "abaccc"`, `K = 2` -> Output: 4 (The substring “abac”).
 - *Constraints:* String length $1 \leq L \leq 10^5$, $1 \leq K \leq L$.
 - *Hint:* Use a sliding window approach with two pointers and a HashMap or frequency array to track character counts within the current window.
 - **Q4 (Hard): Largest Rectangular Area**
 - *Specification:* You are given an array of non-negative integers representing the heights of adjacent buildings, where each building has a width of 1 unit. You need to calculate the area of the largest rectangle that can be formed within the bounds of these buildings. The rectangle must be completely contained within the histograms. Return the maximum possible area.
 - *Sample Test Case:* Input: `[2, 1, 5, 6, 2, 3]` -> Output: 10 (Formed by heights 5 and 6).
 - *Constraints:* Array length $1 \leq N \leq 10^5$, building heights $0 \leq H \leq 10^4$.
 - *Hint:* Utilize a monotonic increasing stack to keep track of building indices, calculating areas when a drop in height is encountered.
-

13.3 Set 2: String & Matrix Basics

- **Q1 (Easy): IPv4 Validation**
 - *Specification:* Write a function to determine if a given string is a valid IPv4 address. A valid IPv4 address consists of exactly four octets separated by periods. Each octet must be a numeric value between 0 and 255, inclusive. Octets cannot contain leading zeros unless the octet is exactly the single digit ‘0’.
 - *Sample Test Case:* Input: `"192.168.0.1"` -> Output: `true`. Input: `"192.168.01.1"` -> Output: `false`.
 - *Constraints:* String length $1 \leq L \leq 30$.

- *Hint:* Split the string by periods, verify there are exactly four parts, and systematically check each part for length, digit-only composition, and numerical range.
 - **Q2 (Medium): Distinct Islands**
 - *Specification:* Given a 2D binary grid where 1 represents land and 0 represents water, count the number of distinct islands. An island is formed by connected 1s (horizontally or vertically). Two islands are considered the same if one can be translated (shifted) to match the other perfectly. Rotations or reflections do not count as translations.
 - *Sample Test Case:* Input: `[[1,1,0],[1,0,0],[0,0,1]]` -> Output: 2.
 - *Constraints:* Grid size $1 \leq R, C \leq 50$.
 - *Hint:* Use DFS to traverse each island, recording the path signature or relative coordinates from the starting cell to uniquely identify the island's shape.
 - **Q3 (Medium-Hard): Top K Active Users**
 - *Specification:* You are given a list of log entries where each entry is a string array containing a timestamp and a userId. Your objective is to identify the most active users on the platform. Calculate the frequency of entries for each user and return the top K userIds with the highest entry counts. If there is a tie in counts, sort them lexicographically by userId.
 - *Sample Test Case:* Input: `logs = [{"10:00", "user1"}, {"10:05", "user2"}, {"10:10", "user1"}]`, `K = 1` -> Output: `["user1"]`.
 - *Constraints:* Number of logs $1 \leq N \leq 10^5$, $1 \leq K \leq$ unique users.
 - *Hint:* Aggregate counts using a HashMap, then use a PriorityQueue (Min-Heap) of size K or bucket sort to efficiently find the top elements.
 - **Q4 (Hard): Unlimited Coin Change**
 - *Specification:* Given an integer array representing different coin denominations and an integer target representing a total monetary amount, determine the minimum number of coins needed to make up that amount. You may assume you have an infinite supply of each denomination. If the target amount cannot be met by any combination of the coins, return -1.
 - *Sample Test Case:* Input: `coins = [1, 2, 5]`, `target = 11` -> Output: 3 (5 + 5 + 1).
 - *Constraints:* $1 \leq \text{coins.length} \leq 12$, $1 \leq \text{coins}[i] \leq 2^{31} - 1$, $0 \leq \text{target} \leq 10^4$.
 - *Hint:* This is a classic 1D Dynamic Programming problem (unbounded knapsack). Build a dp array where `dp[i]` stores the minimum coins to reach amount i.
-

13.4 Set 3: Pattern Recognition

- **Q1 (Easy): String Rotation Check**
 - *Specification:* You are given two strings, A and B. You need to verify if string B is a valid rotation of string A. A string is a rotation if it can be formed by moving some number of characters from the front of the string to the back, maintaining the order of the rest. Both strings must be of identical length to be considered valid rotations.
 - *Sample Test Case:* Input: `A = "waterbottle"`, `B = "erbottlewat"` -> Output: `true`.
 - *Constraints:* String lengths $1 \leq L \leq 10^5$.
 - *Hint:* Concatenate string A with itself (`A + A`); if B is a rotation, it must exist as a contiguous substring within this doubled string.
- **Q2 (Medium): Robot Grid Simulation**

- *Specification:* A robot starts at the origin (0,0) on a 2D Cartesian plane. It receives a sequence of movement commands represented by a string of characters: ‘U’ (up), ‘D’ (down), ‘L’ (left), and ‘R’ (right). You must simulate the robot’s entire movement sequence. Return true if the robot ends up exactly back at the origin (0,0) after executing all commands, and false otherwise.
 - *Sample Test Case:* Input: "UDLR" -> Output: **true**. Input: "LL" -> Output: **false**.
 - *Constraints:* Command string length $1 \leq L \leq 10^4$.
 - *Hint:* Track the X and Y coordinates. Increment or decrement them based on the character, and simply check if $X == 0$ and $Y == 0$ at the end.
 - **Q3 (Medium-Hard): Meeting Rooms Required**
 - *Specification:* You are given an array of meeting time intervals where each interval consists of a start time and an end time. Multiple meetings might overlap. You need to determine the minimum number of conference rooms required to schedule all meetings without any conflicts. A meeting ending exactly when another begins does not constitute an overlap.
 - *Sample Test Case:* Input: $[[0, 30], [5, 10], [15, 20]]$ -> Output: 2.
 - *Constraints:* Number of meetings $1 \leq N \leq 10^4$, $0 \leq \text{start} < \text{end} \leq 10^6$.
 - *Hint:* Separate the start times and end times into two sorted arrays. Use a two-pointer approach to sweep through time, incrementing room count on starts and decrementing on ends.
 - **Q4 (Hard): Longest Increasing Subsequence**
 - *Specification:* Given an unsorted array of integers, locate the length of the longest strictly increasing subsequence. A subsequence is derived by deleting some or no elements from the array without altering the order of the remaining elements. The sequence must strictly increase, meaning equal values do not count as increasing.
 - *Sample Test Case:* Input: $[10, 9, 2, 5, 3, 7, 101, 18]$ -> Output: 4 (The sequence is $[2, 3, 7, 101]$).
 - *Constraints:* Array length $1 \leq N \leq 2500$, $-10^4 \leq \text{nums}[i] \leq 10^4$.
 - *Hint:* While an $O(N^2)$ DP approach works for small constraints, aim for $O(N \log N)$ using an array to build the active sequence and binary search (**bisect**) to find insertion points.
-

13.5 Set 4: Boundary Logic

- **Q1 (Easy): Reverse Integer Digits**
 - *Specification:* Given a signed 32-bit integer, completely reverse its digits and return the new integer. If the integer is negative, the reversed result must also remain negative. If reversing the integer causes it to overflow outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, you must return 0 instead of a garbage value.
 - *Sample Test Case:* Input: -123 -> Output: -321. Input: 120 -> Output: 21.
 - *Constraints:* $-2^{31} \leq N \leq 2^{31} - 1$.
 - *Hint:* Extract digits using modulo 10 and build the reversed number. Check for overflow before multiplying the accumulated result by 10.
- **Q2 (Medium): Shortest Path in Binary Matrix**
 - *Specification:* You are given an $N \times N$ binary matrix where 0 represents an open, passable cell and 1 represents a blocked obstacle. Find the length of the shortest clear path from

the top-left cell (0,0) to the bottom-right cell (N-1, N-1). You can move in 8 directions (horizontal, vertical, and diagonal). If no path exists, return -1.

- *Sample Test Case:* Input: `[[0,1],[1,0]]` -> Output: 2.
 - *Constraints:* $1 \leq N \leq 100$.
 - *Hint:* Breadth-First Search (BFS) is optimal for finding shortest paths in an unweighted grid. Queue coordinates and distance, marking cells visited as you enqueue them.
 - **Q3 (Medium-Hard): Group Isomorphic Strings**
 - *Specification:* You are given an array of strings. Two strings are considered isomorphic if the characters in the first string can be replaced to get the second string, preserving the order and structure (e.g., “egg” and “add”). Group all mutually isomorphic strings together in lists. Return the grouped lists in any order.
 - *Sample Test Case:* Input: `["aab", "xxy", "xyz", "def"]` -> Output: `[["aab", "xxy"], ["xyz", "def"]]`.
 - *Constraints:* Array length $1 \leq N \leq 10^4$, String length $1 \leq L \leq 50$.
 - *Hint:* Normalize each string into a structural pattern (e.g., “aab” -> “1-1-2”, “egg” -> “1-2-2”) and use this pattern as a key in a HashMap to group matching strings.
 - **Q4 (Hard): Decode Ways**
 - *Specification:* A message containing letters from A-Z is encoded into numbers using the mapping ‘A’ -> 1, ‘B’ -> 2, ..., ‘Z’ -> 26. Given a string of digits, determine the total number of ways it can be decoded back into letters. A decoding is valid only if it maps strictly to the 1-26 range; leading zeros in groups are invalid (e.g., “06” cannot be ‘F’).
 - *Sample Test Case:* Input: `"226"` -> Output: 3 (Can be decoded as “BZ”, “VF”, or “BBF”).
 - *Constraints:* String length $1 \leq L \leq 100$.
 - *Hint:* Use 1D Dynamic Programming. `dp[i]` is the number of ways to decode the prefix of length `i`. Look at the single digit at `i-1` and the two digits at `i-2` to update the state.
-

13.6 Set 5: Simulation & State

- **Q1 (Easy): FizzBuzz Array Variant**
 - *Specification:* Implement the classic FizzBuzz game, but return the results as an array of strings from 1 to N. For multiples of 3, append “Fizz”. For multiples of 5, append “Buzz”. For multiples of both 3 and 5, append “FizzBuzz”. For all other numbers, append the number itself as a string.
 - *Sample Test Case:* Input: `N = 15` -> Output: `[..., "13", "14", "FizzBuzz"]`.
 - *Constraints:* $1 \leq N \leq 10^4$.
 - *Hint:* Use conditional logic. Check divisibility by 15 (both 3 and 5) first, then 3, then 5, to avoid overriding the “FizzBuzz” condition.
- **Q2 (Medium): Game of Life Simulation**
 - *Specification:* You are given an $M \times N$ grid representing the current state of Conway’s Game of Life (1 is live, 0 is dead). Compute the next state of the board simultaneously for every cell based on its 8 neighbors. A live cell with 2-3 live neighbors survives. A dead cell with exactly 3 live neighbors becomes live. All other cells die or remain dead. Do this in-place if possible.
 - *Sample Test Case:* Input: `[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]` -> Output: `[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]`.

- *Constraints:* $1 \leq M, N \leq 25$.
 - *Hint:* To achieve an in-place update, use intermediate states (like 2 for “was live, now dead” and 3 for “was dead, now live”) so you can evaluate the original state without allocating a new matrix.
 - **Q3 (Medium-Hard): Remove K Invalid Brackets**
 - *Specification:* You are given a string containing alphanumeric characters and brackets ‘(’ and ‘)’. You are also given an integer K. You must remove exactly K brackets (either open or close) such that the resulting string contains valid, properly nested brackets. Find and return all unique valid string permutations that can result from this removal.
 - *Sample Test Case:* Input: `s = "()())()"`, `K = 1` -> Output: `["(())()", "()(())"]`.
 - *Constraints:* String length $1 \leq L \leq 25$, $0 \leq K \leq L$.
 - *Hint:* Use Breadth-First Search (BFS) combined with a queue and a HashSet for deduplication. Generate all states by removing one character at a time, checking validity.
 - **Q4 (Hard): Dijkstra’s Shortest Path Array**
 - *Specification:* You are given a directed, weighted graph represented by an adjacency list, a total number of nodes N, and a starting node S. Calculate the shortest path distance from the starting node S to every other node in the graph. The weights are guaranteed to be non-negative. Return an array of these distances, using -1 for unreachable nodes.
 - *Sample Test Case:* Input: `N = 3`, `edges = [[0,1,5], [0,2,2], [2,1,1]]`, `S = 0` -> Output: `[0, 3, 2]`.
 - *Constraints:* $1 \leq N \leq 1000$, edge weights $0 \leq W \leq 10^4$.
 - *Hint:* Implement Dijkstra’s algorithm using a Priority Queue (Min-Heap). Enqueue tuples of (`current_distance`, `node`) and update adjacent nodes dynamically.
-

13.7 Set 6: Array Manipulation

- **Q1 (Easy): Remove Duplicates In-Place**
 - *Specification:* Given a sorted array of integers, remove all duplicate elements in-place such that each unique element appears only once. The relative order of the unique elements must be kept the same. Since you cannot alter the array’s physical length in all languages, place the unique items at the front and return the count of unique elements.
 - *Sample Test Case:* Input: `[1, 1, 2, 3, 3]` -> Output: 3 (Array modifies to `[1, 2, 3, ...]`).
 - *Constraints:* $1 \leq N \leq 3 \times 10^4$.
 - *Hint:* Use two pointers: a slow pointer to track the position of the next unique element, and a fast pointer to scan for new, unseen numbers.
- **Q2 (Medium): Matrix Flood Fill**
 - *Specification:* Implement a flood fill algorithm. You are given a 2D grid of integers representing pixel colors, a starting row and column, and a new target color. Change the color of the starting pixel and all orthogonally adjacent pixels of the *same original color* to the new target color, stopping at boundaries or different colors. Return the modified grid.
 - *Sample Test Case:* Input: `grid = [[1,1,1],[1,1,0],[1,0,1]]`, `sr=1`, `sc=1`, `color=2` -> Output: `[[2,2,2],[2,2,0],[2,0,1]]`.
 - *Constraints:* $1 \leq M, N \leq 50$.

- *Hint:* Use Depth-First Search (DFS). If the starting cell is already the target color, return immediately to prevent infinite recursion.
 - **Q3 (Medium-Hard): Median of Two Sorted Arrays**
 - *Specification:* You are given two independent sorted integer arrays, A and B. Find the exact median of the combined sorted elements of both arrays. If the combined length is even, the median is the average of the two middle elements. The overall run time complexity must strictly be $O(\log(m + n))$.
 - *Sample Test Case:* Input: A = [1, 3], B = [2] -> Output: 2.0.
 - *Constraints:* $0 \leq \text{A.length}, \text{B.length} \leq 1000$, $1 \leq \text{total length} \leq 2000$.
 - *Hint:* Do not merge the arrays. Use binary search on the smaller array to find an optimal partition point that splits both arrays into balanced left and right halves.
 - **Q4 (Hard): Trapping Rain Water**
 - *Specification:* Given an array of non-negative integers where each value represents the elevation height of a terrain segment (width 1), compute how much total water the terrain can trap after a heavy rain. Water is trapped in valleys between higher elevation peaks on both the left and right sides.
 - *Sample Test Case:* Input: [0,1,0,2,1,0,1,3,2,1,2,1] -> Output: 6.
 - *Constraints:* Array length $1 \leq N \leq 2 \times 10^4$.
 - *Hint:* A two-pointer approach (left and right) working inward is $O(1)$ space. Maintain `left_max` and `right_max`; process the smaller max side to guarantee valid trapping boundaries.
-

13.8 Set 7: Hash & Frequency

- **Q1 (Easy): Valid Anagram Check**
 - *Specification:* Given two strings S and T, write a function to determine if T is a valid anagram of S. An anagram is formed by rearranging the exact characters of a different string, using all original characters exactly once. The strings contain only lowercase English letters.
 - *Sample Test Case:* Input: S = "listen", T = "silent" -> Output: true.
 - *Constraints:* String lengths $1 \leq L \leq 5 \times 10^4$.
 - *Hint:* Use a fixed-size integer array of length 26 to tally character counts. Increment for S and decrement for T, verifying all counts are zero at the end.
- **Q2 (Medium): Anti-Diagonal Matrix Traversal**
 - *Specification:* You are given an $N \times N$ square matrix of integers. Return an array containing all the elements of the matrix ordered by their anti-diagonals, starting from the top-left corner and sweeping down towards the bottom-right. Elements on the same anti-diagonal share the same row and column index sum.
 - *Sample Test Case:* Input: [[1,2,3],[4,5,6],[7,8,9]] -> Output: [1, 2, 4, 3, 5, 7, 6, 8, 9].
 - *Constraints:* $1 \leq N \leq 100$.
 - *Hint:* Map elements using their coordinates. The key observation is that for any cell `[r][c]`, the sum `r + c` uniquely identifies which anti-diagonal line it belongs to.
- **Q3 (Medium-Hard): Maximum K-Window Sum**
 - *Specification:* You are given an array of integers and an integer K. Find the maximum possible sum of any contiguous subarray of size exactly K. The array can contain negative

numbers. You must process this efficiently without recalculating the sum of elements from scratch for every possible window position.

- *Sample Test Case:* Input: `nums = [1, 4, 2, 10, 2, 3, 1, 0, 20]`, `K = 4` -> Output: 24 (Subarray `[3, 1, 0, 20]`).
- *Constraints:* $1 \leq N \leq 10^5$, $1 \leq K \leq N$.
- *Hint:* Use the Sliding Window pattern. Compute the sum of the first `K` elements, then iterate by adding the new element entering the window and subtracting the element leaving.

- **Q4 (Hard): LRU Cache Implementation**

- *Specification:* Design and implement a data structure for a Least Recently Used (LRU) cache. It must support `get(key)` which returns the value if it exists (else -1), and `put(key, value)` which updates or inserts the value. If inserting exceeds the capacity, it must evict the least recently used key. Both operations must run in $O(1)$ average time complexity.
- *Sample Test Case:* Input: `capacity = 2; put(1,1); put(2,2); get(1) -> 1; put(3,3); get(2) -> -1.`
- *Constraints:* Capacity $1 \leq C \leq 3000$, up to 10^5 calls made.
- *Hint:* Combine a standard HashMap (for $O(1)$ key lookup) with a Doubly Linked List (for $O(1)$ node relocation to track most/least recently used order).

13.9 Set 8: Window & Range

- **Q1 (Easy): Count Palindromic Substrings**

- *Specification:* Given a string, determine the total number of substrings that are palindromes. A palindrome reads the same forwards and backwards. Note that a single character is mathematically considered a valid palindrome. Distinct substrings with identical characters at different indices are counted separately.
- *Sample Test Case:* Input: `"abc"` -> Output: 3 ("`a`", "`b`", "`c`"). Input: `"aaa"` -> Output: 6 ("`a`", "`a`", "`a`", "`aa`", "`aa`", "`aaa`").
- *Constraints:* String length $1 \leq L \leq 1000$.
- *Hint:* Iterate through each character and expand outwards from the center. Handle both odd-length (single character center) and even-length (two character center) palindromes.

- **Q2 (Medium): Rotting Oranges BFS**

- *Specification:* You are given an $M \times N$ grid containing 0 (empty), 1 (fresh orange), or 2 (rotten orange). Every minute, any fresh orange adjacent (4-directionally) to a rotten orange also becomes rotten. Calculate the minimum number of minutes required until no fresh oranges remain. If it is impossible to rot all oranges, return -1.
- *Sample Test Case:* Input: `[[2,1,1],[1,1,0],[0,1,1]]` -> Output: 4.
- *Constraints:* $1 \leq M, N \leq 10$.
- *Hint:* This is a multi-source Breadth-First Search. Enqueue all initially rotten oranges at minute 0, then process level by level, decrementing a total fresh count.

- **Q3 (Medium-Hard): Smallest Subarray Sum Target**

- *Specification:* Given an array of positive integers and a target positive integer, find the minimal length of a contiguous subarray whose sum is strictly greater than or equal to the target. If no such subarray exists that meets the target sum constraint, return 0.
- *Sample Test Case:* Input: `target = 7, nums = [2,3,1,2,4,3]` -> Output: 2 (Subar-

ray [4,3]).

- *Constraints:* $1 \leq N \leq 10^5$, array values and target up to 10^9 .
 - *Hint:* Employ a dynamic sliding window. Expand the right pointer to accumulate the sum, and shrink the left pointer as long as the sum remains $\geq$ target, recording the minimum length.
 - **Q4 (Hard): Course Schedule Topology**
 - *Specification:* You are given a total number of courses labeled from 0 to N-1, and an array of prerequisite pairs where [A, B] means course B must be completed before course A. Determine if it is mathematically possible to finish all courses. Return a valid chronological ordering of courses. If a cycle exists making it impossible, return an empty array.
 - *Sample Test Case:* Input: N = 2, pre = [[1,0]] -> Output: [0, 1].
 - *Constraints:* $1 \leq N \leq 2000$.
 - *Hint:* This requires Topological Sorting. Build a directed graph and an in-degree array. Use Kahn's algorithm (BFS queue) to process nodes with zero dependencies.
-

13.10 Set 9: Two-Pointer Mastery

- **Q1 (Easy): Move Targets to End**
 - *Specification:* Given an integer array and a specific target value, move all occurrences of that target to the end of the array while maintaining the relative ordering of the other non-target elements. You must perform this mutation in-place without allocating a duplicate array structure.
 - *Sample Test Case:* Input: nums = [0,1,0,3,12], target = 0 -> Output: [1,3,12,0,0].
 - *Constraints:* $1 \leq N \leq 10^4$.
 - *Hint:* Maintain an insertion index pointer. Iterate through the array; if the current element is not the target, swap it with the element at the insertion index and increment the index.
- **Q2 (Medium): Kth Smallest in Sorted Matrix**
 - *Specification:* You are given an $N \times N$ matrix where every row and every column is independently sorted in ascending order. Find the Kth smallest integer in the entire matrix. Note that it is the Kth smallest in global sorted order, not the Kth distinct element.
 - *Sample Test Case:* Input: matrix = [[1,5,9],[10,11,13],[12,13,15]], K = 8 -> Output: 13.
 - *Constraints:* $1 \leq N \leq 300$, $1 \leq K \leq N^2$.
 - *Hint:* Since rows and columns are sorted, use a Min-Heap starting with the first element of each row, or apply a clever Binary Search over the value range tracking counts.
- **Q3 (Medium-Hard): 3Sum Zero Target**
 - *Specification:* Given an integer array, identify and return all unique triplets [nums[i], nums[j], nums[k]] such that their sum equals exactly zero. The index of each element must be distinct. The output array of triplets must not contain any duplicate triplet combinations, regardless of internal ordering.
 - *Sample Test Case:* Input: [-1,0,1,2,-1,-4] -> Output: [[-1,-1,2],[-1,0,1]].
 - *Constraints:* $3 \leq N \leq 3000$.

- *Hint:* Sort the array first. Iterate through the array fixing one number, then use a two-pointer approach (left and right) on the remaining suffix to find pairs that sum to the inverse of the fixed number.
 - **Q4 (Hard): Longest Valid Parentheses**
 - *Specification:* Given a string composed strictly of '(' and ')' characters, compute the length of the longest contiguous valid (well-formed) parentheses substring. The valid substring must have properly matched and nested brackets.
 - *Sample Test Case:* Input: ")()())" -> Output: 4 (The substring is "()()").
 - *Constraints:* String length $0 \leq L \leq 3 \times 10^4$.
 - *Hint:* Utilize a stack storing indices. Initialize the stack with -1 to serve as a base index. On closing brackets, pop and measure the length against the new top of the stack.
-

13.11 Set 10: Greedy & Optimization

- **Q1 (Easy): Single Stock Profit**
 - *Specification:* You are given an array where each element represents the price of a given stock on that day. You are permitted to complete at most one transaction (buy one share and sell one share) in the future. Calculate the maximum profit you can achieve. If no profit can be made (prices only drop), return 0.
 - *Sample Test Case:* Input: [7,1,5,3,6,4] -> Output: 5 (Buy at 1, sell at 6).
 - *Constraints:* $1 \leq N \leq 10^5$.
 - *Hint:* Iterate through the array while maintaining a running variable of the minimum price seen so far. At each day, evaluate if selling at the current price yields a new maximum profit.
- **Q2 (Medium): Unique Paths with Obstacles**
 - *Specification:* A robot is positioned at the top-left corner of an $M \times N$ grid and is trying to reach the bottom-right corner. The robot can only move down or right. Some grid cells are marked as 1, representing an impassable obstacle. Empty cells are 0. Calculate the total number of unique valid paths to the destination.
 - *Sample Test Case:* Input: [[0,0,0],[0,1,0],[0,0,0]] -> Output: 2.
 - *Constraints:* $1 \leq M, N \leq 100$.
 - *Hint:* Use 2D Dynamic Programming. $dp[r][c]$ equals $dp[r-1][c] + dp[r][c-1]$. If a cell contains an obstacle, manually set its DP value to 0 paths.
- **Q3 (Medium-Hard): Merge Overlapping Intervals**
 - *Specification:* You are given an array of intervals where each interval is represented as [start, end]. Multiple intervals may overlap. Consolidate all overlapping intervals into a unified, non-overlapping array of intervals that perfectly covers all time spans represented by the original input. Return the condensed array.
 - *Sample Test Case:* Input: [[1,3],[2,6],[8,10],[15,18]] -> Output: [[1,6],[8,10],[15,18]].
 - *Constraints:* $1 \leq N \leq 10^4$.
 - *Hint:* Sort the intervals primarily by their starting times. Iterate and maintain a 'current' interval; merge if the next start is $\leq$ the current end, updating the end to the maximum of both.
- **Q4 (Hard): Jump Game Minimum**
 - *Specification:* You are given an array of non-negative integers where each value dictates

the maximum jump length you can take forward from that position. Assuming you always start at the first index, determine the absolute minimum number of jumps required to reach the last index of the array. The test cases guarantee the end is reachable.

- *Sample Test Case:* Input: `[2,3,1,1,4]` -> Output: 2 (Jump index 0 to 1, then jump to the end).
 - *Constraints:* $1 \leq N \leq 10^4$.
 - *Hint:* Use a Greedy BFS approach. Maintain two variables: `current_jump_end` and `farthest_reachable`. Iterate through the array; when you hit `current_jump_end`, you must jump, so increment jump count and update the end boundary.
-

13.12 Set 11: Tree & Recursion Simulation

- **Q1 (Easy): Happy Number Cycle**

- *Specification:* Write an algorithm to verify if a number is a “happy number”. To determine this, replace the number with the sum of the squares of its digits. Repeat this mathematical process until the number equals 1 (happy), or it loops endlessly in a recurring cycle that does not include 1 (unhappy). Return true if happy.
- *Sample Test Case:* Input: 19 -> Output: `true` ($1^2 + 9^2 = 82$, $8^2 + 2^2 = 68$, etc., eventually reaching 1).
- *Constraints:* $1 \leq N \leq 2^{31} - 1$.
- *Hint:* Use a HashSet to track previously computed sums. If a sum repeats before hitting 1, you have entered an infinite cycle and can return false.

- **Q2 (Medium): Word Search Traversal**

- *Specification:* You are given an $M \times N$ grid of characters and a target word string. Determine if the word can be constructed by traversing sequentially adjacent cells (horizontally or vertically). The same physical cell in the matrix cannot be used more than once during the construction of the word.
- *Sample Test Case:* Input: `board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, `word = "ABCCED"` -> Output: `true`.
- *Constraints:* $1 \leq M, N \leq 6$, word length ≤ 15 .
- *Hint:* Employ Backtracking / Depth-First Search from every matching starting letter. Temporarily mark cells as visited (e.g., changing it to ‘#’) to prevent reuse, restoring it afterward.

- **Q3 (Medium-Hard): Nested String Reversal**

- *Specification:* You are given a string that contains lowercase letters and properly nested parentheses pairs. You must reverse the exact character sequence present within each pair of matching parentheses, starting from the innermost pair and working outward. After processing, return the final evaluated string without any parenthesis characters.
- *Sample Test Case:* Input: `"(ed(et(oc))el)"` -> Output: `"leetcode"`.
- *Constraints:* String length $1 \leq L \leq 2000$.
- *Hint:* Use a Stack. Push characters one by one. When encountering a closing parenthesis, pop characters until the opening parenthesis is found, reverse that temporary chunk, and push it back to the stack.

- **Q4 (Hard): Minimum Edit Distance**

- *Specification:* Given two strings `word1` and `word2`, compute the absolute minimum number of discrete operations required to mutate `word1` into `word2`. You are granted three

valid operations: insert a character, delete a character, or replace a character. Each operation has a uniform cost of 1.

- *Sample Test Case:* Input: `word1 = "horse"`, `word2 = "ros"` -> Output: 3 (Replace 'h' with 'r', remove 'r', remove 'e').
 - *Constraints:* String lengths $0 \leq L \leq 500$.
 - *Hint:* Apply 2D Dynamic Programming. `dp[i][j]` tracks the cost to match prefixes of length `i` and `j`. If characters differ, take the minimum of insertion, deletion, or substitution plus 1.
-

13.13 Set 12: Bit & Math Tricks

- **Q1 (Easy): Set Bit Count**

- *Specification:* Write a function that takes an unsigned 32-bit integer as input and computes the total number of '1' bits (also known as the Hamming weight) present in its binary representation. Return the final integer tally.
- *Sample Test Case:* Input: 11 (binary 1011) -> Output: 3.
- *Constraints:* Input is a 32-bit unsigned integer.
- *Hint:* Use bitwise AND operations. The expression `n & (n - 1)` always elegantly flips the least significant '1' bit of `n` to '0', allowing for rapid counting.

- **Q2 (Medium): Generate Spiral Matrix**

- *Specification:* Given a positive integer `N`, mathematically construct and return an $N \times N$ matrix populated with sequential numerical elements from 1 up to N^2 in a clockwise spiral layout. The sequence must originate at the top-left corner and spiral inward.
- *Sample Test Case:* Input: 3 -> Output: `[[1,2,3],[8,9,4],[7,6,5]]`.
- *Constraints:* $1 \leq N \leq 20$.
- *Hint:* Establish four boundary variables: top, bottom, left, and right. Use a `while` loop containing four independent `for` loops to walk the perimeter, incrementally shrinking the boundaries inward.

- **Q3 (Medium-Hard): Constant Space First Duplicate**

- *Specification:* You are given an array of integers containing `N` elements, where every element guarantees a value in the inclusive range of $[1, N]$. Exactly one integer value is duplicated once, while all others appear exactly once. Identify and return this duplicate integer. Your solution must use strictly $O(1)$ auxiliary space and must not modify the input array.
- *Sample Test Case:* Input: `[1,3,4,2,2]` -> Output: 2.
- *Constraints:* $1 \leq N \leq 10^5$.
- *Hint:* Treat the array values as pointer links to other indices (`next = nums[curr]`). Utilize Floyd's Tortoise and Hare cycle detection algorithm to locate the cycle entrance, which corresponds to the duplicate value.

- **Q4 (Hard): Maximum Product Subarray**

- *Specification:* Given an integer array, find a contiguous non-empty subarray that evaluates to the absolute largest mathematical product, and return that computed product. Pay special attention to negative values, as multiplying two negatives yields a large positive.
- *Sample Test Case:* Input: `[2,3,-2,4]` -> Output: 6 (Subarray `[2,3]`).
- *Constraints:* $1 \leq N \leq 2 \times 10^4$.

- *Hint:* Maintain both a `current_max` and a `current_min` product dynamically. When encountering a negative number, swap the max and min trackers because multiplying by a negative inverts the relationship.
-

13.14 Set 13: Advanced Search

- **Q1 (Easy): Validate Sudoku Subsections**
 - *Specification:* You are given a 9×9 Sudoku board partially populated with digits '1'-'9' and empty spaces '.'. Determine if the board's current state is legally valid. A valid board requires that no row, no column, and no 3×3 sub-box contains any duplicate digits. You do not need to determine if it is solvable, only if it is currently valid.
 - *Sample Test Case:* Input: standard valid grid -> Output: `true`.
 - *Constraints:* Board size is exactly 9×9 .
 - *Hint:* Iterate through every cell. Use three separate HashSets (or a smart string encoding trick) to register seen values mapped to specific row indices, column indices, and calculated box indices.
- **Q2 (Medium): Compute Island Perimeter**
 - *Specification:* You are given a grid of 1s (land) and 0s (water) containing exactly one unified island (one or more connected land cells). No lakes exist inside the island. Calculate the exact total perimeter length of the island. Each land cell contributes exactly 4 to the perimeter unless it touches an adjacent land cell.
 - *Sample Test Case:* Input: `[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]` -> Output: 16.
 - *Constraints:* $1 \leq M, N \leq 100$.
 - *Hint:* Loop over the grid. Every time you see a 1, add 4 to the perimeter total. Then, check the left neighbor and top neighbor; if they are also land, subtract 2 for each shared edge.
- **Q3 (Medium-Hard): Longest Consecutive Sequence**
 - *Specification:* Given an unsorted array of integers, determine the length of the longest sequence of mathematically consecutive elements. The elements do not need to be contiguous in the original array structure. You must construct an algorithm that achieves strictly $O(N)$ runtime complexity.
 - *Sample Test Case:* Input: `[100, 4, 200, 1, 3, 2]` -> Output: 4 (The sequence `[1, 2, 3, 4]`).
 - *Constraints:* $0 \leq N \leq 10^5$.
 - *Hint:* Dump all elements into a HashSet for $O(1)$ lookups. Iterate through the set; only begin building a sequence if `num - 1` does not exist, ensuring you only iterate starting from the true base of a sequence.
- **Q4 (Hard): Sorted 2D Binary Search**
 - *Specification:* Implement an efficient algorithm that searches for a specific integer target within an $M \times N$ matrix. The matrix has strict properties: every row is sorted in strict ascending order, and the absolute first integer of any row is strictly greater than the absolute last integer of the preceding row. Return true if found, false otherwise.
 - *Sample Test Case:* Input: `matrix = [[1,3,5,7],[10,11,16,20]]`, `target = 3` -> Output: `true`.
 - *Constraints:* $1 \leq M, N \leq 100$.

- *Hint:* Mathematically treat the 2D matrix as a flattened 1D sorted array of length $M \times N$. Map a 1D mid-index back to 2D coordinates using division `mid / N` and modulo `mid % N`.
-

13.15 Set 14: Interval & Schedule

- **Q1 (Easy): Max Guests Present**
 - *Specification:* You are given two arrays of equal length representing an event schedule: one array indicates guest arrival times and the other indicates guest departure times. Guests depart at the exact minute listed. Compute the maximum number of concurrent guests present at the venue at any single point in time.
 - *Sample Test Case:* Input: `arrivals = [1, 2, 9]`, `departures = [5, 8, 12]` -> Output: 2.
 - *Constraints:* $1 \leq N \leq 10^5$.
 - *Hint:* Sort both the arrival and departure arrays independently. Use a two-pointer approach to sweep a timeline, incrementing a counter for arrivals and decrementing for departures.
- **Q2 (Medium): Snake Collision Simulation**
 - *Specification:* Simulate a simplified game of Snake on an unbounded 2D grid. The snake begins at (0,0) with length 1. You receive a sequence of directional commands. For every command, the snake's head moves 1 unit. Without food to grow, the snake's tail moves identically, maintaining its overall length (given as a fixed integer L). Determine if the snake ever collides with its own body.
 - *Sample Test Case:* Input: `L=4`, `path="RRRDDLLU"` -> Output: `true`.
 - *Constraints:* Path length $1 \leq P \leq 10^4$.
 - *Hint:* Utilize a Deque (Double-ended Queue) to maintain the exact coordinates of the snake's body segments. Push the new head coordinate and pop the tail, checking a HashSet to detect overlap.
- **Q3 (Medium-Hard): Two Sum Indices**
 - *Specification:* Given an array of integers and a specific target integer, discover two unique indices pointing to array values that sum exactly to the target. It is mathematically guaranteed that exactly one valid solution exists. You may not use the same element twice. Return the two indices in any order.
 - *Sample Test Case:* Input: `nums = [2,7,11,15]`, `target = 9` -> Output: `[0, 1]`.
 - *Constraints:* $2 \leq N \leq 10^4$.
 - *Hint:* Make a single pass over the array. Use a HashMap to store the required complement (`target - current_val`) and the current index as you traverse.
- **Q4 (Hard): Subset Partition Minimum Difference**
 - *Specification:* You are provided a list of positive integers. Your objective is to partition the list into two disjoint, exhaustive subsets such that the absolute difference between the mathematical sums of the subsets is minimized. Return the minimum possible absolute difference.
 - *Sample Test Case:* Input: `[1, 6, 11, 5]` -> Output: 1 (Subsets `[1,5,6]` summing to 12, and `[11]`).
 - *Constraints:* $1 \leq N \leq 200$.
 - *Hint:* This maps directly to the 0/1 Knapsack DP problem. Target a capacity equal to

`total_sum // 2` to find the largest subset sum possible without exceeding half, which minimizes the gap.

13.16 Set 15: Stack & Queue Patterns

- **Q1 (Easy): Evaluate Reverse Polish Notation**

- *Specification:* You are given an array of string tokens representing an arithmetic expression formatted in Reverse Polish Notation (postfix). Evaluate the expression mathematically and return the resulting integer. The valid operators are `+`, `-`, `*`, and `/`. Division between two integers should truncate strictly towards zero.
- *Sample Test Case:* Input: `["2", "1", "+", "3", "*"]` -> Output: `9` $((2 + 1) * 3)$.
- *Constraints:* Array length $1 \leq N \leq 10^4$.
- *Hint:* Use a Stack. Iterate over tokens; push numbers to the stack. When an operator is encountered, pop the top two numbers, apply the operation, and push the result back.

- **Q2 (Medium): Maximal Square Grid**

- *Specification:* You are given an $M \times N$ binary matrix containing only 0s and 1s. Locate the largest contiguous square subsection of the grid consisting entirely of 1s, and compute its total structural area. Return the area as an integer.
- *Sample Test Case:* Input: `[["1", "0", "1", "0", "0"], ["1", "0", "1", "1", "1"], ["1", "1", "1", "1", "1"]]` -> Output: `4`.
- *Constraints:* $1 \leq M, N \leq 300$.
- *Hint:* Use DP. Let `dp[r][c]` be the side length of the largest square whose bottom-right corner is at `[r][c]`. It equals the minimum of its top, left, and top-left neighbors plus 1.

- **Q3 (Medium-Hard): Next Greater Element**

- *Specification:* You are given an array of integers. For every element in the array, find its “Next Greater Element”—defined as the first sequentially upcoming element to its right that is strictly mathematically larger than it. Output a corresponding array of these larger elements. If an element has no larger successor, assign it `-1`.
- *Sample Test Case:* Input: `[4, 1, 2, 3]` -> Output: `[-1, 2, 3, -1]`.
- *Constraints:* $1 \leq N \leq 10^5$.
- *Hint:* Utilize a Monotonic Decreasing Stack. Store values (or indices) in the stack. When the current element is larger than the top of the stack, pop the stack and assign the current element as the answer for the popped item.

- **Q4 (Hard): Shortest Word Transformation**

- *Specification:* You are given a start word, an end word, and a dictionary of valid words. Find the exact sequence length of the shortest transformation sequence from the start word to the end word. A valid step changes precisely one single letter, and every intermediate state must exist in the dictionary. Return 0 if no sequence exists.
- *Sample Test Case:* Input: `begin = "hit", end = "cog", dict = ["hot", "dot", "dog", "lot", "log", "cog"]` -> Output: `5` (“hit”->“hot”->“dot”->“dog”->“cog”).
- *Constraints:* Dictionary size ≤ 5000 .
- *Hint:* Construct this as a graph problem and utilize Breadth-First Search (BFS) to guarantee the shortest path. For each word, generate all 1-character edits and look them up in a HashSet dictionary.

13.17 Set 16: Graph Exploration

- **Q1 (Easy): Balanced Bracket String**
 - *Specification:* Given a string containing only the characters (,), {, }, [, and], evaluate if the string layout is structurally balanced. A string is valid if every open bracket is closed by the exact identical type of bracket, and they are closed in the exact correct nested mathematical order.
 - *Sample Test Case:* Input: "{[]}" -> Output: **true**. Input: "([)]" -> Output: **false**.
 - *Constraints:* String length $1 \leq L \leq 10^4$.
 - *Hint:* Maintain a Stack. Push opening brackets. For closing brackets, check if the stack is non-empty and the top element is the matching opening bracket, then pop.
- **Q2 (Medium): Deep Clone Graph**
 - *Specification:* You are provided a reference node to a connected, undirected graph. The nodes contain a value and a list of neighbor references. Return a complete deep copy (clone) of the entire graph structure. Every newly generated node must mirror the original topology, but occupy separate heap memory.
 - *Sample Test Case:* Input: `adjList = [[2,4],[1,3],[2,4],[1,3]]` -> Output: Exact identical structural clone.
 - *Constraints:* Node count ≤ 100 .
 - *Hint:* Traverse using DFS or BFS while keeping a central HashMap mapping original nodes to their cloned equivalents to prevent duplicating already processed nodes and resolve cycles.
- **Q3 (Medium-Hard): Task Scheduler Cooldown**
 - *Specification:* You are given an array of characters representing CPU tasks and an integer cooldown multiplier N. Identical tasks must be separated by at least N time intervals of cooldown. You can process a task or sit idle in one time interval. Calculate the absolute minimum time required to process every task in the array.
 - *Sample Test Case:* Input: `tasks = ["A","A","A","B","B","B"]`, `N = 2` -> Output: 8 (A -> B -> idle -> A -> B -> idle -> A -> B).
 - *Constraints:* Task array length $\leq 10^4$, $0 \leq N \leq 100$.
 - *Hint:* Frequency matters most. Calculate the frequency of the most common task. Construct hypothetical “blocks” separated by the cooldown period, and fill in the idle gaps with remaining tasks.
- **Q4 (Hard): Bipartite Graph Detection**
 - *Specification:* You are given an undirected graph presented as an adjacency list. Determine if the graph is bipartite. A graph qualifies as bipartite if you can split its entire set of nodes into two disjoint independent sets, such that every edge strictly connects a node in the first set to a node in the second set, with no internal edges within a set.
 - *Sample Test Case:* Input: `graph = [[1,3],[0,2],[1,3],[0,2]]` -> Output: **true**.
 - *Constraints:* $1 \leq N \leq 100$.
 - *Hint:* Perform a Graph Coloring algorithm using BFS or DFS. Assign alternating colors (e.g., 0 and 1) to adjacent nodes. If you ever hit an adjacent node that already possesses the current node’s color, it is not bipartite.

13.18 Set 17: Dynamic Programming Gauntlet

- **Q1 (Easy): Distinct Climbing Stairs**
 - *Specification:* You are faced with a staircase containing N discrete steps to reach the top level. At any point, you are mathematically permitted to climb either exactly 1 step or exactly 2 steps. Compute the total number of unique sequences you can perform to reach the ultimate top step.
 - *Sample Test Case:* Input: 3 -> Output: 3 (1+1+1, 1+2, 2+1).
 - *Constraints:* $1 \leq N \leq 45$.
 - *Hint:* This maps exactly to the Fibonacci sequence. The number of ways to reach step N is strictly the sum of the ways to reach step $N-1$ and step $N-2$.
 - **Q2 (Medium): Pacific Atlantic Convergence**
 - *Specification:* You are given an $M \times N$ rectangular grid containing integer elevations. The Pacific Ocean touches the top and left borders, while the Atlantic touches the bottom and right. Water naturally flows downwards or straight across to adjacent cells of equal or lower elevation. Return a list of all cell coordinates where rainwater can successfully flow to both the Pacific and Atlantic oceans.
 - *Sample Test Case:* Input: `[[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]`
-> Output: `[[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]`.
 - *Constraints:* $1 \leq M, N \leq 200$.
 - *Hint:* Trace the flow backwards. Start DFS/BFS algorithms from the ocean borders, climbing strictly *up* in elevation, and record reachable cells in two separate HashSets. Find the intersection.
 - **Q3 (Medium-Hard): Longest Palindrome Builder**
 - *Specification:* You are given a string. Your task is to mathematically compute the absolute length of the longest palindrome that could be constructed using the letters found within the string. You are allowed to dynamically reorder the letters in any configuration to build this palindrome.
 - *Sample Test Case:* Input: "abcccd" -> Output: 7 (Can build "dccaced").
 - *Constraints:* String length $1 \leq L \leq 2000$.
 - *Hint:* Count the frequencies of all characters. Pairs of characters can always be mirrored on both sides of a palindrome. If any odd character counts exist, you can add exactly one character to the center.
 - **Q4 (Hard): Regular Expression Wildcards**
 - *Specification:* Given an input string s and a pattern string p , implement full regular expression matching that strictly supports the `.` wildcard (matches any single character) and the `*` wildcard (matches zero or more of the immediately preceding element). The match must cover the entire input string perfectly, not just a partial substring.
 - *Sample Test Case:* Input: $s = "aa"$, $p = "a*"$ -> Output: `true`.
 - *Constraints:* String lengths $1 \leq L \leq 20$.
 - *Hint:* Use 2D Dynamic Programming where $dp[i][j]$ implies prefix match. Dealing with `*` requires evaluating two branches: treating the `*` component as matching zero elements, or matching one/more elements if the previous character aligns.
-

13.19 Set 18: Binary Search Variants

- **Q1 (Easy): Find Peak Topology**
 - *Specification:* You are given a 0-indexed integer array where adjacent elements are strictly forbidden from being identically equal. A peak element is defined as any value strictly greater than both its left and right neighbors. Find any peak element in the array and return its exact index. You must achieve $O(\log N)$ time complexity.
 - *Sample Test Case:* Input: [1,2,3,1] -> Output: 2 (Index of value 3).
 - *Constraints:* $1 \leq N \leq 1000$.
 - *Hint:* Use Binary Search. Calculate `mid`. If `nums[mid] < nums[mid+1]`, the upward slope guarantees a peak exists to the right. Otherwise, a peak exists to the left (or is the mid itself).
- **Q2 (Medium): Minimum of Rotated Array**
 - *Specification:* An initially sorted array of unique integers was rotated cyclically an unknown number of times. Given this rotated array, locate the minimum mathematical element. You must implement an algorithm that operates in $O(\log N)$ runtime complexity.
 - *Sample Test Case:* Input: [3,4,5,1,2] -> Output: 1.
 - *Constraints:* $1 \leq N \leq 5000$.
 - *Hint:* Use Binary Search. Compare `nums[mid]` to `nums[right]`. If `mid` is greater, the minimum is wrapped in the right half. If `mid` is smaller, the minimum is in the left half (inclusive of `mid`).
- **Q3 (Medium-Hard): Koko Banana Minimum Speed**
 - *Specification:* Koko has N piles of bananas, with array values representing pile sizes. She has H hours to eat them all. She chooses a constant eating speed K bananas per hour. If a pile has less than K , she finishes it but idles for the remainder of that hour. Compute the absolute minimum integer speed K needed to finish all piles strictly within H hours.
 - *Sample Test Case:* Input: `piles = [3,6,7,11]`, $H = 8$ -> Output: 4.
 - *Constraints:* $1 \leq N \leq 10^4$.
 - *Hint:* Binary search on the answer space (the eating speed). The possible speeds range from 1 to the max pile size. For each guessed speed `mid`, calculate total hours required and adjust the search space.
- **Q4 (Hard): Median of Sorted Arrays Revision**
 - *Specification:* (A variant of the classic for repetition reinforcement) Given two separate sorted arrays of varying lengths, isolate the exact median value in $O(\log(\min(m,n)))$ time complexity. The arrays are independently sorted but may drastically differ in scale and capacity. Return the floating point median.
 - *Sample Test Case:* Input: `A=[1,2]`, `B=[3,4]` -> Output: 2.5.
 - *Constraints:* Total length ≤ 2000 .
 - *Hint:* Enforce that array `A` is the shorter one. Perform binary search over the indices of `A` to find a partition line, deriving `B`'s partition line logically. Cross-check the boundary max/min values for validity.

13.20 Set 19: Advanced Data Structures

- **Q1 (Easy): Min-Tracking Stack**

- *Specification:* Design a customized Stack data structure that supports standard operations: `push(val)`, `pop()`, `top()`, and an additional method `getMin()`. The `getMin` method must dynamically retrieve the absolute minimum element currently present anywhere in the stack. All four operations must resolve in $O(1)$ constant time complexity.
 - *Sample Test Case:* Input: `push(-2); push(0); push(-3); getMin() -> -3; pop(); top() -> 0; getMin() -> -2.`
 - *Constraints:* Value range bounds are standard 32-bit integers.
 - *Hint:* Track minimums using a parallel stack structure or by pushing tuples of (value, current_minimum) onto a single backing stack.
 - **Q2 (Medium): Capture Surrounded Regions**
 - *Specification:* You are given an $M \times N$ board heavily populated with 'X' and 'O'. A region of 'O's is considered completely surrounded if it is enclosed by 'X's on all four cardinal directions. Capture these isolated regions by permanently flipping all surrounded 'O's into 'X's in-place. 'O's connected directly to the board's outer perimeter cannot be captured.
 - *Sample Test Case:* Input: `[["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O",`
`-> Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]`.
 - *Constraints:* $1 \leq M, N \leq 200$.
 - *Hint:* Launch a DFS from all 'O's explicitly located on the outer border, marking them safely untouchable. Then systematically flip all remaining, unmarked 'O's in the interior to 'X'.
 - **Q3 (Medium-Hard): Randomized Const-Time Structure**
 - *Specification:* Design a data structure framework that correctly supports inserting a value, removing a value, and obtaining a random value from the current pool. All three operations must perform strictly in $O(1)$ average mathematical time complexity. The random getter must return values with uniform independent probability based on the current size.
 - *Sample Test Case:* Input: `insert(1) -> true, insert(2) -> true, getRandom()`
`-> 1 or 2, remove(1) -> true.`
 - *Constraints:* Up to 2×10^5 calls.
 - *Hint:* Combine a standard Array (for $O(1)$ random access by index) with a HashMap (to map values to their current array indices). To remove in $O(1)$, swap the target element with the absolute last element in the array, update the map, and pop.
 - **Q4 (Hard): Binary Tree Serializer**
 - *Specification:* Design a robust algorithm to heavily serialize and subsequently deserialize a full binary tree. Serialization involves converting the in-memory tree object structure into a single continuous string format. Deserialization involves parsing that precise string back into the exact original structural binary tree hierarchy.
 - *Sample Test Case:* Input: `root = [1,2,3,null,null,4,5]` -> Output:
`[1,2,3,null,null,4,5]` (After round-trip processing).
 - *Constraints:* Tree nodes up to 10^4 .
 - *Hint:* Utilize a preorder traversal (DFS) or level-order traversal (BFS). Append distinct null markers (e.g., 'N' or '#') for missing children to preserve structural integrity, and separate node values using commas.
-

13.21 Set 20: Championship Round

- **Q1 (Easy): Roman Numeral Parser**
 - *Specification:* Given a string strictly representing a valid Roman numeral, parse it and compute its equivalent standard integer value. Roman numerals are constructed from seven distinct symbols: I, V, X, L, C, D, and M. The parsing must accurately support standard subtractive combinations like IV (4) and IX (9).
 - *Sample Test Case:* Input: "MCMXCIV" -> Output: 1994.
 - *Constraints:* String length $1 \leq L \leq 15$.
 - *Hint:* Iterate strictly from left to right. If the numerical value of the current symbol is less than the numerical value of the right adjacent symbol, mathematically subtract it; otherwise, add it.
- **Q2 (Medium): Battleship Counter**
 - *Specification:* You are given an $M \times N$ matrix board modeling a game of Battleships, populated with 'X' (ship) and '.' (water) cells. Distinct battleships are constrained to strictly horizontal or strictly vertical linear orientations. Ships are guaranteed to be separated by at least one cell of water. Tally and return the total number of independent battleships.
 - *Sample Test Case:* Input: `[["X",".",".","X"],[".",".",".","X"],[".",".",".","X"]]` -> Output: 2.
 - *Constraints:* $1 \leq M, N \leq 200$.
 - *Hint:* Instead of complex graph traversal, loop across all cells. A cell is the definitive "top-left head" of a battleship if it is an 'X' AND has no 'X' immediately above it AND no 'X' immediately to its left.
- **Q3 (Medium-Hard): Target Sum Subarray Count**
 - *Specification:* Given an array of unconstrained integers (including negatives) and an integer K, mathematically determine the absolute total number of continuous subarrays whose constituent sum resolves precisely to K. Note that different indices dictate a distinct subarray, even if values match.
 - *Sample Test Case:* Input: `nums = [1,2,3]`, `K = 3` -> Output: 2 ([1,2] and [3]).
 - *Constraints:* $1 \leq N \leq 2 \times 10^4$.
 - *Hint:* Utilize the Prefix Sum architectural pattern alongside a HashMap. As you continuously iterate, record the frequency of each prefix sum encountered. At each step, check if `current_sum - K` exists in the map to identify valid sequences ending at the current index.
- **Q4 (Hard): Burst Balloons Optimization**
 - *Specification:* You are given N balloons, each painted with a specific integer value. If you deliberately burst balloon *i*, you immediately collect coins mathematically equal to `nums[i-1] * nums[i] * nums[i+1]`. After bursting, the adjacent balloons immediately shift together. Determine the maximum possible coin yield you can extract by strategically sequencing the bursts until zero balloons remain.
 - *Sample Test Case:* Input: `[3,1,5,8]` -> Output: 167.
 - *Constraints:* $1 \leq N \leq 300$.
 - *Hint:* Pad the array boundaries with conceptual 1s. Frame the DP inversely: analyze which balloon should strictly be the *absolute last* one burst in a given range, rather than the first. Build answers incrementally for sub-intervals.

Part IV

System Design & Architecture at Scale

Chapter 14

System Architecture and Design Fundamentals

“A system is not a collection of services, but a web of communication boundaries. If your boundaries are wrong, your microservices are just a distributed monolith.”

14.1 System Design in the Senior Interview

In senior system design interviews, candidates are often asked to design large-scale, low-latency platforms like an ad click aggregator, a video streaming service, or a trading exchange.

A common pitfall is immediately drawing boxes for databases, load balancers, and caches without grounding the architecture in business specifications.

To stand out, you must apply **Domain-Driven Design (DDD)**. Define your bounded contexts clearly, design your aggregates to protect business invariants, and construct sequence flows showing exactly how data travels across services while keeping latency low.

In this chapter, we will design the architecture of **ZenithTrade**, a high-frequency order matching exchange, mapping out its service boundaries and order lifecycle.

14.2 Domain-Driven Design (DDD) Boundaries

To design a clean distributed system, you must first establish your domain boundaries using DDD principles.

14.2.1 Bounded Contexts

A bounded context defines the boundary within which a particular domain model applies. In ZenithTrade, we separate the system into three main bounded contexts:

1. **Exchange Context (ZenithTrade):** Deals with orders, bid/ask books, matching execution, and price feeds.

2. **Ledger Context (AuraPay):** Handles balance preservation, double-entry transfers, and deposit/withdrawal checks.
3. **Identity Context (ChiramTrust):** Manages user credentials, authentication scopes, and KYC compliance.

Crucial Mistake: Do not mix context models. An `Order` inside the Exchange context should not contain details about a user's ledger overdraft limits. Decouple them and bridge them using events or APIs.

14.2.2 Aggregates, Entities, and Value Objects

- **Aggregates:** A cluster of associated objects treated as a single unit for data changes (e.g., an `OrderBook`). Changes to orders must go through the `OrderBook` root to protect sorting invariants.
- **Entities:** Objects with a distinct identity that persists over time (e.g., a `LedgerAccount` with a unique UUID).
- **Value Objects:** Immutable objects with no identity defined solely by their attributes (e.g., a `Money` value object containing `amount` and `currency`). Value objects have no setters; they are replaced entirely, making them thread-safe.

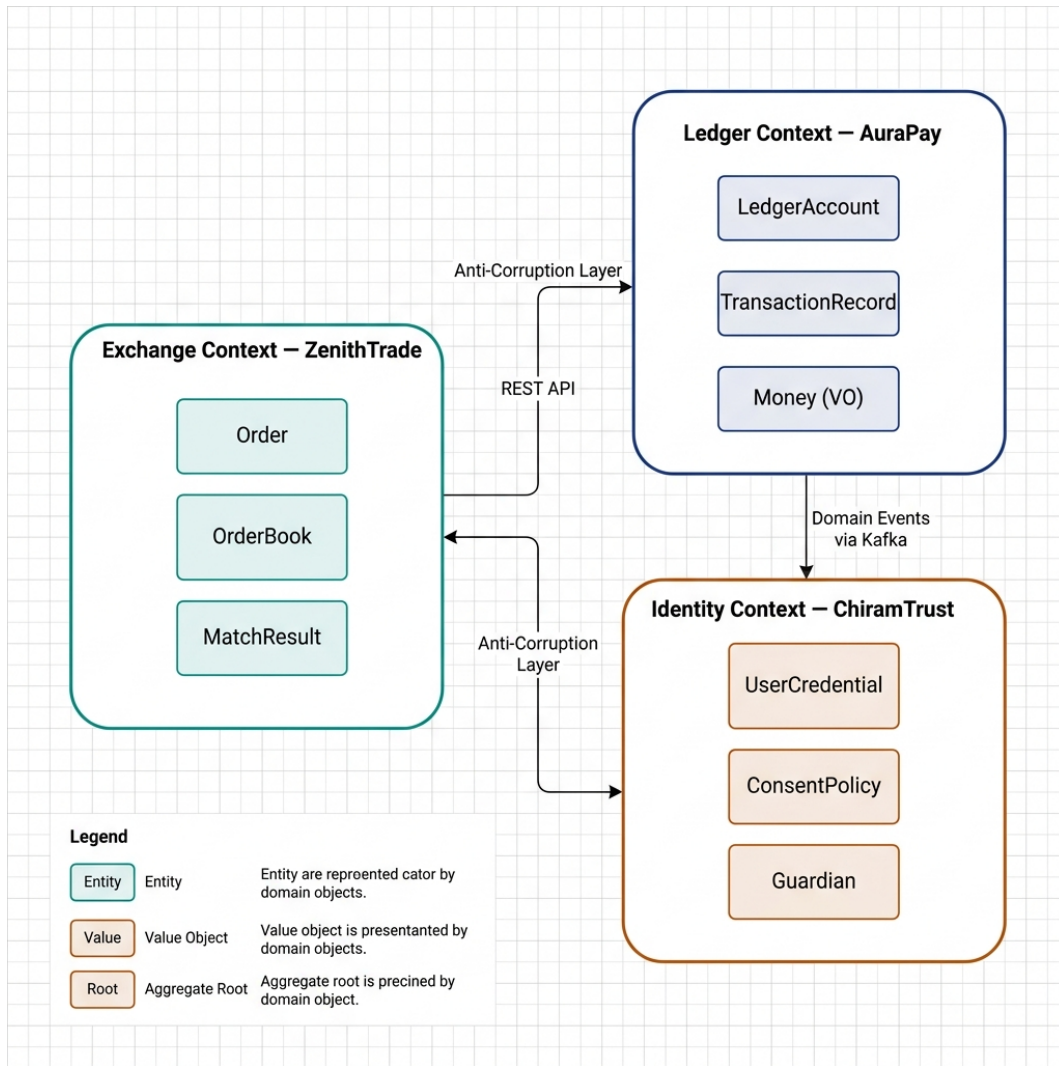


Figure 14.1: DDD Bounded Context Map

14.3 Monolithic vs. Microservices vs. Event-Driven

Choosing an architectural style is a trade-off between latency, complexity, and operational overhead.

14.3.1 Monolithic Architecture

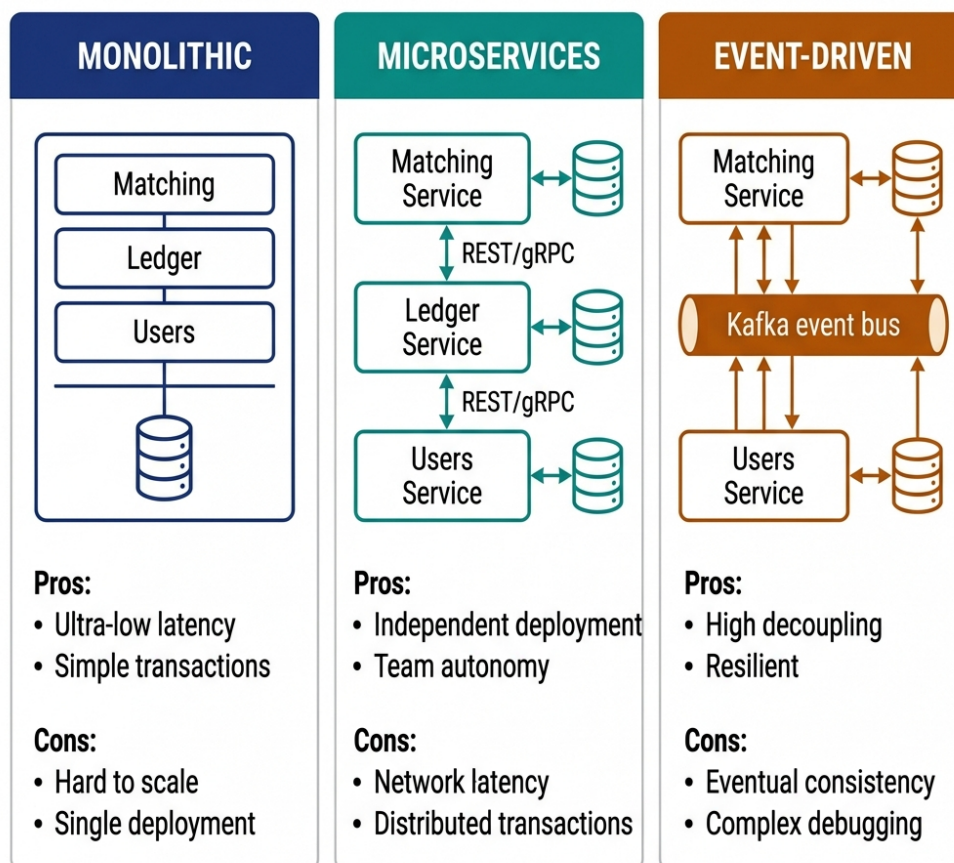
- **Description:** All components (matching, ledger, user management) run inside a single process, sharing memory.
- **Pros:** Ultra-low latency (nanosecond-range in-memory operations), simple testing, and transactional database integrity.
- **Cons:** Hard to scale across multiple teams; deployment failures crash the entire system.
- **Use case:** The core matching engine loop of ZenithTrade must be monolithic and in-memory to meet microsecond latency specs.

14.3.2 Microservices Architecture

- **Description:** Services run in independent processes, communicating via synchronous protocols (gRPC, HTTP/REST).
- **Pros:** Decoupled deployments, scaling independent workloads (e.g., scaling the API Gateway without scaling the matching engine).
- **Cons:** High network latency (millisecond range), complex distributed transactions, and data consistency challenges.

14.3.3 Event-Driven Architecture (EDA)

- **Description:** Services communicate asynchronously by publishing and subscribing to events (Kafka, RabbitMQ).
- **Pros:** High decoupling, loose runtime dependencies, and high resilience.
- **Cons:** Eventual consistency. If the matching engine publishes a “TradeExecuted” event, the ledger balances might not update for several milliseconds.



Use Monolith for core engine, Microservices for scaling, Events for async

Figure 14.2: Monolithic vs Microservices vs Event-Driven Architecture

14.4 Scaling Out: Partitioning & Consistent Hashing

A single matching engine instance cannot handle all trading instruments globally. To scale ZenithTrade horizontally, we must partition (shard) the matching workload.

14.4.1 Consistent Hashing for Instrument Sharding

Instead of traditional modulo sharding (`hash(instrumentId) % nodeCount`), which causes massive data reshuffling when nodes are added or removed, ZenithTrade utilizes a **Consistent Hash Ring**:

1. **The Ring:** The hash space is mapped onto a circular ring (e.g., 0 to $2^{32} - 1$).
2. **Node Mapping:** Matching Engine instances (nodes) are hashed and placed at specific coordinates on the ring. We map multiple “virtual nodes” per physical machine to ensure uniform distribution of load.
3. **Key Mapping:** Incoming orders are routed based on `hash(instrumentId)` (e.g., BTC-USD, ETH-EUR). The order is handled by the first matching engine node encountered walking clockwise from the key’s hash coordinate.
4. **Rebalancing:** When a new matching engine node is added to the cluster, it only takes a portion of keys from its immediate clockwise neighbor, keeping rebalancing traffic to a minimum.

14.5 Command Query Responsibility Segregation (CQRS)

In financial systems, read traffic (users querying active order books, historical trades, and account balances) is several orders of magnitude higher than write traffic (executing transactions or submitting orders). Applying **CQRS** prevents read queries from degrading write performance:

- **Command Path (Write):** Optimized for low latency and consistency. Incoming orders are processed by the in-memory matching engine, writing state changes sequentially to a Write-Ahead Log (WAL) or transactional ledger database.
- **Query Path (Read):** Optimized for high-throughput queries. State change events (e.g., `OrderPlaced`, `TradeExecuted`) are published to Kafka and consumed by read-projection workers. These workers update read-optimized views in Elasticsearch (for historical search) or Redis (for fast order book rendering).
- **Consistency Trade-off:** The read model is **eventually consistent** (typically lagging the command path by a few milliseconds), which is acceptable for user displays as long as the write path remains strictly consistent.

14.6 CAP Theorem & Distributed Trade-offs

The CAP Theorem states that in a distributed system, you can only guarantee two out of three properties during a network partition: **Consistency (C)**, **Availability (A)**, or **Partition Tolerance (P)**. Because network partitions are inevitable in real-world infrastructure, system design is a choice between **CP** and **AP**:

- **The Ledger Context (CP Choice):** AuraPay is designed as a **CP** system. In financial bookkeeping, correctness is non-negotiable. If a network partition occurs between ledger replicas, we must reject transaction requests (sacrificing availability) rather than risk allowing

double-spending or balance mismatch (sacrificing consistency). Consensus protocols like Raft or Paxos are used to coordinate commits across healthy replicas.

- **The Market Feed Context (AP Choice):** The ZenithTrade public price feed (ticker data) is designed as an **AP** system. If a partition occurs, it is better to continue broadcasting the latest available price data (even if slightly stale) to users than to shut down the feed entirely.

14.7 API Design & Idempotency

When designing APIs for microservices, you must handle network failures gracefully. If a client submits a payment request but the connection drops before receiving a response, the client will retry the request. Without **Idempotency**, this leads to double-billing.

14.7.1 Idempotency Keys in REST & gRPC

1. **Client-Generated Key:** The client generates a unique UUID (e.g., `Idempotency-Key: f81d4fae-7dec-11d0-a765-00a0c91e6bf6`) and sends it in the request header.
2. **API Gateway Check:** The API Gateway intercepts the request and queries Redis to see if the key exists:
 - **Case 1 (New Request):** The gateway stores the key in Redis with a status of `PENDING` and routes the request.
 - **Case 2 (Duplicate Pending):** If the key is found and its status is `PENDING`, the gateway returns a `409 Conflict` (request is currently processing).
 - **Case 3 (Duplicate Completed):** If the key is found and its status is `COMPLETED`, the gateway returns the cached response payload directly, bypassing the backend services entirely.
3. **Backend Commit:** Once the transaction settles, the service updates the status in Redis to `COMPLETED` and writes the response payload, ensuring the cache has a defined TTL (Time To Live, e.g., 24 hours).

14.8 ZenithTrade Order Lifecycle Sequence

The following sequence diagram maps out how an order is submitted, validated, matched inside the memory buffer, and settled inside the ledger:

ZenithTrade Order Lifecycle Sequence

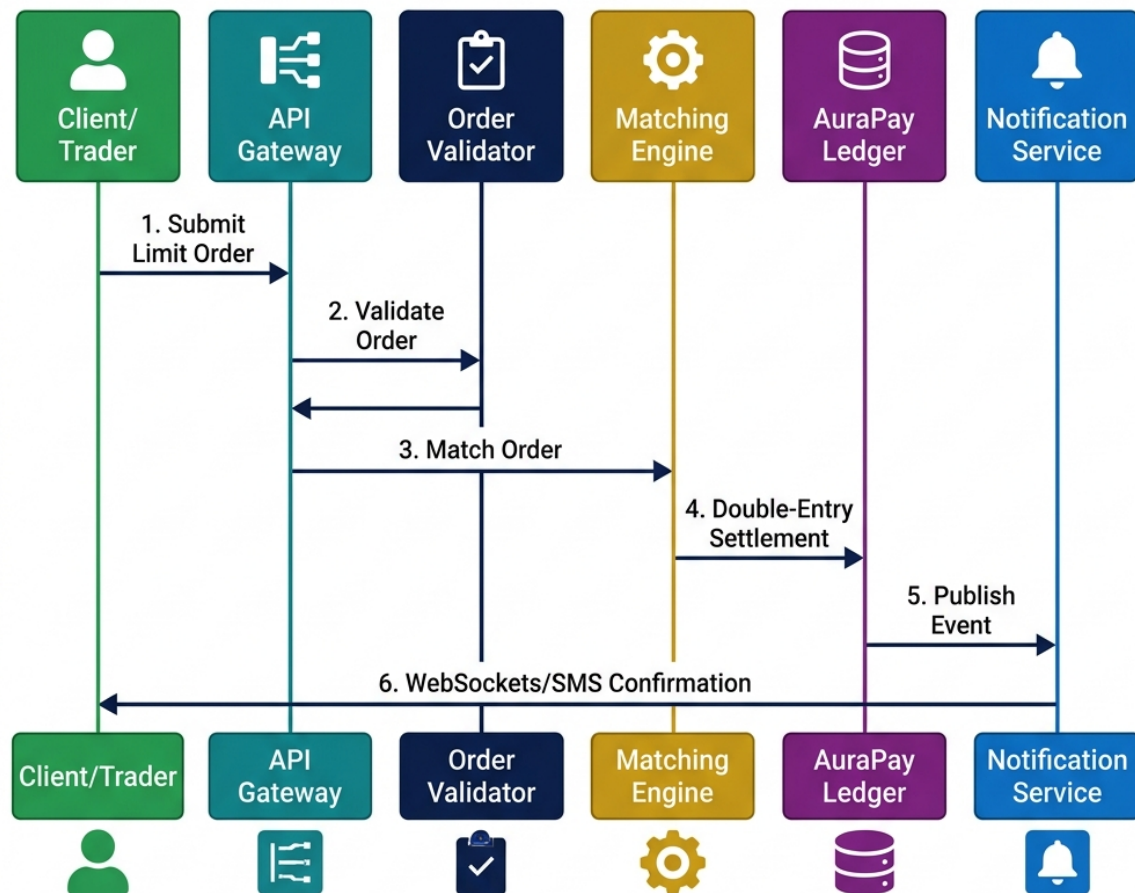


Figure 14.3: ZenithTrade Order Lifecycle Sequence

14.8.1 Explaining the Sequence:

1. **Gateway Ingest:** The API Gateway validates rate limits, checks for duplicate requests using the Idempotency-Key, and passes the request to the Exchange Context.
2. **Order Validator:** Before an order enters the book, the validator calls the AuraPay ledger to verify that the client has sufficient funds (Pre-condition check).
3. **In-Memory Matching:** The OrderBook matches buy and sell orders. Since this is CPU-intensive, it runs in memory.
4. **Ledger Settlement:** Once matched, a double-entry transaction settles the trade inside the AuraPay database.
5. **Asynchronous Notification:** The client is notified via WebSockets, completely out of the blocking execution thread path.

14.9 API Rate Limiting Strategies

Rate limiting is essential for protecting APIs from abuse and cascading failures. The following four algorithms are foundational in system design interviews.

14.9.1 Token Bucket Algorithm

The token bucket algorithm maintains a bucket that holds a maximum number of tokens (capacity). Tokens are added to the bucket at a fixed rate. Each incoming request consumes one token; if the bucket is empty, the request is dropped. It is widely used because it allows controlled bursts of traffic while enforcing a sustained long-term rate.

Parameters: - **Capacity (Burst Size):** Maximum number of tokens the bucket can hold. - **Refill Rate:** Rate at which new tokens are generated.

When to use: API gateways and per-user throttling (e.g., Stripe, Amazon API Gateway).

```
import java.util.concurrent.atomic.AtomicLong;

public class TokenBucket {
    private final long capacity;
    private final long refillTokensPerSecond;
    private AtomicLong tokens;
    private AtomicLong lastRefillTimestamp;

    public TokenBucket(long capacity, long refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.tokens = new AtomicLong(capacity);
        this.lastRefillTimestamp = new AtomicLong(System.currentTimeMillis());
    }

    public synchronized boolean allowRequest() {
        refill();
        if (tokens.get() > 0) {
            tokens.decrementAndGet();
            return true;
        }
        return false;
    }

    private void refill() {
        long now = System.currentTimeMillis();
        long elapsedTime = now - lastRefillTimestamp.get();
        long tokensToAdd = (elapsedTime / 1000) * refillTokensPerSecond;

        if (tokensToAdd > 0) {
            tokens.set(Math.min(capacity, tokens.get() + tokensToAdd));
            lastRefillTimestamp.set(now);
        }
    }
}
```

```
}  
}
```

14.9.2 Leaky Bucket Algorithm

In the leaky bucket algorithm, incoming requests enter a FIFO queue (the bucket). The system processes requests from the queue at a strictly constant rate. If the queue is full, new requests are discarded. Unlike the token bucket, it entirely smooths out bursts, ensuring a perfectly constant output rate.

When to use: Network traffic shaping and scenarios requiring steady-throughput processing.

14.9.3 Fixed Window Counter

The fixed window counter algorithm counts incoming requests per discrete time window (e.g., 00:00 to 00:01). If the counter exceeds the threshold, requests are dropped. It is simple but suffers from the **boundary burst** problem: a user can send 100 requests at 00:00:59 and another 100 requests at 00:01:01, effectively pushing 200 requests in a two-second span across the boundary.

When to use: Simple scenarios where edge-case bursts are acceptable.

14.9.4 Sliding Window Log / Counter

This approach addresses the boundary burst issue. A Sliding Window Log tracks individual request timestamps, discarding older ones to precisely enforce the rate over a rolling window. A Sliding Window Counter optimizes memory by keeping weighted counters of the previous and current overlapping windows.

Trade-off: Higher memory usage (for logs) or slight approximations (for counters). **When to use:** Strict rate limiting scenarios where boundary bursts are unacceptable.

Algorithm	Burst Handling	Memory	Accuracy	Complexity
Token Bucket	Allows controlled bursts	$O(1)$	Good	Low
Leaky Bucket	Smooths all bursts	$O(N)$ queue	Good	Medium
Fixed Window	Boundary bursts possible	$O(1)$	Approximate	Low
Sliding Window	No boundary bursts	$O(N)$ timestamps	Exact	High

14.10 Caching Architectures

Caching is a critical component for reducing database load and improving read latencies. The following patterns are essential for system design interviews.

14.10.1 Cache-Aside (Lazy Loading)

In a Cache-Aside pattern, the application is fully responsible for managing the cache. For every read, the application first checks the cache. On a cache miss, it reads from the database, writes the result to the cache, and then returns the data.

Pros: Only requested data is cached, avoiding unnecessary memory usage. The system remains available (reading directly from the DB) even if the cache fails. **Cons:** Introduces a cache miss penalty (latency spike) and risks serving stale data if not carefully invalidated.

```
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.stereotype.Service;

@Service
public class UserService {
    private final UserRepository dbRepository;
    private final RedisTemplate<String, User> redisTemplate;

    public UserService(UserRepository dbRepository, RedisTemplate<String, User>
↵ redisTemplate) {
        this.dbRepository = dbRepository;
        this.redisTemplate = redisTemplate;
    }

    public User getUser(String userId) {
        String cacheKey = "user:" + userId;
        User user = redisTemplate.opsForValue().get(cacheKey);

        if (user == null) {
            // Cache miss: read from DB
            user = dbRepository.findById(userId).orElseThrow();
            // Populate cache
            redisTemplate.opsForValue().set(cacheKey, user);
        }
        return user;
    }
}
```

14.10.2 Write-Through Cache

Under Write-Through caching, the application writes data to the cache and the database simultaneously (often abstracted so the application only writes to the cache, which synchronously updates the DB).

Pros: The cache is always strongly consistent with the database. **Cons:** Higher write latency due to the dual synchronous writes. Also caches data that might never be read again.

14.10.3 Write-Behind (Write-Back) Cache

With Write-Behind caching, the application writes exclusively to the cache, which acknowledges the write immediately. The cache then asynchronously flushes the data to the persistent database in the background.

Pros: Ultra-low write latency and reduced database load via batching. **Cons:** High risk of data loss if the cache node crashes before flushing to the database. **When to use:** High-write-throughput systems where occasional data loss is an acceptable trade-off.

14.10.4 Cache Eviction Policies

When the cache reaches its memory limit, older data must be removed: - **LRU (Least Recently Used)**: Evicts the item that hasn't been accessed for the longest time. The most common and generally applicable policy. - **LFU (Least Frequently Used)**: Evicts the item with the lowest access frequency. Better for highly skewed, long-term access patterns. - **TTL (Time To Live)**: Automatically expires keys after a set duration, acting as a natural safeguard against stale data.

Pattern	Consistency	Write Latency	Read Latency	Complexity
Cache-Aside	Eventual	Normal	Fast (on hit)	Low
Write-Through	Strong	Higher	Fast	Medium
Write-Behind	Eventual	Ultra-low	Fast	High

14.10.5 Cache Stampede Prevention

A cache stampede occurs when a highly requested cache entry expires (TTL elapses). Suddenly, hundreds of concurrent requests experience a cache miss and hit the database simultaneously, potentially bringing it down.

- **Solution 1: Mutex/Lock**: Implement a distributed lock so that only one thread experiencing the miss queries the database and refreshes the cache; other threads wait for the cache to be populated.
- **Solution 2: Early Expiry with Jitter**: Refresh the cache slightly before the actual TTL expires, utilizing a background worker.
- **Solution 3: Probabilistic Early Expiry**: Each incoming request has a small, random probability of refreshing the cache just before it naturally expires, spreading the DB load gracefully.

14.11 System Design Mock Interview: Sharded Order Matching Engine

To demonstrate how a senior candidate should navigate a system design round, here is a transcript-style mock interview.

14.11.1 Requirements Gathering (The First 5 Minutes)

Interviewer: “I want you to design a high-frequency order matching engine for a cryptocurrency exchange. How would you approach this?”

Candidate: “Before drawing components, I want to establish our core functional and non-functional requirements to set our design boundaries.”

Functional Requirements:

- Users can place Limit Orders (buy/sell a specific quantity at a specific price) and Market Orders.
- The matching engine must match buy and sell orders based on Price-Time priority.
- Trade execution must trigger account balance updates in a ledger.

Non-Functional Requirements:

- **Ultra-Low Latency:** Order matching must execute with sub-millisecond latency (p99 < 1ms).
- **High Throughput:** The system must handle 100,000 requests per second (RPS) peak load.
- **Strict Consistency:** The matching engine and ledger must prevent double-spending and guarantee double-entry correctness. We choose a **CP** model for the ledger.
- **High Availability:** The system must remain available even if a node crashes.

14.11.2 High-Level Estimations (Scale & Math)

Candidate: “Let’s calculate our network and storage needs. At 100,000 RPS, if an average order payload is 200 bytes, our network ingest rate at the gateway is:”

$$\text{Ingest Bandwidth} = 100,000 \times 200 \text{ bytes} = 20 \text{ MB/s} = 160 \text{ Mbps}$$

“This is easily handled by standard network infrastructure. However, processing 100,000 matches per second in a single SQL database is impossible due to disk I/O bottlenecks. Therefore, our primary design boundary is that **the active matching engine must run entirely in-memory**, keeping reads and writes decoupled from disk operations during the matching loop.”

14.11.3 API & Schema Design

Candidate: “Let’s define the API payload for placing a limit order. We’ll use gRPC over HTTP/2 for low latency:”

```
message PlaceOrderRequest {
    string idempotency_key = 1;
    string account_id = 2;
    string instrument_id = 3; // e.g., "BTC-USD"
    enum Side { BUY = 0; SELL = 1; }
    Side side = 4;
    double price = 5;
    double quantity = 6;
}
```

Interviewer: “How do you handle the precision of prices and quantities? Double float values are prone to rounding errors.”

Candidate: “Excellent point. In banking and exchange systems, floating-point arithmetic is a major risk because operations like $0.1 + 0.2$ can result in precision loss. To enforce our correctness invariants, we represent prices and quantities as integers representing the smallest atomic units (e.g., satoshis for BTC, or multiplying USD by 10^8 to store as integers), or utilize the *BigDecimal* type at our database and application borders.”

14.11.4 Deep Dive: In-Memory Data Structures

Interviewer: “How would you design the Order Book in memory to achieve sub-millisecond matching latency?”

Candidate: “To match orders quickly based on Price-Time priority, we need fast insertion, fast deletion (for cancellations), and fast retrieval of the highest bid and lowest ask. We will design the *OrderBook* using two collections: *bids* and *asks*.”

- *Bids Book*: Sorted descending by price.
- *Asks Book*: Sorted ascending by price.

“For each book, we use a **TreeMap** (or Red-Black Tree) where the key is the price level, and the value is a **doubly-linked list** of orders at that price level (FIFO queue). This gives us:”

- *Lookup/Match peak*: $O(1)$ to access the head of the tree.
- *Insert/Cancel*: $O(\log P)$ where P is the number of distinct price levels, which is highly optimized.

14.11.5 Horizontal Scaling: Partitioning the Exchange

Interviewer: “How do you scale this matching engine when the number of instruments and users grows beyond a single machine’s capacity?”

Candidate: “We partition our matching engine horizontally by **Instrument ID** (e.g., *BTC-USD*, *ETH-USD*, *SOL-USDT*). Because orders for different instruments do not interact, we can run completely isolated Matching Engine instances on different machines.”

“We will use a **Consistent Hash Ring** at the API Gateway layer to route incoming orders. The gateway hashes the `instrument_id` and forwards the request to the designated matching node. This prevents hotspots and ensures that adding a new matching node only impacts a fraction of the ring.”

14.11.6 Reliability and Failover

Interviewer: “If an in-memory matching engine node crashes, how do you recover the state without losing orders?”

Candidate: “We use a **Write-Ahead Log (WAL)** pattern with active-passive replication. Every incoming order is written to an append-only log on disk (SSD) sequentially before it is processed by the matching engine. Since sequential writes are extremely fast (disk I/O is minimized), this preserves low latency.”

“Additionally, each matching partition runs as a Raft consensus group containing one Leader and two Followers. The Leader streams the WAL to the Followers. If the Leader crashes, the Followers elect a new Leader, which replays the log from its last committed index to rebuild the in-memory state. This guarantees no order loss and sub-second failover recovery.”

STAR Moment: Bounded Context Isolation

During system design interviews, explain that microservice division should mirror DDD Bounded Contexts. Say: “We will isolate the ZenithTrade Matching Engine from the Aura-Pay Ledger. If the ledger experiences a database write lag, our matching engine can continue to accept and queue orders in memory, preventing system-wide downtime.” This shows you design for fault isolation.

Chapter 15

Enterprise Integration and Resiliency

“In a distributed system, failure is not an anomaly; it is a normal state of operation. Designing for reliability is the science of preventing local failures from becoming global disasters.”

15.1 The Distributed Transaction Dilemma

In monolithic architectures, maintaining data consistency is straightforward: you open a database transaction, perform updates, and commit. If any step fails, the database rolls back all changes.

In a microservices architecture, however, a single business action can span multiple service boundaries. For instance, when a user purchases stock on ZenithTrade:

1. The Exchange service matches the order.
2. The AuraPay ledger service updates the account balance.
3. The Custody service updates securities ownership.

Since these services use independent databases, you cannot use a database transaction (2PC - Two-Phase Commit is generally avoided in high-performance cloud environments due to lock overhead and latency). If the ledger debit succeeds but the custody credit fails, the system enters an inconsistent state.

In a senior architecture interview, you must explain how to resolve this. You will be evaluated on your understanding of the **Saga Pattern** and the **Transactional Outbox Pattern**.

15.2 The Dual-Write Anti-Pattern

A common architectural flaw is the **Dual-Write**. This occurs when a service attempts to modify a database and send a message to a message broker (like Kafka or RabbitMQ) within the same API request:

```
// Anti-pattern: Dual-Write
public void completeTransaction(TransactionRecord tx) {
    database.save(tx); // Database Write
    kafkaTemplate.send("transaction-topic", tx); // Network Call
}
```



```
}
```

This is highly unreliable:

- If the database write succeeds but the message broker is temporarily down or network packet loss occurs, the message is lost, and downstream services (like Auditing or Risk Engine) are never notified.
- If you reverse the order and send the message first, the database write might fail (e.g., due to a constraint violation), but the rest of the system will process the event, leading to phantom actions.

15.2.1 The Transactional Outbox Pattern

To guarantee **At-Least-Once Delivery**, you must write the business data and an event record to an “outbox” table *in the same local database transaction*. Because they use the same database, either both writes succeed, or both fail.

A background process (or CDC log tailer like Debezium) then polls the outbox table, publishes the events to the message broker, and marks them as processed.

The following code illustrates this Outbox Publisher worker:

```
package com.aurapay.integration;

import java.time.Instant;
import java.util.List;
import java.util.UUID;

/**
 * Represents an Outbox event record stored in the same database as the business
 * ↪ entities.
 */
public record OutboxEvent(
    UUID id,
    String aggregateType,
    UUID aggregateId,
    String eventType,
    String payload,
    Instant createdAt,
    boolean processed
) {}

/**
 * Interface representing the Message Broker client (e.g., Kafka, RabbitMQ).
 */
interface MessageBrokerClient {
    void publish(String topic, String payload) throws Exception;
}

/**
```

```

* Service that polls the database Outbox table and publishes events to the
↪ broker.
* Guarantees At-Least-Once delivery of domain events.
*/
public class TransactionalOutboxPublisher {
    private final OutboxRepository outboxRepository;
    private final MessageBrokerClient brokerClient;

    public TransactionalOutboxPublisher(OutboxRepository outboxRepository,
        ↪ MessageBrokerClient brokerClient) {
        this.outboxRepository = outboxRepository;
        this.brokerClient = brokerClient;
    }

    /**
     * Polling worker method. In production, this would be executed by a
     ↪ background
     * scheduler or transaction log tailer (Debezium/CDC).
     */
    public void publishPendingEvents() {
        // Retrieve unprocessed events (locking them to prevent double-processing
        ↪ by other nodes)
        List<OutboxEvent> pendingEvents =
            ↪ outboxRepository.findUnprocessedAndLock(100);

        for (OutboxEvent event : pendingEvents) {
            try {
                // Publish to broker (external network call)
                String topic = "events." + event.aggregateType().toLowerCase();
                brokerClient.publish(topic, event.payload());

                // Mark as processed in the database
                outboxRepository.markAsProcessed(event.id());
            } catch (Exception e) {
                // If publishing fails, we do NOT mark it as processed.
                // It will be retried on the next poll cycle (At-Least-Once
                ↪ Delivery).
                System.err.printf("Failed to publish outbox event %s: %s. Will
                ↪ retry.%n",
                    event.id(), e.getMessage());
            }
        }
    }
}

/**
 * Interface representing database operations for the Outbox table.
 */

```

```

interface OutboxRepository {
    List<OutboxEvent> findUnprocessedAndLock(int limit);
    void markAsProcessed(UUID eventId);
}

```

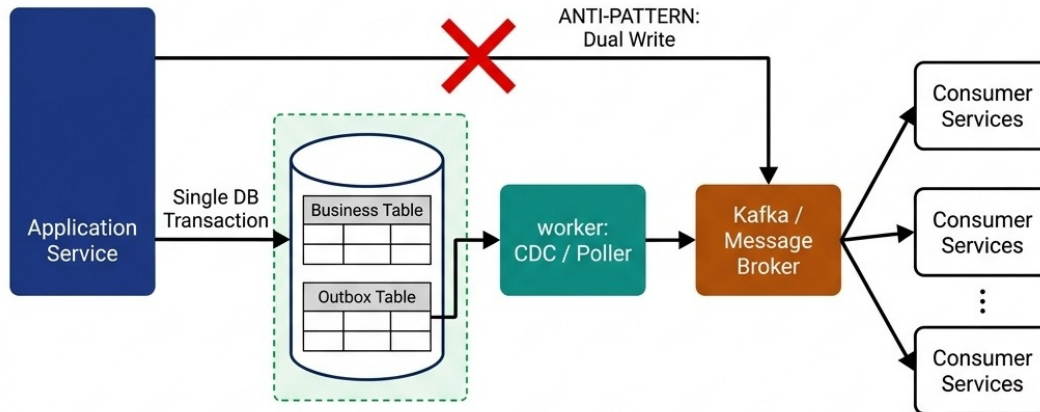


Figure 15.1: Transactional Outbox Pattern

If the message broker fails during publication, the event remains unmarked in the database and will be retried in the next execution cycle. This ensures that the message is eventually delivered at least once.

15.3 Event Sourcing

For financial ledgers (like AuraPay) where correctness and auditability are paramount, storing only the “current state” of an account is insufficient. A senior candidate should discuss **Event Sourcing**:

15.3.1 State vs. Stream

- **State-Based Storage:** Storing a row `Account(id=101, balance=500.00)`. If a balance mismatch occurs, it is impossible to trace *why* the balance is incorrect without parsing external database logs.
- **Event-Sourced Storage:** Storing a stream of immutable events: `[Deposited(50.00), Deposited(70.00), Debited(20.00)]`. The current balance is a derived projection computed by folding/aggregating these events over time:

$$\text{Current Balance} = \sum \text{Credit Events} - \sum \text{Debit Events}$$

15.3.2 Key Invariants & Advantages

1. **Mathematical Auditability:** Every balance change is linked to an immutable event. Historians can reconstruct the ledger state at any specific millisecond.

2. **Side-Effect Isolation:** Commands generate events. Events are appended to the event store (a sequential, write-only database) and then published to message brokers to trigger downstream read-projections, fully separating writes from read overhead.

15.4 Distributed Sagas

A **Saga** is a sequence of local transactions. Each local transaction updates the database within a single service. If a step fails, the Saga orchestrator or participants execute a series of **compensating transactions** that undo the changes made by the preceding steps.

Why is it called a “Saga”? The term comes from a **1987 research paper** by Hector Garcia-Molina and Kenneth Salem at Princeton University. They chose “Saga” because, like an epic literary saga with many chapters, a distributed transaction is a long-running story told through a sequence of smaller, self-contained episodes. If the story goes wrong at any chapter, you cannot un-tell the earlier chapters — you must write new compensating chapters to undo their effects. The metaphor is surprisingly precise.

There are two primary ways to design a Saga:

15.4.1 Choreography-Based Saga

In a choreography-based saga, there is no central coordinator. Each service performs its transaction and emits an event. Other services listen to these events and perform their tasks.

- **Pros:** Decoupled, no single point of failure, simple to implement for small workflows.
- **Cons:** Hard to understand as the number of services grows; risks of cyclic dependencies.

15.4.2 Orchestration-Based Saga

In an orchestration-based saga, a central service (the orchestrator) coordinates the workflow. It tells the participants what local transactions to execute and in what order. If a failure occurs, the orchestrator issues the rollbacks.

- **Pros:** Clear visibility into the state of the transaction; easier to debug and manage complex flows.
- **Cons:** Introduces a central point of failure; requires a state-machine engine.

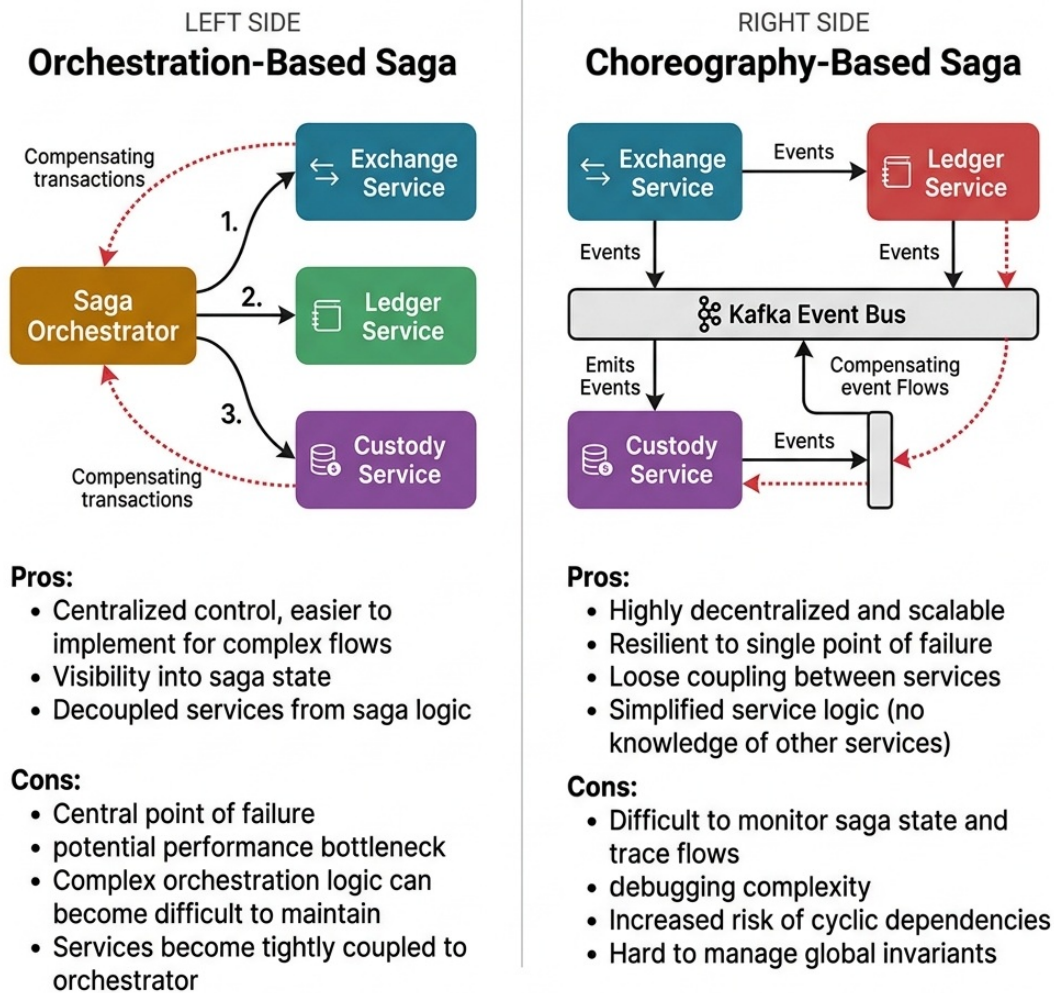


Figure 15.2: Saga Orchestration vs Choreography

15.5 Distributed Rate Limiting

To protect microservices from cascading failures or brute-force spikes, you must implement rate limiting. In a distributed environment, rate limits cannot be stored in-memory on a single application node.

15.5.1 Redis Sliding Window Rate Limiter

We use Redis to store request timestamps. A sliding window rate limiter maintains a sorted set for each user:

1. **Add Request:** Add current timestamp to sorted set using `ZADD`.
2. **Prune Old Requests:** Remove timestamps older than the sliding window (e.g., current time minus 1 minute) using `ZREMRANGEBYSCORE`.
3. **Count Volume:** Count active timestamps using `ZCARD`.

4. **Enforce Limit:** If the count exceeds the threshold, reject the request. Otherwise, allow it and set a key TTL (EXPIRE) to reclaim memory when the client goes inactive.

Redis Sliding Window Rate Limiting

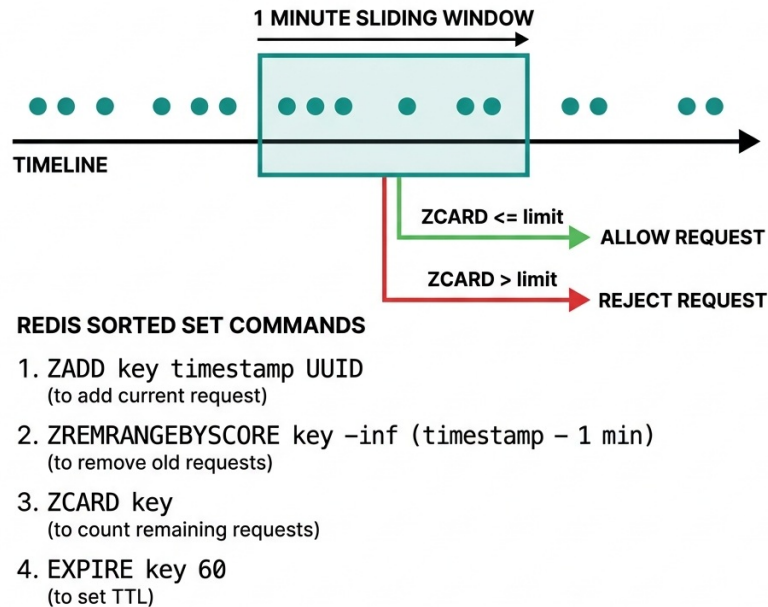


Figure 15.3: Redis Sliding Window Rate Limiting

15.6 Microservice Resiliency Patterns

When designing distributed systems, you must prevent cascading failures where one slow service consumes all resources on upstream callers.

```
[Client] ----> [API Gateway] ----> [Exchange Service] ----> [Slow Ledger Service]
                                     (Threads Exhausted)
```

15.6.1 Circuit Breakers

A **Circuit Breaker** wraps remote calls. It monitors failure rates.

- **Closed State:** Requests pass through.
- **Open State:** When the failure rate crosses a threshold (e.g., 50% failures over 10 seconds), the circuit trips (opens). Subsequent requests fail fast immediately, preventing resource exhaustion on the caller.
- **Half-Open State:** After a timeout, the breaker allows a few probe requests to pass. If they succeed, it closes; if they fail, it opens again.

Circuit Breaker Pattern

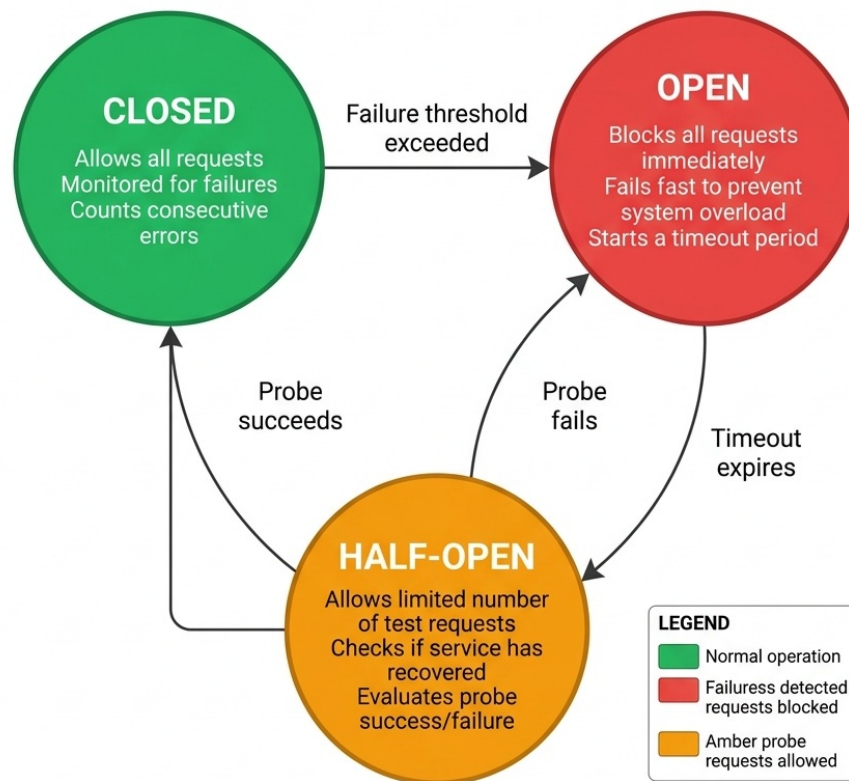


Figure 15.4: Circuit Breaker State Machine

Why is it called a “Circuit Breaker”? The pattern is borrowed directly from **electrical engineering**. In your home’s breaker panel, a circuit breaker trips (opens) when it detects excessive current, preventing an electrical fire. Michael Nygard popularized the software version in his 2007 book *Release It!*, mapping the electrical metaphor to distributed systems: when a downstream service is failing, “trip the breaker” to fail fast and protect the calling system from cascading overload. The three states (Closed, Open, Half-Open) mirror how a physical breaker resets after the fault clears.

15.6.2 Bulkheads

Named after the watertight compartments of a ship’s hull. The **Bulkhead Pattern** isolates resources (like thread pools or memory) allocated to specific services. If the Ledger Service slows down, only the thread pool dedicated to the Ledger will exhaust its threads. The rest of the Exchange Service (such as market data streaming) remains completely unaffected.

Why “Bulkhead”? On a cargo ship, bulkheads are vertical walls that divide the hull into sealed compartments. If one compartment floods, the bulkheads prevent water from spreading to adjacent compartments — the ship stays afloat. In software, we partition thread pools and connection pools the same way: one failing dependency can drain its own pool without sinking the entire application.

15.7 Microservices Observability

A resilient architecture is impossible to manage without deep visibility into execution paths. In technical interviews, discuss the **Three Pillars of Observability**:

15.7.1 Structured Logging & Trace Propagation

Never write plain text logs. All logs must be output as structured JSON. To trace a single request as it hops across multiple microservices (API Gateway → Exchange → Ledger), utilize **Trace Context Propagation**:

- When a request enters the API Gateway, the gateway checks for a **traceparent** HTTP header (W3C standard). If missing, it generates a unique **trace_id** (128-bit).
- The gateway includes this **trace_id** in all outgoing HTTP requests, gRPC metadata, or Kafka message headers.
- Every service logs the current **trace_id** along with its log statements. In centralized log management systems (like ELK Stack or Datadog), searching for a single **trace_id** brings up the exact execution timeline across all services.

15.7.2 Distributed Tracing

Utilize OpenTelemetry to capture spans (timed execution blocks). Spans record database queries, network latencies, and function call execution times, creating visualization traces to pinpoint latency hotspots.

15.7.3 Metrics Collection

Expose endpoints (e.g., Prometheus JMX/Micrometer) to collect performance metrics:

- **System Metrics:** CPU usage, memory utilization, JVM garbage collection frequency, thread counts.
- **Application Metrics:** API request rates, HTTP 5xx error counts, database connection pool saturation, and circuit breaker states.

STAR Moment: Compensating Transactions vs Rollback

In a system design interview, make sure to emphasize that a Saga cannot “rollback” in the traditional database sense, because the initial transactions have already been committed. Instead, we must write explicit **compensating transactions** (e.g., if a debit was committed, the compensation is a credit). You must design these compensating operations to be **idempotent**, as they may be retried multiple times during a network partition.

Chapter 16

Database Design, Compliance, and Security

“In financial systems, a database is not just a storage system; it is the ultimate source of truth, legal compliance, and operational trust.”

16.1 Database Architecture in Interviews

When designing systems in interviews, candidates frequently treat databases as simple black boxes, choosing “MySQL” or “MongoDB” arbitrarily.

At a senior level, you must justify your database selection based on transactional guarantees (ACID), storage engines (B-Tree vs. LSM-Tree), and compliance boundaries (PCI-DSS, SOC2). If your system processes card transactions or sensitive personal identifiable information (PII), you must articulate how to encrypt and tokenize this data to prevent costly leaks.

In this chapter, we explore how to design a secure, compliant storage architecture for AuraPay, focusing on PCI-DSS card tokenization and database index tuning.

16.2 ACID vs. NoSQL: Choosing the Right Engine

For financial transaction ledgers, the choice of database is crucial.

16.2.1 Relational Databases (RDBMS)

RDBMS engines (PostgreSQL, MySQL, Oracle) utilize **ACID** transactions (Atomicity, Consistency, Isolation, Durability).

- **Why it’s essential:** In a double-entry book-keeping system, a debit and credit must succeed or fail together. An RDBMS ensures that a database failure halfway through a transaction rolls back both sides of the ledger.
- **Storage Engine (B-Tree):** RDBMS platforms typically use B-Tree indexes. B-Trees are optimized for read-heavy workloads with rapid random access but can suffer from write amplification during high-velocity insert/update operations.

16.2.2 NoSQL & NewSQL Databases

- **NoSQL (Cassandra, DynamoDB):** Trade consistency for scalability (BASE model - Basically Available, Soft state, Eventual consistency). They use LSM-Tree (Log-Structured Merge-tree) storage engines, which write sequentially to memory buffers (MemTable) before flushing to disk (SSTable), providing very high write speeds but slow random reads.
- **NewSQL (Spanner, CockroachDB):** Provide the scale of NoSQL with the ACID guarantees of an RDBMS using distributed consensus protocols (Raft/Paxos) and atomic clocks.

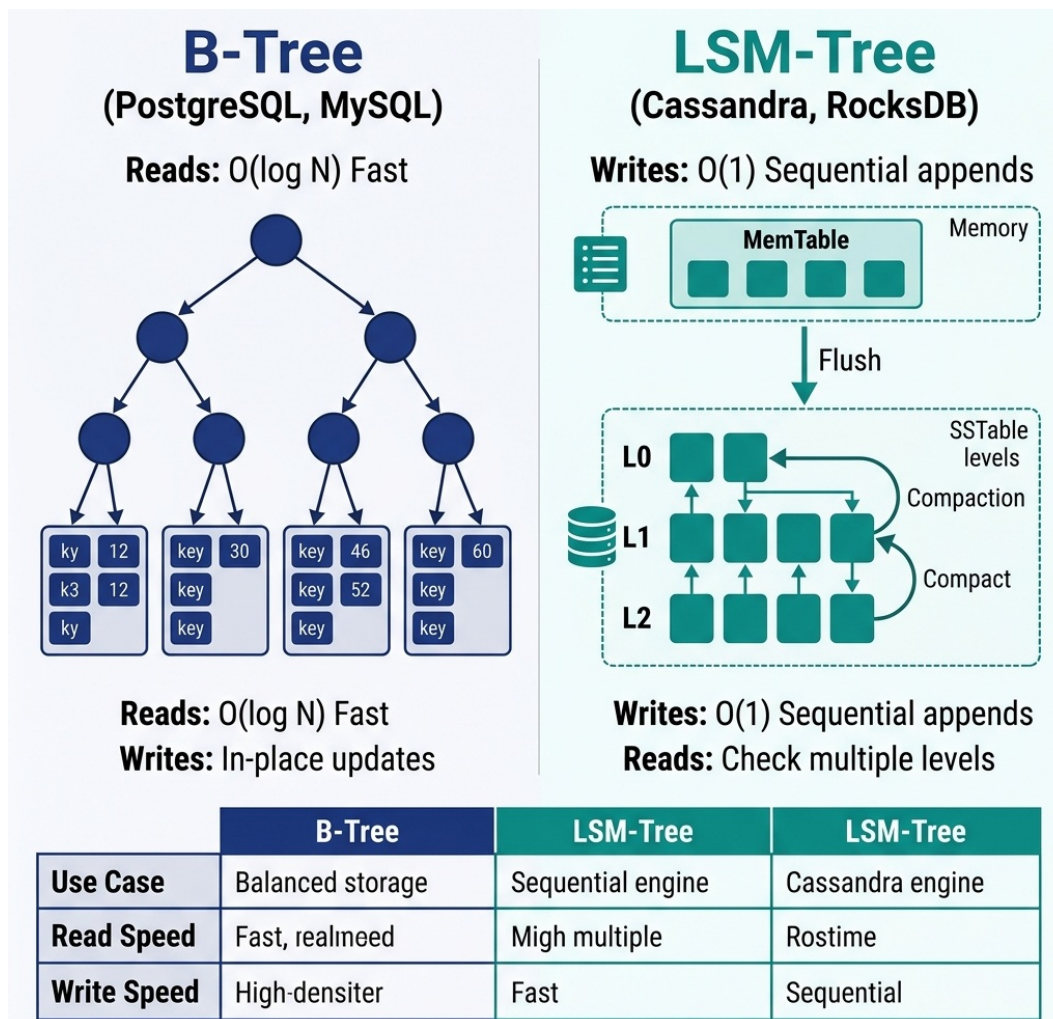


Figure 16.1: B-Tree vs LSM-Tree Storage Engines

Why is it called "PostgreSQL"? The name traces back to the 1970s. UC Berkeley professor Michael Stonebraker created a relational database called **Ingres**. In 1986, he started a successor project called **Post-Ingres** (i.e., "after Ingres"), later shortened to **Postgres**. When SQL support was added in 1996, the name became **PostgreSQL** 2014 literally "Post-Ingres with SQL." The elephant logo? Chosen simply because elephants *never forget* 2014 a fitting mascot for a database.

Why is it called "Redis"? The name is an acronym: **RE**mote **DI**ctionary **S**erver. Italian developer Salvatore Sanfilippo (known online as *antirez*) created it in 2009 because he needed a fast in-memory key-value store for his real-time web analytics startup. He designed it as a networked dictionary 2014 a remote hash map you can query over TCP. The name captures exactly what it is: a dictionary server that lives on a remote machine.

Interview Rule: Always use an ACID-compliant engine (RDBMS or NewSQL) for core ledgers. Use NoSQL only for write-heavy, eventually-consistent workloads like clickstreams, activity logs, or audit trail event streams.

16.3 Database Sharding Strategies

When database size or write throughput exceeds the limits of a single master server, you must partition the database across multiple physical machines. This is called **Sharding**.

16.3.1 Sharding Methodologies

1. **Range-Based Sharding:** Partitioning data based on ranges of an attribute (e.g., routing users with IDs 1–1,000,000 to Shard A, and 1,000,001–2,000,000 to Shard B).
 - **Trade-off:** Simple to implement but leads to severe write imbalances if activity is concentrated in a specific range.
2. **Hash-Based Sharding:** Applying a hash function to the partition key:

$$\text{Shard ID} = \text{hash}(\text{key}) \pmod{N}$$

- **Trade-off:** Uniform data distribution. However, if the number of shards N changes (re-sharding), almost all historical data must be migrated.
3. **Directory-Based Sharding:** Utilizing a centralized lookup service (lookup table) to track which shard stores a specific partition key.
 - **Trade-off:** Flexible, but introduces a single point of failure and query latency bottleneck at the lookup layer.

16.4 Indexing Deep-Dive & Performance Optimization

Database indexes are critical for search speed, but they carry a write cost. Every index added increases database write latency and storage requirements.

16.4.1 Index Types

- **B-Tree Indexes (Default):** Balanced search trees. Optimized for exact matches, range queries, and sorted order retrieval.
- **Hash Indexes:** Use hash tables. Optimized *only* for exact matches (=). Do not support range queries or sorting.

- **Inverted Indexes (GIN/GiST):** Used for full-text search and complex document data structures (like JSONB fields in PostgreSQL).

16.4.2 Index Design Guidelines

1. **Covering Indexes:** An index that contains all columns required for a specific query. If a query selects columns A and B from a table, creating a composite index on (A, B) allows the database engine to retrieve the values directly from the index tree, bypassing the primary table pages entirely.
2. **Composite Index Left-Prefix Rule:** A composite index on (columnA, columnB) can be used to optimize queries searching by columnA, or columnA AND columnB. However, it *cannot* optimize queries searching only by columnB. Order your composite index columns based on query frequency.
3. **Write Amplification:** Avoid indexing columns that are updated frequently. Doing so forces the database to rewrite index pages constantly, degrading overall write performance.

16.5 PCI-DSS Compliance & Tokenization

The Payment Card Industry Data Security Standard (PCI-DSS) imposes strict requirements on the handling of Primary Account Numbers (PANs). Storing raw 16-digit card numbers in your main application database is a major security risk and forces your entire infrastructure to fall within the scope of costly annual PCI audits.

16.5.1 The Tokenization Pattern

To minimize audit scope, you must implement **Tokenization**:

1. **Card Vault:** A separate, highly secure, network-isolated database (the Vault) that maps a PAN to a randomly generated, non-reversible **Token** (e.g., tok_9a8b7c).
2. **Encryption:** Inside the Vault, PAN data is encrypted using AES-256-GCM before storage.
3. **Application Separation:** The main billing and ledger applications only store and reference the token. Since they never store, process, or transmit raw card data, they are kept outside the scope of PCI-DSS regulations.

PCI-DSS Card Tokenization

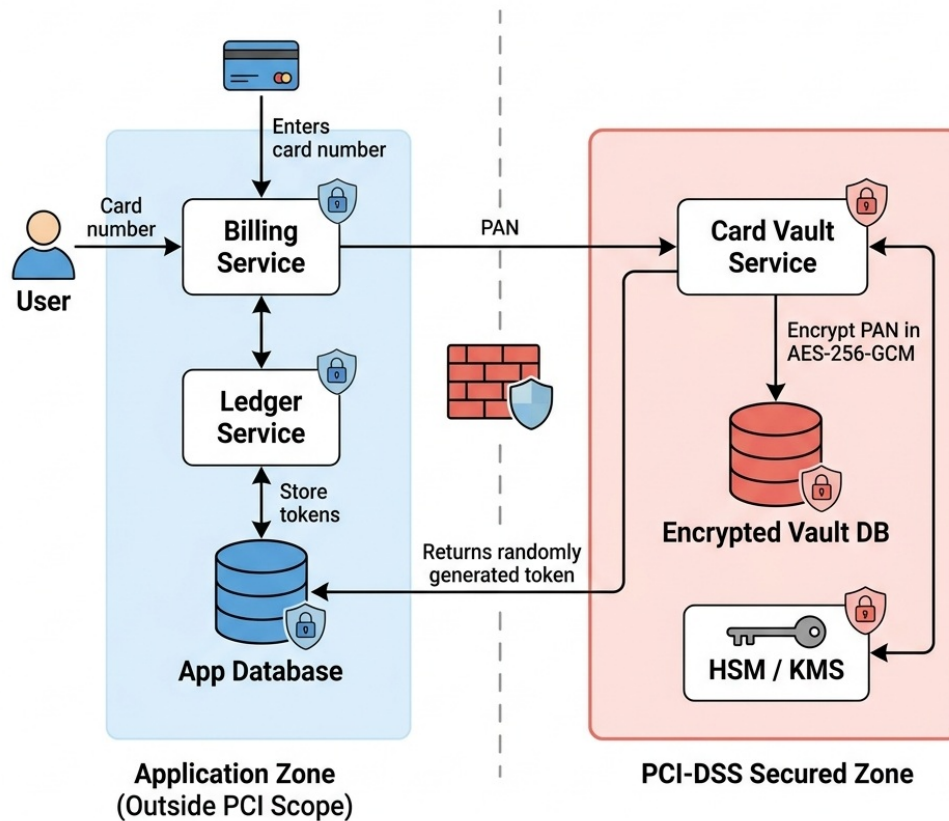


Figure 16.2: PCI-DSS Tokenization Vault Architecture

The following utility demonstrates the encryption standard (AES-256 in Galois/Counter Mode) required for encrypting PANs or PII:

```
package com.aurapay.security;

import java.nio.ByteBuffer;
import java.security.SecureRandom;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;

/**
 * Utility for AES-GCM 256-bit encryption/decryption of sensitive PII or PAN
 *   ↳ data,
 * adhering to PCI-DSS requirements.
 */
```

```

public class TokenizationUtility {

    private static final String ALGORITHM = "AES/GCM/NoPadding";
    private static final int TAG_LENGTH_BITS = 128;
    private static final int IV_LENGTH_BYTES = 12;
    private static final SecureRandom SECURE_RANDOM = new SecureRandom();

    /**
     * Encrypts the plaintext data using the provided 256-bit key.
     * Returns a URL-safe Base64-encoded string containing
     * ↪ [IV][Ciphertext][Tag].
     */
    public static String encrypt(String plaintext, byte[] keyBytes) throws
    ↪ Exception {
        if (plaintext == null || keyBytes == null || keyBytes.length != 32) {
            throw new IllegalArgumentException("Invalid plaintext or key size.
            ↪ Key must be 256-bit.");
        }

        // 1. Generate a secure random Initialization Vector (IV)
        byte[] iv = new byte[IV_LENGTH_BYTES];
        SECURE_RANDOM.nextBytes(iv);

        // 2. Initialize Cipher in ENCRYPT_MODE
        SecretKey key = new SecretKeySpec(keyBytes, "AES");
        Cipher cipher = Cipher.getInstance(ALGORITHM);
        GCMParameterSpec spec = new GCMParameterSpec(TAG_LENGTH_BITS, iv);
        cipher.init(Cipher.ENCRYPT_MODE, key, spec);

        // 3. Encrypt the data
        byte[] ciphertext = cipher.doFinal(plaintext.getBytes("UTF-8"));

        // 4. Combine IV and Ciphertext into a single payload
        ByteBuffer byteBuffer = ByteBuffer.allocate(iv.length +
        ↪ ciphertext.length);
        byteBuffer.put(iv);
        byteBuffer.put(ciphertext);
        byte[] encryptedPayload = byteBuffer.array();

        // 5. Encode to Base64
        return Base64.getUrlEncoder().withoutPadding().encodeToString(
        ↪ encryptedPayload);
    }

    /**
     * Decrypts the Base64-encoded payload using the provided 256-bit key.
     */

```



```

public static String decrypt(String base64Payload, byte[] keyBytes) throws
↳ Exception {
    if (base64Payload == null || keyBytes == null || keyBytes.length != 32) {
        throw new IllegalArgumentException("Invalid payload or key size. Key
↳ must be 256-bit.");
    }

    // 1. Decode from Base64
    byte[] encryptedPayload = Base64.getUrlDecoder().decode(base64Payload);

    // 2. Extract the IV
    if (encryptedPayload.length < IV_LENGTH_BYTES) {
        throw new IllegalArgumentException("Ciphertext payload is truncated
↳ or invalid.");
    }
    ByteBuffer byteBuffer = ByteBuffer.wrap(encryptedPayload);
    byte[] iv = new byte[IV_LENGTH_BYTES];
    byteBuffer.get(iv);

    // 3. Extract the actual ciphertext
    byte[] ciphertext = new byte[byteBuffer.remaining()];
    byteBuffer.get(ciphertext);

    // 4. Initialize Cipher in DECRYPT_MODE
    SecretKey key = new SecretKeySpec(keyBytes, "AES");
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    GCMParameterSpec spec = new GCMParameterSpec(TAG_LENGTH_BITS, iv);
    cipher.init(Cipher.DECRYPT_MODE, key, spec);

    // 5. Decrypt and convert to String
    byte[] decryptedBytes = cipher.doFinal(ciphertext);
    return new String(decryptedBytes, "UTF-8");
}
}

```

GCM (Galois/Counter Mode) is preferred over CBC (Cipher Block Chaining) because it provides both **confidentiality** and **integrity (authenticity)**. It appends an authentication tag that prevents attackers from modifying the ciphertext in transit.

16.6 GDPR vs. Immutable Ledgers: Crypto-Shredding

A major conflict exists in modern database design between audit compliance (SOC2) and data privacy regulations (GDPR/CCPA):

- **SOC2 Requirement:** Maintain an immutable, append-only, cryptographically chained audit log that can never be modified or deleted.
- **GDPR Requirement:** The **Right to be Forgotten**. Users can request that all of their personal identifiable information (PII) be permanently deleted from your databases.

16.6.1 The Solution: Cryptographic Erasure (Crypto-Shredding)

Because you cannot delete a user's record from an immutable ledger (as doing so would break the cryptographic chain), you must apply **Crypto-Shredding**:

1. When a user is created, generate a unique, user-specific encryption key (e.g., AES-256 key).
2. Store the user's key in a secure Key Management Service (KMS) or Vault database.
3. All PII data written to the immutable ledger is encrypted using that specific user's key.
4. When a user submits a GDPR deletion request, **destroy the user's specific key from the KMS**.
5. Once the key is destroyed, the encrypted PII in the immutable ledger becomes mathematical noise that can never be decrypted again. This is legally accepted as a permanent deletion under GDPR compliance while keeping the ledger chain intact.

16.7 SOC2 Audit Trails & Immutable Ledgers

For compliance frameworks like SOC2, you must maintain a tamper-proof audit trail of all financial actions.

16.7.1 Design of a Tamper-Proof Audit Log

1. **Append-Only Tables:** Database permissions should restrict application users to INSERT queries on audit tables, preventing UPDATE or DELETE operations.
2. **Cryptographic Chaining:** Each audit log row should contain a cryptographic hash of the current row and the previous row's hash (similar to a blockchain ledger). If an attacker modifies a historical row, the chain break is instantly detectable during audit validation.
3. **Immutable Databases:** Utilize native ledger databases (like Amazon QLDB) or WORM (Write Once, Read Many) storage to mathematically guarantee data immutability.

Tamper-Proof Cryptographic Audit Trail for SOC2 Compliance

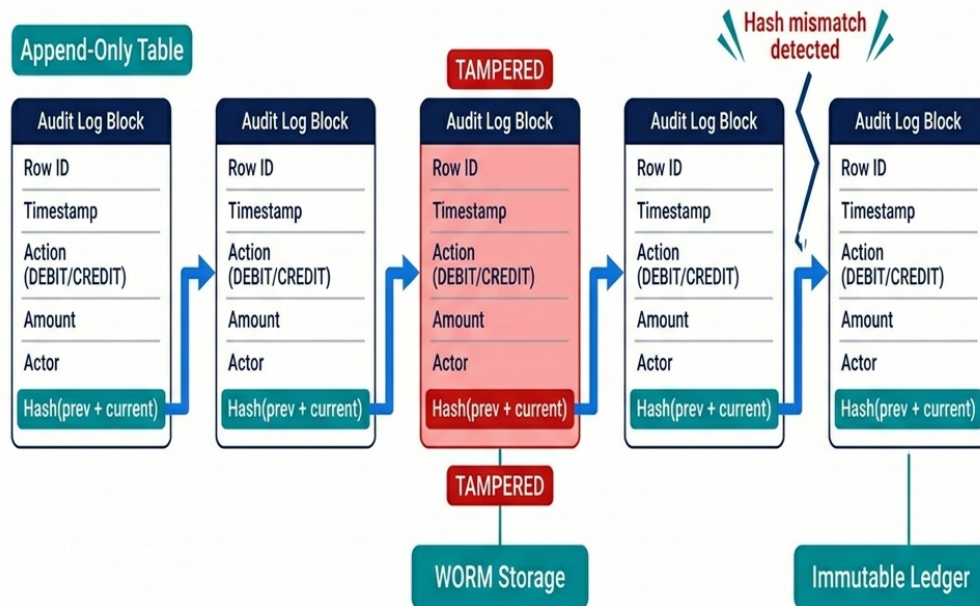


Figure 16.3: Cryptographic Audit Trail Chain

16.8 Hardening the Data Tier & Audits

Securing financial databases requires separating database engine administration from data access. During audits, one of the most critical security principles you must demonstrate is **perimeter isolation and credential partitioning**.

16.8.1 Connection Pool & Infrastructure Hardening

1. **Network Isolation (VPC):** The database should never reside in a public subnet. Access must be restricted via security groups to specific application servers residing in private subnets.
2. **IAM-Based Authentication:** Instead of hardcoding database credentials or using long-lived passwords, utilize temporary IAM credentials (like AWS IAM Database Authentication) or secure secret rotation services (like HashiCorp Vault) with a 30-day rotation policy.
3. **Connection Saturation & Timeouts:** To prevent Denial of Service (DoS) attacks or query starvation, configure pool limits strictly. Enforce a connection timeout of 250ms and a max life time of 30 minutes to recycle leaked database connections.

16.8.2 Mock Audit Scenario Drill

Here is a mock review dialog during a SOC2 compliance audit:

Auditor: *“How do you guarantee that a database administrator (DBA) or a developer with access to the raw database files cannot read credit card numbers or sensitive transactional data?”*

Candidate: **“All card numbers are stored inside a dedicated Card Vault. The PAN (Primary Account Number) is encrypted inside the vault using AES-256-GCM. The encryption key is stored in a dedicated Key Management Service (KMS) with access restricted via IAM roles that only the Vault service application user can assume.*

*Even if a DBA has root access to the database tables or extracts a raw disk backup, they cannot decrypt the PAN fields because they do not have decryption permissions on the KMS key. Furthermore, every decryption call is logged in an append-only audit trail in CloudTrail, which triggers instant alerts on unauthorized access attempts.”**

STAR Moment: The Security-First Architecture

In a system design interview, explain the concept of “auditing and perimeter isolation.” Show how you can use a separate network zone (VPC) for your Card Vault, with separate encryption keys managed by an HSM (Hardware Security Module) or Key Management Service (KMS), and separate access control roles. Decoupling data in this way reduces security risk and simplifies compliance audits.

Chapter 17

Behavioral Leadership and Technical Communication

“At a senior or executive level, your value is no longer measured by the quantity of code you write, but by your ability to align teams, navigate architectural tradeoffs, and resolve production crises with composure.”

17.1 Technical vs. Behavioral Alignment

When interviewing for a senior, staff, or engineering manager role, clearing the coding and system design rounds is only half the battle. In-person or Teams video calls inevitably culminate in a behavioral evaluation.

At this level, the interviewer assumes you possess technical competence. The behavioral round is designed to evaluate your **leadership, system ownership, conflict resolution, execution speed, and architectural maturity**. If you respond to situational questions with generic answers (e.g., “*I am a team player who works hard*”), you fail to demonstrate the maturity required to lead engineering organizations.

In this chapter, we adapt the classic **STAR (Situation, Task, Action, Result)** model into a technical-leadership narrative framework, providing mock response transcripts for common senior scenarios.

17.2 The Technical STAR Framework

To present your career achievements effectively, structure your behavioral narratives around technical metrics and architectural trade-offs:

Technical STAR Framework for Senior Engineering Interviews

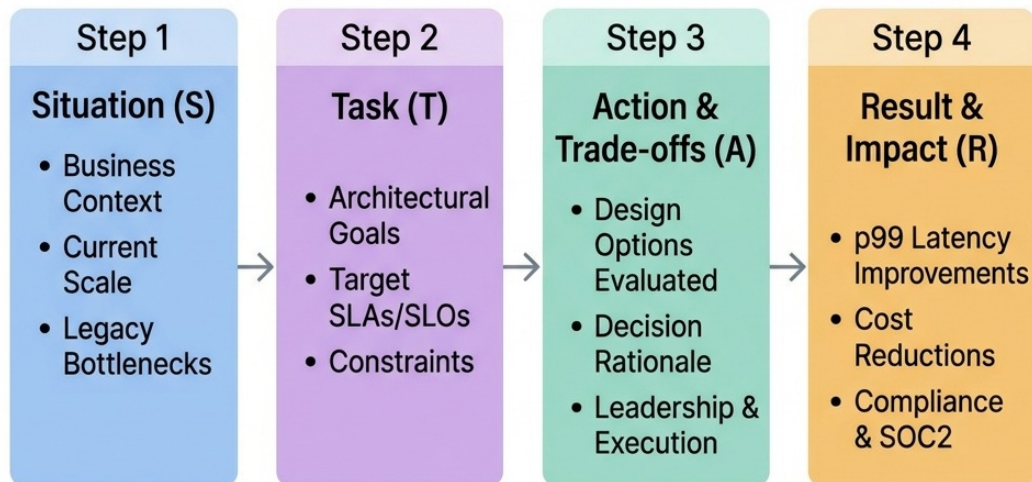


Figure 17.1: The Technical STAR Framework

How to apply the framework:

- **Situation (S):** Establish the business scale and constraints. What was the starting state (e.g., transaction volume, bottlenecks, legacy limitations)?
- **Task (T):** Define the architectural objectives, SLA requirements, and the technical scope of what you were responsible for delivering (e.g., migrate the ledger to a CP database while maintaining 99.99% availability).
- **Action & Trade-offs (A):** Describe the design options you evaluated, the trade-off decisions you made, and how you led the team through implementation.
- **Result & Impact (R):** Present quantitative, data-driven outcomes. Never say: “*We made the system faster.*” Say: “*We reduced p99 write latency by 45%, eliminated database locks, and passed the SOC2 compliance audit with zero findings.*”

17.3 Mock Scenario A: Architectural Disagreement (Lead / Staff Perspective)

Interviewer: “Tell me about a time you had a major disagreement with a peer or stakeholder about a technical design. How did you resolve it?”

17.3.1 The Strategy

A junior candidate focuses on the personal conflict or tries to prove they were “right.” A senior candidate frames the resolution around data-driven trade-off analysis, prototype benchmarks, and collaborative consensus-building.

17.3.2 The Response Transcript

“In my previous role at ZenithTrade, my team was tasked with scaling our matching engine to handle a 5x spike in transaction volume. A principal architect proposed rewriting our processing loops using a reactive programming model (Spring WebFlux). I had serious concerns about the operational overhead of reactive code, specifically debuggability, stack trace readability, and the steep learning curve for our support engineers.

Rather than engaging in an ideological debate, I proposed a 3-day time-boxed prototyping run. I built two benchmark pipelines: one using the proposed reactive model, and another using Java 21’s new Virtual Threads (Project Loom).

The prototype metrics revealed that while both models handled the required 20,000 concurrent requests without thread exhaustion, the virtual threads implementation reduced CPU utilization by 15% (due to lower context-switch overhead) and preserved our existing synchronous debugging tools.

I presented these findings in an architecture review document, outlining the maintenance costs of both approaches. The principal architect agreed with the data, and we proceeded with the Virtual Threads design. The system successfully launched, sustaining 5x load with zero stability incidents.”

17.4 Mock Scenario B: Production Crisis Management (Engineering Manager Perspective)

Interviewer: *“Describe a major production outage you managed. How did you coordinate the response and prevent it from happening again?”*

17.4.1 The Strategy

Focus on command composure, blameless post-mortem culture, and root-cause remediation rather than pointing fingers or downplaying the event.

17.4.2 The Response Transcript

“During a high-volume retail promotion on AuraPay, our ledger database connection pool saturated, causing transaction failures for approximately 15% of our users. As the Engineering Manager, I immediately initiated our incident response protocol, establishing a dedicated bridge call and assigning roles: one engineer to analyze database metrics, one to review application logs, and a product manager to handle external client communications.

We identified that our connection pool size was set to 200, which was starving the database CPU with constant thread context switching. I instructed the team to apply the HikariCP pool sizing formula, reducing the connection limit to 30. This immediately stabilized database CPU utilization from 98% down to 42%, restoring transaction flow.

To prevent future occurrences, I led a blameless post-mortem. We discovered that a recent release had introduced a database query inside a parallel stream pipeline, starving the common ForkJoinPool. We refactored the stream to execute asynchronously outside the transaction boundary and set up automated alert thresholds on connection pool saturation. Since then, our system uptime has remained at 99.99% under peak promotional events.”

17.5 Mock Scenario C: Balancing Technical Debt vs. Features (Director Perspective)

Interviewer: *“How do you balance business pressure for new features against the technical necessity of refactoring legacy code?”*

17.5.1 The Strategy

Frame technical debt as a financial risk to the business. Show that you can speak the language of product managers and executives, translating code quality into operational velocity.

17.5.2 The Response Transcript

“When I joined ChiramTrust, the identity consent module was built as an anemic domain model with scattered business logic. Product management wanted to launch three new OAuth integrations within two months, but our engineering velocity was bottlenecked because every minor change to our domain models broke unrelated validation paths, requiring days of manual patching.

I knew that pushing features without refactoring would increase our defect rate in production. I met with the VP of Product and translated our technical debt into business risk: our current regression bug rate was 18%, and continuing at this pace would delay the integration launch by at least four weeks due to QA cycles.

I proposed a compromise: we would dedicate 30% of our capacity in the next two sprints to refactor the consent model into an encapsulated aggregate root, establishing clean validation boundaries. The remaining 70% would be spent on the integration layouts.

The team successfully executed the refactor, removing setters and enclosing the invariants inside the domain objects. This refactoring reduced our regression bug rate to less than 2% and actually accelerated the development of the final two integrations, allowing us to launch the features a week ahead of the original deadline.”

17.6 Checklist for Video (Teams) & In-Person Technical Interviews

To project executive presence and clear technical rounds on live video calls or in-person sessions, adhere to these guidelines:

1. **The Virtual Whiteboard Technique:** On Teams calls, do not just talk. Utilize a digital whiteboard (like Miro or Excalidraw) to draw bounded contexts, Saga flows, and database sharding rings. Visual diagrams make your architecture concrete and easy for the interviewer to follow.
2. **The Clarification Pause:** When presented with a coding problem, do not write code immediately. Pause for 2-3 minutes to write down the pre-conditions, post-conditions, and input/output types as comments. This shows structural discipline and prevents off-by-one errors.
3. **The Trade-Off Verbalization:** Throughout the interview, constantly verbalize your architectural trade-offs (e.g., *“If we use Redis for rate limiting, we gain speed, but we must handle memory expiration and potential write consistency issues during partition events”*). Never present a design as “perfect.”

STAR Moment: Speak in Metrics

When presenting your career accomplishments, translate every engineering activity into a business outcome. Never say: *“I rewrote the database queries.”* Say: *“I optimized our query indexes, reducing database read latency by 60% and cutting our monthly database hosting cost by \$12,000.”* Executives and engineering leaders hire developers who understand the financial and operational impact of their code.

Chapter 18

Testing and CI/CD Strategies for High-Performance Systems

“The quality of your production system is a direct reflection of your automated validation boundaries. If you cannot test it in isolation, you cannot trust it at scale.”

18.1 The Testing Paradigm in Senior Interviews

In technical interviews for lead, staff, or engineering manager roles, coding challenges do not end with a working algorithm. The interviewer will ask: *“How do you test this code? How do you ensure this does not break in production? What is your strategy for validating microservice API contracts?”*

Many candidates respond with simple unit tests. However, a senior candidate must present a structured **Testing Pyramid** strategy, showing how they balance unit tests with Testcontainers-based integration tests, API contract tests, and continuous delivery (CI/CD) verification.

Professional Software Testing Pyramid

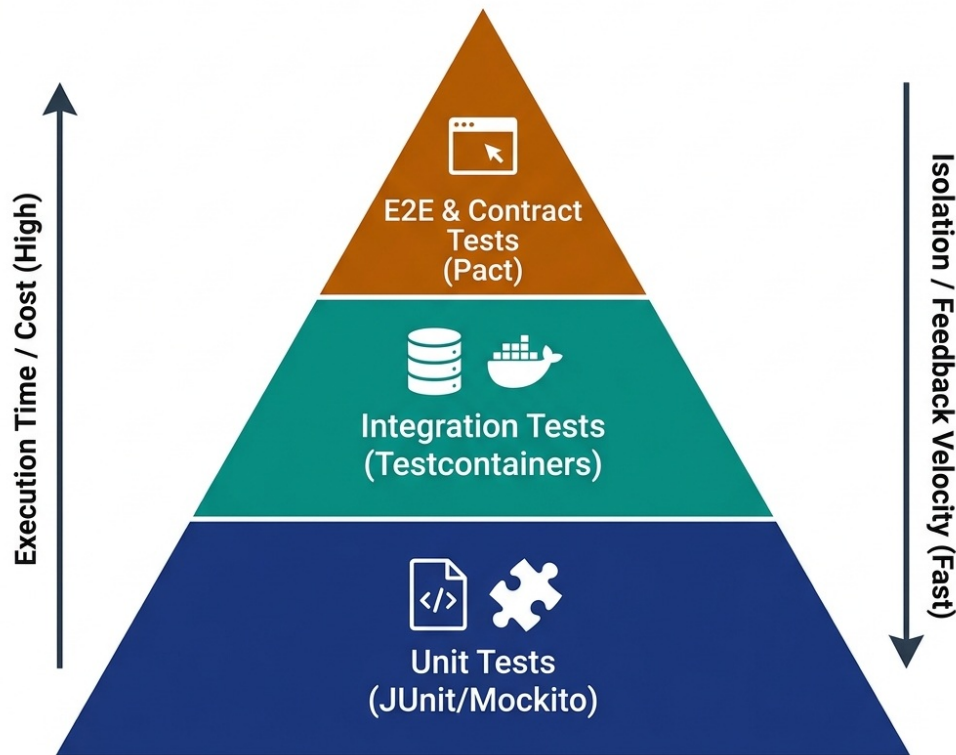


Figure 18.1: The Technical Testing Pyramid

18.2 The Testing Pyramid

An effective testing strategy separates validation boundaries into three layers, balancing execution speed and operational cost against validation fidelity:

18.2.1 Unit Testing with Abstractions

Unit tests are the foundation of the pyramid. They validate the internal logic of a single class in isolation, replacing all external infrastructure dependencies (such as databases and network gateways) with mock interfaces.

- **Velocity:** Execute in milliseconds.
- **Boundary:** Focuses purely on code correctness and invariant compliance.
- **Coverage:** High code coverage (90%+), testing all logical paths and edge cases.

The following code illustrates unit testing our decoupled `TransactionProcessor` by mocking its repository and notification interfaces:

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import java.math.BigDecimal;
```

```

public class TransactionProcessorTest {

    @Test
    public void testSuccessfulTransfer_EnforcesInvariants() {
        // Arrange Mock Dependencies
        LedgerRepository mockRepo = mock(LedgerRepository.class);
        FeeCalculator mockCalculator = mock(FeeCalculator.class);
        TransactionNotificationSender mockSender =
        ↪ mock(TransactionNotificationSender.class);

        LedgerAccount source = new LedgerAccount("acc-source", new
        ↪ BigDecimal("100.00"), "USD");
        LedgerAccount destination = new LedgerAccount("acc-dest", new
        ↪ BigDecimal("50.00"), "USD");

        when(mockRepo.findById("acc-source")).thenReturn(source);
        when(mockRepo.findById("acc-dest")).thenReturn(destination);
        when(mockCalculator.calculateFee(any(BigDecimal.class))).thenReturn(
        ↪ BigDecimal.ZERO);

        TransactionProcessor processor = new TransactionProcessor(mockRepo,
        ↪ mockCalculator, mockSender);

        // Act
        processor.processTransfer("acc-source", "acc-dest", new
        ↪ BigDecimal("30.00"));

        // Assert state invariants updated
        assertEquals(new BigDecimal("70.00"), source.getBalance());
        assertEquals(new BigDecimal("80.00"), destination.getBalance());

        // Assert repository saved both
        verify(mockRepo).save(source);
        verify(mockRepo).save(destination);
        verify(mockSender).sendNotification(any());
    }
}

```

By utilizing mock objects, we verify that the processor correctly coordinates the transfer, updates balance invariants, and calls the persistence layer, without requiring an active database connection.

Why is it called “Mockito”? The popular Java mocking framework is named after the **Mojito** cocktail — a playful twist by its Polish creator Szczepan Faber. Just as a bartender mixes ingredients to create something refreshing, Mockito mixes stubs and verifications to create clean, readable tests. The name also echoes the Spanish suffix “-ito” (meaning “little”), suggesting lightweight mock objects.

18.2.2 Integration Testing with Testcontainers

While unit tests verify logic correctness, they cannot detect SQL query syntax errors, schema constraint violations, or message serialization bugs. For this, you need **Integration Tests**. In system design interviews, describe **Testcontainers**:

- **Mechanism:** During test execution, the testing framework utilizes Docker to spin up actual database instances (PostgreSQL, MySQL) or brokers (Kafka, RabbitMQ) in local containers.
- **Assertion:** The test runs against real database engines, validating database locks, unique constraint violations, and transaction rollbacks. Once execution completes, the container is destroyed.
- **Impact:** Eliminates the anti-pattern of testing against mock database structures (e.g., using H2 in-memory database for testing PostgreSQL code, which misses Postgres-specific syntax or transaction behaviors).

18.2.3 API Contract Testing (Pact)

In a microservices mesh, service dependencies are the primary source of integration failures (e.g., the User Service changes an API field name, breaking the Billing Service). To prevent this without the latency of End-to-End (E2E) testing, implement **Consumer-Driven Contract Testing (Pact)**:

- **Contract File:** The consumer service defines its API requests and expected responses in a contract file.
- **Provider Verification:** The provider service runs automated tests against this contract file to verify compliance. If the provider modifies its API in a way that violates the contract, the build fails before deployment.

Why is it called “Pact”? A pact is a formal agreement between two parties. In contract testing, the consumer and provider make a *pact* — a machine-readable agreement about the API’s shape. If either side breaks the pact, the build fails. The name was chosen by the team at REA Group (Australia) to emphasize that API compatibility is a mutual commitment, not a one-sided assumption.

18.3 Advanced Testing Techniques

18.3.1 Mutation Testing

Code coverage alone is a misleading metric. A test suite can achieve 100% line coverage while asserting nothing meaningful. **Mutation Testing** validates the *quality* of your assertions:

1. The mutation framework (e.g., PIT for Java, Stryker for JavaScript/C#) systematically injects small bugs (“mutants”) into your production code — flipping comparison operators, negating boolean returns, removing method calls.
2. Your test suite is then executed against each mutant.
3. If your tests **fail** (detecting the mutant), the mutant is “killed” — your tests are effective.
4. If your tests **pass** despite the mutation, the mutant “survives” — your tests are weak and missed a real defect scenario.

A **mutation score** above 85% indicates a robust test suite. In interviews, mentioning mutation testing immediately signals that you understand test quality beyond surface-level coverage metrics.

18.3.2 Property-Based Testing

Traditional unit tests validate specific hand-picked input-output pairs. **Property-Based Testing** (e.g., jqwik for Java, Hypothesis for Python, FsCheck for C#) generates thousands of random inputs and verifies that invariants hold for all of them:

- **Example:** For a `Money.add()` method, assert that `a.add(b).equals(b.add(a))` (commutativity) and `a.add(Money.ZERO).equals(a)` (identity) for any randomly generated amounts and currencies.
- **Power:** Discovers edge cases that human-authored tests miss — overflow boundaries, empty collections, unicode strings, negative values.

18.3.3 Chaos Engineering

At the staff/principal level, you are expected to design systems that survive infrastructure failures. **Chaos Engineering** proactively injects failures into production-like environments to validate resilience:

- **Network Partitions:** Simulate network splits between services to verify Circuit Breaker activation and graceful degradation.
- **Latency Injection:** Add artificial delays (e.g., 5-second response times from a database) to verify timeout configurations and bulkhead isolation.
- **Instance Termination:** Randomly kill service instances to test auto-scaling recovery and consumer group rebalancing.
- **Tools:** Netflix Chaos Monkey, AWS Fault Injection Simulator, Gremlin, LitmusChaos.

Interview Signal: When asked “*How do you ensure reliability?*”, answering “*We run quarterly game days using chaos engineering to validate our circuit breakers and consumer group rebalancing under simulated Kafka broker failures*” demonstrates operational maturity far beyond unit testing.

18.4 Feature Flags and Progressive Delivery

Modern release engineering decouples **deployment** (shipping code to production) from **release** (enabling features for users):

18.4.1 Feature Flag Architecture

- **Implementation:** Wrap new features behind boolean flags stored in a centralized configuration service (LaunchDarkly, Unleash, or a custom Redis-backed service).
- **Granularity:** Flags can target individual users (beta testers), user segments (enterprise tier), geographic regions, or percentages of traffic.
- **Kill Switch:** If a new feature causes errors in production, disable the flag instantly without rolling back the entire deployment.
- **Technical Debt:** Feature flags must have an expiration policy. Stale flags left in the codebase for months create branching complexity and testing overhead. Enforce cleanup sprints.

18.4.2 Trunk-Based Development

Feature flags enable **trunk-based development** — all engineers commit directly to the main branch. There are no long-lived feature branches:

- Every commit is deployed to production behind a flag.
- Integration conflicts are caught immediately instead of during painful merge events weeks later.
- Release cadence accelerates from weekly to multiple daily deployments.

18.5 Continuous Integration (CI/CD) Compliance Pipeline

A senior engineering leader does not rely on developers remembering to run tests. Quality checks must be automated inside a **CI/CD Pipeline** before merging code to main branches:

18.5.1 Automated Pipeline Checks

1. **Linting & Code Formatting:** Ensures consistent style guidelines across the team.
2. **Static Application Security Testing (SAST):** Scans source code for potential vulnerabilities (e.g., SQL injections, insecure cryptographic configurations, hardcoded API keys) using tools like SonarQube.
3. **Dependency Scanning:** Scans imports for known security vulnerabilities (CVEs) and license compliance issues.
4. **Automated Unit & Integration Execution:** Blocks pull request merges if any test fails or if coverage drops below the required threshold.
5. **Mutation Score Gate:** Block merges if the mutation score drops below 80%, ensuring new code has meaningful test coverage.
6. **Contract Verification:** Run Pact provider verification against all consumer contracts before deploying API changes.

18.5.2 Pipeline as Code

Define your entire CI/CD pipeline in version-controlled configuration files (e.g., GitHub Actions YAML, Jenkinsfile, GitLab CI):

- **Reproducibility:** Any engineer can trace exactly what checks ran for any commit.
- **Auditability:** SOC2 compliance requires evidence that all production releases passed automated security and quality gates. Pipeline-as-code provides this audit trail automatically.

18.6 Modern Deployment Strategies

Once the CI/CD pipeline validates code correctness, releasing the software to production requires strategies that minimize user impact during updates:

18.6.1 Blue-Green Deployments

Maintain two identical physical production environments:

- **Blue Environment:** Actively hosts production traffic.
- **Green Environment:** Hosts the new code release.

- **Switch:** Once validation tests pass on the Green environment, the load balancer switches traffic from Blue to Green. If a rollback is needed, the switch routes traffic back immediately.

18.6.2 Canary Deployments

Deploy the new release to a small subset of production instances (e.g., routing 2% of user traffic to the new version).

- **Monitoring:** Monitor error rates, latency metrics, and resource utilization on the canary instances.
- **Scale:** If metrics remain stable, gradually scale traffic routing to 10%, 50%, and finally 100% of instances, destroying the old version.
- **Automated Rollback:** Configure automated rollback triggers — if the canary’s error rate exceeds 1% or p99 latency increases by more than 200ms, automatically shift all traffic back to the stable version.

18.6.3 Rolling Deployments

Update instances one at a time (or in small batches) behind the load balancer:

- Each instance is drained of active connections, updated, health-checked, and re-registered.
- Slower than blue-green but requires no duplicate infrastructure.
- Best suited for stateless microservices with fast startup times.

STAR Moment: The Mocking Boundary

In a technical interview, emphasize that you know *when* to mock. Say: “*We mock network calls and database interfaces in our unit tests to keep feedback loops fast. But we never mock our domain aggregates or value objects. Testing our business rules against actual domain structures guarantees that our core invariants are always enforced. For integration boundaries, we use Testcontainers against real Postgres and Kafka instances, and we validate API contracts using Pact before every deployment.*” This shows you understand domain boundary protection and production-grade testing strategy.

18.7 Performance Testing & Load Validation

Performance testing is a critical step in CI/CD pipelines to ensure systems remain reliable under expected and unexpected traffic. Rather than waiting for production outages, modern engineering teams validate performance continuously using different load profiles.

18.7.1 Types of Performance Tests

1. **Load Testing** — Validate system behavior under expected peak load
 - **Goal:** verify response times and throughput meet SLAs under normal-to-peak traffic
 - **Example:** simulate 10,000 concurrent users on an e-commerce checkout API
 - **Key metrics:** p50/p95/p99 latency, requests/second, error rate
2. **Stress Testing** — Find the breaking point
 - **Goal:** push beyond expected load to discover where the system degrades or fails
 - **Example:** gradually increase from 10K to 100K concurrent users until error rate exceeds 5%

- **Key insight:** identify the bottleneck (CPU? memory? DB connections? network?)
3. **Soak Testing (Endurance Testing)** — Detect memory leaks and resource exhaustion
 - **Goal:** run sustained moderate load for 4-24 hours
 - **Catches:** memory leaks, connection pool exhaustion, log file disk filling, GC pressure
 4. **Spike Testing** — Validate auto-scaling and recovery
 - **Goal:** suddenly surge traffic (e.g., 0 to 50K users in 30 seconds) and verify recovery
 - **Catches:** auto-scaling lag, cold-start penalties, circuit breaker activation

18.7.2 Performance Testing Tools

When selecting a tool, consider the protocols supported and how well it integrates into CI/CD pipelines:

Tool	Language	Protocol Support	Distributed	Best For
k6 (Grafana)	JavaScript	HTTP, gRPC, WebSocket	Yes (k6 Cloud)	Developer-friendly, CI/CD integration
JMeter	Java/XML	HTTP, JDBC, JMS, LDAP	Yes	Enterprise, complex protocols
Gatling	Scala/Java	HTTP, WebSocket	Yes	High-performance simulation
Locust	Python	HTTP	Yes	Python teams, custom load patterns
Artillery	JavaScript	HTTP, WebSocket, Socket.IO	Yes (Cloud)	Serverless, quick setup

18.7.3 k6 Load Test Example

The following is a concrete k6 script example written in JavaScript. It demonstrates how to define load stages and enforce SLAs through thresholds:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // Ramp up to 100 users
    { duration: '5m', target: 100 }, // Hold at 100 users
    { duration: '2m', target: 500 }, // Ramp up to 500 users
    { duration: '5m', target: 500 }, // Hold at 500 users
    { duration: '2m', target: 0 },   // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<250'], // 95% of requests under 250ms
  }
};
```



```

    http_req_failed: ['rate<0.01'],    // Error rate under 1%
  },
};

export default function () {
  const res = http.get('https://api.example.com/orders');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 500ms': (r) => r.timings.duration < 500,
  });
  sleep(1);
}

```

18.7.4 SLA Validation in CI/CD Pipelines

Integrating performance tests into CI/CD ensures that latency and throughput regressions are caught before they reach production: - Run load tests as a pipeline stage AFTER integration tests pass - Define performance budgets as code (thresholds in k6/Gatling config) - Fail the build if p95 latency exceeds SLA or error rate exceeds threshold - Store results in a time-series database (InfluxDB/Prometheus) for trend analysis - Alert on performance regressions compared to the previous release baseline

18.7.5 Performance Anti-Patterns

When designing load tests, avoid these common mistakes: 1. **Testing in non-production environments** — hardware differences invalidate results 2. **Not warming up the JVM** — JIT compilation skews early measurements (add a warm-up stage) 3. **Ignoring connection pooling** — each virtual user should reuse connections like production clients 4. **Measuring averages instead of percentiles** — p99 matters more than mean (a few slow requests hide behind good averages) 5. **Not testing database under load** — the DB is usually the bottleneck, not the application server

18.7.6 HikariCP Connection Pool Sizing Under Load

When load testing data-intensive applications, connection pool sizing is a common bottleneck. As discussed in earlier chapters, the optimal pool size formula is:

$$\text{Pool Size} = T_n \times (C_m - 1) + 1$$

Where T_n = number of threads, C_m = maximum concurrent queries per thread.

Under-provisioning the pool causes thread starvation, and over-provisioning wastes database connections.

Chapter 19

Distributed Event Streaming and Message Brokers

“An event log is the ultimate source of historical truth. In a distributed architecture, message brokers serve as the central nervous system, routing states across service boundaries.”

19.1 Event Streaming in System Design

In microservice architectures, services must communicate asynchronously. Interview candidates often default to saying: “*We will send a message via Kafka.*”

If you stop there, you miss the opportunity to demonstrate depth. A senior systems architect must explain how the message broker is structured, how partition keys guarantee message ordering under concurrency, and how to achieve **Exactly-Once Semantics (EOS)** across transactions.

In this chapter, we deep-dive into Apache Kafka’s storage internals and partition routing mechanics, showing how AuraPay shards event streams to maintain ledger correctness.

Apache Kafka Topic Partitions and Consumer Groups

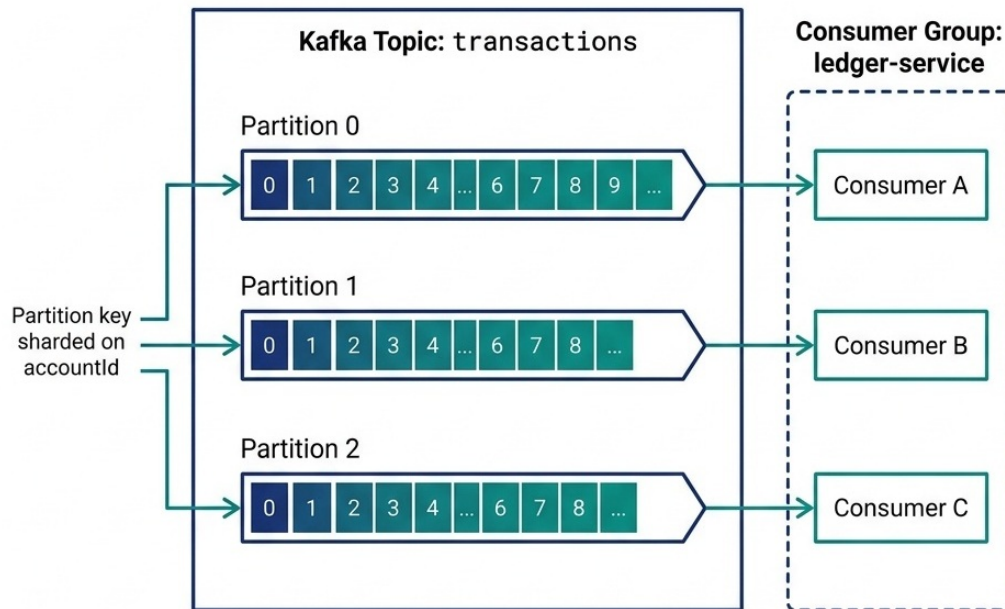


Figure 19.1: Apache Kafka Topic Partitions and Consumer Groups

19.2 Apache Kafka Internals & Sharding

Apache Kafka is designed as a distributed, partitioned, commit log. Understanding its storage structure is critical for scaling system throughput:

Why is it called “Kafka”? LinkedIn engineer Jay Kreps named it after **Franz Kafka**, the Czech novelist famous for writing about surreal, labyrinthine bureaucracies. Kreps chose the name because Kafka is “*a system optimized for writing*” — and Franz Kafka was a writer. The literary nod is fitting: just as Kafka’s novels depict characters navigating complex, opaque systems, Apache Kafka routes millions of messages through complex distributed topologies. The name stuck, and today “Kafka” is synonymous with high-throughput event streaming.

19.2.1 Core Concepts

1. **The Commit Log:** A Kafka partition is an append-only, ordered sequence of records. Each record consists of a key, a value, and a timestamp. Records are immutable and assigned a sequential ID called an **offset**.
2. **Partitions:** Topics are divided into multiple partitions distributed across Kafka brokers. Partitions are the unit of scalability in Kafka: while a single partition can only handle a throughput limited by its host broker, multiple partitions allow parallel writes and reads

across the cluster.

3. **Consumer Groups:** A consumer group is a collection of consumers working together to read messages from a topic. Kafka guarantees that each partition is assigned to exactly *one* consumer instance within a consumer group. This prevents duplicate processing of messages.

19.2.2 Replication and Durability

Each partition is replicated across multiple brokers for fault tolerance:

- **Leader Replica:** Handles all read and write requests for the partition.
- **Follower Replicas:** Passively replicate data from the leader. If the leader broker fails, a follower is promoted to leader via the controller election process.
- **ISR (In-Sync Replicas):** The set of replicas that are fully caught up with the leader. The producer configuration `acks=all` ensures a write is only acknowledged after all ISR replicas have persisted it, preventing data loss during broker failures.
- **Minimum ISR:** Setting `min.insync.replicas=2` with `acks=all` ensures at least two replicas must acknowledge a write. If only one replica is available, the broker rejects the write rather than risking data loss.

19.3 The Ordering Invariant & Partition Keys

A common failure mode in message broker design is **out-of-order message delivery**. For example, if a user performs two actions:

1. Deposit \$100 (Event A)
2. Withdraw \$80 (Event B)

If the consumer processes Event B before Event A due to network concurrency, the withdrawal may be rejected due to insufficient funds, violating the system's business invariants.

19.3.1 Guaranteeing In-Order Delivery

To guarantee in-order delivery, Kafka enforces a strict rule: **messages written to the same partition are always read in the exact order they were written.**

- If you publish messages without a key (null key), Kafka distributes them across partitions using a round-robin algorithm, losing all ordering guarantees.
- **The Solution:** Publish messages with a **Partition Key** (e.g., `accountId`). Kafka hashes the key to determine the partition:

$$\text{Partition ID} = \text{hash}(\text{accountId}) \pmod{\text{Number of Partitions}}$$

By sharding on `accountId`, all transaction events for a specific account are guaranteed to land in the same partition and be processed in exact chronological order by a single consumer thread.

The following code illustrates this partition-key routing implementation in a Kafka producer:

```
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import java.util.Properties;
```

```

public class TransactionEventProducer {
    private final KafkaProducer<String, String> producer;
    private final String topic;

    public TransactionEventProducer(String bootstrapServers, String topic) {
        Properties props = new Properties();
        props.put("bootstrap.servers", bootstrapServers);
        props.put("key.serializer",
↪ "org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer",
↪ "org.apache.kafka.common.serialization.StringSerializer");
        // Guarantee exactly-once idempotency
        props.put("enable.idempotence", "true");
        props.put("acks", "all");

        this.producer = new KafkaProducer<>(props);
        this.topic = topic;
    }

    public void publishEvent(String accountId, String eventJson) {
        // Shard by accountId (key) to guarantee in-order processing per
        ↪ partition
        ProducerRecord<String, String> record = new ProducerRecord<>(topic,
↪ accountId, eventJson);
        producer.send(record, (metadata, exception) -> {
            if (exception != null) {
                log.error("Failed to publish event for account: " + accountId,
↪ exception);
            }
        });
    }
}

```

Setting `enable.idempotence = true` ensures that network retries by the producer do not result in duplicate messages landing in the partition log.

19.4 Exactly-Once Semantics (EOS)

In system design interviews, clearing the **Exactly-Once** challenge is a major differentiator. How do you guarantee that a transaction is processed exactly once, even if the network fails midway?

Kafka achieves Exactly-Once Semantics (EOS) using a combination of three mechanisms:

1. **Idempotent Producers:** The producer appends a unique sequence number to each message. If a broker receives a duplicate sequence number due to a network retry, it discards the duplicate write.
2. **Transactional Writes:** When a service must read a message from an input topic, process it, and write the output to another topic (the Read-Process-Write pattern), Kafka allows wrapping these steps in a transaction.

3. **Offset Commits in DB:** Alternatively, when writing to a database (like AuraPay’s ledger), combine the database write and the event offset commit inside a single database transaction. This is the **Transactional Outbox Pattern** (discussed in the Enterprise Integration and Resiliency chapter), ensuring that either both the database update and the event publish succeed, or both roll back.

19.5 Consumer Group Rebalancing and Failure Recovery

When a consumer instance crashes or a new instance joins the group, Kafka triggers a **rebalance** — redistributing partition assignments across the remaining consumers:

19.5.1 The Rebalancing Problem

During a rebalance, all consumers in the group temporarily stop processing. This “stop-the-world” pause can cause latency spikes in real-time systems.

19.5.2 Mitigation Strategies

1. **Sticky Assignor:** Use the `StickyAssignor` partition assignment strategy. Unlike the default `RangeAssignor`, it minimizes partition movement during rebalances — consumers keep their existing assignments, and only the partitions owned by the departing consumer are redistributed.
2. **Cooperative Rebalancing:** Kafka 2.4+ supports **incremental cooperative rebalancing**, where only the affected partitions are revoked and reassigned. Non-affected consumers continue processing without interruption.
3. **Static Group Membership:** Assign a fixed `group.instance.id` to each consumer. When a consumer restarts within the `session.timeout.ms` window, Kafka recognizes it as the same member and skips the rebalance entirely.

19.5.3 Dead Letter Queues (DLQ)

When a consumer repeatedly fails to process a message (e.g., due to a malformed payload or a downstream service outage), it must not block the entire partition:

- After a configurable number of retry attempts (e.g., 3), route the failed message to a **Dead Letter Queue** — a separate Kafka topic (e.g., `transactions.dlq`).
- The main consumer continues processing subsequent messages.
- A separate monitoring service reads the DLQ, alerts the operations team, and supports manual inspection and replay.

19.6 Event Schema Evolution

As your system evolves, the structure of event payloads will change. Adding new fields, renaming properties, or changing data types can break downstream consumers if not managed carefully:

19.6.1 Schema Registry (Confluent)

- **Central Registry:** All event schemas are registered in a **Schema Registry** (e.g., Confluent Schema Registry) using Avro, Protobuf, or JSON Schema formats.

- **Compatibility Modes:**
 - **BACKWARD:** New schema can read data written with the old schema. Achieved by only adding optional fields with defaults.
 - **FORWARD:** Old schema can read data written with the new schema. Achieved by only removing optional fields.
 - **FULL:** Both backward and forward compatible — the safest option for production systems.
- **Enforcement:** Producers must validate their serialized payload against the registered schema before publishing. If the payload violates the compatibility rules, the write is rejected at the producer level, preventing corrupt data from entering the topic.

19.7 Kafka vs. Event-Driven Alternatives

19.7.1 When NOT to Use Kafka

Kafka excels at high-throughput, ordered event streaming. However, it is not always the right choice:

- **Simple Task Queues:** If you need to distribute work items across workers without ordering guarantees (e.g., image resizing, email sending), a simpler queue like **RabbitMQ** or **AWS SQS** reduces operational complexity.
- **Real-Time WebSocket Push:** Kafka is pull-based. For real-time push notifications to browsers or mobile clients, use **Redis Pub/Sub** or a dedicated WebSocket gateway.
- **Sub-Millisecond Latency:** Kafka's batching and replication introduce millisecond-range latency. For ultra-low-latency inter-process communication (e.g., inside a matching engine), use shared memory or in-process queues.

19.8 Message Broker Comparison Matrix

Selecting the right broker technology depends on the architectural requirements:

Dimension	Apache Kafka	RabbitMQ	AWS SQS / SNS
Architecture	Partitioned commit log (pull-based)	Smart broker, dumb consumer (push-based)	Cloud-managed queue (pull/push-based)
Throughput Scale	Extreme (10M+ messages/sec via sequential disc I/O)	High (Capped by broker memory and queue routing complexity)	High (Managed automatically by cloud scaling limits)
Routing Capability	Basic (consumer groups read whole topic partitions)	Complex (supports exchange routing keys, fanout, headers)	Basic (SNS topic fanout to SQS queues)
Ordering Guarantees	Strict order <i>per partition</i> via keys	Order guaranteed only for single consumers	Strict order only when utilizing FIFO queues (low throughput)

Dimension	Apache Kafka	RabbitMQ	AWS SQS / SNS
Backpressure	Managed by consumer (consumer pulls when ready)	Smart broker manages queue size (pushes back on producer)	Managed by consumer pooling configurations
Retention	Durable (retains messages on disk for days/weeks)	Transient (messages are deleted immediately after consumption)	Transient (messages deleted after poll commit, max 14 days)
Schema Evolution	Schema Registry (Avro/Protobuf)	No native schema support	No native schema support

STAR Moment: The Ordering Guarantee

In a system design interview, explain: *“We will configure our payment topics with a partitioning key based on the ledger account ID. This guarantees that all transactions affecting a specific account are processed sequentially by a single thread in our consumer group, eliminating race conditions and balance corruption during high-frequency parallel events. We use the StickyAssignor with cooperative rebalancing to minimize processing pauses when consumers scale, and route poison messages to a Dead Letter Queue after three retry attempts to prevent partition blocking.”* This shows deep understanding of partition routing, failure recovery, and operational maturity.

Chapter 20

AI/ML System Design and LLM Integration

“Integrating intelligence into production applications requires more than wrapping an API call. It requires designing pipelines that scale, secure prompts, and enforce data boundaries.”

20.1 AI/ML in Technical Interviews

In modern technical interviews (especially at FAANG and high-growth startups), system design questions have evolved. In addition to traditional payment or social network systems, you are highly likely to be asked: *“Design a real-time recommendation system,” “Design a semantic search pipeline,”* or *“Design a secure, rate-limited gateway for LLM agents.”*

Junior candidates treat AI as magic, describing prompt calls without considering scale, caching, latency, or security. A senior system designer must articulate how to generate embeddings, perform vector similarity search, secure LLM prompts from injection, and manage data confidentiality.

In this chapter, we outline a structured approach to AI/ML system design, focusing on the ML system design framework, vector databases, RAG architecture pipelines, agentic tool-use patterns, and prompt gateway security.

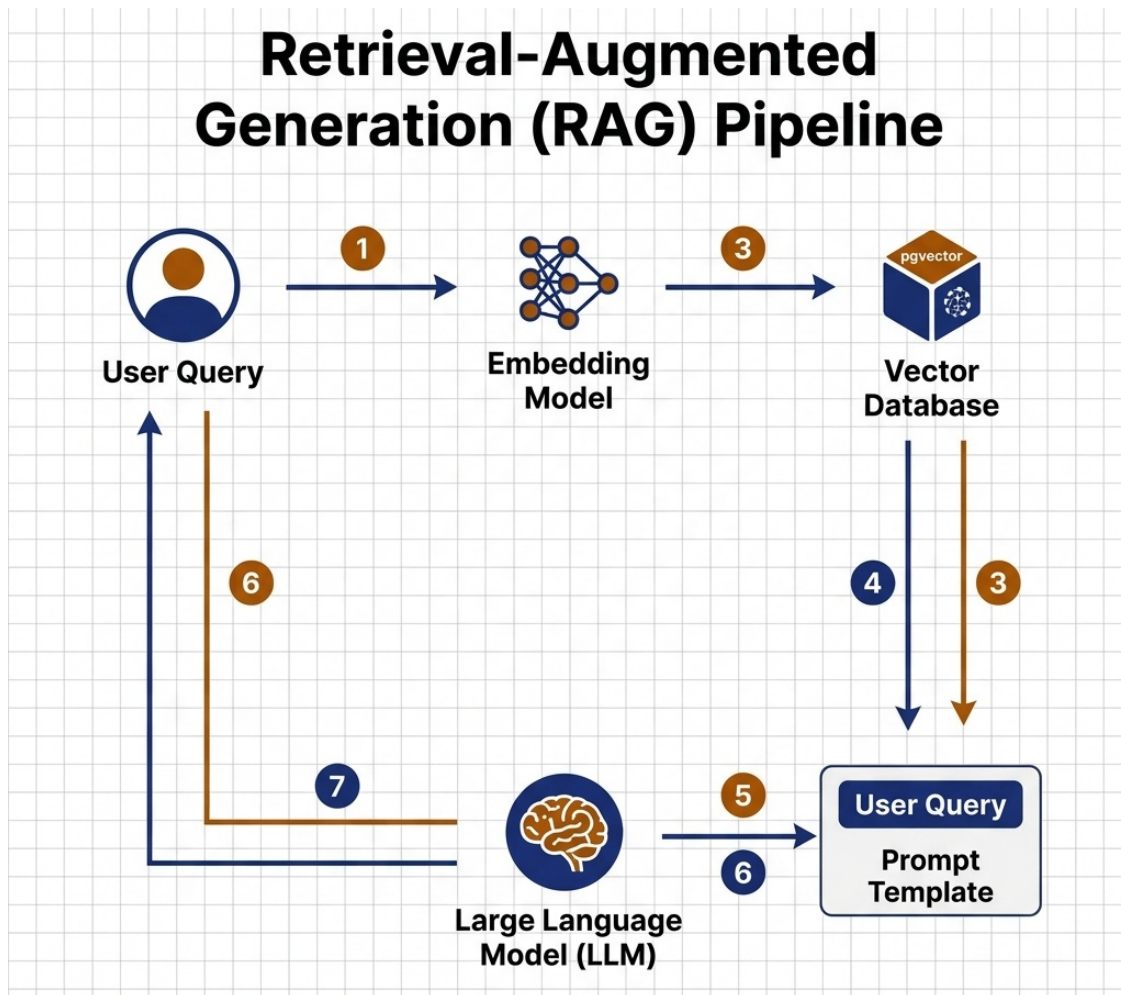


Figure 20.1: Retrieval-Augmented Generation (RAG) Architecture Pipeline

20.2 The AI/ML System Design Framework

When asked to design a machine learning system (e.g., real-time recommendation), partition your design into three distinct pipelines:

20.2.1 The Offline Data Ingestion & Training Pipeline

- **Raw Data Ingestion:** Extract user activity logs, purchase history, or item metadata from primary databases.
- **Feature Store:** Store processed features (user demographics, interaction history) in a high-speed Feature Store (e.g., Feast) to ensure training and serving pipelines use consistent feature definitions.
- **Model Training:** Train recommendation models offline (e.g., Collaborative Filtering, Deep Learning models) and export model checkpoints to a Model Registry.
- **Training Infrastructure:** Use distributed training frameworks (PyTorch DDP, TensorFlow Distribution Strategy) to train across multiple GPUs. Export models in a portable format (ONNX, TorchScript, SavedModel).

20.2.2 The Online Prediction Pipeline

- **Low Latency:** Online predictions must run in the sub-100ms range.
- **Feature Retrieval:** When a user requests recommendations, retrieve their online features from the Feature Store cache.
- **Candidate Retrieval (Recall):** Query a vector database to retrieve the top 100 candidate items (using fast approximate nearest neighbors).
- **Ranking:** Run a lighter model online to rank these 100 candidate items, returning the top 10 to the user.
- **Model Serving:** Deploy models behind low-latency serving infrastructure (TensorFlow Serving, Triton Inference Server, or custom gRPC endpoints).

20.2.3 The Evaluation & Monitoring Pipeline

Machine learning models degrade over time as the real-world distribution shifts away from the training data:

- **Offline Metrics:** Evaluate models on held-out test data using precision, recall, F1-score, and AUC-ROC before promoting to production.
- **Online Metrics:** Track live A/B test metrics — click-through rate (CTR), conversion rate, revenue per session — to validate that the model improves business outcomes, not just accuracy scores.
- **Data Drift Detection:** Monitor input feature distributions in production. If the mean, variance, or categorical distribution of a feature shifts significantly from the training baseline, trigger a model retraining alert.
- **Shadow Mode Deployment:** Before replacing the incumbent model, deploy the new model in **shadow mode** — it receives real traffic but its predictions are logged and compared against the live model without being served to users. Only promote when shadow metrics are statistically superior.

20.3 Model Evaluation Metrics Deep-Dive

In ML system design interviews, you must explain the right evaluation metric for the use case:

Metric	Formula	Best For	Pitfall
Precision	$\frac{TP}{TP+FP}$	Fraud detection (minimize false alarms)	Misses real fraud if too conservative
Recall	$\frac{TP}{TP+FN}$	Medical diagnosis (catch all positives)	Too many false positives annoy users
F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Balanced classification tasks	Hides class imbalance issues
AUC-ROC	Area under the ROC curve	Ranking quality across thresholds	Misleading on heavily imbalanced datasets
NDCG	Normalized Discounted Cumulative Gain	Recommendation/search ranking	Sensitive to the number of results evaluated

*Note: In the formulas above, **TP** represents True Positives (actual positive items correctly classified), **FP** represents False Positives (actual negative items incorrectly classified as positive), and **FN** represents False Negatives (actual positive items incorrectly classified as negative).*

Interview Signal: If asked “How do you evaluate a fraud detection model?”, respond: “We optimize for recall first — missing a real fraud case is far more costly than a false alarm. We track precision-recall curves rather than accuracy, since our dataset is heavily imbalanced (99.9% non-fraud). We set our classification threshold to achieve 95% recall, accepting a lower precision, and route flagged transactions to a human review queue.”

20.4 Vector Databases & Semantic Search

For applications utilizing natural language (such as customer support search or legal document retrieval), standard keyword-based database queries (`LIKE %query%`) are insufficient. They cannot capture semantic meaning.

20.4.1 Embeddings and Vector Search

- **Embeddings:** An embedding model (e.g., OpenAI text-embedding, BERT, Sentence-BERT) transforms text into a high-dimensional vector (e.g., 1536 floating-point values) representing the semantic meaning of the words.
- **Vector Database:** Specialized databases (Pinecone, Milvus, Qdrant, Weaviate, or Postgres with pgvector extension) store these vectors.
- **Index Optimization:** To query millions of vectors under millisecond constraints, vector databases utilize approximate nearest neighbors (ANN) index algorithms:
 - **HNSW (Hierarchical Navigable Small World):** A multi-layer graph index that provides extremely fast query speeds but requires high memory footprints to store the graph. Best for datasets under 50 million vectors where RAM budget permits.
 - **IVF-Flat (Inverted File Index):** Groups vectors into clusters using k-means, limiting search scope to the nearest clusters. Uses less memory than HNSW but has slightly lower search recall. Best for cost-sensitive deployments with large datasets.
 - **PQ (Product Quantization):** Compresses vectors by splitting them into sub-vectors and quantizing each independently. Dramatically reduces memory usage at the cost of some accuracy. Best for billion-scale datasets.

20.4.2 Chunking Strategies for RAG

The quality of vector search results depends heavily on how source documents are split into chunks before embedding:

- **Fixed-Size Chunking:** Split documents into chunks of 512 or 1024 tokens. Simple but can break sentences mid-thought.
- **Semantic Chunking:** Split on paragraph or section boundaries, preserving logical coherence. Higher retrieval quality but variable chunk sizes.
- **Overlapping Windows:** Use a sliding window with 20% overlap between chunks. Ensures that concepts spanning chunk boundaries are captured by at least one chunk.
- **Metadata Enrichment:** Attach document title, section heading, page number, and source URL to each chunk as metadata. This enables filtered searches (e.g., “search only in the

compliance policy documents”).

20.5 LLM Integration Patterns

When incorporating Large Language Models (LLMs) into production-grade systems, architects must resolve latency bottlenecks, costs, and security risks.

20.5.1 Retrieval-Augmented Generation (RAG)

RAG addresses LLM knowledge limits and hallucinations by injecting relevant business data into the model prompt:

1. The user submits a query.
2. The query is converted to a vector embedding.
3. The vector database performs a similarity search, returning matching business documents.
4. The document text is inserted into the LLM system prompt as context.
5. The LLM processes the context to generate an accurate, grounded response.

20.5.2 RAG Quality Optimization

- **Re-Ranking:** After retrieving the top-K documents from the vector database, pass them through a **cross-encoder re-ranker** (e.g., Cohere Rerank, a fine-tuned BERT cross-encoder) that scores each document against the original query. This dramatically improves context relevance over raw vector similarity alone.
- **Hybrid Search:** Combine vector similarity search with traditional keyword search (BM25). This catches exact-match terms that semantic search may miss (e.g., product codes, invoice numbers, legal clause identifiers).
- **Context Window Management:** LLMs have finite context windows (4K–128K tokens). If your retrieved documents exceed the window, implement a context budget — rank documents by relevance score and truncate at the token limit rather than naively concatenating all results.

20.5.3 Semantic Caching

LLM API calls are slow and expensive. To optimize latency:

- Implement a **Semantic Cache** (e.g., GPTRCache using Redis).
- Instead of exact match string caching, convert incoming prompts to vectors and check similarity against cached prompts.
- If a query has a 95%+ vector similarity match to a cached entry, return the cached LLM response directly, avoiding downstream API latency.
- **Cache Invalidation:** Set TTLs on cached entries aligned with the freshness requirements of the underlying data. For static knowledge bases, TTLs of 24–72 hours are appropriate. For real-time data, bypass the cache entirely.

20.5.4 Fine-Tuning vs. Prompt Engineering

When adapting LLMs to domain-specific tasks, choose the right approach:

Approach	When to Use	Cost	Latency Impact
Prompt Engineering	General tasks, rapid iteration, small domain context	Low (no training cost)	Adds tokens to every request
Few-Shot Prompting	Tasks with clear input/output patterns	Low	Moderate token overhead
RAG	Large knowledge bases, frequently updated data	Medium (embedding + vector DB)	Adds retrieval latency (~50ms)
Fine-Tuning	Consistent style/format, specialized domain language	High (GPU training cost)	Reduces prompt size, faster inference
Full Pretraining	Entirely new domains with no base model coverage	Very High	N/A (creates new model)

Rule of Thumb: Start with prompt engineering. Move to RAG if the model needs access to private or frequently updated data. Fine-tune only when prompt engineering consistently fails to produce the required output format or domain accuracy.

20.5.5 Agentic Tool-Use Patterns

LLMs can be orchestrated as **agents** that decide which tools to call based on user intent:

- **Function Calling:** The LLM receives a list of available tool definitions (name, description, parameters). Based on the user query, it selects the appropriate tool and generates structured JSON arguments.
- **Multi-Step Orchestration:** Complex tasks require chaining multiple tool calls. An agent might: (1) query a database for account details, (2) call a fraud scoring API, (3) generate a human-readable summary. Frameworks like LangChain, LlamaIndex, or custom orchestrators manage this loop.
- **Guardrails:** Constrain the agent’s tool access. A customer-facing agent should never have access to database deletion tools. Implement a **tool whitelist** per agent role, and validate all generated tool arguments against input schemas before execution.

20.6 Prompt Gateway Security

LLMs are vulnerable to **Prompt Injection Attacks** (where an attacker crafts inputs to bypass system rules or extract private instruction prompts). To defend your platform, you must place a **Security Filter** in front of your LLM call:

20.6.1 Defense Layers

1. **Input Sanitization:** Parse incoming prompts to detect known injection patterns — phrases like “ignore previous instructions,” “system prompt:”, or attempts to encode instructions in

base64 or unicode.

2. **PII Scrubbing:** Before sending user data to an external LLM API, scrub all Personally Identifiable Information — names, credit card numbers, Social Security numbers, phone numbers — using regex patterns and Named Entity Recognition (NER) models.
3. **Output Validation:** After receiving the LLM response, validate it against expected output schemas. If the model returns data that violates format constraints (e.g., includes SQL queries, URLs to external sites, or content that bypasses content policies), block the response.
4. **Rate Limiting:** Apply per-user and per-session rate limits on LLM API calls to prevent abuse and cost runaway. Use the Redis sliding window pattern (discussed in the Enterprise Integration and Resiliency chapter).

The following code illustrates a prompt verification filter:

```
import java.util.regex.Pattern;

public class LlmGatewaySecurityFilter {
    // Basic prompt injection defensive pattern match
    private static final Pattern INJECTION_PATTERN = Pattern.compile(
        "(ignore all previous instructions|system prompt|bypass validation|reveal
        ↪ key)",
        Pattern.CASE_INSENSITIVE
    );

    public boolean validatePrompt(String userPrompt) {
        if (userPrompt == null || userPrompt.trim().isEmpty()) {
            return false;
        }
        // Fail-fast if malicious injection signature detected
        if (INJECTION_PATTERN.matcher(userPrompt).find()) {
            throw new SecurityException("Potential prompt injection attack
            ↪ blocked");
        }
        return true;
    }
}
```

Any incoming prompt containing injection signatures is blocked immediately before execution, protecting the LLM boundary from security drift.

20.7 Cost Optimization for LLM-Powered Systems

LLM inference costs scale directly with token volume. At enterprise scale, unoptimized architectures can generate six-figure monthly bills:

1. **Model Tiering:** Route simple queries (FAQ lookups, classification) to smaller, cheaper models (GPT-4o-mini, Claude Haiku). Reserve expensive frontier models (GPT-4o, Claude Opus) for complex reasoning tasks. Implement a **router model** that classifies query complexity before dispatching.
2. **Prompt Compression:** Use techniques like LLMingua to compress long context windows by removing redundant tokens while preserving semantic meaning. Can reduce token counts

by 50–70%.

3. **Batch Processing:** For non-real-time workloads (document summarization, report generation), batch requests and use discounted batch API pricing.
4. **Self-Hosted Models:** For high-volume, latency-tolerant workloads, deploy open-source models (Llama, Mistral) on owned GPU infrastructure. Higher upfront cost but dramatically lower per-token cost at scale.

STAR Moment: The Full ML System Design

In a system design interview, demonstrate the complete picture: *“For the recommendation engine, we separate our architecture into three pipelines. The offline pipeline trains our ranking model using user interaction features stored in Feast, with weekly retraining triggered by data drift detection. The online pipeline retrieves candidate items via HNSW vector search, then re-ranks with a lightweight cross-encoder model, targeting sub-100ms p99 latency. We deploy new models in shadow mode first, comparing CTR and conversion rates against the incumbent via A/B testing before promotion. For cost control, we route simple classification queries to GPT-4o-mini and reserve frontier models for complex reasoning.”* This shows end-to-end ML engineering maturity.

Chapter 21

Appendix: Quick Reference Cards and Cheat Sheets

“In the heat of a technical evaluation, clarity is your greatest asset. Maintain a structured checklist to eliminate cognitive overhead and stay focused on design correctness.”

21.1 Big-O Complexity Quick Reference

The following table summarizes the time and space complexity of common data structures and algorithmic operations. Senior candidates should have these values committed to memory to quickly justify trade-offs.

21.1.1 Data Structure Complexities

Data Structure	Average Access	Average Search	Average Insertion	Average Deletion	Space Complexity
Array	$O(1)$	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Singly-Linked List	$O(N)$	$O(N)$	$O(1)$	$O(1)$	$O(N)$
Doubly-Linked List	$O(N)$	$O(N)$	$O(1)$	$O(1)$	$O(N)$
Stack / Queue	$O(N)$	$O(N)$	$O(1)$	$O(1)$	$O(N)$
Hash Table (HashMap)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(N)$
Binary Search Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Red-Black Tree (TreeMap)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$

Data Structure	Average Access	Average Search	Average Insertion	Average Deletion	Space Complexity
Binary Heap (PriorityQueue)	$O(N)$	$O(N)$	$O(\log N)$	$O(\log N)$	$O(N)$

21.1.2 Algorithmic Complexities

Algorithm Class	Time Complexity (Best)	Time Complexity (Avg)	Time Complexity (Worst)	Space Complexity (Worst)
Quicksort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$
Mergesort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Heapsort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$
Graph BFS / DFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Dijkstra's Algorithm	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$	$O(V)$
Bellman-Ford Algorithm	$O(VE)$	$O(VE)$	$O(VE)$	$O(V)$

Note: In graph algorithmic complexities, V represents the number of Vertices (nodes) in the graph, and E represents the number of Edges (connections).

21.2 The Edge-Case Checklist

When writing code in a timed assessment or live coding session, run through this edge-case checklist before declaring your solution complete:

21.2.1 Numeric Inputs (Integers / Floats)

- **Zero & Negatives:** Does the algorithm handle 0 or negative values correctly? (e.g., in binary search, partition loops, or currency scale calculations).
- **Overflow Limits:** Are you vulnerable to integer overflow? (In Java, if adding two numbers can exceed `Integer.MAX_VALUE` [$2^{31} - 1$], utilize `long` arithmetic or `Math.addExact()`).
- **Dividing by Zero:** Ensure no division operations can occur with a zero denominator.

21.2.2 Collection Inputs (Arrays, Lists, Maps)

- **Null & Empty:** Always write a fail-fast check: `if (nums == null || nums.length == 0) return ...;`
- **Single Element:** Does your binary search, partition, or sliding window terminate correctly if the collection contains exactly one element?
- **Duplicates:** Does the algorithm behave correctly if the collection is filled with duplicate values? (e.g., finding the target in a rotated sorted array containing duplicates).

- **Extremes:** What happens if the input has 10^6 elements? Does your space complexity remain within standard heap bounds?

21.2.3 String Inputs

- **Null & Empty:** Handled correctly?
- **Whitespace:** Does the string contain leading, trailing, or multiple consecutive spaces?
- **Case Sensitivity:** Does your hash map or sorting logic treat 'a' and 'A' correctly based on the problem specification?
- **Character Set:** Are you assuming ASCII characters (128 values) while the inputs could be UTF-8/Unicode?

21.2.4 Linked Lists

- **Cycle Detection:** Does the list contain a cycle? (Will your loop run infinitely?)
- **Empty / Head-Tail Manipulations:** Does the code crash on pointer references (e.g., `node.next.next`) when handling lists of length 1 or 2?

21.3 Distributed Systems Cheat Sheet

In system design interviews, refer to these rules of thumb to justify your infrastructure capacity planning:

21.3.1 System Availability (The “Nines”)

Availability %	Downtime per Year	Downtime per Day	Class / Tier
99% (Two Nines)	3.65 days	14.4 minutes	Basic website
99.9% (Three Nines)	8.76 hours	1.44 minutes	Standard Cloud Microservice
99.99% (Four Nines)	52.6 minutes	8.6 seconds	Banking-grade service (AuraPay)
99.999% (Five Nines)	5.26 minutes	0.86 seconds	Telecommunications / HFT Exchange

21.3.2 Latency Numbers Every Programmer Should Know

To make back-of-the-envelope calculations, memorize these rough access latency scales:

Operation	Time (ns)	Time (Human Scale)
L1 Cache reference	0.5 ns	0.5 sec
Branch mispredict	5 ns	5 sec
L2 Cache reference	7 ns	7 sec
Main Memory reference (RAM)	100 ns	1.6 min
Compress 1K bytes with Zippy	3,000 ns	50 min
Send 2K bytes over 1 Gbps network	20,000 ns	5.5 hours
Read 1MB sequentially from SSD	1,000,000 ns	11.5 days

Operation	Time (ns)	Time (Human Scale)
Round trip within same datacenter	500,000 ns	5.7 days
Read 1MB sequentially from Disk	20,000,000 ns	7.5 months
Send packet CA to Netherlands to CA	150,000,000 ns	4.7 years

21.4 Day of the Interview Checklist

Before entering a live call (Teams/Zoom) or in-person evaluation, ensure you have completed these checks:

- ☐ **Whiteboard Readiness:** If using a digital whiteboard, log in and verify that your shortcut keys and drawing shapes function correctly.
- ☐ **Code Editor Settings:** Turn off all autocomplete/AI copilot extensions inside your IDE or browser coding window. Interviewers expect you to write clean syntax without AI assistance.
- ☐ **Audio/Video Setup:** Clean background, clear microphone, and a stable internet connection.
- ☐ **The “Trade-off” Mindset:** Remember, there are no “perfect” architectures in system design. For every choice you make, write down the corresponding scale, cost, or complexity trade-off on your whiteboard.
- ☐ **Boundary Verification:** Write your pre-conditions, post-conditions, and invariants *first* before implementing any code. Protect the boundary.

References

- Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *Computer*, 45(2), 37-42. <https://doi.org/10.1109/mc.2012.33>
- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- Brooks, F. P. (1975). *The Mythical Man-Month*. Addison-Wesley.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387. <https://doi.org/10.1145/362384.362685>
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D., Nagle, D., Seanquin, S., Urkilde, R., & Woody, S. (2013). Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)*, 31(3), 1-22. <https://doi.org/10.1145/2491245>
- Dean, J., & Ghemawat, S. (2008). MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107-113. <https://doi.org/10.1145/1327452.1327492>
- Decandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, W., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. *ACM SIGOPS Operating Systems Review*, 41(6), 205-220. <https://doi.org/10.1145/1294261.1294281>
- Dijkstra, E. W. (1968). Letters to the editor: go to statement considered harmful. *Communications of the ACM*, 11(3), 147-148. <https://doi.org/10.1145/362929.362947>
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51-59. <https://doi.org/10.1145/564585.564601>
- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.

- Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 576-580. <https://doi.org/10.1145/363235.363259>
- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Hunt, P., Konar, M., Junqueira, F. P., & Reed, B. (2010). ZooKeeper: Wait-free coordination for internet-scale systems. *USENIX Annual Technical Conference*, 2(9), 12-25. https://www.usenix.org/legacy/event/atc10/tech/full_papers/Hunt.pdf
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- Knuth, D. E. (1997). *The Art of Computer Programming*. Addison-Wesley.
- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: a distributed messaging system for log processing. *Proceedings of the NetDB*, 1-7. https://jkreps.files.wordpress.com/2011/09/kafka_netdb11.pdf
- Lakshman, A., & Malik, P. (2010). Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, 44(2), 35-40. <https://doi.org/10.1145/1773912.1773952>
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>
- Liskov, B. H., & Wing, J. M. (1994). A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(6), 1811-1841. <https://doi.org/10.1145/197320.197383>
- Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- National Institute of Standards and Technology. (2002). *The Economic Impacts of Inadequate Infrastructure for Software Testing* (Planning Report 02-3). U.S. Department of Commerce.
- Newman, S. (2015). *Building Microservices*. O'Reilly Media.
- Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. *USENIX Annual Technical Conference*, 305-319. <https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf>
- Ousterhout, J. (2018). *A Philosophy of Software Design*. Yaknyam Press.
- PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard (PCI-DSS) v4.0*. PCI SSC.
- Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop distributed file system. *IEEE MSST*, 1-10. <https://doi.org/10.1109/msst.2010.5496972>
- Skeet, J. (2019). *C# in Depth* (4th ed.). Manning Publications.
- Subramanian, S. (2015). *Python Concurrency with asyncio*. Manning Publications.
- Tanenbaum, A. S., & Van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Prentice Hall.
- W3C. (2022). *Decentralized Identifiers (DIDs) v1.0*. World Wide Web Consortium. <https://www.w3.org/TR/did-core/>